



ORACLE
NetSuite

SuiteScript Records Guide

2025.2

January 7, 2026



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

How to Use SuiteScript Records Help	1
SuiteScript Supported Records	2
Activities	14
Activity	14
Event	15
Phone Call	15
Project Task	16
Resource Allocation	19
Task	21
Work Calendar	22
Entities	23
As Charged Project Revenue Rule	24
Bonus	26
Bonus Type	28
Competitor	29
Contact	30
Customer	30
Customer Status	33
Employee	33
Employee Status	35
Employee Type	36
Entity	37
Generic Resource	37
Job	38
Lead	40
Other Name	42
Partner	43
Project	44
Project Status	48
Project Template	49
Project Type	50
Prospect	51
Time-Off Management	52
Vendor	59
Items	60
Using Item Records in SuiteScript	61
Pricing Sublist / Pricing Matrix	63
Analytical Impact	78
Assembly Item	79
Description Item	80
Discount Item	81
Download Item	82
Gift Certificate Item	83
Impact Subcategory	83
Inventory Item	84
Item Group	85
Item Search	86
Kit Item	86
Lot Numbered Assembly Item	87
Lot Numbered Inventory Item	88
Markup Item	89
Non-Inventory Part	89
Other Charge Item	90

Payment Item	91
Planned Order	91
Planning Item Category	92
Planning Item Group	93
Planning Rule Group	94
Planning View	96
Reallocate Items	97
SaaS Metric	98
Serialized Assembly Item	98
Serialized Inventory Item	99
Service	100
Shipping Item	101
Subscription Plan	101
Subscription Term	102
Subtotal Item	103
Supply Chain Snapshot Simulation	104
Supply Change Order	105
Supply Plan Definition	107
Communications	109
Message	109
Note	112
Transactions	113
Assembly Build	116
Assembly Unbuild	117
Bin Transfer	118
Blanket Purchase Order	119
Cash Refund	120
Cash Sale	121
Check	123
Credit Memo	125
Customer Deposit	126
Customer Refund	127
Deposit	129
Deposit Application	132
Employee Expense Source Type	133
Estimate	134
Expense Report	135
Expense Report Policy	136
Fulfillment Request	139
Imported Employee Expense	144
Inventory Adjustment	145
Inventory Cost Revaluation	146
Inventory Count	150
Inventory Status Change	157
Inventory Transfer	158
Invoice	159
Invoice Group	164
Demand Plan Detail Sublist	166
Item Fulfillment	173
Item Receipt	176
Opportunity	178
Order Reservation	179
Paycheck	180
Paycheck Journal	181
Payroll Batch	184

Payroll Batch Employee	187
Period End Journal	188
Project Intercompany Cross Charge Request	189
Purchase Contract	191
Purchase Order	192
Requisition	194
Resource Group	197
Return Authorization	198
Revenue Arrangement	200
Revenue Commitment	202
Revenue Commitment Reversal	203
Sales Channel	203
Sales Order	204
Store Pickup Fulfillment	207
Time Actions	209
Transfer Order	210
Vendor Prepayment	211
Vendor Prepayment Application	214
Vendor Return Authorization	216
Wave	217
Work Order	219
Work Order Close	221
Work Order Completion	222
Work Order Issue	224
Transactions-related Records	225
Advanced Intercompany Journal Entry	226
Balance Transactions by Segments	228
Bin Putaway Worksheet	230
Budget Machine	231
Change Order	235
Charge	236
Credit Card Charge	237
Credit Card Refund	239
Customer Payment	241
Demand Plan Detail Sublist	242
GL Audit Numbering Sequence	249
Intercompany Journal Entry	250
Intercompany Transfer Order	253
Inventory Detail	254
Item Demand Plan	255
Item Supply Plan	257
Journal Entry	259
Landed Cost	261
Manufacturing Operation Task	264
Manufacturing Planned Time	266
Memorized Transaction Definition	267
Multi-Book Accounting Transaction	269
Order Schedule	270
Pick Strategy	271
Pick Task	273
Statistical Journal Entry	276
Subscription	279
Subscription Line	280
Time	280
Transaction Search	282

Unlocked Time Period	283
Usage	284
Vendor Bill	285
Vendor Credit	286
Vendor Payment	287
Weekly Timesheet	290
Support	300
Case	300
Issue	301
Solution	303
Topic	303
File Cabinet	304
File	304
Folder	305
Lists	305
Account	308
Accounting Book	309
Accounting Context	310
Accounting Period	311
Allocation Schedule	312
Amortization Schedule	314
Amortization Template	315
Automated Clearing House	317
Billing Account	318
Billing Class	319
Billing Rate Card	320
Billing Schedule	323
Bin	326
BOM	327
BOM Revision	331
Budget Category	333
Budget Exchange Rate	335
Class	337
Consolidated Exchange Rate	338
Contact Category	339
Contact Role	340
Cost Category	341
Currency	341
Currency Rate	342
Customer Category	344
Customer Message	344
Customer-Subsidiary Relationship	345
Department	346
Employee Change Request	347
Employee Change Request Type	349
Expense Category	350
Fair Value Price	351
Financial Institution	352
Format Profile	354
Gift Certificate	360
Global Account Mapping	360
Global Inventory Relationship	362
Goal	366
Government-Issued ID Tracking	367
Group	371

Inbound Shipment	372
Intercompany Allocation Schedule	375
Inventory Number	377
Item Account Mapping	377
Item Collection	380
Item Collection Item Map	381
Item Location Configuration	382
Item Process Family	383
Item Process Group	384
Item Revision	385
Kudos Feature Records	386
Location	388
Manufacturing Cost Template	388
Manufacturing Routing	389
Merchandise Hierarchy Level	389
Merchandise Hierarchy Node	390
Merchandise Hierarchy Version	391
Nexus	393
Note Type	394
Order Type	394
Other Name Category	395
Partner Category	396
Payment Instruments	396
Payment Method	398
Payroll Item	398
Performance Metric	402
Performance Review	403
Performance Review Schedule	404
Price Book	405
Price Level	405
Price Plan	406
Price Rule	407
Price Rule Entry	408
Pricing Group	410
Project Charge Rule	410
Project Expense Type	413
Project Revenue Rule	414
Project Work Breakdown Structure	415
Recalculate Percent Complete Override	420
Revenue Recognition Event	421
Revenue Recognition Field Map	423
Revenue Recognition Plan	424
Revenue Recognition Schedule	425
Revenue Recognition Template	426
Role	426
Sales Role	427
Sales Tax Item	428
Subsidiary	428
Subsidiary Settings	429
Supply Chain Snapshot	431
System Note	436
Tax Control Account	437
Tax Group	441
Tax Period	443
Tax Type	443

Term	444
Unit of Measure	445
Vendor Category	445
Vendor-Subsidiary Relationship	446
Workplace	447
Zone	447
Customization	449
Custom List	449
Custom Record	450
Custom Segment Fields and Values	452
Custom Transaction	456
Prompt	458
Scheduled Script Instance	460
Script	462
Script Deployment	465
Text Enhance Action	468
Marketing	469
Campaign	469
Campaign Response	470
Campaign Template	471
Coupon Code	472
Email Template	472
Promotion	474
Website	477
CMS Content	477
CMS Content Type	478
CMS Page	480
Commerce Category	482
Website Setup	483
Shopping Cart	486

How to Use SuiteScript Records Help

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The SuiteScript Records Guide must be used in conjunction with the SuiteScript Records Browser. Each record listed in this guide includes a link to the Records Browser. Within the Records Browser, you'll find all available fields, sublists, and search filter fields for that record. The Records Browser also provides field level help for many fields that appears on the record. Within this guide, the description of each record often includes the following types of information:

- **Supported script types:** A description of whether the record is supported in both client and server SuiteScript, and whether the user events are supported
- **Supported functions:** A list of all functions that can be used with the record
- **Field definitions:** A link to the Records Browser and, in some cases, additional explanations about how to work with particular fields
- **Usage notes:** Additional details or context about working with the record



Important: For a list of all NetSuite records that are supported in SuiteScript, see [SuiteScript Supported Records](#).

Working Online

If you're working online, click the links to the SuiteScript Records Browser provided with each record description. The Records Browser will open in your default browser.

Downloading the SuiteScript Records Browser

If you want to use the SuiteScript Records Browser offline, you'll need to download the [SuiteScript Records Browser .zip file](#).

After downloading the .zip file to your local machine, extract the file contents and go to the script directory. To view the Records Browser content, open index.html file in the browser of your choice.

For information about using the Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

SuiteScript Supported Records

Applies to: SuiteScript 2.x | SuiteCloud Developer

Note: The topic applies to all versions of SuiteScript.

The following table lists all NetSuite records that are scriptable with SuiteScript. This table includes:

- clickable record names, which take you to detailed information about the record
- record IDs, which are often referenced in SuiteScript APIs
- scripting levels (where Full scripting level means that the record can be created, updated, copied, deleted, and searched)
- SuiteScript client or server support

Also note that all subrecords are scriptable from the line item level, unless stated otherwise.

In case the information you are looking for is not listed in the table below see the following help topics to find additional information about records:

- [SuiteCloud Reference Tools](#)
- [SuiteScript Records Browser](#)

The following help topics contain additional information about using the SuiteScript Records Browser:

- [Working with the SuiteScript Records Browser](#)
- [How to Use SuiteScript Records Help](#)

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Account	account	List	Full	—	✓
Accounting Book	accountingbook	List	Copy Not Supported	—	✓
Accounting Context	accountingcontext	List	Read, Create, Update, Copy, Delete, and Search	✓	✓
Accounting Period	accountingperiod	List	Read and Search Only	✓	✓
Activity	activity	Activity	Search Only	✓	✓
Address See the help topic Subrecord Scripting Overview .	addressbookaddress	Subrecord	See the help topic About the Address Subrecord .	✓	Server scripts must access through the parent record.
Advanced Intercompany Journal Entry	advintercompanyjournalentry	Transaction	Full	✓	✓
Allocation Schedule	allocationschedule	List	Create, Read, Update, and Delete	✓	✓
Amortization Schedule	amortizationschedule	List	Copy, Create, and Delete Not Allowed	—	✓
Amortization Template	amortizationtemplate	List	Full	—	✓
Analytical Impact	analyticalimpact	Item	Read, Update, Delete, Search	✓	✓
As Charged Project Revenue Rule	aschargedprojectrevenue rule	Entities	Full	✓	✓
Assembly Build	assemblybuild	Transaction	Full	—	✓
Assembly Item	assemblyitem	Item	Full	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Assembly Unbuild	assemblyunbuild	Transaction	Full	—	✓
Automated Clearing House	automatedclearinghouse	List	Copy and Transform Not Supported.	✓	✓
Balance Transactions by Segments	balancetransbysegments	Transaction	Create, Read, and Search	—	✓
Billing Account	billingaccount	List	Full	✓	✓
Billing Class	billingclass	List	Full		✓
Billing Rate Card	billingratecard	List	Read, Create, Update, Delete, and Search Copy and Transform Not Supported	✓	✓
Billing Schedule	billingschedule	List	Full	—	✓
Bin	bin	List	Full	—	✓
Bin Putaway Worksheet	binworksheet	Transaction	Copy and Update Not Supported	—	Dynamic Mode Only
Bin Transfer	bintransfer	Transaction	Full	—	✓
Blanket Purchase Order	blanketpurchaseorder	Transaction	Full	✓	✓
BOM	bom	List	Read, Create, Update, Copy, Delete, and Search	✓	✓
BOM Revision	bomrevision	List	Read, Create, Update, Copy, Delete, and Search	—	✓
Bonus	bonus	Entity	Read, Create, Update, Copy, Delete, and Search	✓	✓
Bonus Type	bonustype	Entity	Read, Create, Update, Copy, Delete, and Search	✓	✓
Budget Category	budgetcategory	List	Read, Create, Update, Delete, and Search	✓	✓
Budget Exchange Rate	budgetexchangerate	List	Read, Update, and Search	✓	✓
Campaign	campaign	Marketing	Full	✓	✓
Campaign Response	campaignresponse	Marketing	Full	✓	✓
Campaign Template	campaigntemplate	Marketing	Search Not Supported	—	✓
Case	supportcase	Support	Full	✓	✓
Cash Refund	cashrefund	Transaction	Full	✓	✓
Cash Sale	cashsale	Transaction	Full	✓	✓
Change Order	subscriptionchangeorder	Transaction	Full	✓	✓
Charge	charge	Transaction	Full	—	✓
Check	check	Transaction	Full	✓	✓
Class	classification	List	Server SuiteScript Only	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
CMS Content	cmscontent	Web Site	Create, Update, and Search Only Attach, Transform, and Copy Not Supported	—	✓
CMS Content Type	cmscontenttype	Web Site	Create, Update, and Search Only Attach, Transform, and Copy Not Supported	—	✓
CMS Page	cmspage	Web Site	Create, Update, and Search Only Attach, Transform, and Copy Not Supported	—	✓
Commerce Category	commercecategory	Website	Attach, Transform, and Copy Not Supported	—	✓
Competitor	competitor	Entity	Full	✓	✓
Consolidated Exchange Rate	consolidatedexchange rate	List	Read, Update, and Search Only Create and Delete Not Supported	✓	✓
Contact	contact	Entity	Full	✓	✓
Contact Category	contactcategory	List	Full	✓	✓
Contact Role	contactrole	List	Full	✓	✓
Cost Category	costcategory	List	Full	✓	✓
Coupon Code	couponcode	Marketing	Full	✓	✓
Credit Card Charge	creditcardcharge	Transaction	Full	—	✓
Credit Card Refund	creditcardrefund	Transaction	Full	—	✓
Credit Memo	creditmemo	Transaction	Full	✓	✓
Currency	currency	List	Full — with Multiple Currencies Feature Enabled Read Only — without Multiple Currencies Feature	—	✓
Currency Rate	currencyrate	List	Create, Read, Copy, and Search Only	✓	✓
Custom List	See Custom List IDs	Custom	Search Only	✓	✓
Custom Record	See Custom Record IDs	Custom	Full	✓	✓
Customer	customer	Entity	Full	✓	✓
Customer Category	customercategory	List	Search Not Supported	✓	✓
Customer Deposit	customerdeposit	Transaction	Full	✓	✓
Customer Message	customermessage	List	Full	✓	✓
Customer Payment	customerpayment	Transaction	Copy and Create Not Allowed	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Customer Refund	customerrefund	Transaction	Full	—	✓
Customer Status	customerstatus	Entity	Full	✓	✓
Customer-Subsidiary Relationship	customersubsidiaryrelationship	List	Full	✓	✓
Department	department	List	Server SuiteScript Only	—	✓
Deposit	deposit	Transaction	Copy Not Supported Dynamic Mode Not Supported	✓	✓
Deposit Application	depositapplication	Transaction	Create Not Allowed	✓	✓
Description Item	descriptionitem	Item	Full	✓	✓
Discount Item	discountitem	Item	Full	✓	✓
Download Item	downloaditem	Item	Full	✓	✓
Driver's License	driverslicense	List	Full	✓	✓ User event scripts are not supported.
Email Template	emailtemplate	Marketing	Search Not Supported	—	✓
Employee	employee	Entity	Full	✓	✓
Employee Change Request	employeechangerquest	List	Copy and Transform Not Supported	✓	✓
Employee Change Request Type	employeechangerequesttype	List	Copy and Transform Not Supported	✓	✓
Employee Expense Source Type	employeeexpensesourcetype	Transaction	Full	✓	✓
Employee Status	employeeestatus	Entity	Transform Not Supported	✓	✓
Employee Type	employeetype	Entity	Transform Not Supported	✓	✓
Entity	entity	Entity	Search Only	—	—
Estimate	estimate	Transaction	Full	✓	✓
Event	calendarevent	Activity	Full	✓	✓
Expense Category	expensecategory	List	Full	—	✓
Expense Report	expensereport	Transaction	Full	✓	✓
Expense Report Policy	expensereportpolicy	Transaction	Read, create, update, copy, delete, and search.	✓	✓
Fair Value Price	fairvaluepricelist	List	Create, Delete, and Search Only Copy Not Allowed	—	✓
File	file	File Cabinet	Search Only	—	✓
Financial Institution	financialinstitution	List	Read, Create, Update, Delete, and Search	—	✓
Folder	folder	File Cabinet	Full	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
					beforeLoad event is not supported on user event scripts.
Format Profile	formatprofile	List	Read, Create, Update, and Delete	✓	✓
Fulfillment Request	fulfillmentrequest	Transaction	Create and Copy Not Supported	✓	✓
General Token See Payment Instruments	generaltoken	List	Copy and Transform Not Supported	✓	✓
Generic Resource	genericresource	Entity	Copy Not Supported	✓	✓
Gift Certificate	giftcertificate	List	Full	✓	✓
Gift Certificate Item	giftcertificateitem	Item	Full	✓	✓
GL Audit Numbering Sequence	glnumberingsequence	Transaction	Read, Create, Update, Copy, Search	✓	✓
Global Account Mapping	globalaccountmapping	List	Full	—	✓
Global Inventory Relationship	globalinventoryrelationship	List	Read, Create, Edit, Delete, Search Copy Not Supported	✓	✓
Goal	goal	List	Full	✓	✓
Group	entitygroup	List	Full	—	✓
Impact Subcategory	impactsubcategory	Item	Read, Create, Update, Delete	✓	✓
Imported Employee Expense	importedemployeeexpense	Transaction	Full	✓	✓
Inbound Shipment	inboundshipment	List	Copy and Transform Not Supported	✓	✓
Intercompany Allocation Schedule	intercompallocationchedule	List	Create, Read, Update, and Delete	✓	✓
Intercompany Journal Entry	intercompanyjournalentry	Transaction	Full	✓	✓
Intercompany Transfer Order	intercompanytransferorder	Transaction	Full	✓	✓
Inventory Adjustment	inventoryadjustment	Transaction	Full	✓	✓
Inventory Cost Revaluation	inventorycostrevaluation	Transaction	Full	✓	✓
Inventory Count	inventorycount	Transaction	Full	✓	✓
Inventory Detail	inventorydetail	Subrecord	See the help topic Creating an Inventory Detail Sublist Subrecord Example .	✓	Server scripts must access through the parent record
Inventory Item (Also referred to in the UI as Inventory Part)	inventoryitem	Item	Full	✓	✓
Inventory Number	inventorynumber	List	Copy, Create, and Delete Not Supported	—	✓
Inventory Status	inventorystatus	List	Full	✓	✓
Inventory Status Change	inventorystatuschange	Transaction	Full	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Inventory Transfer	inventorytransfer	Transaction	Full	—	✓
Invoice	invoice	Transaction	Full	✓	✓
Issue	issue	Support	Full	✓	✓
Issue Product	issueproduct	Support	Copy, transform and search not supported.	✓	✓
Issue Product Version	productversion	Support	Copy, transform and search not supported.	✓	✓
Item Account Mapping	itemaccountmapping	List	Full	—	✓
Item Collection	itemcollection	List	Copy and Transform Not Supported	✓	✓
Item Collection Item Map	itemcollectionmap	List	Update, Copy, and Transform Not Supported	✓	✓
Item Demand Plan	itemdemandplan	Transaction	Copy Not Supported, No Available Transforms	✓	✓
Item Fulfillment	itemfulfillment	Transaction	Copy and Create Not Supported	✓	✓
Item Group	itemgroup	Item	Full	✓	✓
Item Location Configuration	itemlocationconfiguration	List	Read, Create, Edit, Delete, and Search	—	✓
Item Process Family	itemprocessfamily	List	Full	✓	✓
Item Process Group	itemprocessgroup	List	Full	✓	✓
Item Receipt	itemreceipt	Transaction	Copy and Create Not Supported	✓	✓
Item Revision	itemrevision	List	Full	—	✓
Item Search	item	Item	Search Only	—	—
Item Supply Plan	itemsupplyplan	Transaction	Full	✓	✓
Job	job	Entity	Copy and Transform Not Supported	✓	✓
Job Requisition	jobrequisition	Entity	Copy and Transform Not Supported	✓	✓
Journal Entry	journalentry	Transaction	Full	✓	✓
Kit Item	kititem	Item	Full	—	✓
Kudos Feature Records	kudos	List	Full Only the inactive field can be edited.	—	✓
Landed Cost	landedcost	Transaction	Full	—	✓
Lead	lead	Entity	Full	✓	✓
Location	location	List	Server SuiteScript Only	—	✓
Lot Numbered Assembly Item	lotnumberedassemblyitem	Item	Full	✓	✓
Lot Numbered Inventory Item	lotnumberedinventoryitem	Item	Full	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Manufacturing Cost Template	manufacturingcosttemplate	List	Full	—	✓
Manufacturing Operation Task	manufacturingoperationtask	Transaction	Copy Not Supported	✓	✓
Manufacturing Planned Time	mfgplannedtime	Transaction	Search Only	✓	✓
Manufacturing Routing	manufacturingrouting	List	Full	—	✓
Markup Item	markupitem	Item	Full	✓	✓
Memorized Transaction Definition	memdoc	Transaction	Read, Update, Delete, and Search	✓	✓
Merchandise Hierarchy Level	merchandisehierarchylevel	List	Read, Create, Edit, Delete, and Search	—	✓
Merchandise Hierarchy Node	merchandisehierarchynode	List	Read, Create, Edit, Delete, and Search	—	✓
Merchandise Hierarchy Version	merchandisehierarchyversion	List	Read, Create, Edit, Delete, and Search	—	✓
Message	message	Communication	For details, see Message .	—	✓
Multi-Book Accounting Transaction	accountingtransaction	Transaction	Search Only	✓	✓
Nexus	nexus	List	Copy Not Supported	—	✓
Non-Inventory Part	noninventoryitem	Item	Full	✓	✓
Note	note	Communication	Full	—	✓
Note Type	notetype	List	Full	✓	✓
Opportunity	opportunity	Transaction	Full	✓	✓
Order Reservation	orderreservation	Transaction	Created, read, updated, deleted, copied, and searched. It cannot be transformed	✓	✓
Order Schedule	orderschedule	Transaction	See the help topic Subrecord Scripting Overview .	✓	Server scripts must access through the parent record
Order Type	ordertype	List	Full	✓	✓
Organization Value	organizationvalue	List	Full	—	✓
Other Charge Item	otherchargeitem	Item	Full	—	—
Other Government-Issued ID	othergovernmentissuedid	List	Full	✓	✓ User event scripts are not supported.
Other Name	othername	Entity	Full	—	✓
Other Name Category	othernamecategory	List	Full	✓	✓
Partner	partner	Entity	Full	✓	✓
Partner Category	partnercategory	List	Full	✓	✓
Passport	passport	List	Full	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
					User event scripts are not supported.
Paycheck	paycheck	Transaction	Read and Update Only	✓	✓
Paycheck Journal	paycheckjournal	Transaction	Full	✓	✓
Payment Card See Payment Instruments	paymentcard	List	Copy and Transform Not Supported	✓	✓
Payment Card Token See Payment Instruments	paymentcardtoken	List	Copy and Transform Not Supported	✓	✓
Payment Item	paymentitem	Item	Full	✓	✓
Payment Instruments	paymentcard, paymentcardtoken, generaltoken	List	Copy and Transform Not Supported	✓	✓
Payment Method	paymentmethod	List	Full	✓	✓
Payroll Batch	payroll batch	Transaction	Full	✓	✓
Payroll Batch Employee	payrollbatchaddemployees	Transaction	Read and Update Only	✓	✓
Payroll Item	payrollitem	List	Full	✓	✓
Performance Metric	performancemetric	List	Full	✓	✓
Performance Review	performancereview	List	Copy not supported	✓	✓
Performance Review Schedule	performancereview schedule	List	Full	✓	✓
Period End Journal	periodendjournal	Transaction	Read, Search, limited Update	✓	✓
Phone Call	phonecall	Activity	Full	✓	✓
Pick Strategy	pickstrategy	Transaction	Full	✓	✓
Pick Task	picktask	Transaction	Update, Delete and Search Supported Create and Copy Not Supported	✓	✓
Planned Order	plannedorder	Item	Copy and Transform Not Supported	✓	✓
Planning Item Category	planningitemcategory	Item	Transform Not Supported	✓	✓
Planning Item Group	planningitemgroup	Item	Transform Not Supported	✓	✓
Planning Rule Group	planningrulegroup	Item	Transform Not Supported	✓	✓
Price Book	pricebook	List	Full	✓	✓
Price Level	pricelevel	List	Search Not Supported	✓	✓
Price Rule	pricerule	List	Search and Transform Not Supported	✓	✓
Price Rule Entry	priceruleentry	List	Search and Transform Not Supported	✓	✓
Price Plan	priceplan	List	Full	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Pricing Group	pricinggroup	List	Full	✓	✓
Project	job	Entity	Full	✓	✓
Project Charge Rule	chargerule	List	Transform Not Supported	✓	✓
Project Expense Type	projectexpensetype	List	Full	—	✓
Project Revenue Rule	laborbasedprojectrevenue rule fixedamountprojectrevenue rule pctcompleteprojectrevenue rule	List	Copy and Transform Not Supported	✓	✓
Project Status	jobstatus	Entity	Full	✓	✓
Project Task	projecttask	Event	Full	✓	✓
Project Template	projecttemplate	Entity	Copy Not Supported	✓	✓
Project Type	jobtype	Entity	Full	✓	✓
Project Work Breakdown Structure	wbs	List	Copy and Transform not supported	✓	✓
Promotion	promotioncode	Marketing	Full	✓	✓
Prompt	prompt	Customization	Can be created, read, updated, deleted, copied, and searched. It cannot be transformed.	✓	✓
Prospect	prospect	Entity	Full	✓	✓
Purchase Contract	purchasecontract	Transaction	Full	✓	✓
Purchase Order	purchaseorder	Transaction	Full	✓	✓
Reallocate Items	reallocateitem	Item	Exposed for user event scripts only. This record can not be created, loaded, or deleted using scripts.	—	—
Requisition	purchaserequisition	Transaction	Full	✓	✓
Resource Allocation	resourceallocation	Activities	Full	✓	✓
Resource Group	resourcegroup	Transaction	Full	✓	✓
Return Authorization	returnauthorization	Transaction	Full	✓	✓
Revenue Arrangement	revenuearrangement	Transaction	Full	—	✓
Revenue Commitment	revenuecommitment	Transaction	Create Not Supported See Usage Notes .	✓	✓
Revenue Commitment Reversal	revenuecommitmentreversal	Transaction	Create Not Supported See Usage Notes .	✓	✓
Revenue Recognition Event	billingrevenueevent	List	Create, Red, Update, Delete, and Search	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
					User Event Scripts Not Supported
Revenue Recognition Field Map	revrecfieldmapping	List	Full	✓	✓
Revenue Recognition Plan	revrecplan	List	Copy, Create, and Delete Not Supported	—	✓ User Event Scripts Not Supported
Revenue Recognition Schedule	revrecschedule	List	Copy, Create, and Delete Not Supported	—	✓ (User Event Scripts Not Supported)
Revenue Recognition Template	revrectemplate	List	Full	—	✓ (User Event Scripts Not Supported)
Role	role	List	Search Only	—	✓
SaaS Metric	saasmetric	Item	Read, Search	✓	✓
Order Reservation	saleschannel	Transaction	Created, read, updated, deleted, copied, and searched. It cannot be transformed.	✓	✓
Sales Order	salesorder	Transaction	Full	✓	✓
Sales Role	salesrole	List	Full	✓	✓
Sales Tax Item	salestaxitem	List	Full	—	✓
Scheduled Script Instance	—	Customization	Search Only	—	✓
Script Note that this covers these script type records: Bundle Installation Script, Client Script, Map/Reduce Script, Mass Update Script, Portlet, Restlet, Suitelet, User Event Script, and Workflow Action Script.	Need Internal ID. See Script description	Customization	Copy, Create, and Delete Not Supported See Usage Notes	—	✓
Script Deployment	Need Internal ID. See Script Deployment description	Customization	See Usage Notes	—	✓
Serialized Assembly Item	serializedassemblyitem	Item	Full	✓	✓
Serialized Inventory Item	serializedinventoryitem	Item	Full	✓	✓
Service	serviceitem	Item	Full	✓	✓
Shipping Item	shipitem	Item	Copy and Create Not Supported	—	✓
Shopping Cart	shoppingcart	Website	Search Only	—	✓
Solution	solution	Support	Full	✓	✓
Statistical Journal Entry	statisticaljournalentry	Transaction	Full	✓	✓
Store Pickup Fulfillment	storepickupfulfillment	Transaction	Copy and Create Not Supported	✓	✓
Subscription	subscription	Transaction	Full	✓	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Subscription Line	subscriptionline	Transaction	Create and Delete Not Supported	✓	✓
Subscription Plan	subscriptionplan	Item	Full	✓	✓
Subscription Term	subscriptionterm	Item	Copy, Create, Delete, Read, Search, and Update	✓	✓
Subsidiary	subsidiary	List	Full	✓	✓
Subsidiary Settings	subsidiarysettings	List	Read, Update, and Search Search is available only through SuiteAnalytics Workbook.	—	✓
Subtotal Item	subtotalitem	Item	Full	✓	✓
Supply Chain Snapshot	supplychainsnapshot	List	Read, Create, Edit, Delete, and Search	✓	✓
Supply Chain Snapshot Simulation	supplychainsnapshotsimulation	Item	Read, Create, Update, Delete, Search	✓	✓
Supply Change Order	supplychangeorder	Item	Create, Copy, and Transform Not Supported	✓	✓
Supply Plan Definition	supplyplandefinition	Item	Transform Not Supported	✓	✓
System Note	systemnote (SYSTEM_NOTE for SuiteScript 2.0 search enum)	List	Search Only	✓	✓
Task	task	Activity	Full	✓	✓
Tax Control Account	taxacct	List	Delete and Search Not Supported	✓	✓
Tax Group	taxgroup	List	Full	—	✓
Tax Period	taxperiod	List	Full	—	✓
Tax Type	taxtype	List	Full	—	✓
Term	term	List	Search Not Supported	✓	✓
Text Enhance Action	textenhanceaction	Customization	Can be created, read, updated, deleted, copied, and searched. It cannot be transformed.	✓	✓
Time	timebill	Transaction	Full	✓	✓
Time-Off Change	timeoffchange	Entity	Copy and Transform Not Supported	—	✓ User Event Triggering not supported.
Time-Off Plan	timeoffplan	Entity	Copy and Transform Not Supported	—	✓ User Event Triggering not supported.
Time-Off Request	timeoffrequest	Entity	Copy and Transform Not Supported	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
					User Event Triggering not supported.
Time-Off Rule	timeoffrule	Entity	Copy and Transform Not Supported	—	✓ User Event Triggering is not supported.
Time-Off Type	timeofftype	Entity	Copy and Transform Not Supported	—	✓ User Event Triggering not supported.
Topic	topic	Support	Full	—	✓
Transaction Search	transaction	Transaction	Search Only	✓	✓
Transfer Order	transferorder	Transaction	Full	✓	✓
Unit of Measure	unitstype	List	Full	—	✓
Unlocked Time Period	unlockedtimeperiod	Transaction	Read, Create, Update, Delete, Copy, and Search	✓	✓
Usage	usage	Transaction	Full	—	—
Vendor	vendor	Entity	Full	✓	✓
Vendor Bill	vendorbill	Transaction	Full	✓	✓
Vendor Category	vendorcategory	List	Search Not Supported	✓	✓
Vendor Credit	vendorcredit	Transaction	Full	✓	✓
Vendor Payment	vendorpayment	Transaction	Full	✓	✓
Vendor Prepayment	vendoreprepayment	Transaction	Read, Create, Update, Delete, and Search	✓	✓
Vendor Prepayment Application	vendoreprepaymentapplication	Transaction	Read, Transform, Update, Delete, and Search	✓	✓
Vendor Return Authorization	vendorreturnauthorization	Transaction	Full	✓	✓
Vendor-Subsidiary Relationship	vendorsubsidiaryrelationship	List	Full	✓	✓
Wave	wave	Transaction	Read, Create, Update, Delete, and Search	✓	✓
Website Setup	website	Website	Full	—	✓ Not supported in beforeLoad user event scripts.
Weekly Timesheet	timesheet	Transaction	Full	✓	✓
Work Order	workorder	Transaction	Full	✓	✓
Work Order Close	workorderclose	Transaction	Copy and Create Not Supported.	—	✓
Work Order Completion	workordercompletion	Transaction	Copy and Create Not Supported	—	✓

Record Name	Record ID	Record Category	Scripting Level	Scriptable in Client SuiteScript	Scriptable in Server SuiteScript
Work Order Issue	workorderissue	Transaction	Copy and Create Not Supported	—	✓
Workplace	workplace	List	Full	✓	✓
Zone	zone	List	Full	✓	✓

Activities

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following activity records are scriptable in SuiteScript:

- [Activity](#)
- [Event](#)
- [Phone Call](#)
- [Project Task](#)
- [Resource Allocation](#)
- [Task](#)
- [Work Calendar](#)

Activity

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is activity.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The activity record is scriptable in both server and client SuiteScript.

Supported Functions

The activity record can only be searched using SuiteScript.

Event

Applies to: SuiteScript 2.x | SuiteCloud Developer

Events are scheduled activities that are automatically added to your calendar when created.

For help working with this record in the user interface, see the help topic [Creating a New Event Record](#).

The internal ID for this record is calendarevent. Note that setting recurring events in SuiteScript is not currently supported.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The event record is scriptable in both server and client SuiteScript.

Supported Functions

The event record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

When the Time Tracking feature is enabled, the TimeItemList sublist is available. This list is used to track employee time associated with the phone call, including payroll, billing, and project fields. It is an inline editor sublist.

Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Phone Call

Applies to: SuiteScript 2.x | SuiteCloud Developer

Phone calls records are used to document phone call activity. All information submitted for a phone call record is stored on a record in the phone call list, on the customer record who calls and on any contact's records referenced in the call contact list.

For help working with this record in the user interface, see the help topic [Working with Phone Calls](#).

The internal ID for this record is `phonecall`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The phone call record is scriptable in both server and client SuiteScript.

Supported Functions

The phone call record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

When the Time Tracking feature is enabled, the `TimeItemList` sublist is available. This list is used to track employee time associated with the phone call, including payroll, billing, and project fields. It is an inline editor sublist.

 **Important:** If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Project Task

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The project task record can be used to keep track of specific activities and milestones associated with a project.

The internal ID for this record is `projecttask`.

The project task record is available when the Project Management feature is enabled at Setup > Company > Enable Features, on the Company subtab. When the feature is enabled, you can access the project task record in the UI by going to an existing project and clicking the New Project Task or New Milestone button.

For help working with this record in the user interface, see the help topic [Project Tasks](#).

Project task records cannot be created as standalone records. Rather, you create a project task for a specific project record, and the task remains attached to that record. For information about working with the project record in SuiteScript, see [Project](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Project Tasks Versus Milestone Tasks

Every project task record is designated as either a project task or a milestone task. A project task is used to represent an activity, whereas a milestone task is used to represent a checkpoint in the overall progress of the project.

Although they are the same type of record, a milestone task cannot have values in the `estimatedwork` body or sublist fields. Therefore, if you create a project task record and do not include any estimated work, the record is automatically saved as a milestone task. If you do include estimated work, the record is saved as a project task.

These same rules apply to the updating of records as well. In other words:

- To convert a project task into a milestone task, clear the `estimatedwork` body field and all values from the Assignees sublist.
- To convert a milestone task into a project task, add a value to the `estimatedwork` body field or add at least one record to the Assignees sublist, with a positive value of estimated work.

Note that the `estimatedwork` body field is populated by the sum of estimated work listed for Assignees. If you explicitly set a value for the `estimatedwork` body field, and you also include Assignees, the value you specify for the body field is overwritten based on the sublist's `estimatedwork` values. If you do not include Assignees, you can explicitly assign a value to the `estimatedwork` body field.

Supported Script Types

The project record is scriptable in server SuiteScript only.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The project record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Field Definitions

When creating new project tasks you must set the Project (*company*) field to the project/job ID. Project tasks are not standalone records, and therefore must be associated with a specific project.

For other details on body fields and sublist fields, See the [SuiteScript Records Browser](#), which lists all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Code Sample

The following example shows how you might create both a project task and a milestone task.

```

1 // create a project
2 var project = record.create({
3   type: record.Type.JOB,
4   isDynamic: true
5 });
6 project.setValue({
7   fieldId: 'companyname',
8   value: 'Reconstruction'
9 });
10 project.setValue({
11   fieldId: 'subsidiary',
12   value: 1
13 });
14 var projectId = project.save();
15
16 // create a project task
17 var task = record.create({
18   type: record.Type.PROJECT_TASK
19 });
20 task.setValue({
21   fieldId: 'estimatedwork',
22   value: 2
23 });
24 task.setValue({
25   sublistId: 'title',
26   value: 'Remove old furniture'
27 });
28 task.setValue({
29   sublistId: 'company',
30   value: projectId
31 });
32 var task1Id = task.save();
33
34 // create another task depending on the first one with Finish-To-Start dependency
35 task = record.create({
36   type: record.Type.PROJECT_TASK
37 });
38 task.setValue({
39   fieldId: 'estimatedwork',
40   value: 5
41 });
42 task.setValue({
43   fieldId: 'title',
44   value: 'Paint walls'
45 });
46 task.setValue({
47   fieldId: 'company',
48   value: projectId
49 });
50 task.selectNewLine({
51   sublistId: 'predecessor'
52 });
53 task.setCurrentSublistValue({
54   sublistId: 'predecessor',
55   fieldId: 'task',
56   value: task1Id
57 });
58 task.setCurrentSublistValue({
59   sublistId: 'predecessor',
60   fieldId: 'type',
61   value: 'FS'

```

```

62 });
63 task.commitLine({
64     sublistId: 'predecessor'
65 });
66 var task2Id = task.save();
67
68 // create a milestone
69 task = record.create({
70     type: record.Type.PROJECT_TASK,
71     isDynamic: true
72 });
73 task.setValue({
74     fieldId: 'estimatedwork',
75     value: 0
76 });
77 task.setValue({
78     fieldId: 'title',
79     value: 'Verify painting after the walls dry out'
80 });
81 task.setValue({
82     sublistId: 'company',
83     value: projectId
84 });
85 task.selectNewLine({
86     sublistId: 'predecessor'
87 });
88 task.setCurrentSublistValue({
89     sublistId: 'predecessor',
90     fieldId: 'task',
91     value: task2Id
92 });
93 task.setCurrentSublistValue({
94     sublistId: 'predecessor',
95     fieldId: 'type',
96     value: 'FS'
97 });
98 task.commitLine({
99     sublistId: 'predecessor'
100 });
101 var milestoneId = task.save();

```

Resource Allocation

Applies to: SuiteScript 2.x | SuiteCloud Developer



Important: For information about the availability of the Resource Allocations feature, please contact your account representative.

The resource allocation record supports reserving an employee's time for a particular project.

The internal ID for this record is resourceallocation.

In the UI, you access this record by going to Activities > Scheduling > Resource Allocations. Alternatively, you can view the resource allocations for a specific project through the project record's Resources subtab (Lists > Relationships > Projects).

This record is available only if the Resource Allocations feature has been enabled at Setup > Company > Enable Features, on the Company subtab. Note that this option will not be visible unless your account has been provisioned for this feature. For more details, contact your account representative.

For help working with this record in the user interface, see the help topic [Resource Allocations](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client SuiteScript and server SuiteScript

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The resource allocation record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Field Definitions

To create a new resource allocation record, you must reference two existing NetSuite records, as follows:

- `project` — A reference to a project record defined at Lists > Relationships > Projects. Note that in some NetSuite accounts, projects are referred to as jobs.
- `allocationresource` — A reference to an employee record, defined at Lists > Employees > Employees, for which the Project Record option has been checked. The Project Resource box is located on the Human Resources subtab of the employee record.

You are also required to provide values for several other fields. Refer to the [SuiteScript Records Browser](#) for the internal IDs of all fields associated with this record.

Note also that `numberhours` and `percentoffime` are read-only fields. They are returned when you load the record, but they cannot be modified.

For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

Code Samples

The following example shows how to create a resource allocation record:

```
1 var AMOUNT = "10.0";
2 var PROJECT1 = "43";
3 var RESOURCE1 = "45";
4 var ALLOCATIONTYPE1 = "1"; // Hard
5 var ALLOCATIONTYPE2 = "2"; // Soft
6 var ALLOCATIONUNIT1 = "H"; // Hours
7 var ALLOCATIONUNIT2 = "P"; // Percent of Time
```

```

8  var NOTES1 = "My notes 1";
9  var STARTDATE1 = "4/20/2013";
10 var ENDDATE1 = "4/28/2013";
11
12 var newRecord = record.create ({
13     type: 'resourceallocation'
14 });
15
16 newRecord.setValue ({
17     fieldId: 'allocationamount',
18     value: AMOUNT
19 });
20 newRecord.setValue ({
21     fieldId: 'allocationresource',
22     value: RESOURCE1
23 });
24 newRecord.setValue ({
25     fieldId: 'allocationtype',
26     value: ALLOCATIONTYPE1
27 });
28 newRecord.setValue ({
29     fieldId: 'allocationunit',
30     value: ALLOCATIONUNIT1
31 });
32 newRecord.setValue ({
33     fieldId: 'startdate',
34     value: STARTDATE1
35 });
36 newRecord.setValue ({
37     fieldId: 'notes',
38     value: NOTES1
39 });
40 newRecord.setValue ({
41     fieldId: 'project',
42     value: PROJECT1
43 });
44 newRecord.setValue ({
45     fieldId: 'enddate',
46     value: ENDDATE1
47 });
48 var recId = newRecord.save();

```

Task

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Tasks are activities that need to be completed. Use the task record to add new tasks for individuals, companies or contacts and to modify those records.

For help working with this record in the user interface, see the help topic [Creating a Project Task Record](#).

The internal ID for this record is task.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ [N/record Module](#)

Supported Script Types

The task record scriptable in both client SuiteScript and server SuiteScript.

Supported Functions

The resource allocation record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

When the Time Tracking feature is enabled, the TimeItemList sublist is available. This list is used to track employee time associated with the task, including payroll, billing, and project fields. It is an inline editor sublist.



Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Work Calendar

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You can set up work calendars to track and manage the work capacity for employees and vendors you assign as resources on projects. Knowing the work capacity for each employee helps you to schedule resources for project tasks.

Also, any employee or vendor you plan to assign as a project resource must have a work calendar assigned on their record.

For help working with this record in the user interface, see the following topics:

- [Project Resource Work Calendars](#)
- [Setting Up a Work Calendar](#)
- [Assigning a Resource Work Calendar](#)

The internal ID for this record is workcalendar.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Entities

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Entity Record Restrictions for Scripting on Administrator Role

This section describes entity record restrictions for scripting on the Administrator and Full Access roles. Note that these restrictions apply to all versions of SuiteScript and all NetSuite environments (see the help topic [Understanding NetSuite Account Types](#)).

You cannot perform these actions with SuiteScript:

- Create an entity record where any role is set to Administrator or Full Access
- Delete an entity record where any role is set to Administrator or Full Access
- Edit an entity record so that the entity gains an Administrator or Full Access role
- Edit an entity record so that the entity loses an Administrator or Full Access role
- Edit the password or email field value on an entity record where any role is set to Administrator or Full Access

These restrictions contribute to improved security. Scripts that break them throw an error message: Script Security Violation: Unauthorized attempt to <operation> entity with <role> role by SuiteScript!



Important: As of 2019.1, changes have been made to the Full Access role, in preparation for its deprecation. For details, see the help topic [Full Access Role \(Deprecated\)](#).

Entity Record Types

The following entity records are scriptable in SuiteScript:

- [As Charged Project Revenue Rule](#)
- [Bonus](#)
- [Bonus Type](#)
- [Competitor](#)
- [Contact](#)
- [Customer](#)
- [Customer Status](#)

- Employee
- Employee Status
- Employee Type
- Entity
- Generic Resource
- Job
- Job Requisition
- Lead
- Other Name
- Partner
- Project
- Project Status
- Project Template
- Project Type
- Prospect
- Time-Off Management
 - Time-Off Change
 - Time-Off Plan
 - Time-Off Request
 - Time-Off Rule
 - Time-Off Type
- Vendor

As Charged Project Revenue Rule

i Applies to: SuiteScript 2.x | SuiteCloud Developer

As charged project revenue rules recognize the revenue from actual project charges as they are created rather than when they are billed. It is available when the Projects Management, Charge-Based Billing and Advanced Revenue Management features are enabled.

For help working with this record in the UI, see the help topics [Creating As Charged Project Revenue Rules](#) and [Charge-Based Project Billing](#).

The internal ID for this record is `aschargedprojectrevenuerule`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The As Charge Project Revenue rule record is scriptable in server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The system notes sublist is search-only. The chargerule sublist contains only one editable column: chargerule. Other columns are read only: amount, type, billingitem, description.

Code Samples

The following sample shows how to create a new As Charged Revenue rule.

```

1  require(['N/record'], function(record) {
2      var asChargedRevRule = record.create({
3          type: 'aschargedprojectrevenue',
4          isDynamic: true
5      });
6
7      asChargedRevRule.setText({
8          fieldId: 'name',
9          text: 'As Charged Revenue Rule name'
10     });
11     asChargedRevRule.setValue({
12         fieldId: 'project',
13         value: 456
14     });
15     asChargedRevRule.selectNewLine({
16         sublistId: 'chargerule'
17     });
18     asChargedRevRule.setCurrentSublistValue({
19         sublistId: 'chargerule',
20         fieldId: 'chargerule',
21         value: '3'
22     });
23     asChargedRevRule.commitLine({
24         sublistId: 'chargerule'
25     });
26     var asChargedRevRuleId = asChargedRevRule.save();
27 });

```

Related Topics

- [Projects](#)
- [Creating As Charged Project Revenue Rules](#)
- [Charge-Based Project Billing](#)
- [Creating a Project Resource Role](#)
- [Using Project Management](#)
- [Working with the SuiteScript Records Browser](#)
- [SuiteCloud Supported Records](#)

■ Entities

Bonus

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the bonus record to award different types of bonuses to your employees. To use bonus records, the Compensation Tracking feature must be enabled in your account. For help working with this record in the user interface, see the help topic [Recording Base Pay Compensation for an Employee](#).

The internal ID for this record is bonus.

See [Bonus Record Action](#) for the action associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

📘 Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The bonus record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Copy
- Delete
- Search

Usage Notes

If a payroll item is associated with the bonus type for the bonus record:

- The update function is not available.
- SuitePeople U.S. Payroll determines the status of the bonus. Workflows cannot see these statuses, so you should not use status changes as event triggers.

Script Sample

The following sample shows how to award a bonus.

```

1 require(['N/record'], function (record) {
2   var employeeId = 659; // id of selected employee
3   var bonusTypeId = 1; // id of previously created bonus type NOTE: bonus type must have same subsidiary as selected employee
4   var percentage = 25; // set percentage of employee salary for given bonus
5   var bonusStatus = 'none'; // bonus status ('none', 'ready_to_pay', 'being_processed', 'paid', 'cancelled')
6   var awardDate = new Date('12/24/2020'); // date in MM/DD/YYYY format
7
8   var bonus = record.create({
9     type: record.Type.BONUS
10  });
11
12  bonus.setValue({
13    fieldId: 'bonusemployee',
14    value: employeeId
15  });
16  bonus.setValue({
17    fieldId: 'bonustype',
18    value: bonusTypeId
19  });
20  bonus.setValue({
21    fieldId: 'bonusawarddate',
22    value: awardDate
23  });
24  bonus.setValue({
25    fieldId: 'bonusstatus',
26    value: bonusStatus
27  });
28  bonus.setValue({
29    fieldId: 'bonusamountpercentage',
30    value: percentage
31  });
32
33  bonus.save();
34 });

```

Bonus Record Action

i Applies to: SuiteScript 2.x | SuiteCloud Developer

cancel

Corresponding UI Button	Exclude from Payroll
Action Description	Excludes the existing bonus from the upcoming payroll run. The action.ALL_QUALIFIED_INSTANCES constant is currently not supported.
Returns	void
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2020.1

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action.

Bonus Cancel Action Syntax

```

1 require(['N/action'], function(action) {
2     // action & action list loading
3     var actionList = action.find({
4         recordType: 'bonus'
5     });
6     var actionObj = action.get({
7         recordType: 'bonus',
8         id: 'cancel'
9     });
10    // action execution
11    var result = actionObj.execute({
12        recordId: 1
13    });
14    var result = action.execute({
15        recordType: 'bonus',
16        id: 'cancel',
17        params: {
18            recordId: 2
19        }
20    });
21 });

```

Bonus Type

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the bonus type record to define different types of bonuses within your organization's compensation plans. For example, you might have holiday bonuses, sign-on bonuses, or merit bonuses. To use bonus records, the Compensation Tracking feature must be enabled in your account. For help working with this record in the user interface, see the help topic [Recording Base Pay Compensation for an Employee](#).

The internal ID for this record is bonustype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The bonus type record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Copy
- Delete
- Search

Script Sample

The following sample shows how to create a bonus type record.

```

1 require(['N/record'], function (record) {
2   var bonusType = record.create({
3     type: record.Type.BONUS_TYPE
4   });
5   bonusType.setValue({
6     fieldId: 'name',
7     value: 'Annual Bonus'
8   });
9   bonusType.setValue({
10    fieldId: 'subsidiary',
11    value: 2
12  });
13  bonusType.save();
14 });

```

Competitor

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can create competitor records to track how other businesses in your industry impact your sales.

If you use NetSuite or NetCRM and have the Opportunities feature enabled, selecting a competitor on an opportunity record lets everyone working with that opportunity know who they are competing with. By being aware of the strengths and weaknesses of the competition, sales reps can make educated offers and close more deals.

For help working with this record in the user interface, see the help topics [Competitors](#) and [Opportunity Records](#).

The internal ID for this record is `competitor`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The competitor record is scriptable in both client and server SuiteScript.

Supported Functions

The competitor record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Contact

Applies to: SuiteScript 2.x | SuiteCloud Developer

Contacts represent people or companies that you deal with in the daily activity of your business. Use the contact record to create, modify, or delete contacts and associate a contact to a parent record.

For help working with this record in the user interface, see the help topic [Contacts](#).

The internal ID for this record is contact.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The contact record is scriptable in both client and server SuiteScript.

Supported Functions

The contact record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Customer

Applies to: SuiteScript 2.x | SuiteCloud Developer

Customer records allow you to track all the following types of information about your customers in one place, depending on the features you have enabled.

For help working with this record in the user interface, see the help topic [Customers](#).

The internal ID for this record is `customer`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer record is scriptable in both client and server SuiteScript.

Note: The customer center role does not support scripting capabilities. You must log into a valid account role to write and deploy SuiteScripts.

Supported Functions

The customer record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

Notes on Scripting Customer Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
<code>datecreated</code>	Date Created	This is a system-generated field that marks the date the record was created in NetSuite. You cannot change or override this field. Tip: If you need to capture “date created” information that is not related to the date the record was created in NetSuite, create a custom field and set it to auto-default to today's date.
<code>password</code>	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
<code>password2</code>	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts

Field Internal ID	Field UI Label	Note
		for external role users (for example, shoppers, online form users (anonymous users), customer center).

Search Filters and Search Columns

ccnumber	Credit Card Number	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
----------	--------------------	--

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Access / contactroles sublist is included in the customer record on the Access subtab. It is an inline editor subtab. The Access / contract roles sublist is related to the Contact / contactroles sublist. For details, see the contactroles sublist in the [SuiteScript Records Browser](#).

The Time Tracking sublist is included with the customer record. It is an inline editor sublist.



Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Notes on Scripting Customer Sublists

You can update the contactroles sublist to provide Customer Center access to contacts. You can provide access to contacts that already exist in NetSuite and that have already been attached to a customer that already exists in NetSuite. The workflow is as follows: 1) Add customer. 2) Add contacts. 3) Attach contacts to customer. 4) Update customer with contact access information.

The fields in this sublist map to the fields on the Access subtab in the UI. These fields include: a boolean field that indicates whether a contact has access to NetSuite, contact name key field, email address used to log in to NetSuite, password used to log in to NetSuite, NetSuite role (Customer Center), and a boolean field that indicates whether the contact should receive a notification email when access changes are made. If this Notify field is set to true, an email is sent.

Transform Types

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Target Record Type	Target Record Internal ID	Field Defaults
Cash Sale	cashsale	billdate
Customer Payment	customerpayment	—
Quote	estimate	—
Invoice	invoice	billdate
Opportunity	opportunity	—
Sales Order	salesorder	—

Customer Status

Applies to: SuiteScript 2.x | SuiteCloud Developer

The customer status record describes a lead, prospect, or a customer's stage in the sales cycle. To create a new customer status record, go to Setup > Sales > Setup Tasks > Customer Statuses > New.

For help working with this record in the user interface, see the help topic [Customers](#).

The internal ID for this record is customerstatus.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer status record is scriptable in both client and server SuiteScript.

Supported Functions

The customer status record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Employee

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the user interface, see the help topic [Employee Information Management](#).

The internal ID for this record is employee.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The employee record is scriptable in both client and server SuiteScript.

Supported Functions

The employee record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
password	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
password2	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Scripting when Advanced Employee Permissions is Enabled

When the Advanced Employee Permissions feature is enabled keep the following in mind:

- Before submit scripts can be run, either as the role that has manipulated the record or as an administrator, the script only has access to the information that has been submitted from the browser. In the case of an employee record being edited, the before submit script only has access to the fields or sublists the role editing the record is permitted to edit. Therefore, you cannot assume that all fields and sublists on the employee record are available in a before submit script.
- After submit scripts can be run as administrator, which means complete access to an employee record. When the Advanced Employee Permissions feature is enabled, perform actions, such as setting the value of a field, as a role that is known to have the appropriate level of access or as an administrator.

For information about this feature, see the help topic [Advanced Employee Permissions](#).

Transform Types

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Target Record Type	Target Record Internal ID	Field Defaults
Expense Report	expensereport	—

Target Record Type	Target Record Internal ID	Field Defaults
Time	timebill	—

Employee Status

Applies to: SuiteScript 2.x | SuiteCloud Developer

Employee status values can be used to categorize employees' work status. Example values include part-time, full-time, temporary, regular. These values display on the Human Resources subtab of employee records.

For help working with this record in the user interface, see the help topic [Setting Up Employee Related Lists](#).

The internal ID for this record is employeestatus.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The employee status record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Copy
- Delete
- Search

Script Sample

The following sample shows how to create an employee status record.

```

1 require(['N/record'], function (record) {
2     var myRecord = record.create({
3         type: record.Type.EMPLOYEE_STATUS
4     });
5     myRecord.setValue({
6         fieldId: 'name',
7         value: Active
8     });
9     myRecord.save();
10 });

```

Employee Type

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Employee type values are used to categorize employees for descriptive or reporting purposes. These values display on the Human Resources subtab of employee records. System-defined values include the following: Contractor, Exempt (in the U.S., an employee not entitled to overtime pay), Officer, Owner, Regular Employee, and Statutory Employee.

For help working with this record in the user interface, see the help topic [Setting Up Employee Related Lists](#).

The internal ID for this record is `employeetype`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The employee record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Copy
- Delete

- Search

Script Sample

The following sample shows how to create an employee type record.

```

1 require(['N/record'], function (record) {
2   var myRecord = record.create({
3     type: record.Type.EMPLOYEE_TYPE
4   });
5   myRecord.setValue({
6     fieldId: 'name',
7     value: Exempt
8   });
9   myRecord.save();
10 });

```

Entity

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is entity.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The entity record is only exposed in search.

Supported Functions

Only search is supported for the entity record.

Generic Resource

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Generic resource records can be used as placeholders when project managers and resource managers are planning a project in NetSuite. This feature enables resource allocations and project task assignments to be made when a specific resource may not yet be identified.

For help working with this record in the user interface, see the help topic [Generic Resources](#).

The internal ID for this record is genericresource.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Use a generic resource as a placeholder for resource allocation and project task assignment. This record is primarily used when a specific resource is not available. See the help topic [Generic Resources](#) for additional information.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The generic resource record is supported in all client and server scripts.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Edit
- Delete
- Search

 **Note:** Copy is not supported.

Job

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The Job Management feature must be enabled to script with this record.

The internal ID for this record is job.

For help working with this record in the user interface, see the help topic [Managing Jobs](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The job record is scriptable in both client and server SuiteScript.

Supported Functions

The job record can be read, created, updated, searched, and deleted using SuiteScript. It cannot be copied or transformed.

Usage Notes

The Assigned Employees sublist and Job Requisition sublist are not available to SuiteScript.

Code Sample

```

1 function beforeLoad(type, form, request){
2   // rec in all following statements has been previous defined (created/loaded) as a Job record
3   rec.setValue({
4     fieldId: 'title',
5     value: 'abc'
6   });
7   rec.setValue({
8     fieldId: 'description',
9     value: 'test description'
10  });
11  rec.setValue({
12    fieldId: 'isinactive',
13    value: 'F'
14  });
15 }
```

Job Requisition

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Job Requisitions feature must be enabled to script with this record.

The internal ID for this record is jobrequisition.

For help working with this record in the user interface, see the help topic [Job Requisitions](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The job requisition record is scriptable in both client and server SuiteScript.

Supported Functions

The job requisition record can be read, created, updated, searched, and deleted using SuiteScript. It cannot be copied or transformed.

Code Samples

```

1  function beforeLoad(type, form, request){
2  // rec in all following statements has been previous defined (created/loaded) as a Job Requisition record
3      rec.setValue({
4          fieldId: 'title',
5          value: 'abc'
6      });
7      rec.setValue({
8          fieldId: 'description',
9          value: 'test description'
10     });
11     rec.setValue({
12         fieldId: 'isinactive',
13         value: 'F'
14     });
15 }
16
17 function beforeSubmit(type, form, request){
18     rec.setValue({
19         fieldId: 'subsidiary',
20         value: '1'
21     });
22     rec.setValue({
23         fieldId: 'department',
24         value: '2'
25     });
26     rec.setValue({
27         fieldId: 'location',
28         value: '3'
29     });
30 }

```

Lead

Applies to: SuiteScript 2.x | SuiteCloud Developer

Leads are companies or individuals who represent potential customers. In NetSuite, leads are the first step in the sales cycle that progresses to prospect and then to customer.

For help working with this record in the user interface, see the help topic [Lead Management](#).

The internal ID for this record is `lead`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The lead record is scriptable in both client and server SuiteScript.

Supported Functions

The lead record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
<code>datecreated</code>	Date Created	This is a system-generated field that marks the date the record was created in NetSuite. You cannot change or override this field. Tip: If you need to capture “date created” information that is not related to the date the record was created in NetSuite, create a custom field and set it to auto-default to today's date.
<code>password</code>	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in <code>beforeSubmit</code> user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
<code>password2</code>	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in <code>beforeSubmit</code> user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
Search Filters and Search Columns		
<code>ccnumber</code>	Credit Card Number	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in <code>beforeSubmit</code> user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Access / contactroles sublist is included in the customer record on the Access subtab. It is an inline editor subtab. The Access / contract roles sublist is related to the Contact / contactroles sublist. For details, see the contactroles sublist in the [SuiteScript Records Browser](#).

The Time Tracking sublist is included with the lead record. It is an inline editor sublist.



Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Transform Types

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Target Record Name	Target Record Internal ID	Field Defaults
Opportunity	opportunity	—

Other Name

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The list of other name records is a collection of records for people or companies who are not vendors, customers or employees. This enables you to keep records for other people or companies if you write checks to or receive deposits from them.

For example, your company might donate money to a favorite charity, so you create an other name record for the charity. You might also list your owners and partners here if they contribute or withdraw equity.

For help working with this record in the user interface, see the help topic [Other Name Records](#).

The internal ID for this record is othername.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The other name record is scriptable in server SuiteScript only.

Supported Functions

The lead record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Partner

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A partner is a company you have a business agreement with who isn't a customer or a vendor.

For help working with this record in the user interface, see the help topic [Managing Partners](#).

The internal ID for this record is partner.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The partner record is scriptable in both client and server SuiteScript.

Supported Functions

The partner record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

Notes on Scripting Partner Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
password	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts

Field Internal ID	Field UI Label	Note
		for external role users (for example, shoppers, online form users (anonymous users), customer center).
password2	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Notes on Scripting Partner Sublists

You can update the contactaccessroles sublist to provide Partner Center access to contacts. You can provide access to contacts that already exist in NetSuite and that have already been attached to a partner that already exists in NetSuite. The workflow is as follows: 1) Add partner. 2) Add contacts. 3) Attach contacts to partner. 4) Update partner with contact access information.

The fields in this sublist map to the fields on the Access subtab in the UI. These fields include: a boolean field that indicates whether a contact has access to NetSuite, contact name key field, email address used to log in to NetSuite, password used to log in to NetSuite, NetSuite role (Partner Center), and a boolean field that indicates whether the contact should receive a notification email when access changes are made. If this Notify field is set to true, an email is sent.

The Access / contactroles sublist is included in the customer record on the Access subtab. It is an inline editor subtab. The Access / contract roles sublist is related to the Contact / contactroles sublist. For details, see the contactroles sublist in the [SuiteScript Records Browser](#).

Project

Applies to: SuiteScript 2.x | SuiteCloud Developer

Use the project record to manage company initiatives.

To use the project record, you must have the Projects feature enabled at Setup > Company > Enable Features, on the Company subtab. If you plan to do advanced project tracking, you must also enable Project Management. If you do not see the Project Management box, your company must first purchase the Project Management add-on from NetSuite.

To access the project record in the UI, choose Lists > Relationships > Projects(or Jobs).

For help working with this record in the user interface, see the help topic [Projects](#).

The internal ID for this record is job.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project record is scriptable in both client and server SuiteScript.

Supported Functions

The project record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Field Definitions

Note that the `datecreated` body field is a system-generated field that marks the date the record was created in NetSuite. You cannot change or override this field. If you need to capture “date created” information that is not related to the date the record was created in NetSuite, create a custom field and set it to auto-default to today's date.

For more details on available fields, see the [SuiteScript Records Browser](#), which lists all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Usage Notes

The Time Tracking sublist is included with the project (job) record. It is an inline editor sublist.



Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Adding a Resource With Multiple Roles

When using SuiteScript, the behavior of the Resources sublist differs slightly from the behavior in the UI. Specifically, in the UI, each line must have a unique value in the Name field. Further, in the UI, you can specify more than one role for each resource.

Name *	Email	Role *
Kelly, Taylor	ktaylor@wolfe.com	Project Manager, Designer, Assistant Manager

With SuiteScript, if you want to add a resource that has two different roles, you set up your code as if you are adding two sublist records for that resource -- one for each role. The `jobresource` values are not required to be unique. The following example illustrates this technique. In this example, only two unique resources are being added, but because one resource has two roles, that resource is represented twice.

```
1 | proj = record.create({
2 |   type: record.Type.JOB
3 | });
```

```

4 proj.setValue({
5     fieldId: 'companyname',
6     value: 'Launch'
7 });
8 proj.setValue({
9     fieldId: 'subsidiary',
10    value: '1'
11 });
12 proj.selectNewLine({
13     sublistId: 'jobresources'
14 });
15 proj.setCurrentSublistText({
16     sublistId: 'jobresources',
17     fieldId: 'jobresource',
18     text: 'Employee Resource 1'
19 });
20 proj.setCurrentSublistText({
21     sublistId: 'jobresources',
22     fieldId: 'role',
23     text: 'Staff'
24 });
25 proj.commitLine({
26     sublistId: 'jobresources'
27 });
28
29 proj.selectNewLine({
30     sublistId: 'jobresources'
31 });
32 proj.setCurrentSublistText({
33     sublistId: 'jobresources',
34     fieldId: 'jobresource',
35     text: 'Employee Resource 1'
36 });
37 proj.setCurrentSublistText({
38     sublistId: 'jobresources',
39     fieldId: 'role',
40     text: 'Project Manager'
41 });
42 proj.commitLine({
43     sublistId: 'jobresources'
44 });
45
46 var id = proj.save();
47 var savedProj = recordload ({
48     type: record.Type.JOB,
49     id: id
50 });
51 savedProj.selectLine({
52     sublistId: 'jobresources',
53     line: '1'
54 });
55 savedProj.setCurrentSublistText({
56     sublistId: 'jobresources',
57     fieldId: 'jobresource',
58     text: 'Employee Resource 2'
59 });
60 savedProj.commitLine({
61     sublistId: 'jobresources'
62 });
63 savedProj.save();

```

Note that after you complete the add operation, the Resources sublist in the UI looks the same as it would if you had manually added one line for the resource, with multiple roles specified on that line, as shown in the illustration above.

Scripting Projects with Advanced Revenue Management

The following table lists the scriptable fields associated with the Project record and advanced revenue management.

Field	Type	Internal ID
Accounting Period	List/Record	accountingperiod
Comments	Text Area	comments
Percentage Complete	Percent	percent

Before you begin working with advanced revenue management programmatically, see the help topic [Setup for Advanced Revenue Management \(Essentials\)](#).

For help working with this record in the user interface, see the help topic [Advanced Revenue Management \(Essentials\) for Projects](#).

Project Records Actions

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Mark Project as Completely Billed

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Mark Project as Completely Billed record action type changes the state of Project record to marked as completely billed and it triggers adjusting of recognized revenue to be reconciled with actual charges. This action is available on the record only when the record meets the following 4 conditions:

- Project status must be Closed (or other status, which sets the field bAllowCompletelyBilledOnPrjct = "T")
- The project uses Billing Type = Charge Based Project
- The project uses a Project Revenue Rule
- The project is not marked as completely billed yet

For help working with this record in the UI, see the help topic [Marking a Project as Completely Billed](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

Input Parameters

RecordId is a default input parameter for this action. No additional input parameters are necessary.

Return Object

When the action is successful, only the default response object is returned.

```

1 | {
2 |   "response": {},
3 |   "notifications": []
4 | }
```

API Usage Charts

The Mark Project as Completely Billed record action provides two SQL updates of one column. The governance is 5 units. For usage in the bulk governance, it is 50 units.

Code Samples

The following sample show how to use this API:

```

1  require(['N/action'], function(actionMod) {
2  // Action list and action loading
3  const actionList = actionMod.find({recordType: 'job'});
4  const actionObj = actionMod.get({recordType: 'job', id: 'markprojectascompletelybilled'});
5
6  // Action execution sync using object
7  const result = actionObj.execute({recordId: 1});
8  console.log(result);
9
10 // Action execution sync using object, another version
11 const result = actionObj({recordId: 1});
12 console.log(result);
13
14 // Action execution sync without object
15 const result = actionMod.execute({recordType: 'job', id: 'markprojectascompletelybilled', params: {recordId: 1}});
16 console.log(result);
17
18 // Client side example async
19 const recId = 1;
20
21 actionMod.find.promise({recordType: 'job', recordId: recId}).then(function(actions) {
22     actions.markprojectascompletelybilled.promise().then(function(result) {
23         console.log("Project id:" + recId + " has been marked as completely billed.");
24     }).catch(function (error) {
25         console.log(error);
26     });
27 });
28
29 // Server side example
30 var result = actionMod.execute({recordType: 'job', id: 'markprojectascompletelybilled', params: {recordId: 1}});
31
32 // Usage in bulk
33 var projectsToBeMarked = [{recordId: 1}, {recordId: 2}];
34
35 var taskId = actionMod.executeBulk({recordType: 'job', id: 'markprojectascompletelybilled', params: projectsToBeMarked});
36
37 var taskStatus = actionMod.getBulkStatus(taskId);
38 console.log(taskStatus);
39
40 });

```

Project Status

Applies to: SuiteScript 2.x | SuiteCloud Developer

The project status record indicates the progress of the project.

You can create new statuses at Setup > Accounting > Accounting Lists > New > Project Status.

To use the project status and [Project](#) records, you must have the Projects feature enabled at Setup > Company > Enable Features, on the Company subtab. If you plan to do advanced project tracking, you must also enable Project Management. If you do not see the Project Management box, your company must first purchase the Project Management add-on from NetSuite.

For help on how this record is used with the [Project](#) record in the user interface, see the help topic [Creating a Project Record](#).

The internal ID for this record is jobstatus.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project status record is scriptable in both client and server SuiteScript.

Supported Functions

The project status record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Project Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

Project Templates enable you to create project records in NetSuite for projects your business performs repeatedly.

To use the project template and [Project](#) records, you must have the Projects feature enabled at Setup > Company > Enable Features, on the Company subtab. If you plan to do advanced project tracking, you must also enable Project Management. If you do not see the Project Management box, your company must first purchase the Project Management add-on from NetSuite.

For help working with this record in the user interface, see the help topic [Project Templates](#).

The internal ID for this record is projecttemplate.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Use a project template as a standard starting point for projects and project items. Each record instance is reusable. See the help topic [Project Templates](#) for information about using project templates in the UI.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project template record is supported in all client and server scripts.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Edit
- Delete
- Search

 **Note:** Copy is not supported.

Project Type

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

To use the project type and [Project](#) records, you must have the Projects feature enabled at Setup > Company > Enable Features, on the Company subtab. If you plan to do advanced project tracking, you must also enable Project Management. If you do not see the Project Management box, your company must first purchase the Project Management add-on from NetSuite.

For help working with this record in the user interface, see the help topic [Basic Projects](#).

The internal ID for this record is jobtype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project type record is scriptable in both client and server SuiteScript.

Supported Functions

The project type record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Prospect

Applies to: SuiteScript 2.x | SuiteCloud Developer

Prospect records enable you to track all the information you need to convert a prospect into a customer.

The internal ID for this record is prospect.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The prospect record is scriptable in both client and server SuiteScript.

Supported Functions

The prospect record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
password	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Field Internal ID	Field UI Label	Note
password2	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Search Filters and Search Columns

ccnumber	Credit Card Number	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
----------	--------------------	--

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Access / contactroles sublist is included in the customer record on the Access subtab. It is an inline editor subtab. The Access / contract roles sublist is related to the Contact / contactroles sublist. For details, see the contactroles sublist in the [SuiteScript Records Browser](#).

The Time Tracking sublist is included with the prospect record. It is an inline editor sublist.



Important: If any case, task, or event record has more than 9500 time entries in the Time Tracking sublist, all cases include a static list of time entries. These static lists are not accessible to scripting. Attempts to script on Time Tracking sublists that are static lists of time entries will not return accurate results.

Transform Types

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Target Record Name	Target Record Internal ID	Field Defaults
Estimate/Quote	estimate	—
Opportunity	opportunity	—
Sales Order	salesorder	—

Time-Off Management

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following record types are related to the Time-Off Management feature:

- [Time-Off Change](#)
- [Time-Off Plan](#)
- [Time-Off Request](#)
- [Time-Off Rule](#)
- [Time-Off Type](#)

Time-Off Change

The internal ID for this record is `timeoffchange`.

For help working with this record in the user interface, see the help topic [Viewing an Employee's Time-Off Balance](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The time-off change record is scriptable in server SuiteScript only.

Note that triggering of user events is not supported for this record.

Supported Functions

The time-off change record is not fully scriptable. Transform and copy are not permitted.

Code Samples

The following example shows how to create a time-off change.

```

1  ...
2
3  var timeOffChange = record.create({
4      type: record.Type.TIME_OFF_CHANGE
5  });
6  timeOffChange.setValue({
7      fieldId: 'timeofftype',
8      value: '1' // ID of time-off type. In this example, ID of 1 is used.
9  });
10 timeOffChange.setValue({
11     fieldId: 'description',
12     value: 'Time-Off Type Name'
13 });
14 timeOffChange.setValue({
15     fieldId: 'dateapplied',
16     value: '12/25/2016'
17 });
18 timeOffChange.setValue({
19     fieldId: 'amount',
20     value: '1'
21 });
22 timeOffChange.setValue({
23     fieldId: 'timeoffunit',
24     value: '1' // ID of unit from drop-down

```

```

25 });
26 var id = timeOffChangeReq.save();
27
28 ...

```

Time-Off Plan

The internal ID for this record is `timeoffplan`.

For help working with this record in the user interface, see the help topic [Creating a Time-Off Plan](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The time-off plan record is scriptable in server SuiteScript only.

Note that triggering of user events is not supported for this record.

Supported Functions

The time-off plan record can be read, created, updated, searched, and deleted using SuiteScript. It cannot be copied or transformed.

Code Samples

The following example shows how to create a time-off plan and modify an existing time-off plan.

```

1  ...
2  //Create the Time-Off Plan record
3  var timeOffPlan = record.create({
4      type: record.Type.TIME_OFF_PLAN
5  });
6  timeOffPlan.setValue({
7      fieldId: 'name',
8      value: 'new Time-Off Plan'
9  });
10 timeOffPlan.setValue({
11     fieldId: 'isinactive',
12     value: 'F'
13 });
14 timeOffPlan.setValue({
15     fieldId: 'includefutureaccruals',
16     value: 'F'
17 });

```



```

18 timeOffPlan.setValue({
19     fieldId: 'subsidiary',
20     value: '1' // Use ID of subsidiary, location, department, or class. In this example, ID of 1 is used.
21 });
22 timeOffPlan.setValue({
23     fieldId: 'location',
24     value: '1'
25 });
26 timeOffPlan.setValue({
27     fieldId: 'department',
28     value: '1'
29 });
30 timeOffPlan.setValue({
31     fieldId: 'class',
32     value: '1'
33 });
34 timeOffPlan.setValue({
35     fieldId: 'startmonth',
36     value: '1'
37 });
38
39 var id = timeOffPlan.save();
40
41 //Modify existing Time-Off Plan record
42 var timeOffPlan = record.load({
43     type: record.Type.TIME_OFF_PLAN,
44     id: id
45 });
46 timeOffPlan.setValue({
47     fieldId: 'isinactive',
48     value: '1' // Inactivate plan
49 });
50 timeOffPlan.save();
51
52 ...

```

Time-Off Request

The internal ID for this record is timeoffrequest.

For help working with this record in the user interface, see the help topics [Submitting Time-Off Requests](#) and [Approving or Rejecting Time-Off Requests](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Note: To submit time-off requests, employees must have a start date and time-off plan assigned to them on the employee record.

Supported Script Types

The time-off request record is scriptable in server SuiteScript only.

Note that triggering of user events is not supported for this record.

Supported Functions

The time-off request record is not fully scriptable. Transform and copy are not permitted.

Code Samples

The following example shows how to create and approve a time-off request.

```

1  ...
2  //Create Time-Off Request
3  var timeOffRequest = record.create({
4      type: record.Type.TIME_OFF_REQUEST
5  });
6  timeOffRequest.setValue({
7      fieldId: 'employee',
8      value: '25'
9  });
10 timeOffRequest.setValue({
11     fieldId: 'startdate',
12     value: '12/25/2016'
13 });
14 timeOffRequest.setValue({
15     fieldId: 'enddate',
16     value: '12/26/2016'
17 });
18 timeOffRequest.selectNewLine({
19     sublistId: 'detail'
20 });
21 timeOffRequest.setCurrentSublistValue({
22     sublistId: 'detail',
23     fieldId: 'timeoffdate',
24     value: '12/25/2016'
25 });
26 timeOffRequest.setCurrentSublistValue({
27     sublistId: 'detail',
28     fieldId: 'timeofftype',
29     value: '1'
30 });
31 timeOffRequest.setCurrentSublistValue({
32     sublistId: 'detail',
33     fieldId: 'amountoftime',
34     value: '1.0'
35 });
36 timeOffRequest.commitLine({
37     sublistId: 'detail'
38 });
39 var id = timeOffRequest.save();
40
41 //Approve Time-Off Request
42 var timeOffRequest = record.load({
43     type: record.Type.TIME_OFF_REQUEST,
44     id: id // ID is the ID number of the request
45 });
46 timeOffRequest.setValue({
47     fieldId: 'approvalstatus',
48     value: '8' // Approval status is 8
49 });
50 timeOffRequest.save();
51 ...

```

Time-Off Rule

The internal ID for this record is timeoffrule.

For help working with this record in the user interface, see the help topic [Creating a Time-Off Rule](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The time-off rule record is scriptable in server SuiteScript only.

Note that triggering of user events is not supported for this record.

Supported Functions

The time-off rule record is not fully scriptable. Copy and search are not permitted.

Code Samples

The following code snippets show how to create a time-off rule.

```

1  ...
2
3  //Create Time-Off Rule to add to existing Time-Off Plan record.
4  var timeOffRule = record.create({
5      type: record.Type.TIME_OFF_RULE
6  });
7  timeOffRule.setValue({
8      fieldId: 'openingbalancefornewemployees',
9      value: 'T'
10 });
11 timeOffRule.setValue({
12     fieldId: 'entitlement',
13     value: '1'
14 });
15 timeOffRule.setValue({
16     fieldId: 'maximumtenure',
17     value: '1'
18 });
19 timeOffRule.setValue({
20     fieldId: 'minimumtenure',
21     value: '0'
22 });
23 timeOffRule.setValue({
24     fieldId: 'maximumtenureunit',
25     value: '1' // ID of unit available in drop-down. In this example, ID of 1 is used.
26 });
27 timeOffRule.setValue({
28     fieldId: 'minimumtenureunit',
29     value: '1'
30 });

```

```

31 timeOffRule.setValue({
32     fieldId: 'entitlementunit',
33     value: '1'
34 });
35 timeOffRule.setValue({
36     fieldId: 'timeofftype',
37     value: '1' // Use ID of Time-Off Type. In this example, ID of 1 is used.
38 });
39 timeOffRule.setValue({
40     fieldId: 'timeoffplan',
41     value: '1' // Use ID of Time-Off Plan you want to assign a rule to. In this example, ID of 1 is used.
42 });
43 var id = timeOffRule.save();
44
45 ...

```

Time-Off Type

The internal ID for this record is timeofftype.

For help working with this record in the user interface, see the help topic [Creating a Time-Off Type](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The time-off type record is scriptable in server SuiteScript only.

Note that triggering of user events is not supported for this record.

Supported Functions

The time-off type record is not fully scriptable. Transform and copy are not permitted.

Code Samples

The following example shows how to create a time-off type.

```

1 ...
2
3 //Create Time-Off Type
4 var timeOffType = record.create({
5     type: record.Type.TIME_OFF_TYPE
6 });

```

```

7  timeOffType.setValue({
8      fieldId: 'name',
9      value: 'Time-Off Type Name'
10 });
11 timeOffType.setValue({
12     fieldId: 'displayname',
13     value: 'Time-Off Type Name'
14 });
15 timeOffType.setValue({
16     fieldId: 'istrackonly',
17     value: 'T'
18 });
19 timeOffType.setValue({
20     fieldId: 'isautogeneratetimesentry',
21     value: 'T'
22 });
23 timeOffType.setValue({
24     fieldId: 'minimumincrement',
25     value: '1'
26 });
27 timeOffType.setValue({
28     fieldId: 'incrementunit',
29     value: '1' // ID of unit available in drop-down. In this example, ID of 1 is used.
30 });
31
32 timeOffType.setValue({
33     fieldId: 'payrollitem',
34     value: '1' // Use ID of payroll item in the drop-down. In this example, ID of 1 is used.
35 });
36 var id = timeOffType.save();
37
38 ...

```

Vendor

Applies to: SuiteScript 2.x | SuiteCloud Developer

A vendor is a company or person you purchase goods and services from. Vendor records track information about your vendors and enable you to view past transactions and communications with them.

For help working with this record in the user interface, see the help topic [Vendor Records Overview](#).

The internal ID for this record is vendor.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor record is scriptable in both client and server SuiteScript.

Supported Functions

The vendor record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
password	Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
password2	Confirm Password	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Transform Types

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Target Record Name	Target Record Internal ID	Field Defaults
Purchase Order	purchaseorder	—
Vendor Bill	vendorbill	—
Vendor Payment	vendorpayment	—

Items

Applies to: SuiteScript 2.x | SuiteCloud Developer

- [Using Item Records in SuiteScript](#)
- [Pricing Sublist / Pricing Matrix](#)
- [Analytical Impact](#)
- [Assembly Item](#)
- [Description Item](#)
- [Discount Item](#)
- [Download Item](#)

- [Gift Certificate Item](#)
- [Impact Subcategory](#)
- [Inventory Item](#)
- [Item Search](#)
- [Item Group](#)
- [Kit Item](#)
- [Lot Numbered Assembly Item](#)
- [Lot Numbered Inventory Item](#)
- [Markup Item](#)
- [Non-Inventory Part](#)
- [Other Charge Item](#)
- [Payment Item](#)
- [Planned Order](#)
- [Planning Item Category](#)
- [Planning Item Group](#)
- [Planning Rule Group](#)
- [Planning View](#)
- [Reallocate Items](#)
- [SaaS Metric](#)
- [Serialized Assembly Item](#)
- [Serialized Inventory Item](#)
- [Service](#)
- [Shipping Item](#)
- [Subscription Plan](#)
- [Subscription Term](#)
- [Subtotal Item](#)
- [Supply Chain Snapshot Simulation](#)
- [Supply Change Order](#)
- [Supply Plan Definition](#)

Using Item Records in SuiteScript

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

This section includes the following topics:

- [Loading Item Types](#)
- [Filtering Items by Type](#)
- [Looking Up the Subsidiary Field](#)

■ Advanced Revenue Management Scripting with Items

For information about working with items in the UI, see the help topics [Using Item Records](#) and [Item Types](#).

Loading Item Types

When using `record.load(options)`, you can:

- set the type parameter to `record.Type.INVENTORY_ITEM` to load the following types of item records: `inventoryitem`, `lotnumberedinventoryitem`, `serializedinventoryitem`
- set the type parameter to `record.Type.ASSEMBLY_ITEM` to load the following types of item records: `assemblyitem`, `lotnumberedassemblyitem`, `serializedassemblyitem`

Filtering Items by Type

The following are valid search filter item type IDs. Note that the item filter IDs are case-sensitive.

Item Type IDs	
Assembly	Markup
Description	NonInvPart
Discount	OthCharge
DwnLdItem	Payment
EndGroup	Service
GiftCert	ShipItem
Group	Subtotal
InvPart	TaxGroup
Kit	TaxItem

To use these IDs:

1. Create a script that will search for items of a specific type or types (for example, search for all non-inventory items).
2. Next, see any of the valid SuiteScript item type IDs.

Sample Code

```

1 //Create a script that will search for all non-inventory part items
2 function searchnoninventorypart()
3 {
4     var searchType = search.Type.NON_INVENTORY_ITEM;
5     var mySearch = search.create({
6         type: searchType,
7         columns: [{
8             name: 'internalId'
9         }],
10        filters: [{
11            name: ' ',
12            operator: search.Operator.ANYOF,
13            values: 'NonInvPart'
14        }]
15    })

```



```

15 |     });
16 | }

```

Looking Up the Subsidiary Field

The [search.lookupFields\(options\)](#) method lets you retrieve the current value of a field on a record. Usually, single select fields are returned as an object with value and text properties. Multiselect fields are returned as an object with a set of value: text pairs.

The subsidiary field on an item record is a multiselect field. However, when you use [search.lookupFields\(options\)](#) to retrieve the value of this field, the field is returned as a single value: text pair instead of a set of pairs. The returned value: text pair represents the first subsidiary in the list.

Advanced Revenue Management Scripting with Items

The item record (internal ID `item`) contains several additional accounting fields associated with the Advanced Revenue Management (Essentials) and Advanced Revenue Management (Revenue Allocation) features. The following table lists these scriptable fields.

Field	Type	Internal ID
Create Revenue Plans On	List/Record	createrevenueplanson
Item Revenue Category	List/Record	itemrevenuecategory
Revenue Allocation Group	List/Record	revenueallocationgroup
Revenue Recognition Rule	List/Record	revenuerecognitionrule

The **Item Revenue Category** and **Revenue Allocation Group** fields appear only when the Advanced Revenue Management (Revenue Allocation) feature is enabled.

Before you begin working with advanced revenue management programmatically, see the help topics [Setup for Advanced Revenue Management \(Essentials\)](#) and [Setup for Advanced Revenue Management \(Revenue Allocation\)](#).

For help working with this record in the UI, see the help topic [Item Configuration for Advanced Revenue Management \(Essentials\) and \(Revenue Allocation\)](#).

Pricing Sublist / Pricing Matrix

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The Pricing sublist is often referred to as a pricing matrix, since there can be multiple prices specified for an item as determined by one or more price levels and one or more quantity levels.

Functionally, the Pricing sublist shares many of the characteristics of list sublists. However, scripting to the Pricing sublist is not like scripting to other sublists in NetSuite. For this reason, you should read the following topics to learn about using SuiteScript on the Pricing sublist.

- [The Pricing Matrix](#)
- [Pricing Sublist Feature Dependencies](#)

- [Pricing Sublist Internal IDs](#)
- [Pricing Sublist Code Sample](#)
- [Matrix Sublist APIs and Standard Sublist APIs](#)

Note: The screen shots in this section depict the NetSuite UI prior to Version 2010 Release 2.

For general information about item pricing, see these topics in the NetSuite Help Center:

- [Item Pricing](#)
- [Setting Up Item Pricing](#)
- [Using Item Records](#)

To see which records the Pricing sublist appears on, see [Records that Include the Pricing Sublist](#).

The Pricing Matrix

Depending on the features enabled in your account, the Pricing sublist on many item records can resemble a matrix of rows and columns of various prices (see figure). To access the price values on a per-row, per-column basis, you must use matrix APIs. These APIs are a subset of the standard APIs that are more commonly used when scripting with other sublists. See [Matrix Sublist APIs and Standard Sublist APIs](#) for more information.

Note: Non-matrix sublist APIs can also be used on the Pricing sublist. However, they are not used to get or set values considered to be part of the pricing matrix. For information about when to use matrix and non-matrix APIs on the Pricing sublist, see [Matrix Sublist APIs and Standard Sublist APIs](#).

The figure below provides an overview of the rows and columns considered to be the pricing matrix. As previously stated, the configuration of the Pricing sublist greatly depends on the features enabled in your account. However, regardless of the features set, all configurations will have some variation of a row / column matrix layout like the one shown below.

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

This code snippet shows the kind of values you will typically set when working with price values in the pricing matrix. The internal ID of the Pricing sublist, as well as its field IDs, will change depending on the features enabled in your account.

Note: See [Pricing Sublist Feature Dependencies](#) and [Pricing Sublist Internal IDs](#) for more information.

Example

```
1 record.getMatrixSublistValue ({
2   sublistId: 'price',
3   fieldId: 'price',
4   column: 1,
5   line: 2
6 });
```

In this sample you:

1. Specify the sublist internal ID (price).
2. Specify the internal ID of the pricing field (which will generally be price).



Important: Although the UI labels in this figure show field names such as Alternate Price 1, Alternate Price 2, and Online Price, the internal ID for the fieldId parameter is still price. The only exception is described in [Pricing Sublist Field IDs](#) for the currency field.

3. Specify the line number (row) of the price you want to get (in this sample, you are getting the value in row 2 – this is the price for **Alternate Price 1**).
4. Specify the column number you want to get the value for (in this sample, you are getting the value in column 1 for **Alternate Price 1**).

Pricing

Quantity Pricing Schedule: Pricing Schedule 1 Calculate Quantity Discounts: By Line Quantity Pricing Group:

☒ Use Marginal Rates

USA • British pound • Canadian Dollar • Euro • Yen •		Qty 0	Qty 100	Qty 200	Qty 300	Qty
Price Level	Default Discount %					
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

1	Column one
2	Row two

Price is typically the internal ID for the fieldId parameter.

Pricing Sublist Feature Dependencies

There are three features that, if enabled or disabled in your account, can affect:

- the overall functionality of the Pricing sublist

- the appearance of the Pricing sublist in the UI
- the internal IDs that are referenced in SuiteScript

You can check which of these features are enabled by looking in the UI, or by calling the [runtime.isFeatureInEffect\(options\)](#) method and specifying the feature internal ID.

The features that affect the Pricing sublist are:

- [Multiple Currencies](#)
- [Multiple Prices](#)
- [Quantity Pricing](#)

If none of these features is enabled in your account, then there is no Pricing sublist on the item record, and the field that holds the item price appears on the Basic subtab as Sales Price. The internal ID for this field is rate. Do not use sublist methods to set or get values on rate. Instead, use field methods.

Multiple Currencies

Enable this feature when you want to set item prices in more than one currency. You then specify separate pricing for each currency. On the Pricing sublist you will see subtabs with the name of the currencies specified in your account (see figure).



Important: See [Pricing Sublist ID](#) to learn how to determine the internal ID of the Pricing sublist based on whether the Multiple Currencies feature is enabled.

The screenshot shows the NetSuite Pricing subtab. At the top, there are dropdowns for 'Quantity Pricing Schedule' (set to 'Pricing Schedule 1'), 'Calculate Quantity Discounts' (set to 'By Line Quantity'), and 'Pricing Group'. A checkbox 'Use Marginal Rates' is checked. Below these, a row of currency tabs is visible: 'USA', 'British pound', 'Canadian Dollar', 'Euro', and 'Yen'. The 'USA' tab is selected and highlighted with a red box. Below the tabs is a table with columns for 'Price Level', 'Default Discount %', and quantity-based price columns (Qty 0, Qty 100, Qty 200, Qty 300, Qty). The table contains rows for 'Base Price', 'Alpha price level', 'Alternate Price 1', 'Alternate Price 2', 'Alternate Price 3', 'Alternate Price 4' (with a 10.0% discount), 'Beta price level' (with a 50.0% discount), 'Wholesale Price', and 'Online Price'. Each row has input fields for the prices in each quantity column.

In the UI, you can check if this feature is enabled by looking at the Pricing sublist itself or by going to Setup > Company > Enable Features. On the Company subtab, the Multiple Currencies box will be checked if this feature is enabled.



Note: Only a NetSuite administrator can enable the Multiple Currencies feature.

In SuiteScript, you can get the feature status by writing something similar to:

```
1 var multiCurrency = runtime.isFeatureInEffect ({
2   feature: 'MULTICURRENCY'
3 });
```

Important: See [Pricing Sublist Code Sample](#) for more details.

Multiple Prices

Enable this feature when you want to specify different prices for different conditions or types of customers using Price Levels. A set of standard Price Levels is provided by NetSuite, which you can change or extend to suit your needs.

This figure shows the Pricing sublist with the Multiple Prices feature enabled.

Note: You can specify multiple prices for the same item.

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1 | Calculate Quantity Discounts: By Line Quantity | Pricing Group:

☒ Use Marginal Rates

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

By comparison, this figure shows the Pricing sublist with the Multiple Prices feature disabled. You can set only one price for the item.

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1 | ☒ Use Marginal Rates | Calculate Quantity Discounts: By Line Quantity

Price Levels -

Currency	Qty 0	Qty 100	Qty 200	Qty 300	Qty
USA	50.00	50.00	52.50	55.00	
British pound					
Canadian Dollar					
Euro					
Yen					

In the UI, you can check if this feature is enabled by looking at the Pricing sublist itself or by going to Setup > Company > Enable Features. On the Transactions subtab, the Multiple Prices box will be checked if this feature is enabled.

Note: Only a NetSuite administrator can enable the Multiple Prices feature.

In SuiteScript, you can get the feature status by writing something similar to:

```
1 | var multiPrice = runtime.isFeatureInEffect ({
2 |     feature: 'MULTIPRICE'
3 | });
```



Important: See [Pricing Sublist Code Sample](#) for more details.

Quantity Pricing

Enable this feature when you want to vary the item price based on the quantity of items sold. Specifically, this feature lets you specify different quantity levels with different prices at each quantity level.

The following figure shows the Pricing sublist with the Quantity Pricing feature enabled.



Note: When the Quantity Pricing feature is enabled, an administrator can specify the number of Qty columns that appear on the Pricing sublist. The following figure shows that four Qty columns have been specified. Set the Qty preference by going to Setup > Accounting > Accounting Preferences > Items & Transactions. In the Maximum # of Quantity-based Price Levels field, specify the number of columns.

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1

Calculate Quantity Discounts: By Line Quantity

Pricing Group:

☒ Use Marginal Rates

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

Item pricing is determined by values specified in the Qty fields.

By comparison, the following figure shows the Pricing sublist with the Quantity Pricing feature disabled. Item prices are not determined by the quantities specified.

No Qty fields exist in the following image:

▼ Pricing

Pricing Group:

Price Level	Default Discount %	Amount
Base Price		50.00
Alpha price level		75.00
Alternate Price 1		
Alternate Price 2		
Alternate Price 3		
Alternate Price 4	10.0%	55.00
Beta price Level	50.0%	75.00
Wholesale Price		
Online Price		

In the UI, you can check if this feature is enabled by looking at the Pricing sublist itself or by going to Setup > Company > Enable Features. On the Transactions subtab, the Quantity Pricing box will be checked if this feature is enabled.

Note: Only a NetSuite administrator can enable this feature.

In SuiteScript, you can get the feature status by writing something similar to:

```
1 var quantityPricing = runtime.isFeatureInEffect ({
2   feature: 'QUANTITYPRICING'
3 });
```

Important: See [Pricing Sublist Code Sample](#) for more details.

Pricing Sublist Internal IDs

As discussed in [Pricing Sublist Feature Dependencies](#), the Pricing sublist looks and functions differently depending on the features set in your account.

See [Pricing Sublist ID](#) for the internal ID of the Pricing sublist depending on features enabled in your account.

See [Pricing Sublist Field IDs](#) for all other field IDs associated with this sublist.

Pricing Sublist ID

In SuiteScript, the internal ID of the Pricing sublist is determined by the features enabled in your NetSuite account.

If the Multiple Currencies feature is not enabled in your account, the internal ID for the Pricing sublist is price. This means that you will set the sublist parameter in APIs such as [Record.getMatrixHeaderField\(options\)](#) and [Record.setMatrixHeaderValue\(options\)](#) to price.

If Multiple Currencies is enabled, then there are separate Pricing sublists per currency as shown in the following figure.

The screenshot shows the NetSuite Pricing Sublist interface. At the top, there are dropdowns for 'Quantity Pricing Schedule' (set to 'Pricing Schedule 1'), 'Calculate Quantity Discounts' (set to 'By Line Quantity'), and 'Pricing Group'. A checkbox 'Use Marginal Rates' is checked. Below these, there are tabs for different currencies: 'USA', 'British pound', 'Canadian Dollar', 'Euro', and 'Yen'. The 'USA' tab is selected and highlighted with a red box. Below the tabs is a table with columns for 'Price Level', 'Default Discount %', and 'Qty' (with sub-columns for 0, 100, 200, 300, and an empty 'Qty' column). The table contains several rows of pricing data, including 'Base Price', 'Alpha price level', 'Alternate Price 1', 'Alternate Price 2', 'Alternate Price 3', 'Alternate Price 4' (with a 10.0% discount), 'Beta price level' (with a 50.0% discount), 'Wholesale Price', and 'Online Price'.

Each currency pricing list will have its own internal ID. For example, the internal ID for the currency called USA will be price1. This ID reflects the internal ID of the sublist (price) and the internal ID of the USA currency (1).

The internal ID for the Canadian dollar sublist will be price3. This reflects the internal ID of the sublist (price) and the internal ID of the Canadian dollar currency (3).

This figure shows the currencies that have been set in this account. The Internal ID for each currency is the numeric value appended to price. When the Multiple Currencies feature is enabled, you can see the internal ID for each currency by going to Lists > Accounting > Currencies.

Currencies						
New Refresh ☐ Show Inactives						
Internal ID	Name	ISO Code	Base Currency	Inactive	Automatic Update	
1	USA	USD	Yes	No	Yes	
2	British pound	GBP	No	No	Yes	
3	Canadian Dollar	CAD	No	No	Yes	
4	Euro	EUR	No	No	Yes	
5	Yen	JPY	No	No	No	

Based on the internal ID shown in the preceding figure, set the sublistId parameter in APIs such as [Record.getMatrixHeaderField\(options\)](#) as follows (fieldId and column parameter values are examples values):

```

1 // if scripting on the USA dollar tab:
2   record.getMatrixHeaderField ({
3     sublistId: price1,
4     fieldId: fldname,
5     column: column
6   })
7 // if scripting on the British pound tab:
8   record.getMatrixHeaderField ({
9     sublistId: price2,
10    fieldId: fldname,
11    column: column
12  })
13 // if scripting on the Canadian dollar tab:
14   record.getMatrixHeaderField ({
15     sublistId: price3,
16     fieldId: fldname,
17     column: column
18   })
19 // if scripting on the Euro tab:
20   record.getMatrixHeaderField ({
21     sublistId: price4,
22     fieldId: fldname,
23     column: column
24   })

```

For topics related to this topic, see [Pricing Sublist Feature Dependencies](#).

Pricing Sublist Field IDs

This table provides the internal IDs for all fields associated with the Pricing sublist. Field types are categorized as matrix fields, sublist fields, and body fields.

Field UI Label	Field Internal ID	Field Type	Required	Field Notes
Matrix Fields				
Base Price	price	string	true	The price for that level and that quantity. See Figure 1 – Matrix Fields .

Field UI Label	Field Internal ID	Field Type	Required	Field Notes
Qty				 Important: When using matrix APIs (that is, any API that has the word Matrix in its name), price will generally be the value specified for the fieldId parameter. The exception to this is if the Multiple Currencies feature is enabled in your account.
Sublist Fields				
Default Discount %	discount	—	—	See Figure 2 – Sublist Fields . See also Standard Sublist APIs for a code sample that references this internal ID.
—	currency	—	—	The currency field is a hidden field. It is not visible in the UI. See Figure 2 – Sublist Fields. This field is scriptable only when the Multiple Currencies feature is enabled. The internal IDs for the fieldId parameter in matrix APIs will be price1currency, price2currency, pricecurrency3, etc., to reflect the currency internal ID.
Price Level	pricelevel	—	—	The pricelevel for this price. See Figure 2 – Sublist Fields. See also Standard Sublist APIs for a code sample that references this internal ID.
Body Fields				
Quantity Pricing Schedule	quantitypricingschedule	select	false	If a quantity pricing schedule has been specified in the UI drop-down, item prices will be calculated for the pricing matrix according to the specified schedule. See Figure 3 – Body Fields. Also note that the value of the quantity pricing schedule sets the value of the Calculate Quantity Discounts (overallquantitypricingtype) field. Use field methods in the N/record Module or N/currentRecord Module module to access this field.
Calculate Quantity Discounts	overallquantitypricingtype	select	false	Used to determine the quantity amount at the time the item is priced on the order. (This field does not change price settings in the matrix). See Figure 3 – Body Fields. Use field methods in the N/record Module or N/currentRecord Module module to access this field.
Use Marginal Rates	usemarginalrates	checkbox	false	Used to determine how the quantity discounts are applied at the time the item is priced on the order with a specified quantity. (This field does not change the price settings in the matrix.) See Figure 3 – Body Fields .

Field UI Label	Field Internal ID	Field Type	Required	Field Notes
				Use field methods in the N/record Module or N/currentRecord Module module to access this field.
Pricing Group	pricinggroup	select	false	Used to provide customer- specific pricing. Could affect the pricing at the time the item is placed on the order and associated with a specific customer. See Figure 3 – Body Fields . Use field methods in the N/record Module or N/currentRecord Module module to access this field.

Figure 1. Figure 1 – Matrix Fields

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1

Calculate Quantity Discounts: By Line Quantity

Pricing Group:

☒ Use Marginal Rates

USA • British pound • Canadian Dollar • Euro • Yen •

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

Pricing sublist matrix field.

Figure 2. Figure 2 – Sublist Fields

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1

Calculate Quantity Discounts: By Line Quantity

Pricing Group:

☒ Use Marginal Rates

USA • British pound • Canadian Dollar • Euro • Yen •

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

Use non-matrix sublist APIs to get or set values for these pricing sublist fields.

Figure 3. Figure 3 – Body Fields

▼ Pricing

Quantity Pricing Schedule
Pricing Schedule 1

Calculate Quantity Discounts
By Line Quantity

Pricing Group

☒ Use Marginal Rates

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

Use standard field APIs to get or set values for Pricing sublist body fields.

Pricing Sublist Code Sample

This sample shows how to determine which pricing-related features are enabled in your account. It then shows how to programmatically determine the internal ID for the Pricing sublist itself, and then verify whether a Quantity Schedule has been applied to the items in this list. The script also shows how to set item prices and quantity levels depending on various conditions set within the pricing matrix.

Note: If your browser is inserting scroll bars in this code sample, maximize your browser window, or expand the main frame in which this sample appears.

```

1 // Check the features enabled in the account. See Pricing Sublist Feature Dependencies for
2 // details on why this is important.
3 var multiCurrency = runtime.isFeatureInEffect({
4     feature: 'MULTICURRENCY'
5 });
6 var multiPrice = runtime.isFeatureInEffect({
7     feature: 'MULTIPRICE'
8 });
9 var quantityPricing = runtime.isFeatureInEffect({
10     feature: 'QUANTITYPRICING'
11 });
12
13 // Set the name of the Price sublist based on which features are enabled and currency type.
14 // See Pricing Sublist Internal IDs for details on why this is important.
15 var priceID;
16 var currencyID = 'EUR';
17
18 // Set the ID for the sublist and the price field. Note that if all pricing-related features
19 // are disabled, you will set the price in the rate field. See Pricing Sublist Feature Dependencies
20 // for details.
21 if (multiCurrency === true && multiPrice === false && quantityPricing === false)
22     priceID = 'rate';
23 else {
24     priceID = 'price';
25     if (multiCurrency === true){
26         var internalId = search.create ({
27             type: search.Type.CURRENCY,
28             filters: [{
29                 name: 'symbol',
30                 operator: search.Operator.CONTAINS,
31                 values: currencyID
32             }],
33         });
34         priceID = priceID + internalId;

```

```

35     }
36 }
37
38 // Verify that the item is using a Quantity Schedule
39 // If a Quantity Schedule is used, only the base price needs to be set.
40 // All other prices will be set according to the schedule.
41 var itemRecord = record.load ({
42     type: record.Type.INVENTORY_ITEM,
43     id: itemId
44 });
45 var qtyPriceSchedule = itemRecord.getValue({
46     fieldId: 'quantitypricingschedule'
47 });
48
49 // Set the base price
50 var basePrice = 100;
51
52 // You must select, set, and then commit the sublist line you want to change.
53 itemRecord.selectLine({
54     sublistId: priceID,
55     line: 1
56 });
57 itemRecord.setCurrentMatrixSublistValue({
58     sublistId: priceID,
59     fieldId: 'price',
60     column: 1,
61     value: basePrice
62 });
63 itemRecord.commitLine({
64     sublistId: priceID
65 });
66
67 // Get the number of columns in the price matrix
68 // Each column represents a different quantity level
69 columnCount = itemRecord.getMatrixHeaderCount({
70     sublistId: priceID,
71     fieldId: 'price'
72 });
73
74 // Set the base price in each quantity of the price matrix for a specific sublist, for example, currency
75
76 // Set the base price in each quantity
77 for (var j = 1; j <= columnCount; j++){
78     // Set the price for this cell of the matrix
79     itemRecord.selectLine({
80         sublistId: priceID,
81         line: 1
82     });
83     itemRecord.setCurrentMatrixSublistValue({
84         sublistId: priceID,
85         fieldId: 'price',
86         column: j,
87         value: currencyBasePrice
88     });
89     itemRecord.commitLine({
90         sublistId: priceID
91     });
92 }
93
94 // Display the full price matrix for a specific currency as an HTML table
95
96 // get the size of the matrix
97 var quantityLevels = itemRecord.getMatrixHeaderCount({
98     sublistId: priceID,
99     fieldId: 'price'
100 });
101 var priceLevels = itemRecord.getLineCount({
102     sublistId: priceID
103 });
104 var priceName = "";
105 var priceNameField = 'pricelevel';
106 var itemPrice = 0;
107 var fieldObj = null;

```

```

108
109 // create an xml table to present the results
110 var strName = '<table>';
111 if (quantityLevels > 1){
112     strName += '<tr>';
113     // write out the quantity levels as the first row
114     for (var j = 1; j <= quantityLevels; j++){
115         strName += '<td>';
116         fieldObj = itemRecord.getMatrixHeaderField( {
117             sublistId: priceID,
118             fieldId: 'price',
119             column: j
120         });
121         if (fieldObj != null){
122             strName += fieldObj.getLabel();
123         }
124         strName += j;
125         strName += '</td>';
126         strName += '<td>';
127
128         // this matrix API obtains the value of the Quantity level
129         strName += itemRecord.getMatrixHeaderValue({
130             sublistId: priceID,
131             fieldId: 'price',
132             column: j
133         });
134         strName += '</td>';
135     }
136     strName += '</tr>';
137 }
138
139 // loop through the matrix one row at a time
140 for (var i = 1; i <= priceLevels; i++){
141     strName += '<tr>';
142     // loop through each column of the matrix
143     for (j = 1; j <= quantityLevels; j++){
144         // get the price for this cell of the matrix
145         itemPrice = itemRecord.getLineItemMatrixValue( priceID, 'price', i, j);
146         // Get the name of the price level. Note: you will use a standard
147         // sublist API and not a matrix API for this.
148         priceName = itemRecord.getSublistText({
149             sublistId: priceID,
150             fieldId: priceNameField,
151             line: i
152         });
153
154         strName += '<td>';
155         strName += priceName;
156         strName += '</td>';
157         strName += '<td>';
158         strName += itemPrice;
159         strName += '</td>';
160     }
161     strName += '</tr>';
162 }
163 strName += '</table>';

```

Matrix Sublist APIs and Standard Sublist APIs

To get or set values in the pricing matrix, use matrix APIs (within [N/record Module](#) and [N/currentRecord Module](#) modules).

If you want to get or set non-matrix fields, use all non-matrix sublist or field (within the [N/record Module](#) or [N/currentRecord Module](#) modules), depending on which fields you are trying to access.

Note: See [The Pricing Matrix](#) for information about the pricing matrix. See [Pricing Sublist Field IDs](#) for information about the differences among matrix, non-matrix, and body sublist fields.

Matrix APIs

The following are considered to be matrix APIs for use on the Pricing sublist. Use these APIs to get or set matrix fields. See [Pricing Sublist Field IDs](#) to learn which fields are considered matrix fields.

Click the following links to see the API documentation for each matrix API. Also see the figures below for a visual representation for where on the pricing matrix each matrix API runs.

- [Record.findMatrixSublistLineWithValue\(options\)](#)
- [Record.getCurrentMatrixSublistValue\(options\)](#)
- [Record.getMatrixHeaderCount\(options\)](#)
- [Record.getMatrixHeaderField\(options\)](#)
- [Record.getMatrixHeaderValue\(options\)](#)
- [Record.getMatrixSublistField\(options\)](#)
- [Record.getMatrixSublistValue\(options\)](#)
- [Record.setCurrentMatrixSublistValue\(options\)](#)
- [Record.setMatrixHeaderValue\(options\)](#)
- [Record.setMatrixSublistValue\(options\)](#)
- [CurrentRecord.findMatrixSublistLineWithValue\(options\)](#)
- [CurrentRecord.getCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.getMatrixHeaderCount\(options\)](#)
- [CurrentRecord.getMatrixHeaderField\(options\)](#)
- [CurrentRecord.getMatrixHeaderValue\(options\)](#)
- [CurrentRecord.getMatrixSublistField\(options\)](#)
- [CurrentRecord.getMatrixSublistValue\(options\)](#)
- [CurrentRecord.setCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.setMatrixHeaderValue\(options\)](#)
- [CurrentRecord.setMatrixSublistValue\(options\)](#)

▼ Pricing

Quantity Pricing Schedule: Calculate Quantity Discounts: Pricing Group:

☒ Use Marginal Rates

USA • British pound • Canadian Dollar • Euro • Yen •		Qty 0	Qty 100	Qty 200	Qty 300	Qty
Price Level	Default Discount %					
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

On matrix header fields, use [Record.getMatrixHeaderField\(options\)](#), [Record.getMatrixHeaderValue\(options\)](#), and [Record.getMatrixHeaderCount\(options\)](#).

When on an existing line in the matrix, use [Record.getCurrentMatrixSublistValue\(options\)](#) and [Record.setCurrentMatrixSublistValue\(options\)](#) to get and set the price on that line in the specific column, respectively.

▼ Pricing

Quantity Pricing Schedule: Pricing Schedule 1 Calculate Quantity Discounts: By Line Quantity Pricing Group:

☒ Use Marginal Rates

USA • British pound • Canadian Dollar • Euro • Yen •

Price Level	Default Discount %	Qty 0	Qty 100	Qty 200	Qty 300	Qty
Base Price		50.00	50.00	52.50	55.00	
Alpha price level		75.00	75.00	78.75	82.50	
Alternate Price 1						
Alternate Price 2						
Alternate Price 3						
Alternate Price 4	10.0%	55.00	55.00	57.75	60.50	
Beta price Level	50.0%	75.00	75.00	78.75	82.50	
Wholesale Price						
Online Price						

For all other lines in the matrix, use [Record.getMatrixSublistValue\(options\)](#) and [Record.findMatrixSublistLineWithValue\(options\)](#).

Standard Sublist APIs

If you want to reference the other fields in the Pricing sublist, such as currency, name, or discount, use the existing [Record.getSublistValue\(options\)](#) and pass in the existing fieldId (for example: price1currency). Also see [Pricing Sublist Field IDs](#), which specifies which fields on the Pricing sublist can be set using standard sublist APIs (within the [N/record Module](#) and [N/currentRecord Module](#) modules).

Example:

```

1 // load an item record
2 var record = record.load ({
3   type: record.Type.INVENTORY_ITEM,
4   id: 536 // for example - use your own record id
5 });
6
7 // get the value of the currency field on line 2
8 var currency = record.getSublistValue({
9   sublistId: 'price1',
10  fieldId: 'currency',
11  line: '2'
12 });
13
14 // get the value of the pricelevelname field on line 2
15 var pricelevelname2 = record.getSublistValue({
16   sublistId: 'price1',
17   fieldId: 'pricelevel',
18   line: '2'
19 });
20 var pricelevelname3 = record.getSublistValue({
21   sublistId: 'price1',
22   fieldId: 'pricelevel',
23   line: '3'
24 });
25 var pricelevelname4 = record.getSublistValue({
26   sublistId: 'price1',
27   fieldId: 'pricelevel',

```

```

28     line: '4'
29   });
30
31   // returns the discount from line item 2
32   var discount2 = record.getSublistValue({
33     sublistId: 'price1',
34     fieldId: 'discount',
35     line: '2'
36   });
37   var discount3 = record.getSublistValue({
38     sublistId: 'price1',
39     fieldId: 'discount',
40     line: '3'
41   });
42   var discount4 = record.getSublistValue({
43     sublistId: 'price1',
44     fieldId: 'discount',
45     line: '4'
46   });

```

Records that Include the Pricing Sublist

The Pricing sublist appears on the following records that are scriptable using SuiteScript:

- Assembly Item
- Lot Numbered Assembly Item
- Serialized Assembly Item
- Lot Numbered Inventory Item
- Serialized Inventory Item
- Gift Certificate Item
- Kit Item
- Inventory Item

Analytical Impact

Applies to: SuiteScript 2.x | SuiteCloud Developer

The analytical impact record is available only when the SaaS Metrics feature is enabled at Setup > Company > Enable Features (Administrator), on the Analytics subtab. With this feature enabled, you can open the item you want to collect data about, set a start date, and start tracking SaaS metrics for transactions and subscriptions that contain that item. After the item is set up, source records that contain the item can generate Analytical Impact records.

If you need historical reporting of SaaS metrics for transactions that already include these items, you can set the start date in the past. Adding an end date helps to prevent errors in reporting and gives you greater control in defining what's included in the report.

The internal ID for this record is `analyticalimpact`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The analytical impact record is supported in both server and client SuiteScript.

Supported Functions

The analytical impact record is scriptable. It can be read, updated, deleted, and searched using SuiteScript, with the following exceptions:

- the template is stored, so is not scriptable
- the System Notes sublist is search-only

Code Sample

The following sample shows how to edit an analytical impact event, monthly impact amount, impact category and impact subcategory.

```

1  ///  
2  var analyticalimpactKey = '2';  
3  var adjustamount = '20.00';  
4  var analyticalimpactCategoryKey = '2';  
5  var analyticalimpactSubcategoryKey = '-19';  
6  var record = nlapiLoadRecord('analyticalimpact', analyticalimpactKey, {recordmode: 'dynamic'}); record.selectLineItem('analyticalimpactevent', 1);  
7  var adjustamount = record.getCurrentLineItemValue('analyticalimpactevent', 'adjustmonthlyimpactamount'); alert("Adjust Monthly Impact Amount = " + adjustamount);  
8  record.setCurrentLineItemValue('analyticalimpactevent', 'adjustmonthlyimpactamount', adjustamount); record.setCurrentLineItemValue('analyticalimpactevent', 'adjustimpactcategory', analyticalimpactCategoryKey); record.setCurrentLineItemValue('analyticalimpactevent', 'adjustimpactsubcategory', analyticalimpactSubcategoryKey); record.commitLineItem('analyticalimpactevent');  
9  nlapiSubmitRecord(record);  
10 on sql debugger page, select Api: 1.0

```

Assembly Item

i Applies to: SuiteScript 2.x | SuiteCloud Developer

An assembly item is an inventory item made of several components, but identified as a single item. Assemblies are manufactured by combining raw materials you stock.

You create assembly item records to define the members of an assembly, then NetSuite enables you to track both the raw materials and the assembled items separately.

For example, an Wolfe Electronics sells a computer called Creativo 2400 that they assemble in-house. The Creativo 2400 computer is assembled from these inventory components: one Superior 2 GHz processor, one Creativo 2400 motherboard, 1 GB RAM, 80 GB Hard Drive, one Superior sound card, and one power supply.

NetSuite tracks the stock of the Creativo 2400 and each component item separately. Then, Wolfe can track the stock level of Creativo 2400 in inventory and available to ship to customers, and the quantity of materials available to assemble more.

For help working with this record in the UI, see the help topic [Assembly Items](#).

The internal ID for this record is `assemblyitem`. This record is also sometimes referred to as Build/Assembly.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The assembly item record is scriptable in server SuiteScript only.

Supported Functions

The assembly item record is fully scriptable — it can be created, updated, copied, deleted, and searched using server SuiteScript. It can also be transformed.

Usage Notes

The `taxschedule` field of the Members sublist is visible only in the UI when the Advanced Taxes feature is enabled.

The assembly item record includes the [Pricing Sublist / Pricing Matrix](#).

Code Samples

The following sample changes the item name of an assembly.

```
1 var assembly = record.load({
2   type: record.Type.ASSEMBLY_ITEM,
3   id: 123
4 });
5 assembly.setValue({
6   fieldId: 'itemid',
7   value: 'new name'
8 });
9 var recordId = assembly.save();
```

Description Item

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Description line items let you add a sentence- or paragraph-long description on items you are not selling. For example, you may want to enter special shipping instructions or a disclaimer.

Description items have no amount field. They are only used to add text to transactions. They can be used on both purchase and sales transactions.

The item name of a description item does not appear on printed forms, only the descriptive text. Nothing appears in the amount column for description items.

As you create a transaction, select a description item from the items list.

For help working with this record in the UI, see the help topic [Description Items](#).

The internal ID for this record is `descriptionitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The description record is scriptable in both client and server SuiteScript.

Supported Functions

The description record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Discount Item

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Discount item records are used to create discounts you can apply to your transactions.

For help working with this record in the UI, see the help topic [Discount Items](#).

The internal ID for this record is `discountitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

- [N/record Module](#)

Supported Script Types

The discount record is scriptable in both client and server SuiteScript.

Supported Functions

The description record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Download Item

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You create download item records for files that you want customers to be able to purchase and download in your Web store.

Customers are charged per download item as opposed to per item. For example, if you want to charge customers for music downloads per song, you would create an item for each song. If you want to charge customers per album, you would create one item and attach each song for the album.

If a download requires a license code, such as a software download, you can add the license code on the Downloads subtab of the customer's record. When a license code is added, the code is available to the customer in the Customer Center or the My Account tab of your website and included in invoice email notification.

For help working with this record in the UI, see the help topic [Download Items](#).

The internal ID for this record is `downloaditem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The download item record is scriptable in both client and server SuiteScript.

Supported Functions

The download item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Gift Certificate Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can create gift certificate items that allow customers to purchase store credit they can send to someone as a gift. The recipient uses the gift certificate code when placing an order through your Web store or entering a transaction with a sales representative. You can set a preference in NetSuite for how you want to generate the gift certificate codes: you can create them yourself, or use a random hash code automatically generated by the system.

Gift certificate codes are not active until the order used to purchase the gift certificate is billed.

For help working with this record in the UI, see the help topic [Gift Certificates](#).

The internal ID for this record is `giftcertificateitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The gift certificate record is scriptable in both client and server SuiteScript.

Supported Functions

The gift certificate item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The gift certificate item record includes the [Pricing Sublist / Pricing Matrix](#).

Impact Subcategory

Applies to: SuiteScript 2.x | SuiteCloud Developer

The impact subcategory record is available only when the SaaS Metrics feature is enabled at Setup > Company > Enable Features (Administrator), on the Analytics subtab. To find this record, go to Setup > Accounting > Accounting Lists > Impact Subcategory. You can filter the accounting list to show only impact subcategories.

The internal ID for this record is `impactsubcategory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The impact subcategory record is supported in both server and client SuiteScript.

Supported Functions

The impact subcategory record is fully scriptable – it can be read, created, updated, and deleted using SuiteScript.

Code Sample

The following sample shows how to create an impact subcategory.

```
1 //== Create Impact
2   Subcategory
3 var record= nlapiCreateRecord('impactsubcategory', {});
4 record.setFieldValue('name','test category 006');
5 record.setFieldValue('isinactive','T');
6 var isKey= nlapiSubmitRecord(record, {"enablesourcing":"true"});
7 alert('Impact subcategory created = ' + isKey);
```

The following sample shows how to edit an impact subcategory.

```
1 //== Edit Impact
2   Subcategory
3 var impactKey = '206';
4 var record = nlapiLoadRecord('impactsubcategory', impactKey , {'recordmode': 'dynamic'});
5 record.setFieldValue('name','test category 006-02');
6 record.setFieldValue('isinactive','F');
7 impactKey= nlapiSubmitRecord(record, {"enablesourcing":"true"});
8 alert('Impact subcategory updated = ' + impactKey);
```

Inventory Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

NetSuite inventory item records enable you to track the quantity and value of your inventory. Your balance sheet will automatically reflect the value of your inventory on hand, and your income statement will automatically reflect the markup you charge for these items.

This record is also referred to as Inventory Part in the UI.

For help working with this record in the UI, see the help topic [Inventory Items](#).

The internal ID for this record is `inventoryitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory item record is scriptable in both client and server SuiteScript.

Supported Functions

The inventory item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The inventory item record includes the [Pricing Sublist / Pricing Matrix](#).

Item Group

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

An item group is stocked and sold as a single unit, but may be made up of several individual items.

For help working with this record in the UI, see the help topic [Item Groups](#).

The internal ID for this record is `itemgroup`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

- [N/record Module](#)

Supported Script Types

The item group record is scriptable in both client and server SuiteScript.

Supported Functions

The item group record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Item Search

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Item Record Management](#).

The internal ID for this record is `item`.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item search record is a search record only.

Supported Functions

The item search record is a search record only. You cannot create, update, delete, or copy this record.

Kit Item

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

Kits or packages let you create items that are collected from other items. You can assign multiple price levels to your kits and even make them available in your website. Whenever you sell a kit, inventory items are deducted from inventory.

For help working with this record in the UI, see the help topic [Kit/Package Items](#).

The internal ID for this record is `kititem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The kit record is scriptable in server SuiteScript.

Supported Functions

The kit record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The taxschedule field of the Members sublist is visible only in the UI when the Advanced Taxes feature is enabled.

The kit item record includes the [Pricing Sublist / Pricing Matrix](#).

Lot Numbered Assembly Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Lot numbered assembly items enable you to build items from raw materials and track the inventory of both the finished items and the raw materials separately. The completed assembly is assigned a lot number to track it as it enters and leaves your inventory.

For help working with this record in the UI, see the help topic [Lot Numbered Items](#). For help working with assembly items in the UI information, see the help topic [Assembly Items](#).

The internal ID for this record is lotnumberedassemblyitem.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The lot numbered assembly item record is scriptable in both client and server SuiteScript.

Supported Functions

The lot numbered assembly item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The taxschedule field of the Members sublist is visible only in the UI when the Advanced Taxes feature is enabled.

The lot numbered assembly item record includes the [Pricing Sublist / Pricing Matrix](#).

Lot Numbered Inventory Item

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Lot numbered inventory items track the purchase, stock, and sale of groups of items by assigning lot numbers. Lot numbered item records track the quantity of items and the specific cost for each lot as products are purchased and sold.

For help working with this record in the UI, see the help topic [Lot Numbered Items](#). For help about using inventory items in the UI, see the help topic [Inventory Items](#).

The internal ID for this record is lotnumberedinventoryitem.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The lot numbered inventory item record is scriptable in both client and server SuiteScript.

Supported Functions

The lot numbered inventory item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The lot numbered inventory item record includes the [Pricing Sublist / Pricing Matrix](#).

Markup Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use markup items to apply an additional charge to an order. Using markup items enables you to track markup amounts without affecting inventory valuation.

For example, you may want to charge a rush fee for completing a service or delivering an item quicker than is usually guaranteed. You can choose to markup the amount for this charge by a flat additional fee.

For help working with this record in the UI, see the help topic [Markup Items](#).

The internal ID for this record is `markupitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The markup record is scriptable in both client and server SuiteScript.

Supported Functions

The markup record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Non-Inventory Part

Applies to: SuiteScript 2.x | SuiteCloud Developer

Items that you always drop ship or other items that you sell or purchase but do not stock can be recorded and tracked as non-inventory items.

For help working with this record in the UI, see the help topic [Non-Inventory Items](#).

The internal ID for this record is `noninventoryitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The non-inventory record is scriptable in both client and server SuiteScript.

Supported Functions

The non-inventory part record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Other Charge Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Other charge items can be used to designate items or services you purchase or sell that do not fall into another type of item.

For help working with this record in the UI, see the help topic [Other Charge Items](#).

The internal ID for this record is `otherchargeitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Functions

The other charge item record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Payment Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can create payment items for types of payments that are made to invoices and should show separately. For example, you may want to create a payment item to specify a down payment amount.

For help working with this record in the UI, see the help topic [Payment Items](#).

The internal ID for this record is `paymentitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The payment record is scriptable in both client and server SuiteScript.

Supported Functions

The payment record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Planned Order

Applies to: SuiteScript 2.x | SuiteCloud Developer

Planned Orders are created by the supply planning process. You can also create a planned order from the supply planning workbench.

For help working with the planned order record in the UI, see the help topic [Creating or Editing a Planned Order](#).

The internal ID for this record is `plannedorder`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The planned order record is scriptable in server and client SuiteScript.

The following initialization parameters are required to create a planned order:

- `supplyplandefinition` – Supply plan definition ID.
- `supplyplanningrun` – Supply planning run internal ID.
- `firmed` – Should always be set to T. Manually created planned orders cannot have Firmed set to F.

Supported Functions

The planned order record can only be created, read, updated, deleted, and searched. It cannot be copied or transformed.

Usage Notes

To work with the planned order record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a planned order record using SuiteScript 2.x:

```

1  require(['N/record'], function(record) {
2      var plannedOrder = record.create({type: 'plannedorder',
3      isDynamic: true,
4      defaultValues: {
5          supplyplandefinition: PLAN_DEFINITION_ID,
6          supplyplanningrun: PLANNING_RUN_ID,
7          firmed: 'T'
8      }});
9      plannedOrder.setValue({fieldId: 'item',          value: ITEM_ID});
10     plannedOrder.setValue({fieldId: 'location',      value: LOCATION_ID});
11     plannedOrder.setValue({fieldId: 'transactiontype', value: 'TrnfrOrd'}); // can be either 'TrnfrOrd', 'WorkOrd' or 'PurchOrd'
12     plannedOrder.setValue({fieldId: 'startdate',     value: START_DATE});
13     plannedOrder.setValue({fieldId: 'enddate',       value: END_DATE});
14     plannedOrder.setValue({fieldId: 'quantity',      value: PLANNED_ORDER_QTY});
15     plannedOrder.setValue({fieldId: 'sourcelocation', value: SOURCE_LOCATION_ID});
16     plannedOrder.save();
17 });

```

Planning Item Category

Applies to: SuiteScript 2.x | SuiteCloud Developer

Planning item categories organize and display items in a hierarchy. Due to the volume of data involved in supply planning, this hierarchy helps you to more easily aggregate and inspect items.

For help working with the planned order record in the UI, see the help topic [Creating Planning Item Categories to an Item](#).

The internal ID for this record is `planningitemcategory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The planning item category record is scriptable in server and client SuiteScript.

Supported Functions

The planning item category record can only be created, read, updated, deleted, searched, and copied. It cannot be transformed.

Usage Notes

To work with the planning item category record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a planning item category record using SuiteScript 2.x:

```
1 | require(['N/record'], function(record) {
2 |     var planningItemCategory = record.create({type : 'planningitemcategory', isDynamic : true});
3 |     planningItemCategory.setValue({fieldId: 'name', value : 'PlanningItem Category'});
4 |     planningItemCategory.save();
5 | });
```

Planning Item Group

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

Planners want to specify which items are in a plan at the item category or item level. A planning item group provides a reusable (across different plans) method for specifying a set of items to be planned together.

For help working with the planning item group record in the UI, see [Planning Item Group](#).

The internal ID for this record is `planningitemgroup`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The planning item group record is scriptable in server and client SuiteScript.

Supported Functions

The planning item group record can only be created, read, updated, deleted, searched, and copied. It cannot be transformed.

Supported Sublist

Item sublist record ID – ITEM_FS_TAB

Usage Notes

To work with the planning item group record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a planning item group record using SuiteScript 2.x:

```
1 require(['N/record'], function(record) {
2   var planningItemGroup = record.create({type : 'planningitemgroup', isDynamic : true});
3   planningItemGroup.setValue({fieldId: 'name', value : 'Planning Item Group Name'});
4
5   planningItemGroup.selectNewLine({sublistId : 'planningitems'});
6   planningItemGroup.setCurrentSublistValue({sublistId : 'planningitems', fieldId : 'itemtype', value : 'ITEM', ignore
7     FieldChange : false});
8   planningItemGroup.setCurrentSublistValue({sublistId : 'planningitems', fieldId : 'itemcategory', value : ITEM_ID, ignore
9     FieldChange : false});
10  planningItemGroup.commitLine({sublistId : 'planningitems'});
11  planningItemGroup.save();
12 });
```

Planning Rule Group

Applies to: SuiteScript 2.x | SuiteCloud Developer

Planning rule groups enable planners to create rules that govern the supply planning process to define how items and item categories are replenished. In NetSuite 2020.2, you can only apply these rules to

transfer orders. For example, you can define where to source supply for an item when it is required at a specified location.

For help working with the planned order record in the UI, see [Planning Rule Group](#).

The internal ID for this record is `planningrulegroup`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The planning rule group record is scriptable in server and client SuiteScript.

Supported Functions

The planning rule group record can only be created, read, updated, deleted, searched, and copied. It cannot be transformed.

Supported Sublist

Rule sublist record ID – `RULE_FS_TAB`

Usage Notes

To work with the planning rule group record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a planning rule group record using SuiteScript 2.x:

```

1  require(['N/record'], function(record) {
2      var planningRuleGroup = record.create({type : 'planningrulegroup', isDynamic : true});
3      planningRuleGroup.setValue({fieldId : 'name', value : 'Planning Rule Group Name'});
4
5      planningRuleGroup.selectNewLine({sublistId : 'planningrules'});
6      planningRuleGroup.setCurrentSublistValue({sublistId : 'planningrules', fieldId : 'itemtype', value : 'ITEM',
7      ignoreFieldChange : false});
8      planningRuleGroup.setCurrentSublistValue({sublistId : 'planningrules', fieldId : 'itemcategory', value : ITEM_ID,
9      ignoreFieldChange : false});
10     planningRuleGroup.setCurrentSublistValue({sublistId : 'planningrules', fieldId : 'location', value : LOCATION_ID,
11     ignoreFieldChange : false});
12     planningRuleGroup.setCurrentSublistValue({sublistId : 'planningrules', fieldId : 'sourcelocation', value : SOURCE_LO
13     CATION_ID, ignoreFieldChange : false});
14     planningRuleGroup.setCurrentSublistValue({sublistId : 'planningrules', fieldId : 'leadtime', value : LEAD_TIME,
15     ignoreFieldChange : false});
16     planningRuleGroup.commitLine({sublistId : 'planningrules'});

```

```

12 |
13 |     planningRuleGroup.save();
14 | });

```

Planning View

i Applies to: SuiteScript 2.x | SuiteCloud Developer

NetSuite Supply Planning enables you to create a Planning Workbench View which can consist of pre-set filters and an embedded item saved search. The item saved search can include item-related criteria, such as item location and preferred supplier.

The internal ID for this subrecord is `planningview`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Prerequisites

The Material Requirements Planning feature must be enabled.

Supported Script Types

The planning view record is scriptable in both client and server SuiteScript.

Supported Functions

The planning view record can be created, read, edited, copied, deleted, searched, and transformed using SuiteScript.

Code Samples

The following SuiteScript 2.x sample shows how to create an inventory count record with an associated inventory detail record for a lot-numbered or serialized item.

```

1 | require(['N/record'], function(record) {
2 |     // Create an object to hold name data for the contact
3 |     const nameData = {
4 |         exception: 'PAST_DUE_SUPPLY',
5 |         supplyreplenishmenttype: 'MRP',
6 |         action: 'CANCEL',
7 |         demand : 'FORECAST_DEMAND',

```

```

8      supply : 'PLANNED_WORK_ORDER',
9      relativeenddays : '180'
10 };
11
12 // Create a planning view record
13 var objRecord = record.create({
14     type: 'planningview',
15     isDynamic: true
16 });
17
18 objRecord.setValue({
19     fieldId: 'name',
20     value: 'Planning View 2'
21 });
22
23 for (var key in nameData) {
24     if (nameData.hasOwnProperty(key)) {
25         objRecord.setValue({
26             fieldId: key,
27             value: nameData[key]
28         });
29     }
30 }
31 var recordId = objRecord.save();
32 });

```

Reallocate Items

i Applies to: SuiteScript 2.x | SuiteCloud Developer

NetSuite sometimes automatically allocates items from inventory to commit them to fill orders. Inventory may be automatically committed in the following cases:

- As each sales order is created or approved, your account automatically allocates inventory from the item's quantity available.
- Inventory is automatically committed to fill backorders when goods are received from your vendors.
- The inventory generated from assembly work orders is automatically allocated.

For help working with this record in the UI, see the help topic [Reallocating Items](#).

The internal ID for this record is `reallocateitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The `reallocate items` record is only exposed in user event scripts. You can run `beforeLoad`, `beforeSubmit`, and `afterSubmit` user event scripts on this record.

Supported Functions

The reallocate items record can only be copied and searched. It cannot be created, updated, or deleted using scripts.

SaaS Metric

Applies to: SuiteScript 2.x | SuiteCloud Developer

Use the saas metric record to track Software as a Service (SaaS) metrics reporting.

The saas metric record is available only when the SaaS Metrics feature is enabled at Setup > Company > Enable Features (Administrator), on the Analytics subtab. With this feature enabled, you can open the item you want to collect data about, set a start date, and start tracking SaaS metrics for transactions and subscriptions that contain that item.

If you need historical reporting of SaaS metrics for transactions that already include these items, you can set the start date in the past. Adding an end date helps to prevent errors in reporting and gives you greater control in defining what's included in the report.

The internal ID for this record is saasmetric.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The saas metric record is not supported in server or client SuiteScript.

Supported Functions

The saas metric record is partially scriptable – it can be read and searched using SuiteScript.

Serialized Assembly Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Serialized assembly items enable you to build items from raw materials and track the inventory of both finished items and the raw materials separately. The completed assembly is assigned a serial number to track it as it enters and leaves your inventory. Serialized assembly items are available on sales transactions and inventory adjustment transactions. They are not available on purchase transactions.

For help working with this record in the UI, see the help topic [Serial Numbered Items](#). For help about using assembly items in the UI, see the help topic [Assembly Items](#).

The internal ID for this record is `serializedassemblyitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The serialized assembly item record is scriptable in both client and server SuiteScript.

Supported Functions

The serialized assembly record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The serialized assembly item record includes the [Pricing Sublist / Pricing Matrix](#).

Serialized Inventory Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Serialized inventory item records are used to track information about items which you maintain a stock of. Note that you must first enable serialized inventory items in your NetSuite account before you can access this record type. To enable serialized inventory items, go to Setup > Company > Enable Features. On the Items & Inventory subtab, under Inventory, select the Serialized Inventory box.

For help working with this record in the UI, see the help topic [Serial Numbered Items](#). For help about using inventory items in the UI, see the help topic [Inventory Items](#).

The internal ID for this record is `serializedinventoryitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The serialized inventory item record is scriptable in both client and server SuiteScript.

Supported Functions

The serialized inventory record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The serialized inventory item record includes the [Pricing Sublist / Pricing Matrix](#).

Service

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A service item is an item you create to track time and record billable hours.

For help working with this record in the UI, see the help topic [Service Items](#).

The internal ID for this record is `serviceitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The service record is scriptable in both client and server SuiteScript.

Supported Functions

The service record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Shipping Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

A shipping item is a delivery method for a shipping carrier. It describes how to ship an item and can include shipping rate information, handling rates, and rules for shipping and handling. It can also specify when shipping is free.

For help working with this record in the UI, see the help topic [Shipping Items](#).

The internal ID for this record is `shipitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The shipping item record is supported in all server scripts. Client scripting is not supported.

Supported Functions

The shipping item record can only be updated, deleted, and searched. It cannot be created or copied. You can create this record in the UI, however.

Note: Create and copy are not supported. You can create this record only in the UI.

Usage Notes

The following shipping item record components are not scriptable:

- Handling Table
- Shipping Table
- Free Shipping Items Tab

Subscription Plan

Applies to: SuiteScript 2.x | SuiteCloud Developer

Use the subscription plan record to build information for subscriptions.

The subscription plan record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab. With this feature enabled, you can access a list of subscription plan records in the UI by going to Lists > Subscriptions > Subscription Plans.

For help working with this record in the UI, see the help topic [Creating Subscription Plans](#).

The internal ID for this record is subscriptionplan.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subscription plan record is scriptable in server and client SuiteScript.

Supported Functions

The subscription plan record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Subscription Term

Applies to: SuiteScript 2.x | SuiteCloud Developer

Use the subscription term record on the subscription plan and subscription record to define the length of a subscription.

After a subscription term has been created in an account, that term becomes available for use on any future subscription plan or subscription created.

The subscription term record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab.

For help working with this record in the UI, see the help topic [Renewing a Subscription](#).

The internal ID for this record is subscriptionterm.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subscription term record is scriptable in server and client SuiteScript.

Supported Functions

The subscription term record can only be created, read, updated, deleted, searched, and copied. It cannot be transformed.

Code Sample

The following sample shows how to set a six-month subscription term.

```

1  require(['N/record'], function(record) {
2      var subscriptionTerm = record.create({
3          type: "subscriptionTerm",
4          isDynamic: true
5      });
6      subscriptionTerm.setValue({
7          fieldId: "name",
8          value: "TestSubscriptionTerm"
9      });
10     subscriptionTerm.setValue({
11         fieldId: "subscriptiontermduration",
12         value: 6
13     });
14     subscriptionTerm.setValue({
15         fieldId: "subscriptiontermunit",
16         value: "MONTHS"
17     });
18     log.debug(subscriptionTerm.save());
19 });

```

Subtotal Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Subtotal item records are used to separate groups or individual line items if you want to offer a discount or tax on certain line items but not others.

For help working with this record in the UI, see the help topic [Subtotal Items](#).

The internal ID for this record is `subtotalitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subtotal record is scriptable in both client and server SuiteScript.

Supported Functions

The subtotal record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Supply Chain Snapshot Simulation

Applies to: SuiteScript 2.x | SuiteCloud Developer

The supply chain snapshot simulation record is used to provide a simulation of a transaction to determine what the effect that transaction would have on your inventory.

The supply chain snapshot simulation record is only available when the Supply Chain Control Tower feature is enabled at Setup > Company > Enable Features, on the Items & Inventory subtab. When the feature is enabled, you can access a list of supply chain snapshot simulation records in the UI by going to Lists > Supply Chain > Supply Chain Snapshot Simulations.

For help working with the supply chain snapshot record in the UI, see the help topic [Supply Chain Control Tower](#).

The internal ID for this record is supplychainsnapshotsimulation.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The supply chain snapshot simulation record is scriptable in server and client SuiteScript.

Supported Functions

The supply chain snapshot simulation record can be created, updated, deleted, and searched using SuiteScript.

Usage Notes

Item type is not scriptable.

The system notes are search only.

Code Samples

The following sample shows how to create a supply chain snapshot simulation.

```

1  function createSimulation(location, item, quantity, date) {
2      var simulationRecord = nlapiCreateRecord('supplychainsnapshotsimulation', {recordmode: 'dynamic'});
3      simulationRecord.setFieldValue('memo', 'Simulation for item id=' + item);
4      simulationRecord.setFieldValue('transactiontype', 'PurchOrd');
5      simulationRecord.setFieldValue('quantity', quantity);
6      simulationRecord.setFieldValue('status', '5');
7      simulationRecord.setFieldValue('itemnamenum', item);
8      simulationRecord.setFieldValue('supplylocationdate', date);
9      simulationRecord.setFieldValue('supplylocationsubidiary', 1);
10     simulationRecord.setFieldValue('supplylocation', location);
11     var simulationKey = nlapiSubmitRecord(simulationRecord);
12     return simulationKey;
13 }

```

Supply Change Order

Applies to: SuiteScript 2.x | SuiteCloud Developer

Supply change orders are created automatically in the supply planning workbench when you react to an Action message.

For help working with the supply change order record in the UI, see the help topic [Implementing and Approving a Change Order](#).

The internal ID for this record is supplychangeorder.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Supply Change Order Actions](#) for the actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The supply change order record is scriptable in server and client SuiteScript.

Supported Functions

The supply change order record can only be read, updated, deleted, and searched. It cannot be created, copied, or transformed.

Usage Notes

To work with the supply plan order record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a supply change order record using SuiteScript 2.x:

```
1 require(['N/record'], function(record) {
2   var supplyChangeOrderObj = record.load({type : 'supplychangeorder', id: SUPPLY_CHANGE_ORDER_ID, isDynamic : true});
3   supplyChangeOrderObj.setValue({fieldId: 'memo', value : 'Change Order Memo'});
4   supplyChangeOrderObj.setValue({fieldId: 'newdate', value : NEW_DATE});
5   supplyChangeOrderObj.save();
6 });
```

Supply Change Order Actions

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The supply change order record supports the approve and reject actions.

For help working with this record in the UI, see the help topic [Change Orders](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

approve

Corresponding UI Button	Approve all pending.
Action Description	Approve the Supply Change Order with the given recordId.
Governance	This action performs a simple edit, update, and save record. An approve action updates a tranline, which is a necessary step for the feature.

Exception	Exceptions are thrown when tranline fails to update a tranline fails.
Since	2020.2

reject

Corresponding UI Button	Reject all pending. Reject all pending and add a note.
Action Description	Reject the Supply Change Order with the given recordId. Optionally include a note that explains the reason for the rejection.
Governance	This action performs a simple edit, update, and save record.
Exception	Exceptions are thrown when tranline fails to update a tranline fails.
Since	2020.2

Parameters

The recordId is the default input parameter for this action

Syntax

The following code snippet shows the syntax for this action.

```

1  require(['N/action'], function(actionMod) {
2
3      // client example
4      actionMod.getActions.promise({recordType: 'supplychangeorder'}).then(function(actions) {
5          var approveAction = actions.approve;
6          approveAction.promise({recordId: 1}).then(function(result) {
7              });
8          });
9
10     // server example
11     var result = actionMod.executeAction({id: 'approve', recordType: 'supplychangeorder', params: {recordId: 1}});
12 });

```

Supply Plan Definition

Applies to: SuiteScript 2.x | SuiteCloud Developer

Supply Plan Definition enables planners to establish the criteria to be used for supply planning. The plan definition provides the information that drives the supply planning process, such as planning horizon and planning scope. For example: you can run supply planning at multiple facilities at one time. A supply plan definition provides a permanent holding place for your planning criteria and planning results.

For help working with the supply change order record in the UI, see the help topic [Creating a Supply Planning Definition and Schedule](#).

The internal ID for this record is supplyplandefinition.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Supply Plan Definition Record Action](#) for the actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The supply plan definition record is scriptable in server and client SuiteScript.

Supported Functions

The supply plan definition record can only be created, read, updated, deleted, searched, and copied. It cannot be transformed.

Supported Sublist

Scope sublist record ID – SCOPE_FS_TAB

Usage Notes

To work with the supply plan definition record, you must enable material requirements planning. To do this, go to Setup > Company > Enable Features. Check the Material Requirements Planning box under Items & Inventory.

Code Samples

The following sample shows how to create a supply change order record using SuiteScript 2.x:

```

1  require(['N/record'], function(record) {
2      var planDefinition = record.create({type : 'supplyplandefinition', isDynamic : true});
3      planDefinition.setValue({fieldId: 'name', value : 'Plan Definition Name'});
4      planDefinition.setValue({fieldId: 'horizonindays', value : HORIZON_IN_DAYS});
5      planDefinition.setValue({fieldId: 'rulegroup', value : PLANNING_RULE_GROUP_ID});
6      planDefinition.setValue({fieldId: 'itemgroup', value : PLANNING_ITEM_GROUP_ID});
7
8      planDefinition.selectNewLine({sublistId: 'scope'});
9      planDefinition.setCurrentSublistValue({sublistId: 'scope', fieldId: 'location', value : LOCATION_ID, ignoreField
Change : false});
10     planDefinition.setCurrentSublistValue({sublistId: 'scope', fieldId: 'demandsourcetype', value : 'SALESORDER', ignoreField
Change : false});
11     planDefinition.commitLine({sublistId: 'scope'});
12
13     planDefinition.selectNewLine({sublistId: 'scope'});
14     planDefinition.setCurrentSublistValue({sublistId: 'scope', fieldId: 'location', value : LOCATION_ID, ignoreField
Change : false});
15     planDefinition.setCurrentSublistValue({sublistId: 'scope', fieldId: 'demandsourcetype', value : 'DEMANDPLAN', ignoreField
Change : false});
16     planDefinition.commitLine({sublistId: 'scope'});
17
18     planDefinition.save();
19 });

```

Supply Plan Definition Record Action

Applies to: SuiteScript 2.x | SuiteCloud Developer

The supply plan definition record type only supports launch actions.

For help working with this record in the UI, see the help topic [Creating a Supply Planning Definition and Schedule](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

launch

Corresponding UI Button	Launch
Action Description	Click launch to run the supply planning process based on the criteria which will then generate planned orders based on the demand for the items within the planning horizon.
Returns	The record owner can populate zero to many notifications and zero to many response <key, value> pairs.
Exception	After the page is launched, the bulk processing page opens to display any errors.
Since	2020.2

Parameters

The recordId is the default input parameter for the action.

Communications

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following communication records are scriptable in SuiteScript:

- [Message](#)
- [Note](#)

Message

Applies to: SuiteScript 2.x | SuiteCloud Developer

A message is used to record correspondence you have with a specific business. Use the message record to add an email message to an existing customer, contact, or opportunity record. After an email message

has been added to a record, any related emails are automatically attached to the same record and to any recipients of the original email.

To work with the message record, sublist of recipients, and to send email, use the [N/email Module](#).

For help working with this record in the user interface, see the help topic [Sending Email from Records](#).

The internal ID for this record is message.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The message record is scriptable in server SuiteScript only.

Supported Functions

For information about sending email from a record, see the help topic [Sending Email from Records](#).

Note that there are also other types of messages, but those are not supported. In the message that you are creating, you can set all supported fields for email messages before saving. After you save the new message, there are restrictions applied to it. You cannot edit the existing message. It is possible to delete an existing message, if the permissions for the user are set to allow this type of operation. However, you should not delete an existing message because it can cause email communication threads to be displayed incorrectly.

Usage Notes

- Message records can be edited only during the create operation. After they are created and submitted, existing message records cannot be edited.
- Existing message records can be copied and deleted.
- Only beforeLoad and afterSubmit user event scripts will execute on the Message record type when a message is created by an inbound email case capture. Scripts set to execute on a beforeSubmit event will not execute.

For example, if you have a test script like the following deployed to the Message record type:

```

1 function beforeLoad(type, name) {
2     log.debug ({
3         title: 'Before Load',
4         details: 'In beforeLoad function'
5     });
6 }
7 function beforeSubmit(type, name) {

```



```

8      log.debug ({
9          title: 'Before Submit',
10         details: 'In beforeSubmit function'
11     });
12 }
13 function afterSubmit(type, name) {
14     log.debug ({
15         title: 'After Submit',
16         details: 'In afterSubmit function'
17     });
18 }

```

only the beforeLoad(...) and afterSubmit(...) functions will execute if the message was created to respond to an inbound emailed case.

- When creating a new message, you can use the otherrecipientslist sublist to add an email address to the cc or bcc fields. For an example, see [Code Sample](#).
- An entity that is defined as the author or recipient for a newly created message must have an associated email in NetSuite. If the NetSuite record for an entity defined as the author of a message does not include an email, the authoremail field must be defined on the message record. If the NetSuite record for an entity defined as the recipient of a message does not include an email, the recipientemail field must be defined on the message record.

Code Sample

The following SuiteScript 2.x snippet shows how to add an email address to the cc line of a new message, when working in standard (deferredDynamic) mode.

Note: Some of the values in this sample are placeholders. Before using this sample, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record'], function(record) {
6      function makeMessage(){
7          var messageRec = record.create ({
8              type: record.Type.MESSAGE,
9              isDynamic: true
10         });
11
12         messageRec.setValue({
13             fieldId: 'subject',
14             value: 'Test Message'
15         });
16
17         messageRec.setValue({
18             fieldId: 'author',
19             value: 15 //internal id of author name
20         });
21
22         messageRec.setValue({
23             fieldId: 'authoremail',
24             value: 'name@example.com'
25         });
26
27         messageRec.setValue({
28             fieldId: 'recipient',
29             value: 164 //internal ID of recipient
30         });
31
32         messageRec.setValue({

```

```

33     fieldId: 'recipientemail',
34     value: 'name@example.com'
35   });
36
37   messageRec.setValue({
38     fieldId: 'cc',
39     value: 'name@example1.com, name@example2.com'
40   });
41
42   messageRec.setValue({
43     fieldId: 'bcc',
44     value: 'name@example3.com'
45   });
46
47   messageRec.setValue({
48     fieldId: 'message',
49     value: 'test message'
50   });
51
52   var custMessage = messageRec.save();
53
54   log.debug({title: 'message', details: custMessage
55   });
56   }
57   makeMessage();
58   });
59   ...

```

Note

Applies to: SuiteScript 2.x | SuiteCloud Developer

Notes are used to attach information to another record. Use the notes record to create new notes and attach them to a specific record.

The internal ID for this record is note.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The note record is scriptable in server SuiteScript only.

Supported Functions

The note record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Transactions

i Applies to: SuiteScript 2.x | SuiteCloud Developer

i Note: Processed lines for this transaction type contribute to the Monthly Transaction Lines metric that counts toward maximum limits for your NetSuite service tier. For more information, see [Transaction Types Included in Monthly Transaction Lines Metric](#).

A macro can be used for taxable transactions. For more information, see [Transaction Record Macros](#).

The following transaction records are scriptable in SuiteScript.

- [Assembly Build](#)
- [Assembly Unbuild](#)
- [Bin Transfer](#)
- [Blanket Purchase Order](#)
- [Cash Refund](#)
- [Cash Sale](#)
- [Charge](#)
- [Check](#)
- [Credit Memo](#)
- [Customer Deposit](#)
- [Customer Refund](#)
- [Deposit](#)
- [Deposit Application](#)
- [Employee Expense Source Type](#)
- [Estimate](#)
- [Expense Report](#)
- [Expense Report Policy](#)
- [Fulfillment Request](#)
- [Imported Employee Expense](#)
- [Inventory Adjustment](#)
- [Inventory Cost Revaluation](#)
- [Inventory Count](#)
- [Inventory Status Change](#)
- [Inventory Transfer](#)
- [Invoice](#)
- [Invoice Group](#)
- [Item Fulfillment](#)
- [Item Receipt](#)
- [Opportunity](#)
- [Order Reservation](#)
- [Paycheck](#)

- Paycheck Journal
- Period End Journal
- Project Intercompany Cross Charge Request
- Purchase Contract
- Purchase Order
- Requisition
- Resource Group
- Return Authorization
- Revenue Arrangement
- Revenue Commitment
- Revenue Commitment Reversal
- Sales Channel
- Sales Order
- Store Pickup Fulfillment
- Transfer Order
- Vendor Prepayment
- Vendor Prepayment Application
- Vendor Return Authorization
- Wave
- Work Order
- Work Order Close
- Work Order Completion
- Work Order Issue

Transaction Record Macros

The [calculateTax](#) and [getSummaryTaxTotals](#) macros can only be used for the following taxable transactions, if the SuiteTax feature is enabled. If a macro is executed on a non-taxable transaction or the SuiteTax feature is not enabled, an error occurs.

- Cash Refund
- Cash Sale
- Credit Card Charge
- Credit Memo
- Estimate
- Expense Report
- Invoice
- Opportunity
- Purchase Order
- Return Authorization
- Sales Order
- Vendor Bill

- [Vendor Credits](#)
- [Vendor Return Authorization](#)

calculateTax

Macro Description	Calculates taxes on a transaction, same as when a user clicks the Preview Tax button in the UI. Taxes are always calculated when the transaction is saved.
Returns	void
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2016.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro.

Errors

Error Code	Thrown If
SSS_UNSUPPORTED_METHOD	The SuiteTax feature is disabled, or the transaction type of the current record is non-taxable.
SSS_TAX_REGISTRATION_REQUIRED	The subsidiary of the current record does not have a valid tax registration.

Transaction calculateTax Macro Syntax

```

1 | require(['N/currentRecord'],function(currentRecord){
2 |
3 |     currentRecord.executeMacro({id:'calculateTax'});
4 |
5 | });

```

For an example of how to call the calculateTax macro on a sales order record, see the help topic [Call a Macro on a Sales Order Record](#).

getSummaryTaxTotals

Macro Description	Checks whether tax is calculated, iterates through tax details, and gathers all tax type IDs
Returns	An object that contains notification and response, which contains the taxTotals array
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2019.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro.

Errors

Error Code	Thrown If
SSS_UNSUPPORTED_METHOD	The SuiteTax feature is disabled, or the transaction type of the current record is non-taxable.
SSS_TAX_REGISTRATION_REQUIRED	The subsidiary of the current record does not have a valid tax registration.

Transaction getSummaryTaxTotals Macro Syntax

```

1 | require(['N/currentRecord'], function(currentRecord){
2 |
3 |     var taxTotals = currentRecord.executeMacro({id: 'getSummaryTaxTotals'});
4 |
5 | });

```

Sample Return Objects

```

1 | {
2 |
3 |     taxTotals: [
4 |
5 |         {
6 |
7 |             taxTypeId: 123,
8 |
9 |             taxTypeName: 'VAT Standard',
10 |
11 |             taxTotal: '833.33'
12 |         },
13 |
14 |         {
15 |
16 |             taxTypeId: 124,
17 |
18 |             taxTypeName: 'VAT Reduced',
19 |
20 |             taxTotal: '1.99'
21 |         }
22 |     ]
23 |
24 | }
25 |
26 |
27 |

```

Assembly Build

Applies to: SuiteScript 2.x | SuiteCloud Developer

An assembly item is an inventory item made of several components, but identified as a single item. This type of item lets you define the members of an assembly and to separately track both the component

items and the assembled items in inventory. For each assembly build, the assembly item stock level increases and the member items' individual stock levels decrease.

An assembly build transaction records the physical manufacture of an assembly item from component items and the related inventory level changes.

For details about this type of transaction, see the help topic [Building Assembly Items](#). For details about this type of item, see the help topic [Assembly Items](#).

The internal ID for this record is assemblybuild.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Also note that this record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The assembly build record is scriptable in server SuiteScript only.

Supported Functions

The assembly build entry record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Assembly Unbuild

Applies to: SuiteScript 2.x | SuiteCloud Developer

An assembly item is an inventory item made of several components, but identified as a single item. This type of item lets you define the members of an assembly and to separately track both the component items and the assembled items in inventory.

An assembly unbuild transaction records the physical taking apart of an assembly item back into its component items and the related inventory level changes. For each assembly unbuild, the assembly item stock level decreases and the member items' individual stock levels increase.

For details about this type of transaction, see the help topic [Building Assembly Items](#). For details about this type of item, see the help topic [Assembly Items](#).

For help working with this record in the UI, see the help topic [Unbuild Assembly Items](#).

The internal ID for this record is assemblyunbuild.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The assembly unbuild record is scriptable in server SuiteScript only.

Supported Functions

The assembly unbuild entry record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Bin Transfer

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

You can record a bin transfer to move items between bins within a warehouse.

For example, at your East Coast location, you stock item #AB123 in two bins: bin #3003 and #4004. There are currently 100 of the item in each bin. You can enter a bin transfer to record the transfer of 50 items out of bin #3003 and into bin #4004.

Recording a bin transfer does not post to your chart of accounts and has no financial impact. The transfer only updates the quantity on hand in each bin for the items transferred.

On the bin transfer record, identify the item, the bin the item is in, the bin the item will move into and the quantity to be moved.

For help working with this record in the UI, see the help topic [Bin Transfers](#).

The internal ID for this record is bintransfer.

The bin transfer record contains a subrecord: [Inventory Detail](#). For details about subrecords in SuiteScript, see the help topic [SuiteScript 2.x Scripting Subrecords](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The bin transfer is scriptable in server SuiteScript only.

Supported Functions

The bin transfer record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Blanket Purchase Order

i Applies to: SuiteScript 2.x | SuiteCloud Developer

With this record, you can take advantage of fixed pricing for a preset number of items that you will buy during a specific time period. This approach lets you avoid sporadic pricing negotiations with vendors.

This record is available only when the Blanket Purchase Order feature is enabled at Setup > Company > Enable Features, on the Transactions subtab.

In the UI, you access this record at Transactions > Purchases > Enter Blanket Purchase Order. For help working with this record in the UI, see the help topic [Creating a Blanket Purchase Order](#).

The internal ID of this record is `blanketpurchaseorder`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The blanket purchase order record is scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The blanket purchase order record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Cash Refund

Applies to: SuiteScript 2.x | SuiteCloud Developer

A cash refund transaction records the return of money to a customer who immediately paid for goods or services using cash, a check or a credit card.

For help working with this record in the UI, see the help topic [Refunding a Cash Sale](#).

The internal ID for this record is cashrefund.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The cash refund record is scriptable in both client and server SuiteScript.

Supported Functions

The cash refund record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

See the following sections for more details on working with this record:

- [Fields](#)
- [Email Operations](#)

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		

Field Internal ID	Field UI Label	Note
ccnumber	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a beforeLoad user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email Button** is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Cash Sale

Applies to: SuiteScript 2.x | SuiteCloud Developer

A cash sale is a transaction that records the sale of goods or services for which you receive immediate payment. Enter a cash sale when payment for goods or services has been received at the time of delivery.

Record cash sales by adding each item sold from your Items list. Then, cash sale line-items specify the goods and services sold and their sales amounts. The sum of all sales amounts plus any applicable tax equals the total amount paid for this sale.

For help working with this record in the UI, see the help topic [Cash Sales](#).

The internal ID for this record is `cashsale`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The cash sale record is scriptable in both client and server SuiteScript.

Supported Functions

The cash sale record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

See the following sections for more details on working with this record:

- [Fields](#)
- [Email Operations](#)

The following cash sale record sublists are **list** sublists:

- Billable Expenses
- Billable Items
- Billable Time

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Shipping sublist's Shipping Tax Code field, internal ID `shippingtaxcode`, appears only if per-line taxes have been set on the Item sublist.

Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
<code>ccnumber</code>	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in <code>beforeSubmit</code> user event

Field Internal ID	Field UI Label	Note
		scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
isrecurringpayment	Recurring Payment	A value for this field is stored only if the value for paymentmethod is a credit card.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Also note that this record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a beforeLoad user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email Button** is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Check

Applies to: SuiteScript 2.x | SuiteCloud Developer

A check transaction creates and records a check used to pay an expense, records an expense paid in cash and not entered as a bill, or records a non-check debit transaction, such as a debit card transaction, ATM (automated teller machine) transaction, or EFT (electronic funds transfer) payment. A check transaction records an expense directly to your books by debiting the expense account specified in the transaction detail and crediting the bank account the check selected for the check.

For help working with this record in the UI, see the help topic [Checking](#).

The internal ID for this record is check.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The check record is scriptable in both client and server SuiteScript.

Supported Functions

The check record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following sections provide usage notes for specific fields and sublists.

Using Landed Cost Fields

When you create a landed cost category, the associated field IDs for the first category are landedcostamount1 and landedcostsource1. If you create a second category, the IDs will be landedcostamount2 and landedcostsource2.

This pattern increments by one with each additional category. For example, the IDs for the next landed cost category will be landedcostamount3 and landedcostsource3, and so on.

Using the Payee Address Sublist

Payee address data is stored in a subrecord instead of a text field. On the Check form in the UI, the address information is now displayed in a sublist instead of a single field. SuiteScript includes specialized APIs that you must use to script with subrecord data. You should use these subrecord APIs to script with payee address data.

For details about scripting subrecords with SuiteScript 2.x, see:

- [SuiteScript 2.x Scripting Subrecords](#)
- [Record Object Members](#)
- [CurrentRecord Object Members](#)

■ [About the Address Subrecord](#)



Warning: This change may impact existing scripts that reference check payee address data. You should review these scripts to determine whether updates are needed.

Using Profit, Cost, and Amount Fields

The following table provides usage notes for fields on this record related to estimated gross profit and profit percent, estimated extended cost, and amount.

Field UI Label	Note
Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.
Amount	This field is not available using search or lookup for any transactions.

Credit Memo

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A credit memo transaction decreases the amount a customer owes you. This type of transaction can be used to reverse a charge billed to a customer. If a customer receives a credit memo after they have paid an invoice, this memo can be applied to any of the customer's open or future invoices.

For help working with this record in the UI, see the help topic [Customer Credit Memos](#).

The internal ID for this record is `creditmemo`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The credit memo record is scriptable in both client and server SuiteScript.

Supported Functions

The credit card refund record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

NetSuite enables a user to email a copy of a transaction to a customer or another recipient. There are several ways to email a transaction. One way is to open the transaction for viewing and select **Actions > Email**. Additional approaches for emailing transactions are described in [Emailing Transactions](#).

If you deploy a `beforeLoad` user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction record. However, if the companywide preference **Use Popup for Main Transaction Email Button** is selected, a `beforeLoad` script will not execute when a user selects **Activity > Email**. In this case, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this form are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a `beforeLoad` script on emails sent using the popup, deploy the script on the [Message](#) record type.

Note that the **Use Popup for Main Transaction Email Button** preference does not affect other methods of emailing a transaction. For example, a user may email a transaction by using its **Communications** subtab or by using the **Save and Email** button. In these cases, a `beforeLoad` user event script deployed on the transaction still executes, regardless of how the preference is configured. For more information about the **Use Popup for Main Transaction Email Button** preference, see the help topic [Setting Email Preferences](#).

The Multi-Partner Management feature must be enabled in your account for the **Partners** sublist to appear.

Fields

estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Customer Deposit

Applies to: SuiteScript 2.x | SuiteCloud Developer

A customer deposit transaction records the funds received when a customer makes an advance payment for an order. This payment is recorded in the general ledger as a liability until the goods or services are delivered, and does not affect the customer's accounts receivable balance. After the order is filled, the deposit is applied against the invoice.

For help working with this record in the UI, see the help topic [Customer Deposits](#).

The internal ID for this record is `customerdeposit`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer deposit record is scriptable in both client and server SuiteScript.

Supported Functions

The customer deposit record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
<code>ccnumber</code>	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in <code>beforeSubmit</code> user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
<code>isrecurringpayment</code>	Recurring Payment	A value for this field is stored only if the value for <code>paymentmethod</code> is a credit card.

Customer Refund

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `customerrefund`.

For help working with this record in the UI, see the help topic [Customer Refunds](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer refund record is scriptable in server SuiteScript only.

Supported Functions

The customer deposit record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
ccnumber	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

The Deposits sublist is a **list** sublist.

Using the Payee Address Sublist

Payee address data is stored in a subrecord instead of a text field. On the Customer Refund form in the UI, address information is now displayed in a sublist instead of a single field. SuiteScript includes specialized APIs that you must use to script with subrecord data. You should use these subrecord APIs to script with payee address data.

For details about scripting subrecords with SuiteScript 2.x, see:

- [SuiteScript 2.x Scripting Subrecords](#)
- [Record Object Members](#)
- [CurrentRecord Object Members](#)
- [About the Address Subrecord](#)

Warning: This change may impact existing scripts that reference customer refund payee address data. You should review these scripts to determine whether updates are needed.

Deposit

Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the deposit record to adjust the balance of an account.

In the UI, you access this record at Transactions > Bank > Make Deposits.

For help working with this record in the UI, see the help topic [Deposits](#).

The internal ID of this record is `deposit`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The deposit record is scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The deposit record cannot be copied, but otherwise it is fully scriptable. It can be created, updated, deleted, and searched in SuiteScript.

Usage Notes

Be aware of the following:

- To successfully add a new deposit record, you must include at least one line in one of the following three sublists:
 - Payments
 - Other Deposits
 - Cash Back
- If you use the Multi-Book Accounting feature, be aware that the Accounting Book Detail sublist is read-only and not scriptable.
- In some cases, you must specify the `lineId` when depositing payments to ensure the correct `lineId` is used and avoid the `RCRD_HAS_BEEN_CHANGED` error. In particular, journal entries can have a `lineId` that is not 0. If a journal entry is loaded in the line, you are adding `paymentId`. Therefore you must explicitly set the `lineId`.

Code Samples

The following sample shows how to create, load, and delete deposit records.

```

1  function createBankDeposit(cashSaleId){
2
3      var deposit = record.create({
4
5          type: record.Type.DEPOSIT
6
7      });
8
9      deposit.setValue({
10
11          fieldId: 'account',
12
13          value: '1'
14
15      });
16
17      deposit.setValue({
18
19          fieldId: 'subsidiary',
20
21          value: '1'
22
23      });
24
25      deposit.setSublistValue({
26
27          sublistId: 'payment',
28
29          fieldId: 'id',
30
31          line: 1,
32
33          value: cashSaleId // replace this with a valid id
34
35      });
36
37      deposit.setSublistValue({
38
39          sublistId: 'payment',
40
41          fieldId: 'deposit',
42
43          line: 1,
44
45          value: 'T'
46
47      });
48
49      var recordId = deposit.save();
50
51      return recId;
52
53  }
54
55
56
57  function testDeposit() {
58
59      var cashSaleId = null;
60
61      var depositId = null;
62
63      try{
64
65          cashSaleId = createCashSale(); // or type internalId of valid Cash Sale
66
67          depositId = createBankDeposit(cashSaleId);
68

```

```

69     var testdeposit = record.load({
70
71         type: record.Type.DEPOSIT,
72
73         id: depositId
74
75     });
76
77 }
78
79 finally {
80
81     if (depositId != null) {
82
83         record.delete({
84
85             type: record.Type.DEPOSIT,
86
87             id: depositId
88
89         })
90
91     }
92
93     if (cashSaleId != null) {
94
95         record.delete({
96
97             type: record.Type.CASH_SALE,
98
99             id: cashSaleId
100
101         })
102
103     }
104
105 }
106
107 }

```

Disabling or Enabling User Filters

When you create or load a deposit record, you can control whether the script respects the filter preferences set by the user on the record's Deposits > Payments subtab. You control this behavior by using the `disablepaymentfilters` initialization flag.

Set the parameter to true to ignore the preferences and disable filters, so that all the payments are loaded.

Set the parameter to false to indicate that the system should respect the user's preferences and enable the filters. In this case, filters are applied and fewer payments are loaded. If you have a large number of payments, you can set `disablepaymentfilters` to false to improve performance. If you do not set a value for the `disablepaymentfilters` parameter, it defaults to false.

Disabling Filters in SuiteScript 2.x

In SuiteScript 2.x, you can interact with `disablepaymentfilters` when you use the `defaultValues` parameter, which is available for the [record.create\(options\)](#) and [record.load\(options\)](#).

For example, the following SuiteScript 2.x snippet would ignore the user's preferences:

```

1  var newRecord = record.create({
2
3      type: record.Type.DEPOSIT,
4
5      defaultValues: {

```

```

6
7     account: '2',
8
9     disablepaymentfilters: true
10
11 }
12
13 });

```

Specifying Payment IDs

When you create a deposit record, you can choose to specify payment IDs. You control this behavior by using the `deposits` initialization parameter.

Specifying Payment IDs in SuiteScript 2.x

To specify payment IDs in SuiteScript 2.x, use the `deposits` initialization parameter, which is available for `record.create(options)`. The `deposits` parameter is not available for `record.load(options)`.

For example, the following SuiteScript 2.x snippet would select the payment IDs of 818 and 819:

```

1 var myDeposit = record.create({
2
3   type: record.Type.DEPOSIT,
4
5   defaultValues: {
6
7     'deposits': '818,819'
8
9   }
10
11 });

```

Deposit Application

Applies to: SuiteScript 2.x | SuiteCloud Developer

A deposit application transaction applies a customer deposit against an invoice after the order is complete.

For help working with this record in the UI, see the help topic [Applying a Customer Deposit](#).

The internal ID for this record is `depositapplication`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The deposit application record is scriptable in both client and server SuiteScript.

Supported Functions

The deposit application record cannot be created, but otherwise it is fully scriptable. It can be copied, updated, deleted, and searched in SuiteScript.

Usage Notes

You do not use `record.create(options)` to create a deposit application record. Deposit applications are always created when a customer deposit is applied to an invoice. The application can only be created by applying an open customer Deposit from the deposit sublist of the customer payment. On submit, the backend creates a deposit application in the amount applied.

You can use the **doc** field on the apply sublist of the customer payment to get the internal ID of the deposit or invoice.

Employee Expense Source Type

i Applies to: SuiteScript 2.x | SuiteCloud Developer

This transaction is available when the Expense Report feature is enabled at Setup > Company > Enable Features, on the Employees subtab.

For help working with this record in the UI, see the help topic [Expense Reporting](#).

The internal ID for this record is `employeeexpensesourcetype`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The employee expense source type record is scriptable in both client and server SuiteScript.

Supported Functions

The employee expense source type record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Sample Codes

```

1 require(['N/record'], function(record) {
2
3   var srcTypeObj= record.create({type: 'employeeexpensesourcetype', isDynamic: true});
4
5   srcTypeObj.setValue({fieldId: 'name', value: 'My Source Type'});
6
7   srcTypeObj.setValue({fieldId: 'description', value: 'My Description'});
8

```

```
9 | var sourceTypeId = srcTypeObj.save();
10 |
11 | });
```

Estimate

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is estimate.

For help working with this record in the UI, see the help topics [Estimates](#) and [Quotes for Commerce Websites](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The estimate / quote record is scriptable in both client and server SuiteScript.

Supported Functions

The estimate / quote record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

See the following sections for more details on working with this record:

- [Fields](#)
- [Email Operations](#)

Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Field Internal ID	Field UI Label	Note
		When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Also note that this record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Shipping sublist's Shipping Tax Code field, internal ID `shippingtaxcode`, appears only if per-line taxes have been set on the Item sublist.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a `beforeLoad` user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email Button** is enabled, a `beforeLoad` script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a `beforeLoad` script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a `beforeLoad` user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Expense Report

Applies to: SuiteScript 2.x | SuiteCloud Developer

An expense report transaction records an employee's expenses for approval and conversion into a bill. The expense total remains in an unapproved expense account and has no accounting impact until the expense is approved by someone with accounting authority. After an expense report is approved, a bill is created and the expense amount is reflected on the books.

This transaction is available when the Estimates feature is enabled at Setup > Company > Enable Features, on the Employees subtab.

For help working with this record in the UI, see the help topic [Expense Reporting](#).

The internal ID for this record is `expensereport`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The expense report record is scriptable in both client and server SuiteScript.

Supported Functions

The expense report record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Expense Report Policy

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can define policies to limit the expenses your employees can enter. Expense reports must satisfy all defined filters to apply a limit. When you select multiple entries for filters, expense reports must satisfy at least one selection for each applied filter. For example, you create a policy to apply to airfare and meals for your parent subsidiary. The limit applies to any expense report entered by a parent company employee with expenses for airfare or meals. The expense report does not have to include expenses for both airfare and meals.

This transaction is available when the Expense Reports feature is enabled at Setup > Company > Enable Features, on the Employees subtab.

For help working with this record in the UI, see the help topic [Expense Report Policies](#).

The internal ID for this record is `expensereportpolicy`. The internal ID of the new subrecord is `erpolicynotification` and the sublist internal ID is `notification`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The expense report policy record is scriptable in both client and server SuiteScript.

Supported Functions

The expense report policy record supports the following functions: read, create, update, copy, delete, and search.

Code Samples

```

1  /**
2
3  *@NapiVersion 2.x
4
5  */
6
7
8
9  define(['N/record'], function(record) {
10
11     var startDate = new Date();
12
13     var endDate = startDate;
14
15     endDate.setDate(startDate.getDate() + 90);
16
17
18
19     var expenseReportPolicy = record.create({
20
21         type: record.Type.EXPENSE_REPORT_POLICY,
22
23         isDynamic: true
24
25     });
26
27
28
29     expenseReportPolicy.setValue({
30
31         fieldId: 'name',
32
33         value: 'ERpolicy-meals4'
34
35     });
36
37
38
39     expenseReportPolicy.setValue({
40
41         fieldId: 'startdate',
42
43         value: startDate
44
45     });
46
47
48
49     expenseReportPolicy.setValue({
50
51         fieldId: 'enddate',
52
53         value: endDate
54
55     });
56
57
58
59     expenseReportPolicy.setText({
60
61         fieldId: 'subsidiary',
62
63         text: 'UK Sub'
64
65     });
66
67
68
69     expenseReportPolicy.setText({
70

```

```

71     fieldId: 'location',
72
73     text: ['Location UK', 'Location UK 2']
74
75 });
76
77
78
79 expenseReportPolicy.setText({
80
81     fieldId: 'expensecategory',
82
83     text: 'Meals'
84
85 });
86
87
88
89 expenseReportPolicy.setText({
90
91     fieldId: 'limitttype',
92
93     text: 'Per day'
94
95 });
96
97
98
99 expenseReportPolicy.setText({
100
101     fieldId: 'action',
102
103     text: 'Warn but allow expenses outside limit'
104
105 });
106
107
108
109 expenseReportPolicy.setValue({
110
111     fieldId: 'maxamount',
112
113     value: 50
114
115 });
116
117
118
119 expenseReportPolicy.selectNewLine({
120
121     sublistId: 'notification'
122
123 });
124
125
126
127 expenseReportPolicy.setCurrentSublistValue({
128
129     sublistId: 'notification',
130
131     fieldId: 'entity',
132
133     value: 343
134
135 });
136
137
138
139 expenseReportPolicy.commitLine({
140
141     sublistId: 'notification'
142
143 });

```

```

144
145
146
147 var expenseReportPolicyId = expenseReportPolicy.save();
148
149
150
151 // Test of record load
152
153 try {
154
155     var testExpenseReportPolicy = record.load({
156
157         type: record.Type.EXPENSE_REPORT_POLICY,
158
159         id: expenseReportPolicyId
160
161     });
162
163 }
164
165 finally {
166
167     if (expenseReportPolicyId != null) {
168
169         record.delete({
170
171             type: record.Type.EXPENSE_REPORT_POLICY,
172
173             id: expenseReportPolicyId
174
175         })
176
177     }
178
179 }
180
181 });

```

Fulfillment Request

Applies to: SuiteScript 2.x | SuiteCloud Developer

The fulfillment request record represents a request or demand to fulfill a sales order at a fulfillment location. You create a fulfillment request record from a sales order record.

To use this record, you must have the Fulfillment Request feature enabled.

For help working with this record in the UI, see the help topic [Fulfillment Requests](#).

The internal ID for this record is fulfillmentrequest.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Supported Script Types

The fulfillment request record is scriptable in both client and server SuiteScript.

Supported Functions

This record supports the following functions: read, edit, transform, delete, and search.

Note: The create and copy functions are not supported.

Usage Notes

To create a fulfillment request record, you must have a sales order ID. The Fulfillment Choice and Location must also be set on each line in the sales order. Depending on the selected fulfillment choice (Ship or Store Pickup), NetSuite sets the fulfillment request type to Ship or Store Pickup respectively. If there are multiple lines on the sales order with different fulfillment choices or locations, you must create separate fulfillment requests for each location/fulfillment choice combination. If all lines on the sales order have the same fulfillment choice and location, you create a single fulfillment request. Trying to create a fulfillment request with different fulfillment choices or different locations results in an exception.

For example, if a sales order has two lines, with the fulfillment choice on the first line set to Ship, and the fulfillment choice on the second line set to Store Pickup, you need to create two fulfillment request records.

Note: When you automate the creation of fulfillment requests in your account, NetSuite creates the required number of fulfillment requests automatically.

Fulfillment requests have one of several statuses. Only some statuses can be set with SuiteScript. Other statuses are set by NetSuite and are based on the fulfillment state of the line items in the fulfillment request. The following table lists the possible statuses of the fulfillment request record. See the help topic [Fulfillment Request Statuses](#) for more information about fulfillment request statuses.

Status Code	Status Name	Settable
A	New	Yes
B	In Progress	Yes
C	Cancelled	Yes
D	Picked	No
I	Packed	No
E	Partially Fulfilled	No
G	Fulfilled with Exceptions	No
F	Fulfilled	No
R	Rejected	No

You can add fulfillment request exceptions at the line item level to a fulfillment request. To create a fulfillment request exception, you need to supply a quantity, an exception type, and an exception reason. The exception reasons are different for each exception type. The following table lists the exception types and reasons.

Exception Type ID	Exception Type	Exception Reason ID	Exception Reason
1	Picking	-1	Inventory mismatch
		-2	Physical damage

Exception Type ID	Exception Type	Exception Reason ID	Exception Reason
		-3	Other
2	Packing	-4	Packing material shortage
		-5	Other
3	Picking up	-6	Too heavy
		-7	Too big
		-8	Wrong color
		-9	Other
4	Shipping	-10	Shipper can't pick up
		-11	Shipper did not arrive for pick up
		-12	Shipping address issues
		-13	Other

Code Samples

Example 1: Create a fulfillment request from a sales order. This sample shows how to create a fulfillment request from a sales order. The sales order ID is 321654. Use [record.transform\(options\)](#) to transform the sales order record to a fulfillment request.

```

1 var fulfillmentRequestRecord = record.transform({
2
3   fromType: record.Type.SALES_ORDER,
4
5   fromId: 321654,
6
7   toType: record.Type.FULFILLMENT_REQUEST,
8
9   isDynamic: true
10
11 });
12
13 var fulfillmentRequestRecordId = fulfillmentRequestRecord.save();

```

Example 2: Add a fulfillment exception on a fulfillment request. This sample shows how to add one fulfillment exception reason to an existing fulfillment request. The fulfillment request ID is 221100. The fulfillment exception is for the second line item in the fulfillment request.

```

1 var fulfillmentRequestRecord = record.load({
2
3   type: 'fulfillmentrequest',
4
5   id: 221100,
6
7   isDynamic: true
8
9 });
10
11 fulfillmentRequestRecord.selectNewLine({
12
13   sublistId: 'fulfillmentexception'
14
15 });
16
17 fulfillmentRequestRecord.setCurrentSublistValue({

```

```

18     sublistId: 'fulfillmentexception',
19
20     fieldId: 'itemline',
21
22     value: 2
23
24 });
25
26 fulfillmentRequestRecord.setCurrentSublistValue({
27
28     sublistId: 'fulfillmentexception',
29
30     fieldId: 'exceptionquantity',
31
32     value: 1
33
34 });
35
36 fulfillmentRequestRecord.setCurrentSublistValue({
37
38     sublistId: 'fulfillmentexception',
39
40     fieldId: 'exceptiontype',
41
42     value: 1
43
44 });
45
46 fulfillmentRequestRecord.setCurrentSublistValue({
47
48     sublistId: 'fulfillmentexception',
49
50     fieldId: 'exceptionreason',
51
52     value: -1
53
54 });
55
56 fulfillmentRequestRecord.setCurrentSublistValue({
57
58     sublistId: 'fulfillmentexception',
59
60     fieldId: 'exceptioncomments',
61
62     value: fulfillmentRequestExceptionLine.getExceptionComments()
63
64 });
65
66 fulfillmentRequestRecord.commitLine({
67
68     sublistId: 'fulfillmentexception'
69
70 });
71
72
73
74
75 var fulfillmentRequestRecordId = record.save();

```

Example 3: Create fulfillment requests on a sales order with different fulfillment choices. If a sales order has different fulfillment choices or locations at the line level, you first need to get the values of the fulfillment choice and location fields on each line. As you loop through lines in the fulfillment request, you check whether the values on the line are the same as the values on the first line; if they are different, you discard the line.

```

1  var salesOrderID = 321654;
2
3
4
5  var fulfillmentRequestRecord = record.transform({
6

```



```

7   fromType: record.Type.SALES_ORDER,
8
9   fromId: salesOrderID,
10
11  toType: record.Type.FULLFILLMENT_REQUEST,
12
13  isDynamic: true
14
15  });
16
17  var fulfillmentRequestLinesCount = fulfillmentRequestRecord.getLineCount({
18
19    sublistId: 'item'
20
21  });
22
23
24
25  while (fulfillmentRequestLinesCount > 0)
26  {
27
28    var initialLocation = null;
29
30    var initialItemFulfillmentChoice = null;
31
32
33
34
35    for (line = 1; line <= fulfillmentRequestLinesCount; line++) {
36
37      var currentLocation = fulfillmentRequestRecord.getSublistValue({
38
39        sublistId: 'item',
40
41        fieldId: 'location',
42
43        line: line
44
45      });
46
47      var currentItemFulfillmentChoice = fulfillmentRequestRecord.getSublistValue({
48
49        sublistId: 'item',
50
51        fieldId: 'itemfulfillmentchoice',
52
53        line: line
54
55      });
56
57
58
59      if (initialLocation === null) {
60
61        initialLocation = currentLocation;
62
63        initialItemFulfillmentChoice = currentItemFulfillmentChoice;
64
65      }
66
67
68
69      if ((initialLocation === currentLocation) && (initialItemFulfillmentChoice === currentItemFulfillmentChoice)) {
70
71        fulfillmentRequestRecord.setSublistValue({
72
73          sublistId: 'item',
74
75          fieldId: 'itemreceive',
76
77          line: line,
78
79          value: true

```

```

80         });
81     };
82 }
83
84 else {
85     // Discard the line because its location or fulfillment choice is different
86
87     fulfillmentRequestRecord.setSublistValue({
88
89         sublistId: 'item',
90
91         fieldId: 'itemreceive',
92
93         line: line,
94
95         value: false
96     });
97 }
98
99
100
101 }
102
103 }
104
105
106
107 fulfillmentRequestRecordId = fulfillmentRequestRecord.save();
108
109
110
111 var fulfillmentRequestRecord = record.transform({
112     fromType: record.Type.SALES_ORDER,
113
114     fromId: salesOrderID,
115
116     toType: record.Type.FULFILLMENT_REQUEST,
117
118     isDynamic: true
119 });
120
121
122 var fulfillmentRequestLinesCount = fulfillmentRequestRecord.getLineCount({
123     sublistId: 'item'
124 });
125
126
127 });
128
129 }

```

Imported Employee Expense

Applies to: SuiteScript 2.x | SuiteCloud Developer

This transaction is available when the Expense Report feature is enabled at Setup > Company > Enable Features, on the Employees subtab.

For help working with this record in the UI, see the help topic [Expense Reporting](#).

The internal ID for this record is `importedemployeeexpense`. The internal ID of the new subrecord is `importedemplexpensdetail`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The imported employee expense record is scriptable in both client and server SuiteScript.

Supported Functions

The imported employee expense record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Sample Codes

```

1  require(['N/record'], function(record) {
2
3      var recordObj= record.create({type: 'importedemployeeexpense', isDynamic: true});
4
5
6
7      // Testing
8
9      recordObj.setValue({fieldId: 'memo', value: 'Expense memo 2'});
10
11     recordObj.setValue({fieldId: 'billedamount', value: 500});
12
13     recordObj.setValue({fieldId: 'chargedate', value: new Date('9/1/2021')});
14
15     recordObj.setValue({fieldId: 'transactionid', value: 'MyTransactionId'});
16
17
18
19     var sublist = recordObj.getSublist( {sublistId:'impemplexpensedetail'});
20
21
22
23     recordObj.selectNewLine({sublistId:'impemplexpensedetail'});
24
25     recordObj.setCurrentSublistValue({sublistId:'impemplexpensedetail', fieldId: 'fieldname', value: 'Detail Field'});
26
27     recordObj.setCurrentSublistValue({sublistId:'impemplexpensedetail', fieldId: 'fieldvalue', value: 'Detail Value'});
28
29     recordObj.commitLine({sublistId:'impemplexpensedetail'});
30
31 });

```

Inventory Adjustment

Applies to: SuiteScript 2.x | SuiteCloud Developer

The inventory adjustment transaction changes the quantity and value of an inventory item without entering a purchase order. For example, this transaction can be used to account for clerical errors, changes in cost, thefts, or miscounts. If you use the LIFO or FIFO costing methods, this transaction

preserves the quantity and value of an inventory item at the same time preserving the costing history of the item.

For help working with this record in the UI, see the help topic [Inventory Adjustments](#).

The internal ID for this record is `inventoryadjustment`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory adjustment record is scriptable in both client and server SuiteScript.

Supported Functions

The inventory adjustment record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Inventory Cost Revaluation

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The inventory cost revaluation record is used to recalculate the value of items configured to use standard costing.

This record is available when the Standard Costing feature is enabled at Setup > Company > Enable Features, on the Items & Inventory subtab. When the feature is enabled, you can access the inventory cost revaluation record in the UI by choosing Transactions > Inventory > Revalue Inventory Cost.

For help working with this record in the UI, see the help topics [Manually Entering an Inventory Cost Revaluation](#) and [Revalue Standard Cost Inventory](#).

The internal ID for this record is `inventorycostrevaluation`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory cost revaluation record is scriptable in both client SuiteScript and Server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The inventory cost revaluation record can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following sample shows how to create and update inventory cost revaluation records.

```

1  function inventoryCostRevaluation(){
2
3      var recId = null;
4
5      var newId = null;
6
7      var recType = record.Type.INVENTORY_COST_REVALUATION;
8
9      try {
10
11         // Create the record
12
13         var revaluationRec = record.create({
14
15             type: recType,
16
17             isDynamic: true
18
19         });
20
21         revaluationRec.setValue({
22
23             fieldId: 'subsidiary',
24
25             value: '1'
26
27         });
28
29         revaluationRec.setValue({
30
31             fieldId: 'item',
32
33             value: itemId1           // Some assembly item
34
35         });
36
37         revaluationRec.setValue({
38
39             fieldId: 'account',
40
41             value: '1'
42
43         });
44
45         revaluationRec.setValue({
46
47             fieldId: 'location',
48
49             value: '1'
50
51         });
52
53         revaluationRec.setSublistValue({
54
55             sublistId: 'costcomponent',
56
57             fieldId: 'cost',
58
59             line: 1,
60
61             value: '2'

```

```

62     });
63
64
65     revaluationRec.setSublistValue({
66
67         sublistId: 'costcomponent',
68
69         fieldId: 'componentitem',
70
71         line: 1,
72
73         value: componentItemId1      // Some inventory item
74
75     });
76
77     revaluationRec.setSublistValue({
78
79         sublistId: 'costcomponent',
80
81         fieldId: 'quantity',
82
83         line: 1,
84
85         value: '3'
86
87     });
88
89
90
91     recId = revaluationRec.save();
92
93
94
95     var revaluationAddedRec = recordload({
96
97         type: recType,
98
99         id: recId
100
101     });
102
103
104
105     // Update
106
107     // Note you cannot change subsidiary and item
108
109     revaluationAddedRec.setValue({
110
111         fieldId: 'account',
112
113         value: '2'
114
115     });
116
117     revaluationAddedRec.setSublistValue({
118
119         sublistId: 'costcomponent',
120
121         fieldId: 'quantity',
122
123         line: 1,
124
125         value: '5'
126
127     });
128
129     recId2 = revaluationAddedRec.save();
130
131
132
133     var revaluationUpdatedRec = record.load({
134

```

```

135         type: recType,
136
137         id: recId2
138
139     });
140
141
142
143     // Copy
144
145     var copiedRecord = record.copy({
146
147         type: recType,
148
149         id: recId
150
151     });
152
153     var newId = copiedRecord.save();
154
155     copiedRecord = record.load({
156
157         type: recType,
158
159         id: newId
160
161     });
162
163
164
165     var mySearch = search.create({
166
167         type: recType,
168
169         columns: [{
170
171             name: 'memo',
172
173         }, {
174
175             name: 'subsidiary'
176
177         }],
178
179         filters: [{
180
181             name: 'memo',
182
183             operator: 'contains',
184
185             value: 'em'
186
187         }]
188
189     });
190
191
192
193     log.debug({
194
195         details: ''+arguments.callee.name +' passed.'
196
197     });
198
199 }
200
201 catch (e) {
202
203     log.error({
204
205         details: ''+arguments.callee.name +' failed.'
206
207     });

```

```

208     }
209
210
211     finally{
212
213         if (recId != null){
214
215             record.delete({
216
217                 type: recType,
218
219                 id: recId
220
221             });
222
223         }
224
225         if (newId != null){
226
227             record.delete({
228
229                 type: recType,
230
231                 id: newId
232
233             });
234
235         }
236
237     }
238
239 }

```

Inventory Count

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The inventory count record enables you to maintain better inventory accuracy and tighter control of assets.

The inventory count record is available only when the Inventory Count feature is enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.

In the UI, you access the inventory count record at Transactions > Inventory > Enter Inventory Count.

For help working with this record in the UI, see the help topic [Inventory Count](#).

The internal ID for this record is inventorycount.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Inventory Count Record Actions](#) for actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The inventory count record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The inventory count record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

You cannot use SuiteScript to update the Status field on inventory count records. You must set this field by clicking the Start Count button manually on the inventory count record in the UI.

Code Samples

The following samples show how to create an inventory count record with an associated inventory detail record for a lot-numbered or serialized item.

It includes the following:

- The code in Step 1 creates an inventory count record. This code is supported for client or server scripts.
- The code in Step 2 adds an Add Inventory Detail button to the record page before it is loaded. This code is supported for Suitelets, beforeLoad user event scripts, and workflow action scripts. A click of the Add Inventory button calls a client script.
- The code in Step 3 uses an https request to call a Suitelet or RESTlet and refreshes the page. This code is supported for client scripts.
- The code in Step 4 adds an inventory detail subrecord to the inventory count. This code is supported for Suitelets and RESTlets.

Step 1: The following sample creates the inventory count record in dynamic mode.

The following sample shows how to create and update inventory cost revaluation records.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5
6  define(['N/record', 'N/log'], (record, log) => {
7    /**
8     * @param {Object} context Defines the function that is executed
9     * when the page completes loading or when the form is reset.
10   */
11   function pageInit (context) {
12     // Script parameters:
13     const subsidiaryId = 1
  
```

```

14 |     const locationId = 6
15 |     const itemId = 599
16 |     const binNumber = 501
17 |     const inventoryCount = record.create({
18 |       type: record.Type.INVENTORY_COUNT,
19 |       isDynamic: true
20 |     })
21 |     inventoryCount.setValue({
22 |       fieldId: 'subsidiary',
23 |       value: subsidiaryId
24 |     })
25 |     inventoryCount.setValue({
26 |       fieldId: 'location',
27 |       value: locationId
28 |     })
29 |     inventoryCount.selectNewLine({
30 |       sublistId: 'item'
31 |     })
32 |     inventoryCount.setCurrentSublistValue({
33 |       sublistId: 'item',
34 |       fieldId: 'item',
35 |       value: itemId
36 |     })
37 |     inventoryCount.setCurrentSublistValue({
38 |       sublistId: 'item',
39 |       fieldId: 'binnumber',
40 |       value: binNumber
41 |     })
42 |     inventoryCount.commitLine({
43 |       sublistId: 'item'
44 |     })
45 |     inventoryCount.save()
46 |     log.debug({
47 |       title: 'Inventory Count Created: ' + inventoryCount
48 |     })
49 |   } return {
50 |     pageInit: pageInit
51 |   }
52 | })

```

Step 2: The following sample adds an Add Inventory Detail button to the record page before it is loaded.

```

1 | /**
2 |  * @NApiVersion 2.1
3 |  * @NScriptType UserEventScript
4 |  */
5 |
6 | define(['N/ui/serverWidget'], (serverWidget) => {
7 |   /**
8 |    * @param {Object} context Defines the function that is executed before a
9 |    * record is loaded; that is, whenever a read operation occurs on a record,
10 |    * and prior to returning the record or page.
11 |    */
12 |   function beforeLoad (context) {
13 |     const inventoryCount = context.newRecord
14 |     const countStatus = inventoryCount.getValue({ fieldId: 'status' })
15 |     if (countStatus !== 'Started') {
16 |       return
17 |     }
18 |     const itemLineCount = inventoryCount.getLineCount({ sublistId: 'item' })
19 |     let hasInventoryDetail = false
20 |     for (let i = 0; i < itemLineCount; i++) {
21 |       const lineHasInventoryDetail = inventoryCount.hasSublistSubrecord({
22 |         sublistId: 'item',
23 |         fieldId: 'countdetail',
24 |         line: i
25 |       })
26 |       if (lineHasInventoryDetail) {
27 |         hasInventoryDetail = true
28 |         break
29 |       }
30 |     }

```

```

31     if (hasInventoryDetail) {
32         return
33     }
34     const form = context.form
35     form.addButton({
36         id: 'custpage_add_inv_detail',
37         label: 'Add Inventory Detail',
38         functionName: 'addInventoryDetail()'
39     })
40 }
41 return {
42     beforeLoad: beforeLoad
43 }
44 })

```

Step 3: The following sample uses an https request to call a Suitelet or RESTlet that will add the inventory detail, and refreshes the page.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType ClientScript
4   */
5
6  define(['N/runtime', 'N/url', 'N/https', 'N/currentRecord', 'N/log'], (runtime, url, https, currentRecord, log) => {
7      /**
8       * @param {Object} context Defines the function that is executed
9       * when the page completes loading or when the form is reset.
10      *
11      * @param {String} custscriptrestletscriptid Custom script parameter
12      * configured in script deployment.
13      *
14      * @param {String} custscriptrestletscriptdeploymentid Custom script
15      * parameter configured in script deployment.
16      */
17      function pageInit (context) {
18          const script = runtime.getCurrentScript()
19          const restletScriptId = script.getParameter({
20              name: 'custscriptrestletscriptid'
21          })
22          const restletScriptDeploymentId = script.getParameter({
23              name: 'custscriptrestletscriptdeploymentid'
24          })
25          log.debug('Restlet Script ID: ' + restletScriptId)
26          log.debug('Restlet Deployment ID: ' + restletScriptDeploymentId)
27
28          window.addInventoryDetail = function () {
29              const inventoryCountId = currentRecord.get().id
30              const requestData = { inventoryCountId: inventoryCountId }
31              const headers = { 'Content-type': 'application/json' }
32              const restletUrl = url.resolveScript({
33                  scriptId: restletScriptId,
34                  deploymentId: restletScriptDeploymentId,
35                  returnExternalId: false
36              })
37              log.debug('Restlet URL: ' + restletUrl)
38              https.put.promise({
39                  url: restletUrl,
40                  body: requestData,
41                  headers: headers
42              }).then((response) => {
43                  const redirectURL = url.resolveRecord({
44                      recordType: 'inventorycount',
45                      recordId: inventoryCountId,
46                      isEditMode: false
47                  })
48                  window.open(redirectURL, '_self')
49              })
50          }
51      }
52      return {
53          pageInit: pageInit
54      }
55  }

```

```
55 | })
```

Step 4: The following sample adds the inventory detail subrecord to the inventory count record.

```
1  /**
2  * @NApiVersion 2.1
3  * @NScriptType Restlet
4  */
5  define(['N/record', 'N/runtime', 'N/log'], (record, runtime, log) => {
6      /**
7       * @param {Object} requestBody Pass the request body as a JavaScript Object
8       * when the request Content-Type is 'application/json' or 'application/xml'.
9       *
10      * @param {number} custscriptcountquantity Custom script parameter configured
11      * in script deployment.
12      *
13      * @param {number} custscriptinventorynumber Custom script parameter
14      * configured in script deployment.
15      */
16      function put (requestBody) {
17          const script = runtime.getCurrentScript()
18          const countQuantity = script.getParameter({
19              name: 'custscriptcountquantity'
20          })
21          const inventoryNumber = script.getParameter({
22              name: 'custscriptinventorynumber'
23          })
24          log.debug({
25              details: 'Count Quantity: ' + countQuantity
26          })
27          log.debug({
28              details: 'Inventory Number: ' + inventoryNumber
29          })
30          const inventoryCount = record.load({
31              type: 'inventorycount',
32              id: requestBody.inventoryCountId
33          })
34          const itemLineCount = inventoryCount.getLineCount({
35              sublistId: 'item'
36          })
37          for (let i = 0; i < itemLineCount; i++) {
38              inventoryCount.setSublistValue({
39                  sublistId: 'item',
40                  fieldId: 'countquantity',
41                  line: i,
42                  value: countQuantity
43              })
44              const countDetail = inventoryCount.getSublistSubrecord({
45                  sublistId: 'item',
46                  fieldId: 'countdetail',
47                  line: i
48              })
49              countDetail.insertLine({
50                  sublistId: 'inventorydetail',
51                  line: 0
52              })
53              countDetail.setSublistValue({
54                  sublistId: 'inventorydetail',
55                  fieldId: 'inventorynumber',
56                  value: inventoryNumber,
57                  line: 0
58              })
59              countDetail.setSublistValue({
60                  sublistId: 'inventorydetail',
61                  fieldId: 'quantity',
62                  value: countQuantity,
63                  line: 0
64              })
65              inventoryCount.save()
66          }
67      }
68      return {
```

```

69 | put: put
70 | }
71 | })

```

Inventory Count Record Actions

Applies to: SuiteScript 2.x | SuiteCloud Developer

The inventory count transaction type supports the following actions: startcount, completecount, reject, and approve.

For help working with this record in the UI, see [Inventory Count](#).

For information about SuiteScript 2.0 record actions, see the following topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

startcount

Corresponding UI Button	Start Count
Action Description	Changes the status of Open inventory counts to Started.
Returns	Object that contains the inventory count ID
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	20 units
Since	2024.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The allocate action does not support any additional parameters.

completecount

Corresponding UI Button	Complete Count
Action Description	Changes the status of counts from Started to Completed/Pending approval.
Returns	Object that contains the inventory count ID
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Since	2024.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The allocate action does not support any additional parameters.

approve

Corresponding UI Button	Approve
Action Description	Changes the status of completed counts to Approved. Creates inventory adjustments if the count results do not match the current on-hand quantities.
Returns	Object that contains the inventory count ID
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	60 units
Since	2024.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The allocate action does not support any additional parameters.

reject

Corresponding UI Button	Reject
Action Description	Reassigns the Open status to completed counts.
Returns	Object that contains the inventory count ID
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 units
Since	2024.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The allocate action does not support any additional parameters.

Sample Code

Refer to the following samples for using the start count action from the client side and server side:

```

1  require(['N/action'], function(actionMod) {
2
3      // client side example
4      actionMod.getActions.promise({recordType: 'inventorycount'}).then(function(actions) {
5          var startAction = actions.startcount;
6          startAction.promise({recordId: 1}).then(function(result) {
7              console.log("Inventory Count with id " + result.response.id + " has been started");
8          });
9      });
10
11     // server side example
12     var result = actionMod.executeAction({id: 'startcount', recordType: 'inventorycount', params: {recordId: 1}});

```

```
13 | });
```

Inventory Status Change

Applies to: SuiteScript 2.x | SuiteCloud Developer

The inventory status change record is available in NetSuite and NetSuite OneWorld accounts when the Inventory Status feature is enabled. An administrator can enable this feature at Setup > Company > Enable Features, on the Items & Inventory subtab.

For help working with this record in the UI, see the help topic [Inventory Status](#).

The internal ID for the inventory status change record is `inventorystatuschange`. The internal ID for its Items sublist is `inventory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory status change record is scriptable in both client and server SuiteScript.

Supported Functions

The inventory status change record can be created, read, edited, copied, deleted, searched, and transformed using SuiteScript.

Usage Notes

In the UI, you cannot select the same value in the Previous State and Revised State fields. Using SuiteScript, there is no validation if you set the same value for `previousstatus` and `revisedstatus`.

Code Samples

The following sample shows how to create an inventory status change record to move 10 good widgets at your US location to the inspection status.

```
1 var invStatus = record.create({
2
3   type: record.Type.INVENTORY_STATUS_CHANGE,
4
5   isDynamic: true
6
7 });
8
9 invStatus.setText({
10
```

```

11     fieldId: 'location',
12
13     text: 'US Only Location'
14
15 });
16
17 invStatus.setText({
18
19     fieldId: 'previousstatus',
20
21     text: 'Good'
22
23 });
24
25 invStatus.setText({
26
27     fieldId: 'revisedstatus',
28
29     text: 'Inspection'
30
31 });
32
33 invStatus.selectNewLine({
34
35     sublistId: 'inventory'
36
37 });
38
39 invStatus.setCurrentSublistText({
40
41     sublistId: 'inventory',
42
43     fieldId: 'item',
44
45     text: 'Widget'
46
47 });
48
49 invStatus.setCurrentSublistValue({
50
51     sublistId: 'inventory',
52
53     fieldId: 'quantity',
54
55     value: 10
56
57 });
58
59 invStatus.commitLine({
60
61     sublistId: 'inventory'
62
63 });
64
65 var recordId = invStatus.save();

```

Inventory Transfer

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Inventory Transfer transaction posts details about per-location item inventory level changes when items are transferred between two locations. This basic inventory transfer decreases items in the source location and increases them in the receiving location, all in one step.

This transaction is available when the Locations feature and the Multi-Location Inventory (MLI) feature are enabled.

For help working with this record in the UI, see the help topic [Transferring Inventory](#).

The internal ID for this record is `inventorytransfer`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory transfer record is scriptable in server SuiteScript only.

Supported Functions

The inventory detail record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

This record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Invoice

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Invoices](#).

The internal ID for this record is `invoice`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Invoice Record Actions](#) for the actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The invoice record is scriptable in both client and server SuiteScript.

Supported Functions

The invoice record is fully scriptable. It can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

See the following sections for more details on working with this record:

- [Fields](#)
- [Email Operations](#)

Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

This record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

The following invoice record sublists are **list** sublists:

- Billable Expenses
- Billable Items
- Billable Time

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

The Shipping sublist's Shipping Tax Code field, internal ID `shippingtaxcode`, appears only if per-line taxes have been set on the Item sublist.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a `beforeLoad` user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type `email` on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email**

Button is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Invoice Record Actions

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

markforgrouping

Corresponding UI Button	Mark for Grouping
Action Description	Indicates whether an invoice is available for grouping, but does not add this invoice to a group. The Invoice Groups feature must be enabled to use this action. For more information about the Invoice Groups feature, see the help topic Invoice Groups Overview .
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Since	2020.2

The internal ID for this action is **markforgrouping**.

The Mark for Grouping action appears on the [Invoice](#) record type.

For help working with the invoice record in the UI, see the help topic [Invoices](#).

Supported Functions

The invoice record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Possible Error Messages

The Mark for Grouping action generates the following error messages:

- The user does not have permission or the invoice is restricted.
- This invoice is already marked.
- The invoice is partially or fully paid.

- There is no sales order associated with this invoice.

Parameters

- recordid for invoice group
- invoice list

Code Sample

```

1 require(['N/action'], function(actionMod) {
2
3
4
5   var result = actionMod.execute({id: 'markforgrouping', recordType: 'invoice', params: {recordId: 1}});
6
7
8
9   ...
10
11 // Add additional code

```

removeinvoicefromgroup

Corresponding UI Button	Remove From Group
Action Description	Removes an invoice from an existing invoice group. The Invoice Groups feature must be enabled to use this action. For more information about the Invoice Groups feature, see the help topic Invoice Groups Overview .
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Since	2020.2

The internal ID for this action is **removeinvoicefromgroup**.

The Remove From Group action appears on the [Invoice](#) record type.

For help working with the invoice record in the UI, see the help topic [Invoices](#).

Supported Functions

The invoice record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Possible Error Messages

The Remove From Group action generates the following error messages:

- The user does not have permission or the invoice is restricted.
- The operation was completed concurrently.
- The invoice is partially or fully paid.
- This invoice is the only invoice in the invoice group..

Parameters

- recordid for invoice group

Code Sample

```

1 | require(['N/action'], function(actionMod) {
2 |
3 |
4 |
5 | var result = actionMod.execute({id: 'removeinvoicefromgroup', recordType: 'invoice', params: {recordId: 1}});
6 |
7 | ...
8 |
9 | // Add additional code

```

unmarkforgrouping

Corresponding UI Button	Unmark for Grouping
Action Description	Indicates an invoice ineligible for grouping, but it does not remove the invoice from the group. The Invoice Groups feature must be enabled to use this action. For more information about the Invoice Groups feature, see the help topic Invoice Groups Overview .
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Since	2020.2

The internal ID for this action is **unmarkforgrouping**.

For help working with the invoice record in the UI, see the help topic [Invoices](#).

Supported Functions

The invoice record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Possible Error Messages

- The user does not have permission or the invoice is restricted.
- This invoice is already grouped.
- This is already a standalone invoice.

Parameters

- record id for invoice

Code Sample

```

1 | require(['N/action'], function(actionMod) {

```

```
2
3
4
5 var result = actionMod.execute({id: 'unmarkforgrouping', recordType: 'invoice', params: {recordId: 1}});
6
7
8
9 ...
10
11 // Add additional code
```

Invoice Group

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Invoice Groups Overview](#).

The internal ID for this record is **invoicegroup**.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Invoice Group Record Actions](#) for the actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

**Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Invoice Group Record Actions

Applies to: SuiteScript 2.x | SuiteCloud Developer

groupinvoices

Corresponding UI Button	Group Invoices
Action Description	Groups invoices into groups based on the customer name and common criteria. The Invoice Groups feature must be enabled to use this action. For more information about the Invoice Groups feature, see the help topic Invoice Groups Overview .
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Since	2020.2

The internal ID for this action is **groupinvoices**.

The Group Invoices action appears on the [Invoice Groups Overview](#) record type.

For help working with the invoice groups record in the UI, see the help topic [Viewing the Invoice Group Record](#).

Supported Functions

The invoice group record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Possible Error Messages

- There are no invoices listed or some invoices are missing.
- The invoice group does not exist.
- The invoice group is partially or fully paid.
- Some invoices in this group are partially or fully paid.
- The user does not have permission or the invoice is restricted.
- The number of invoices for this group is more than the maximum allowed.

Parameters

- record id for invoice group
- invoice list

Code Sample

```

1  require(['N/action'], function(actionMod) {
2
3
4
5  var result = actionMod.execute({id: 'groupinvoices', recordType: 'invoiceGroup', params: {recordId: 1, invoicelist:'3,4,9'}});
6
7
8
9  ...
10
11 // Add additional code

```

removeinvoicesfromgroup

Corresponding UI Button	Remove From Group
Action Description	Removes invoices from an invoice group. The Invoice Groups feature must be enabled to use this action. For more information about the Invoice Groups feature, see the help topic Invoice Groups Overview .
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	None
Since	2020.2

The internal ID for this action is **removeinvoicesfromgroup**.

The Remove From Group action appears on the [Invoice Group](#) record type.

For help working with the invoice groups record in the UI, see the help topic [Viewing the Invoice Group Record](#).

Supported Functions

The invoice group record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Possible Error Messages

- There are no invoices listed or some invoices are missing.
- The invoice group does not exist.
- The invoice group is partially or fully paid.
- Some invoices in this group are partially or fully paid.
- The user does not have permission or the invoice is restricted.

Parameters

- record id for invoice group
- invoice list

Code Sample

```
1 require(['N/action'], function(actionMod) {
2
3   var result = actionMod.execute({id: 'removeinvoicesfromgroup', recordType: 'invoiceGroup', params: {recordId: 1, in
   voicelist: '3,4,9'}}); });
4
5   ...
6
7   // Add additional code
```

Demand Plan Detail Sublist

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this sublist is **demandplandetail**.

The Demand Plan Detail sublist appears on the [Item Demand Plan](#) record type.

The Demand Plan Detail sublist is a matrix that is similar to the Pricing sublist. This matrix stores projected quantities demanded by date. Each row in the matrix represents a specific month, week, or day, and each column in the matrix represents an expected quantity demand.

For help working with this record in the UI, see the help topic [Demand Planning](#).

Functionally, this sublist shares many of the characteristics of List Sublists. However, scripting with the Demand Plan Detail sublist is not like scripting with most other sublists in NetSuite. You must use [Matrix APIs for the Demand Plan Detail Sublist](#) to access quantity values on a per-row, per-column basis, similar to the way that item pricing values are accessed. These APIs are a subset of the [N/currentRecord Module](#) and [N/record Module](#) APIs more commonly used for scripting with other sublists.

The format of the Demand Plan Detail sublist depends on the values set in body fields for the start date of the plan, the end date of the plan, and the time period to be used (monthly, weekly, or daily). Because of

this dependence, you should work with the Item Demand Plan record and the Demand Plan Detail sublist in dynamic mode. See the help topic [SuiteScript 2.x Standard and Dynamic Modes](#)

Be aware of the following requirements:

- To script with the Item Demand Plan record and the Demand Plan Detail sublist for inventory items, the Demand Planning feature must be enabled. For assembly/BOM items, the Work Orders feature also must be enabled.
- You must Demand plans are supported only for item(s) that have the **supplyreplenishmentmethod** field set to Time Phased. To change this setting, go to the item record and set the Replenishment Method to Time Phased.
- Required body field values must be defined before matrix field values can be edited. In dynamic mode, current values may be retrieved. Start date and end date body fields default to the first day and last day of the current year.

For more details and code samples for each type of demand plan, see the following help topics:

- [Monthly Demand Plan](#)
- [Weekly Demand Plan](#)
- [Daily Demand Plan](#)

Monthly Demand Plan

A monthly demand plan includes a row for each month within the body field start date and end date, and one quantity column for each month.

Manual Item Demand Plan List Search Customize More

Save **Cancel**

Primary Information

Location * Unit of Measure

Item * Memo

Year	Month	Start Date	End Date	View
2024		1.1.2024	31.12.2024	Monthly

Start Date	End Date	Monthly Calculated	Quantity
1.1.2024	31.1.2024		
1.2.2024	29.2.2024		
1.3.2024	31.3.2024		
1.4.2024	30.4.2024		
1.5.2024	31.5.2024		
1.6.2024	30.6.2024		
1.7.2024	31.7.2024		
1.8.2024	31.8.2024		
1.9.2024	30.9.2024		
1.10.2024	31.10.2024		
1.11.2024	30.11.2024		
1.12.2024	31.12.2024		

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the month represented by each row.
 - The enddate is the date of the last day of the month represented by each row.

- The month for row 1 is the month set in the body field start date, the month for row 2 is the next month, and so on, until the month set in the body field end date is reached.
- The values for the quantity field can be set in SuiteScript. For monthly demand plans, the column parameter for this field is always 1.

Monthly Demand Plan Code Sample

The following sample sets quantities for the month of January 2023:

Note: Some of the values in these samples are placeholders. Before using these samples, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
6    const demandplan = record.create({
7      type: record.Type.ITEM_DEMAND_PLAN,
8      isDynamic: true
9    })
10   demandplan.setValue({
11     fieldId: 'demandplancalendartype',
12     value: 'MONTHLY'
13   })
14   demandplan.setValue({
15     fieldId: 'subsidiary',
16     value: 1
17   })
18   demandplan.setValue({
19     fieldId: 'location',
20     value: 1
21   })
22   demandplan.setValue({
23     fieldId: 'item',
24     value: 253
25   })
26   const id = demandplan.save({
27     enableSourcing: true
28   })
29
30   const loadDemandPlan = record.load({
31     type: record.Type.ITEM_DEMAND_PLAN,
32     id: id,
33     isDynamic: true
34   })
35   loadDemandPlan.setValue({
36     fieldId: 'startdate',
37     value: '1/1/2023'
38   })
39   loadDemandPlan.setValue({
40     fieldId: 'enddate',
41     value: '12/31/2023'
42   })
43   loadDemandPlan.selectLine({
44     sublistId: 'demandplandetail',
45     line: '0'
46   })
47   loadDemandPlan.setCurrentSublistValue({
48     sublistId: 'demandplandetail',
49     fieldId: 'quantity_1_',
50     value: 100
51   })
52   loadDemandPlan.selectLine({
53     sublistId: 'demandplandetail',
54     line: '1'

```

```

55 })
56 loadDemandPlan.setCurrentSublistValue({
57   sublistId: 'demandplandetail',
58   fieldId: 'quantity_1_',
59   value: 200
60 })
61 loadDemandPlan.commitLine({
62   sublistId: 'demandplandetail'
63 })
64 loadDemandPlan.save()
65 })

```

Weekly Demand Plan

A weekly demand plan includes a row for each week contained in the time period set by the body field start date and end date, and one quantity column for each week.

Manual Item Demand Plan List Search Customize More

Save Cancel

Primary Information

Location * Unit of Measure

Item * Memo

Year Month Start Date End Date View

Start Date	End Date	Weekly Calculated	Quantity
11.2.2024	7.1.2024		
8.1.2024	14.1.2024		
15.1.2024	21.1.2024		
22.1.2024	28.1.2024		
29.1.2024	4.2.2024		
5.2.2024	11.2.2024		
12.2.2024	18.2.2024		
19.2.2024	25.2.2024		
26.2.2024	3.3.2024		
4.3.2024	10.3.2024		
11.3.2024	17.3.2024		
18.3.2024	24.3.2024		
25.3.2024	31.3.2024		

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the week represented by each row.
 - The enddate is the date of the last day of the week represented by each row.

Note: The first day of the week by default is Sunday, but may vary according to the company preference set for First Day of the Week at Setup > Company > General Preferences.

- The week for row 1 is the week of the date set in the body field start date. Note that unless the body field start date happens to be the first day of the week, the startdate for this first row may precede the body field start date.
- The week for the final sublist row is the week of the date set in the body field end date. Note that unless the body field end date happens to be the last day of the week, the enddate for this last row may be after the body field enddate.

- The values for the quantity field can be set in SuiteScript. For weekly demand plans, the column parameter for this field is always 1.

Weekly Demand Plan Code Sample

The following sample sets quantities for the first two weeks of the year:

Note: Some of the values in these samples are placeholders. Before using these samples, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
6      const demandplan = record.create({
7          type: record.Type.ITEM_DEMAND_PLAN,
8          isDynamic: true
9      })
10     demandplan.setValue({
11         fieldId: 'demandplancalendartype',
12         value: 'WEEKLY'
13     })
14     demandplan.setValue({
15         fieldId: 'subsidiary',
16         value: 1
17     })
18     demandplan.setValue({
19         fieldId: 'location',
20         value: 1
21     })
22     demandplan.setValue({
23         fieldId: 'item',
24         value: 252
25     })
26     const id = demandplan.save({
27         enableSourcing: true
28     })
29
30     const loadDemandPlan = record.load({
31         type: record.Type.ITEM_DEMAND_PLAN,
32         id: id,
33         isDynamic: true
34     })
35     loadDemandPlan.setValue({
36         fieldId: 'startdate',
37         value: '1/1/2023'
38     })
39     loadDemandPlan.setValue({
40         fieldId: 'enddate',
41         value: '12/31/2023'
42     })
43     loadDemandPlan.selectLine({
44         sublistId: 'demandplandetail',
45         line: '0'
46     })
47     loadDemandPlan.setCurrentSublistValue({
48         sublistId: 'demandplandetail',
49         fieldId: 'quantity_1_',
50         value: 100
51     })
52     loadDemandPlan.selectLine({
53         sublistId: 'demandplandetail',
54         line: '1'
55     })
56     loadDemandPlan.setCurrentSublistValue({
57         sublistId: 'demandplandetail',
58         fieldId: 'quantity_1_',

```

```

59     value: 200
60   })
61   loadDemandPlan.commitLine({
62     sublistId: 'demandplandetail'
63   })
64   loadDemandPlan.save()
65 })

```

Daily Demand Plan

A daily demand plan includes a row for each week contained in the time period set by the body field start date and end date, and seven quantity columns for each week, one for each day of the week.

Manual Item Demand Plan List Search Customize More

Save Cancel

Primary Information

Location * Unit of Measure

Item * Memo

Year Month Start Date End Date View

Start Date	End Date	Weekly Calculated	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
11.2.2024	71.2.2024								
8.1.2024	14.1.2024								
15.1.2024	21.1.2024								
22.1.2024	28.1.2024								
29.1.2024	4.2.2024								
5.2.2024	11.2.2024								
12.2.2024	18.2.2024								
19.2.2024	25.2.2024								
26.2.2024	3.3.2024								
4.3.2024	10.3.2024								
11.3.2024	17.3.2024								
18.3.2024	24.3.2024								
25.3.2024	31.3.2024								

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the week represented by each row.
 - The enddate is the date of the last day of the week represented by each row.

Note: The first day of the week by default is Sunday, but may vary according to the company preference set for First Day of the Week at Setup > Company > General Preferences.

- The week for row 1 is the week of the date set in the body field start date. Note that unless the body field start date happens to be the first day of the week, the startdate for this first row may precede the body field start date.
 - The week for the final sublist row is the week of the date set in the body field end date. Note that unless the body field end date happens to be the last day of the week, the enddate for this last row may be after the body field enddate.
- The values for the quantity fields can be set in SuiteScript.
 - The column parameter for a quantity field is 1,2,3,4,5,6, or 7, depending on the day of the week.

- In the screenshot above, the week starts with Sunday, which is the default first day of the week, and in this case, maps to a column parameter of 1. However, 1 does not always map to Sunday; it maps to the first day of the week as set in the company preferences.

Daily Demand Plan Code Sample

Note: Some of the values in these samples are placeholders. Before using these samples, replace all hard-coded values, such as IDs and file paths, with valid values from your NetSuite account. If you run a script with an invalid value, the system may throw an error.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
6    const demandplan = record.create({
7      type: record.Type.ITEM_DEMAND_PLAN,
8      isDynamic: true
9    })
10   demandplan.setValue({
11     fieldId: 'demandplancalendartype',
12     value: 'DAILY'
13   })
14   demandplan.setValue({
15     fieldId: 'subsidiary',
16     value: 1
17   })
18   demandplan.setValue({
19     fieldId: 'location',
20     value: 1
21   })
22   demandplan.setValue({
23     fieldId: 'item',
24     value: 254
25   })
26   const id = demandplan.save({
27     enableSourcing: true
28   })
29
30   const loadDemandPlan = record.load({
31     type: record.Type.ITEM_DEMAND_PLAN,
32     id: id,
33     isDynamic: true
34   })
35   loadDemandPlan.setValue({
36     fieldId: 'startdate',
37     value: '1/1/2023'
38   })
39   loadDemandPlan.setValue({
40     fieldId: 'enddate',
41     value: '12/31/2023'
42   })
43   loadDemandPlan.selectLine({
44     sublistId: 'demandplandetail',
45     line: '0'
46   })
47   loadDemandPlan.setCurrentSublistValue({
48     sublistId: 'demandplandetail',
49     fieldId: 'quantity_1_',
50     value: 100
51   })
52   loadDemandPlan.selectLine({
53     sublistId: 'demandplandetail',
54     line: '1'
55   })
56   loadDemandPlan.setCurrentSublistValue({
57     sublistId: 'demandplandetail',
58     fieldId: 'quantity_1_',

```

```

59     value: 200
60   })
61   loadDemandPlan.commitLine({
62     sublistId: 'demandplandetail'
63   })
64   loadDemandPlan.save()
65 })

```

Matrix APIs for the Demand Plan Detail Sublist

Use the following matrix APIs with the Demand Plan Detail sublist:

- [Record.getCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.getCurrentMatrixSublistValue\(options\)](#)
- [Record.setCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.setCurrentMatrixSublistValue\(options\)](#)

Note: With all APIs listed above, use the [Record.selectLine\(options\)](#) or [CurrentRecord.selectLine\(options\)](#) APIs first to select an existing line.

- [Record.getMatrixSublistField\(options\)](#)
- [CurrentRecord.getMatrixSublistField\(options\)](#)
- [Record.getMatrixSublistValue\(options\)](#)
- [CurrentRecord.getMatrixSublistValue\(options\)](#)
- [Record.findMatrixSublistLineWithValue\(options\)](#)
- [CurrentRecord.findMatrixSublistLineWithValue\(options\)](#)

For more information about APIs, see the help topics [N/currentRecord Module](#) and [N/record Module](#).

Item Fulfillment

Applies to: SuiteScript 2.x | SuiteCloud Developer

An item fulfillment transaction records the shipment of some or all items on an order to the customer. The processes for item fulfillment transactions depend on whether the Advanced Shipping feature is enabled.

- If Advanced Shipping is not enabled, the fulfillment and invoicing processes are combined. When an item fulfillment is created, a related invoice is created at the same time.
- If Advanced Shipping is enabled, fulfillment and invoicing are two independent processes, and shipments can be recorded separately from billing.

For help working with this record in the UI, see the help topic [Order Fulfillment](#).

The internal ID for this record is `itemfulfillment`. Copy and create are not allowed for this record.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The item fulfillment record is scriptable in both client and server SuiteScript.

Supported Functions

The item fulfillment record is partially scriptable. It can be updated, deleted, and searched using SuiteScript. It cannot be created or copied.

Usage Notes

When working with this record, you can set *pick*, *pack*, or *ship* as event trigger types that will execute your user event script. In the NetSuite Help Center, see the help topic [SuiteScript 2.x User Event Script Type](#) for more information.

The Item Fulfillment/Item Receipt sublist is a **list** sublist.

The Shipping sublist is included on the item fulfillment record. This sublist is not currently showing on this record in the [SuiteScript Records Browser](#). To get the internal IDs for the Shipping sublist, open the Records Browser and go to one of the other record types that support this sublist. These records are sales order, cash sale, invoice, and estimate / quote.

The Shipping sublist's Shipping Tax Code field, internal ID shippingtaxcode, appears only if per-line taxes have been set on the Item sublist.

Scripting with Item Fulfillment Return Addresses

The internal ID for the return address subrecord is returnshippingaddress. The following table maps return address subrecord field IDs to prior return address field IDs.

Item Fulfillment Return Address Subrecord ID = returnshippingaddress		Deprecated Return Address Fields	
Field Label	Field ID	Field Label	Field ID
Country	country	Country	returnshipcountry
Attention	attention	—	
Addressee	addressee	—	
Phone	addrphone	—	
Address1	addr1	Address 1	returnshipaddr1
Address2	addr2	Address 2	returnshipaddr2
City	city	City	returnshipcity
State	state	State	returnshipstate
Zip	zip	Zip	returnshipzipcode
Address	addrtext	—	returnaddress
Override	override	—	

The following sample shows how to create an item fulfillment return address using the subrecord.

```

1  require(['N/record'], function(record) {
2
3      var salesOrder = record.transform({
4
5          fromType: record.Type.SALES_ORDER,
6
7          fromId: 1020,
8
9          toType: record.Type.ITEM_FULFILLMENT,
10
11         isDynamic: true,
12     });
13
14
15
16
17     var returnAddressSubrecord = salesOrder.getSubrecord({
18
19         fieldId: 'returnshippingaddress'
20     });
21
22
23
24
25     returnAddressSubrecord.setValue({
26
27         fieldId: 'country',
28
29         value: 'US'
30     });
31
32
33
34
35     returnAddressSubrecord.setValue({
36
37         fieldId: 'attention',
38
39         value: 'Amy Kall'
40     });
41
42
43
44
45     returnAddressSubrecord.setValue({
46
47         fieldId: 'addressee',
48
49         value: 'ABCCCompany'
50     });
51
52
53
54
55     returnAddressSubrecord.setValue({
56
57         fieldId: 'addrphone',
58
59         value: '3256874555'
60     });
61
62
63
64
65     returnAddressSubrecord.setValue({
66
67         fieldId: 'addr1',
68
69         value: '955 Campus Dr Ste 101'
70     });
71

```

```

72
73
74
75     returnAddressSubrecord.setValue({
76
77         fieldId: 'city',
78
79         value: 'San Mateo'
80
81     });
82
83
84
85     returnAddressSubrecord.setValue({
86
87         fieldId: 'state',
88
89         value: 'CA'
90
91     });
92
93
94
95     returnAddressSubrecord.setValue({
96
97         fieldId: 'zip',
98
99         value: '94404'
100
101     });
102
103
104
105     var recordId = salesOrder.save();
106
107 });

```

Item Receipt

i Applies to: SuiteScript 2.x | SuiteCloud Developer

An item receipt transaction records the receipt of returned items from customers. This transaction updates the following information:

- Items on return authorizations are recorded as received.
- Inventory records are updated for the new stock levels.
- Inventory asset accounts are updated with the values of returned items.
- Status of the return is updated.

The item receipt transaction is available when the Advanced Receiving feature is enabled.

For help working with this record in the UI, see the help topics [Receiving a Customer Return](#) and [Handling Returned Items](#).

The internal ID of this record is `itemreceipt`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item receipt record is scriptable in both client and server SuiteScript.

Supported Functions

The item receipt record is partially scriptable. It can be updated, deleted, and searched using SuiteScript. It cannot be created or copied.

Usage Notes

The Item Fulfillment/Item Receipt sublist is a **list** sublist.

Using Landed Cost Fields

When you create a landed cost category, the associated field IDs for the first category are landedcostamount1 and landedcostsource1. If you create a second category, the IDs will be landedcostamount2 and landedcostsource2.

This pattern increments by one with each additional category. For example, the IDs for the next landed cost category will be landedcostamount3 and landedcostsource3, and so on.

Creating Item Receipt Records

You cannot create standalone item receipts using SuiteScript. For example, the following will throw an error:

```
1 var itemReceipt = record.create({
2
3   type: record.Type.ITEM_RECEIPT
4
5 })
```

To create an item receipt, you must use [record.transform\(options\)](#), which transforms the data from one record type, purchase order, for example, into an item receipt. To create an item receipt, your code would be similar to the following:

```
1 // Transform a record with a specific id to a different record type.
2
3 // For example, from PO to Item Receipt
4
5 // Get the object of the transformed record.
6
7 var trecord = record.transform({
8
9   fromType: record.Type.PURCHASE_ORDER,
10
11   fromId: 26,
12
13   toType: record.Type.ITEM_RECEIPT
14
15 });
16
17 qty = trecord.getSublistValue({
18
19   sublistId: 'item',
```

```

20
21     fieldId: 'quantity',
22
23     line: 1
24
25 });
26
27 trecord.setSublistValue({
28
29     sublistId: 'item',
30
31     fieldId: 'quantity',
32
33     line: 1,
34
35     value: '2'
36
37 });
38
39 var id1 = trecord.save ({
40
41     enableSourceing: true
42
43 });
44
45 email.send({
46
47     author: -5,
48
49     recipient: -5,
50
51     subject: 'Transform Email',
52
53     body: 'Original Qty = '
54
55 });

```

Opportunity

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Opportunities represent negotiations with prospects. You must first enable opportunities in your NetSuite account before you can access this record type. To enable opportunities, go to Setup > Company > Enable Features. On the CRM subtab, under Sales, select the Opportunities box.

For help working with this record in the UI, see the help topic [Opportunity Records](#).

The internal ID for this record is opportunity.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The opportunity record is scriptable in both client and server SuiteScript.

Supported Functions

The manufacturing operation task record is fully scriptable. It can be created, copied, updated, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Also note that this record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

Order Reservation

Applies to: SuiteScript 2.x | SuiteCloud Developer

This topic applies to all versions of SuiteScript.

Order reservation works as a virtual container that enables you to save multiple sources of supply for a specific group of future demands. This helps you to ensure that supplies are saved for high priority customers.

For help working with this record in the UI, see the help topic [Supply Allocation Order Management](#).

The internal ID for this record is OrderReservation

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Prerequisites

The Sales Channel Allocation feature must be enabled.

Supported Script Types

The order reservation record is scriptable in both client and server SuiteScript.

Supported Functions

The order reservation record is partially scriptable. It can be created, read, updated, deleted, copied, and searched using SuiteScript. It cannot be transformed.

Usage Notes

The order reservation record has three sublists that are not exposed. The Description field is not exposed. The Supply Allocation and Fulfillment sublists are read-only.

Paycheck

Applies to: SuiteScript 2.x | SuiteCloud Developer

This record is available only when the Payroll feature is enabled at Setup > Company > Enable Features on the Employees subtab.

In the UI, you can access this record at Transactions > Employees > Create Payroll.

For help working with this record in the UI, see the help topics [Creating a Payroll Batch](#) and [Editing an Individual Paycheck from a Payroll Batch](#).

The internal ID for this record is paycheck.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

Supported Functions

The paycheck record is partially scriptable. It can be read and updated only.

Usage Notes

Be aware of the following:

- The paycheck record is created through the payroll batch record.

Code Samples

The following sample shows how to search for paycheck records.

```

1 var mySearch = search.create ({
2
3   type: search.Type.PAYCHECK,
4
5   columns: [{
6
7     name: 'batchnumber',
8
9   }, {
10
11     name: 'employee',
12
13   }],
14
15   filters: [{
16
17     name: 'batchnumber',
18
19     operator: search.Operator.EQUALTO,
20
21     values: 100
22
23   }],
24
25 });

```

Paycheck Journal

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Paycheck Journal record is intended to enable global payroll solutions. You can use it along with the [Payroll Item](#) record to create custom payroll solutions and to support integrations with external payroll systems.

For help working with this record in the UI, see the help topic [Paycheck Journal Feature](#).

The internal ID for this record is paycheckjournal.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Usage Notes

The Paycheck Journal feature must be enabled to work with the Paycheck Journal record.

Code Samples

The following sample creates a paycheck journal transaction.

```

1 // create Paycheck Journal with two earnings and one deduction sublist
2
3 function createPaycheckJournal(){
4
5     var pj = record.create ({
6
7         type: record.Type.PAYCHECK_JOURNAL,
8
9         isDynamic: true
10    });
11
12    pj.setValue({
13
14        fieldId: 'trandate',
15
16        value: '6/10/2012'
17    });
18
19    pj.setValue({
20
21        fieldId: 'employee',
22
23        value: 4 // internal ID of the employee
24    });
25
26    pj.setValue({
27
28        fieldId: 'account',
29
30        value: 28 // internal ID of the account
31    });
32
33    pj.selectNewLine({
34
35        sublistId: 'earning'
36    });
37
38    pj.setCurrentSublistValue({
39
40        sublistId: 'earning',
41
42        fieldId: 'payrollitem',
43
44        value: '102'
45    });
46
47    pj.setCurrentSublistValue({
48
49        sublistId: 'earning',
50
51        fieldId: 'amount',
52
53        value: 20.35
54    });
55
56    pj.commitLine({
57
58        sublistId: 'earning'
59    });
60
61
62
63
64
65
66
67
68

```



```

69     });
70
71
72
73     pj.selectNewLine({
74
75         sublistId: 'earning'
76
77     });
78
79     pj.setCurrentSublistValue({
80
81         sublistId: 'earning',
82
83         fieldId: 'payrollitem',
84
85         value: '102'
86
87     });
88
89     pj.setCurrentSublistValue({
90
91         sublistId: 'earning',
92
93         fieldId: 'amount',
94
95         value: 33.35
96
97     });
98
99     pj.commitLine({
100
101         sublistId: 'earning'
102
103     });
104
105
106
107     pj.selectNewLine({
108
109         sublistId: 'deduction'
110
111     });
112
113     pj.setCurrentSublistValue({
114
115         sublistId: 'deduction',
116
117         fieldId: 'payrollitem',
118
119         value: '103'
120
121     });
122
123     pj.setCurrentSublistValue({
124
125         sublistId: 'deduction',
126
127         fieldId: 'amount',
128
129         value: 444.44
130
131     });
132
133     pj.commitLine({
134
135         sublistId: 'deduction'
136
137     });
138
139
140
141     var recordId = pj.save();

```

```

142 |
143 | }

```

The following sample updates a paycheck journal transaction.

```

1 // update Paycheck journal by setting new amount of line 2 earning list
2
3 // and clearing the deduction list
4
5 function updatePaycheckJournal() {
6
7     var pj = record.load({
8
9         type: record.Type.PAYCHECK_JOURNAL,
10
11         isDynamic: true,
12
13         id: 305
14
15     });
16
17
18
19     pj.setSublistValue({
20
21         sublistId: 'earning',
22
23         fieldId: 'amount',
24
25         line: 2,           // 1 is the line number we intend to update
26
27         value: 444.44
28
29     });
30
31     var lineCount = pj.getLineCount({
32
33         sublistId: 'deduction'
34
35     });
36
37     for (var lineNo = 1; lineNo <= lineCount; lineNo++) {
38
39         pj.removeLine({
40
41             sublistId: 'deduction',
42
43             line: lineNo
44
45         });
46
47     }
48
49     var recId = pj.save();
50
51 }

```

Payroll Batch

Applies to: SuiteScript 2.x | SuiteCloud Developer

This record is available only for SuitePeople U.S. Payroll customers. For more information, see the help topic [SuitePeople Overview](#).

For help working with this record in the UI, see the help topic [Creating a Payroll Batch](#).

The internal ID of this record is payrollbatch.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

The payroll batch record is scriptable in both client and server SuiteScript.

Supported Functions

The payroll batch record is fully scriptable, which means that the record can be created, updated, copied, deleted and searched using SuiteScript.

Usage Notes

Be aware of the following:

- After a payroll batch is created, it is submitted twice: one time to calculate and again to commit.
- To calculate a payroll batch, after the batch is submitted, it needs to be reloaded until one of the following statuses is generated: calculated, committed, or error.
- If you are calculating and committing a large amount of data, you should write a loop to recheck the status of the payroll batch. Refer to the following table for more details.

Payroll Batch Status	Value	Description
Created	A	A new payroll batch has been created.
Calculated	B	A payroll batch has been calculated.
Edited	C	A payroll batch may have been calculated previously, or it is a newly created batch with some paychecks added or calculated.
About to Commit	D	Payroll is about to be committed.
Committed_At_Service	E	Payroll is committed for processing by payroll service, but some records need to be created in NetSuite.
Committed	F	Payroll is committed.
Completed	P	Payroll batch is completed.
Reversed	R	Payroll batch is reversed.
Error	X	Error occurred in payroll batch.

Code Samples

The following sample shows how to create a payroll batch and perform other basic tasks. It also includes how to add an employee to a payroll batch. For more information, see [Payroll Batch Employee](#).

```

1  require(["N/record"], function (record)
2  {
3      var payrollbatch = record.create({
4          type: 'payrollbatch',
5          isDynamic: true
6      });
7
8      payrollbatch.setValue({
9          fieldId: 'offcycle',
10         value: false
11     });
12
13     payrollbatch.setValue({
14         fieldId: 'payfrequency',
15         value: '52'
16     });
17
18     payrollbatch.setValue({
19         fieldId: 'periodending',
20         value: new Date('6/15/2020')
21     });
22
23     var recId = payrollbatch.save();
24
25     // Add employees to the payroll batch
26     // In most cases, you would iterate through the addmorepayeesmachine to find the employees you would add to the batch
27
28     var addemp = payrollbatch.load({
29         type: 'payrollbatchaddemployees',
30         id: id,
31         defaultValues: 'true'
32     });
33
34     addemp.setSublistValue({
35         sublistId: 'addmorepayeesmachine',
36         fieldId: 'payemp',
37         line: 1,
38         value: 'T'
39     });
40
41     var recId2 = addemp.save();
42
43     //Submit the record to be calculated
44     payrollbatch = record.load({
45         type: 'payrollbatch',
46         id: id,
47         defaultValues: 'false'
48     });
49
50     var recId3 = payrollbatch.save();
51
52     //Check the status of the record to ensure that calculating is complete before proceeding
53     do {
54         payrollbatch = record.load({
55             type: 'payrollbatch',
56             id: id,
57             defaultValues: 'false'
58         });
59
60         status = payrollbatch.getValue({
61             fieldId: 'status'
62         });
63     } while ( !(status === 'B' || status === 'E' || status === 'X') );
64
65     //Set the flag to commit the payroll batch
66     if (status === 'B') {

```

```

67     payrollbatch.setValue({
68         fieldId: 'commit',
69         value: 'T'
70     });
71 }
72
73 //Commit
74 var recId3 = payrollbatch.save();
75
76 do {
77     payrollbatch = record.load({
78         type: 'payrollbatch',
79         id: id,
80         defaultValues: 'false'
81     });
82     status = payrollbatch.getValue({
83         fieldId: 'status'
84     });
85 } while ( !(status === 'F' || status === 'E' || status === 'X') );
86 });

```

Payroll Batch Employee

i Applies to: SuiteScript 2.x | SuiteCloud Developer

This record enables you to add employees to the payroll batch.

This record is only available for Payroll customers. For more information, see the help topic [SuitePeople Overview](#).

For help working with this record in the UI, see the help topic [Adding Employees to a Payroll Batch](#).

The internal ID of this record is payrollbatchaddemployees.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

Supported Functions

The payroll batch employee record is read and edit only.

Usage Notes

Be aware of the following:

- You cannot remove or clear an employee through SuiteScript after they have been added to the payroll batch and the record is submitted. If you need to remove an employee from the payroll batch you will need to do this through the UI.

Period End Journal

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

Period End Journal is a transaction record that posts subsidiary consolidation and year end closing to the general ledger. Period end journals are generated automatically when you complete the Create Period End Journals task on the Period Close Checklist.

For help working with this record in the UI, see the help topic [Period End Journal Entries](#).

This record is part of the Period End Journal Entries feature, which is available only in NetSuite OneWorld accounts. The Accounting Periods feature is a prerequisite for the Period End Journal Entries feature. Before you begin working with period end journals programmatically, see the help topic [Setup for Period End Journal Entries](#).

The internal ID for this record is periodendjournal.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The period end journal record is scriptable in server and client SuiteScript. The system process that generates this record does not trigger user events. However, user events can be triggered outside of the process.

Supported Functions

The period end journal record can be read, updated, and searched using SuiteScript. However, update is limited to the Memo fields and any custom fields or custom segments that do not affect the general ledger. When searching for period end journals, Include Period End Transactions must be set to True in the results.

Code Samples

The following sample shows how to update the value of the memo field.

```

1 var periodEndJournal = record.load({
2
3   type: record.Type.PERIOD_END_JOURNAL,
4
5   id: 101,
6
7   isDynamic: true
8
9 });

```

```

10
11 periodEndJournal.setText({
12
13     fieldId: 'memo',
14
15     text: 'Test Memo'
16
17 });
18
19 periodEndJournal.save();

```

Related Topics

- [Period End Journal Entries](#)
- [Working with the SuiteScript Records Browser](#)
- [SuiteCloud Supported Records](#)
- [Transactions](#)

Project Intercompany Cross Charge Request

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Project Intercompany Cross Charge Request record provides the ability to cross-charge amounts between subsidiaries with the integration period-end process. It enables you to set amounts in a currency that you can use for intercompany cross-charges reflected in the Intercompany Cross Charges list, and to request approvals around the record.

To use the Project Intercompany Cross Charge Request record you must enable the Project Management and Intercompany Framework features.

For help working with this record in the UI, see the help topics [Intercompany Framework](#), [Intercompany Cross Charges](#) and [Using Project Management](#).

The internal ID for this record is projecticchargerequest.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The Project Intercompany Cross Charge Request record is scriptable in server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following sample shows how to create a record and how to set up the fields needed. It selects Project Completion as the trigger and adds the amount of 200. The IDs are the values for the currency.

```

1  /**
2
3  * @NApiVersion 2.x
4
5  * @NScriptType UserEventScript
6
7  */
8
9
10
11 define([ 'N/record' ],function(record) {
12
13     function afterSubmit(context){
14
15         // Create the record.
16
17         var rec = record.create({
18
19             type: 'projecticchargerequest',
20
21             isDynamic: true
22
23         });
24
25
26         // Set body fields on the record.
27
28         rec.setValue({
29
30             fieldId: 'fromsubsidiary',
31
32             value: '1'
33
34         });
35
36         rec.setValue({
37
38             fieldId: 'tosubsidiary',
39
40             value: '3'
41
42         });
43
44         rec.setValue({
45
46             fieldId: 'project',
47
48             value: '339'
49
50         });
51
52         rec.setValue({
53
54             fieldId: 'amount',
55
56             value: '200'
57
58         });
59
60         rec.setValue({
61
62             fieldId: 'currency',
63
64             value: '1'
65
66

```



```

67     });
68
69     rec.setValue({
70         fieldId: 'chargetrigger',
71         value: 'ProjectCompletion'
72     });
73
74     // Save the record.
75
76     try {
77         var recId = rec.save();
78
79         log.debug({
80             title: 'Record created successfully',
81             details: 'Id: ' + recId
82         });
83     } catch (e) {
84         log.error({
85             title: e.name,
86             details: e.message
87         });
88     }
89
90     return {
91         afterSubmit: afterSubmit
92     };
93 });

```

Related Topics

- [Intercompany Framework](#)
- [Intercompany Cross Charges](#)
- [Using Project Management](#)
- [Working with the SuiteScript Records Browser](#)
- [SuiteCloud Supported Records](#)
- [Transactions](#)

Purchase Contract

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

This record enables you to take advantage of contracted quantity-based terms and discounts when creating purchase orders.

This record is available only when the Purchase Contracts feature is enabled at Setup > Company > Enable Features, on the Transactions subtab.

In the UI, you access this record at Transactions > Purchases > Enter Purchase Contracts.

For help working with this record in the UI, see the help topic [Creating Purchase Contracts](#).

The internal ID of this record is purchasecontract.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Purchase Order

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

A purchase order transaction authorizes the purchase of goods and services. This transaction tracks items expected to be received, items received, and items yet to be received. A purchase order has no accounting impact until the included items are received. This transaction is enabled when the Purchase Orders feature is enabled.

For help working with this record in the UI, see the help topic [Purchasing](#).

The internal ID for this record is purchaseorder.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The purchase order record is scriptable in both client and server SuiteScript.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

See the following sections for more details on this record:

- [Fields](#)
- [Transforms](#)
- [Email Operations](#)

Fields

estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Transforms

This record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a beforeLoad user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email**

Button is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Requisition

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the requisition record to initiate the purchase process for goods and services needed within your company.

This record is available only when the Requisitions feature is enabled, at Setup > Company > Enable Features, on the Transactions subtab. In the UI, you access this record at Transactions > Purchases/Vendors > Enter Requisitions.

For help working with this record in the UI, see the help topic [Entering a Requisition](#).

The internal ID for this record is purchaserequisition.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The requisition record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The requisition record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Code Samples

The following sample shows how to add a requisition record and perform other basic tasks.

```

1 // Requisition SuiteScript name
2
3 var recType = record.Type.PURCHASE_REQUISITION;
4
5
6
7 // Create and Add Record
8
9 var purchaseReq = record.create({
10
11     type: recType,
12
13     isDynamic: true
14
15 })
16
17 purchaseReq.setValue({
18
19     fieldId: 'location',
20
21     value: 1
22
23 });
24
25 purchaseReq.setSublistValue({
26
27     sublistId: 'expense',
28
29     fieldId: 'account',
30
31     line: 1,
32
33     value: 59
34
35 });
36
37 purchaseReq.setSublistValue({
38
39     sublistId: 'expense',
40
41     fieldId: 'amount',
42
43     line: 1,
44
45     value: 2.3
46
47 });
48
49 var recId = purchaseReq.save();
50
51
52
53 // Load and Update
54
55 purchaseReq = record.load({
56
57     type: recType,
58
59     id: recId
60
61 });
62
63 purchaseReq.setValue({
64
65     fieldId: 'memo',
66
67     value: 'memo'
68

```

```

69 });
70
71 var recId2 = purchaseReq.save();
72
73
74
75 // Search
76
77 var result = search.create({
78
79     type: search.Type.TRANSACTION,
80
81     filters: [{
82
83         name: 'internalId',
84
85         operator: search.Operator.ANYOF,
86
87         values: recId
88
89     }],
90
91     columns: [{
92
93         name: 'memo'
94
95     }]
96
97 });
98
99 // Copy
100
101 var purchaseReq2 = record.copy({
102
103     type: recType,
104
105     id: recId
106
107 });
108
109 var rec2Id = purchaseReq2.save();
110
111
112
113 // Transformation (Initialization)
114
115 var purchaseOrder = record.transform({
116
117     fromType: recType,
118
119     fromId: rec2Id,
120
121     toType: record.Type.PURCHASE_ORDER
122
123 });
124
125
126
127 purchaseOrder.setValue({
128
129     fieldId: 'entity',
130
131     value: '105'
132
133 });
134
135 var poId = purchaseOrder.save();
136
137
138
139 // Delete records
140
141 var delRecId1 = record.delete({

```

```

142     type: record.Type.PURCHASE_ORDER,
143
144     id: poId
145
146 });
147
148 var delRecId2 = record.delete({
149     type: recType,
150
151     id: recId
152
153 });
154
155 var delRecId3 = record.delete({
156     type: recType,
157
158     id: rec2Id
159
160 });
161
162 });
163

```

Resource Group

Applies to: SuiteScript 2.x | SuiteCloud Developer

Resource groups enables you to create a group of resources to be assigned together to a single project task. For example, if you have a group of engineers that typically work on tasks together, you can create a resource group for these engineers. Then, when creating project tasks, you can assign the group to the task instead of each engineer. When assigning groups to a project task, you can choose to divide the estimated work evenly across the members of the resource group or you can assign each member the estimated work entered for the task. It is available when the Project Management feature is enabled.

For help working with this record in the UI, see the help topics [Creating Resource Groups](#) and [Working with Resources in Project Management](#).

The internal ID for this record is resourcegroup.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The Resource Group record is scriptable in server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

There are five fields used for filtering in the UI only – subsidiary, billingclass, class, department, and location.

The system notes sublist is search-only. The resourcegroupmembers sublist contains only one editable column: resource. Other columns are read only: subsidiary, billingclass, unitcost, supervisor, class, department, and location.

Code Samples

The following sample shows how to create a new Resource Group with name and one resource.

```

1  require(['N/record'], function(record) {
2      var resourceGroup = record.create({
3          type: 'resourcegroup',
4          isDynamic: true
5      });
6      resourceGroup.setText({
7          fieldId: 'name',
8          text: 'Resource Group Name'
9      });
10     resourceGroup.selectNewLine({
11         sublistId: 'resourcegroupmembers'
12     });
13     resourceGroup.setCurrentSublistValue({
14         sublistId: 'resourcegroupmembers',
15         fieldId: 'resource',
16         value: '45'
17     });
18     resourceGroup.commitLine({
19         sublistId: 'resourcegroupmembers'
20     });
21     var resourceGroupId = resourceGroup.save();
22 });

```

Related Topics

- [Creating Resource Groups](#)
- [Working with Resources in Project Management](#)
- [Using Project Management](#)
- [Working with the SuiteScript Records Browser](#)
- [SuiteCloud Supported Records](#)
- [Transactions](#)

Return Authorization

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A return authorization transaction, also known as a return materials authorization (RMA), records information about an expected return of items from a customer, including the item IDs, vendors, quantities, and prices (which determine the amounts to be credited or refunded to the customer). This transaction is non-posting. It is available when the Return Authorizations feature is enabled.

For help working with this record in the UI, see the help topic [Customer Return Management](#).

The internal ID for this record is returnauthorization.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The return authorization record is scriptable in both client and server SuiteScript.

Supported Functions

The return authorization record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

See the following sections for more details on working with this record:

- [Transforms](#)
- [Email Operations](#)

Fields

estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

Transforms

This record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a beforeLoad user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email Button** is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Revenue Arrangement

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Revenue Arrangement is a transaction record that contains the details of customer performance obligations for purposes of revenue allocation and recognition.

This record is part of advanced revenue management. To use advanced revenue management, the Accounting Periods feature and Advanced Revenue Management feature must be enabled. Before you begin working with advanced revenue management programmatically, see the help topic [Setup for Advanced Revenue Management \(Essentials\)](#).

For help working with this record in the UI, see the help topic [Revenue Arrangement Management](#).

The internal ID for this record is revenuearrangement.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Revenue Arrangement Record Actions](#) for actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The revenue arrangement record is scriptable in server SuiteScript only.

Supported Functions

The revenue arrangement record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Revenue Arrangement Record Actions

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Revenue Arrangement record currently supports the allocate action.

For help working with this record in the UI, see [Revenue Arrangement](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

allocate

Corresponding UI Button	Allocate
Action Description	Redistributes a transaction amount across revenue elements in a specified revenue arrangement, based on fair value, to obtain the revenue amount. This process is required for revenue arrangements that are not compliant, meaning the amount has not been allocated. Allocation is usually automatic. However, a revenue arrangement may not be compliant because allocation has failed, or the record has been edited to set the Compliant value to false. For more information, see the help topics Revenue Reallocation for Revenue Arrangements and Fair Value and Allocation.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Governance	10 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The allocate action does not support any additional parameters.

Revenue Arrangement Allocate Action Syntax

```
1 | require(['N/action'], function(action) {
```

```

2
3 // action & action list loading
4
5 var actionList = action.find({recordType: 'revenuearrangement'});
6
7 var actionObj = action.get({recordType: 'revenuearrangement', id: 'allocate'});
8
9
10
11 // action execution
12
13 var result = actionObj.execute({recordId: 2});
14
15 var result = actionObj({recordId: 2});
16
17 var result = action.execute({recordType: 'revenuearrangement', id: 'allocate', params: {recordId: 2}});
18
19 });

```

Revenue Commitment

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Using Revenue Commitments](#).

The internal ID for this record is revenuecommitment.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The revenue commitment record is scriptable in client and server SuiteScript.

Supported Functions

The revenue commitment record is partially scriptable. It can be updated, copied, deleted, and searched using SuiteScript. It cannot be created.

Usage Notes

You cannot create this record using the standard [record.create\(options\)](#) function. To create a Revenue Commitment record, you must execute a Sales Order to Revenue Commitment transformation. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

Revenue Commitment Reversal

Applies to: SuiteScript 2.x | SuiteCloud Developer

If a revenue commitment cannot be invoiced, or the revenue and unbilled receivable must be reversed for some reason, a user with sufficient permission must reverse the revenue commitment.

For help working with this record in the UI, see the help topic [Creating Revenue Commitment Reversals](#).

The internal ID for this record is `revenuecommitmentreversal`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The revenue commitment reversal record is scriptable in client and server SuiteScript.

Supported Functions

The revenue commitment reversal record is partially scriptable. It can be updated, copied, deleted, and searched using SuiteScript. It cannot be created.

Usage Notes

You cannot create this record using the standard `record.create(options)` function. To create a Revenue Commitment Reversal record, you must execute a Return Authorization to Revenue Commitment Reversal transformation. Note that the Return Authorization must be approved and received for the transform to work.

In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Sales Channel

Applies to: SuiteScript 2.x | SuiteCloud Developer

A sales channel is a category of sales transaction that you can differentiate by customer type and distribution channel, such as web-shop or retail outlet.

For help working with this record in the UI, see the help topic [Defining a Sales Channel Allocation](#).

The internal ID for this record is SalesChannel

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Prerequisites

The Sales Channel Allocation feature must be enabled.

Supported Script Types

The sales channel record is scriptable in both client and server SuiteScript.

Supported Functions

The sales channel record is partially scriptable. It can be created, read, updated, deleted, copied, and searched using SuiteScript. It cannot be transformed.

Usage Notes

The Description field is not exposed. The Supply Allocation and Fulfillment sublists are read-only.

1 | Create record.

Sales Order

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

A sales order transaction records a commitment to sell goods or services to a customer. Sales orders have no accounting impact until items are shipped or services are completed.

For help working with this record in the UI, see the help topic [Sales Orders](#).

The internal ID for this record is salesorder.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See the [Sales Order Record Macro](#) for the macro associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The sales order record is scriptable in both client and server SuiteScript.

Supported Functions

The sales order record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

See the following sections for more details about working with this record:

- [Fields](#)
- [Email Operations](#)
- [Transforming Sales Orders with the Intercompany Cross-Subsidiary Feature](#)

Fields

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
isrecurringpayment	Recurring Payment	A value for this field is stored only if the value for paymentmethod is a credit card.

Field Internal ID	Field UI Label	Note
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Also note that this record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

The Shipping sublist's Shipping Tax Code field, internal ID shippingtaxcode, appears only if per-line taxes have been set on the Item sublist.

Email Operations

NetSuite enables a user to email a copy of a transaction to a customer or another recipient.

If you deploy a beforeLoad user event script on a transaction, in general the script executes when the transaction is emailed. In these cases, the sending of the email is considered an operation of type email on the transaction. However, if the companywide preference **Use Popup for Main Transaction Email Button** is enabled, a beforeLoad script will not execute in certain cases, depending on how the email is sent. That is, the script will not execute if the user views the transaction and selects Actions > Email, or edits the transaction and sends the email by using the Communications subtab. In these cases, the system displays a popup window that the user can work with to manually edit and send the email. Actions taken with this window are not considered operations on the transaction. If you have your system configured this way, and you want to deploy a beforeLoad script on emails sent using the popup, deploy the script on the [Message](#) record type.

An additional way of emailing a transaction is by editing the transaction and using the Save & Email button. Note that, even if the Use Popup for Main Transaction Email Button preference is enabled, the popup window is not available for emails sent this way. For that reason, a beforeLoad user event script deployed on the transaction will always be triggered by use of the Save & Email button.

For more information about the Use Popup for Main Transaction Email Button preference, see the help topic [Setting Email Preferences](#). For details about emailing transactions, see the help topic [Emailing Transactions](#).

Transforming Sales Orders with the Intercompany Cross-Subsidiary Feature

If the Intercompany Cross-Subsidiary Fulfillment feature is enabled on your account, inventorylocation may become a required transformation parameter when transforming sales orders to item fulfillments.

Sales Order Record Macro

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The sales record type currently supports the autoAssignLocations macro. The Automatic Location Assignment engine processes the sales order and, based on some business rules, sets the best location on each line.

For help working with this record in the UI, see the help topic [Sales Orders](#).

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

autoAssignLocations

Corresponding UI Button	Auto Assign Locations
Macro Description	Assigns the location automatically
Returns	Array with the lines with the assigned location
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2019.2

Macro Syntax

```

1 require(['N/currentRecord'], function(currentRecord) {
2
3     var salesOrder = currentRecord.get();
4
5
6
7     salesOrder.executeMacro({id: 'autoAssignLocations'});
8
9 });

```

Store Pickup Fulfillment

Applies to: SuiteScript 2.x | SuiteCloud Developer

The store pickup fulfillment record is used when you fulfill an order from a location that allows store fulfillment and the fulfillment choice of line items in the order is Store Pickup.

For help working with this record in the UI, see the help topic [Fulfilling Store Pickup Orders](#).

Note: If there are other line items in the sales order with the fulfillment choice set to Ship, you create an item fulfillment record for those lines. See [Item Fulfillment](#) for more information.

The internal ID for this record is storepickupfulfillment.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The store pickup fulfillment record is scriptable in both client and server SuiteScript.

Supported Functions

The store pickup fulfillment record supports the following functions: read, edit, transform, delete, and search.

Note: The create and copy functions are not supported.

Usage Notes

To use this record, you must have the Store Pickup feature enabled. See the help topic [Store Pickup](#) for more information about enabling and configuring the feature in NetSuite.

You should use the Store Pickup feature with the Fulfillment Request feature when possible. A fulfillment request is an intermediate record between the sales order and the store pickup fulfillment. With this intermediate record, you can better manage workload at a store location, as well as add fulfillment exceptions if there are problems fulfilling an item. With the Fulfillment Request feature enabled, you first create a sales order, then transform the sales order record to a fulfillment request record, and finally you transform the fulfillment request record to a store pickup fulfillment request record. See Example 1.

If the Fulfillment Request feature is not enabled, you transform the sales order record to a store pickup fulfillment record. See Example 2.

In both cases, the Fulfillment Choice field must be set to Store Pickup at the line-item level in the sales order.

Code Samples

Example 1: Create a store pickup fulfillment from a sales order. The sales order ID is 321654. The fulfillment request ID is 221100 and is passed to the [record.transform\(options\)](#) API as a hash table.

```
1 var fulfillmentRequestRecord = {"fftreqid" : 221100};
2
3 var storePickupFulfillRecord = record.transform({
4     fromType: record.Type.SALES_ORDER,
5     fromId: 321654,
6     toType: record.Type.STORE_PICKUP_FULFILLMENT,
7     defaultValues: fulfillmentRequestRecord
8 });
9
10
11
12
13
14
15 var storePickupFulfillRecordId = storePickupFulfillRecord.save();
```

Example 2: Create a fulfillment request from a sales order.

```
1 var fulfillmentRequestRecord = record.transform({
2     fromType: record.Type.SALES_ORDER,
3     fromId: 321654,
4     toType: record.Type.FULFILLMENT_REQUEST
5 });
6
7
8
9
10
```

```
11 | var fulfillmentRequestRecordId = fulfillmentRequestRecord.save();
```

Time Actions

Applies to: SuiteScript 2.x | SuiteCloud Developer

The time record type, also known as timebill, currently supports the approve, reject, and submit actions.

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

For help working with this record in the UI, see the help topic [Managing Time Tracking](#).

approve

Corresponding UI Button	Approve
Action Description	Approve the timebill with the specified ID.
Returns	{"id":recordId}, "notifications":[Error, Warning, Information and text]}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The approve action does not support any additional parameters.

reject

Corresponding UI Buttons	Reject Reject with note
Action Description	Reject the timebill with the specified ID. Optionally include a note that explains the reason for the rejection. Available when the Advanced Approvals on Time preference is enabled.
Returns	{"id":recordId}, "notifications":[Error, Warning, Information and text]}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units

Since	2018.2
-------	--------

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The reject action also supports the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
note	string	optional	An explanation of why the timebill was rejected.	2018.1

submit

Corresponding UI Buttons	Submit
Action Description	Submit the timebill with the specified ID for approval. Available when the Advanced Approvals on Time preference is enabled.
Returns	{"id":recordId}, "notifications":[Error, Warning, Information and text]}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The approve action does not support any additional parameters.

Transfer Order

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Transfer Order transaction is used to move inventory between locations when the Multi-Location Inventory (MLI) feature is enabled. Existing integrations with external warehouse management systems can leverage this transaction to manage data about inventory movement between locations.

Transfer orders can initialize item fulfillment and item receipt transactions. See the help topics [Item Fulfillment](#) and [Item Receipt](#) for details about these transactions.

For help working with this record in the UI, see the help topics [Order Fulfillment](#) and [Customer Return Management](#).

The internal ID for this record is transferorder.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Usage Notes

Fields

estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

Vendor Prepayment

Applies to: SuiteScript 2.x | SuiteCloud Developer

A Vendor Prepayment is a posting transaction that impacts the general ledger without offsetting the Accounts Payable account. When the vendor prepayment is applied, the Accounts Payable account is offset. The Vendor Prepayments feature records and tracks payments to vendors before they accept a purchase order for a good or service. You can apply these prepayment amounts against open bills for the vendor.

In NetSuite the vendor prepayment can be associated to a purchase order or not associated to a purchase order (standalone vendor prepayments.)

For help working with this record in the UI, see the help topic [Entering Vendor Prepayments](#).

The internal ID for this record is vendorprepayment.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor prepayment record is scriptable in both client and server SuiteScript.

Supported Fields

For all the internal IDs associated with this record, see the [SuiteScript Records Browser](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Usage Notes

The script user must have the required permission level for the Vendor Prepayment permission. For information about vendor prepayment permissions, read the help topic [Inventory Status](#).

Code Samples

The following example shows how to create a new vendor prepayment transaction for the payee with internal ID 94. The prepayment value is 500 units of the default payee currency:

■ SuiteScript 2.x

```

1  require(['N/record'], function(record) {
2
3      function createAndSaveVendorPrepayment() {
4
5          var objRecord = record.create({
6
7              type: 'vendorprepayment',
8
9              isDynamic: true
10
11          });
12
13          objRecord.setValue({
14
15              fieldId: 'entity',
16
17              value: '94'
18
19          });
20
21          objRecord.setValue({
22
23              fieldId: 'account',
24
25              value: '151'
26
27          });
28
29          objRecord.setValue({
30
31              fieldId: 'payment',
32
33              value: '500'
34
35          });
36
37
38          var recordId = objRecord.save();
39
40      }
41
42      createAndSaveVendorPrepayment();
43
44  }

```

```
45 | });
```

The following example shows how to update an existing vendor prepayment transaction with transaction ID 120:

■ SuiteScript 1.0

```
1 | function updateVendorPrepayment()
2 |
3 | {
4 |
5 |     var id = 120;
6 |
7 |     var memo = 'Updated transaction memo';
8 |
9 |     var vendorPrepayment = nlapiLoadRecord('vendorprepayment', id);
10 |
11 |     vendorPrepayment.setFieldValue('memo', memo);
12 |
13 |     id = nlapiSubmitRecord(vendorPrepayment);
14 |
15 | }
```

■ SuiteScript 2.x

```
1 | require(['N/record'], function(record) {
2 |
3 |     function updateAndSaveVendorPrepayment() {
4 |
5 |
6 |
7 |         var objRecord = record.load({
8 |
9 |             type: 'vendorprepayment',
10 |
11 |             id: 120,
12 |
13 |             isDynamic: true,
14 |
15 |         });
16 |
17 |
18 |
19 |         objRecord.setValue({
20 |
21 |             fieldId: 'memo',
22 |
23 |             value: 'Updated transaction memo 2'
24 |
25 |         });
26 |
27 |
28 |
29 |         var recordId = objRecord.save();
30 |
31 |     }
32 |
33 |     updateAndSaveVendorPrepayment();
34 |
35 | });
```

The following example shows how to delete a vendor prepayment transaction with transaction ID 120:

■ SuiteScript 1.0

```
1 | function deleteVendorPrepayment()
2 |
3 | {
```

```

4
5     var id = 120;
6
7     var vendorPrepayment = nlapiDeleteRecord('vendorprepayment', id);
8
9 }

```

■ SuiteScript 2.x

```

1  require(['N/record'], function(record) {
2
3      function deleteVendorPrepayment() {
4
5          var objRecord = record.delete({
6
7              type: 'vendorprepayment',
8
9              id: 120
10
11          });
12
13      }
14
15      deleteVendorPrepayment();
16
17  });

```

Vendor Prepayment Application

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

After you make a prepayment to a vendor, you must apply the prepayment against the vendor bills. A vendor prepayment application is a posting transaction that impacts the general ledger. The Prepayment account is credited, offsetting the Accounts Payable account. The application decreases the total amount you must pay to your vendor.

You can apply the prepayment manually, or configure NetSuite to apply the prepayment automatically using the Auto-Apply accounting preference.

For help working with this record in the UI, see the help topic [Vendor Prepayment Application](#).

The internal ID for this record is vendorprepaymentapplication.

For information about scripting with this record in SuiteScript, see the following help topics:

- [Vendor Prepayment Application](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor prepayment application record is scriptable in both client and server SuiteScript.

Supported Fields

For all the internal IDs associated with this record, see the [SuiteScript Records Browser](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Usage Notes

The script user must have the required permission level for the Vendor Prepayment Application permission. For information about vendor prepayment permissions, read the help topic [Configuring Permissions for Vendor Prepayments](#).

Code Samples

The following example shows how to transform a vendor prepayment transaction into the vendor prepayment application. The transformation process applies the vendor prepayment. This transformation is required because a vendor prepayment application requires a vendor prepayment.

■ SuiteScript 2.x

This example shows an application of the vendor prepayment with ID 718. During the application, the first vendor bill is selected for application.

```

1  require(['N/record'], function(record) {
2
3      function transformVendorPrepayment() {
4
5          var vendorPrepaymentApplication = record.transform({
6
7              fromType: 'vendorprepayment',
8
9              fromId: 718,
10
11              toType: 'vendorprepaymentapplication',
12
13              isDynamic: true,
14
15          });
16
17
18
19          vendorPrepaymentApplication.selectLine({
20
21              sublistId: 'bill',
22
23              line: 0
24
25          });
26
27
28
29          vendorPrepaymentApplication.setCurrentSublistValue({
30
31              sublistId: 'bill',
32
33              fieldId: 'apply',
34
35              value: true
36
37          });
38
39
40
41          vendorPrepaymentApplication.commitLine({
42
43              sublistId: 'bill'
44

```

```

45     });
46
47
48     var vendorPrepaymentApplicationId = vendorPrepaymentApplication.save();
49
50 }
51
52
53
54
55     transformVendorPrepayment();
56
57
58
59 });

```

Vendor Return Authorization

Applies to: SuiteScript 2.x | SuiteCloud Developer

A vendor return authorization is a non-posting transaction that tracks a return to a vendor, including the items to be returned, their quantities, the approval status, the shipment status, and the amount refunded or credited from the vendor. This type of transaction is available when the Vendor Return Authorizations feature is enabled.

The vendor return process includes four steps: creating a vendor return authorization record, approving or canceling the authorization, shipping items authorized to be returned, and crediting an authorized vendor return.

For help working with this record in the UI, see the help topic [Vendor Return Authorization Overview](#).

The internal ID for this record is vendorreturnauthorization.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor return authorization record is scriptable in both client and server SuiteScript.

Supported Functions

The vendor return authorization record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
usertotal	Amount	This field is not available using search or lookup for any transactions.

Wave

Applies to: SuiteScript 2.x | SuiteCloud Developer

A wave transaction is used to consolidate orders for release to the warehouse. When you release a wave, the items from the order become available for picking and further fulfillment processing on a mobile device. Released waves display pick tickets and pick tasks generated from items in the orders, as well as status updates that enable tracking of fulfillment tasks.

The wave transaction is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Creating Wave Transactions](#).

The internal ID for this transaction is wave.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this transaction.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The wave transaction is scriptable in both client and server SuiteScript.

Supported Functions

The wave transaction is fully scriptable. It can be created, updated, deleted, and searched using SuiteScript.

Usage Notes

For search operations, the wave record supports the [search.create\(options\)](#) method. It does not support the [search.load\(options\)](#) method.

See the following sections for more details about working with this transaction:

- [Fields](#)
- [Subtabs](#)
- [Sublist](#)

Fields

Note the following usage notes for specific fields on this transaction:

- The following fields are read-only: Wave #, Wave Type, Location, Created Date, Released Date, and Completed Date.
- The Custom Form field is shown only in edit mode.
- The Wave Type field is sourced automatically from the Transaction Type field on the Wave Criteria form during wave creation.
- The Picking Type and Status fields cannot be updated on a wave transaction in Released and later statuses.
- The Released Date field initially appears on a wave in Released status.
- The Completed Date field is shown on a wave in Completed status only.

Subtabs

The Pick Tasks subtab initially appears on a wave in Pending Release or Released status. On this subtab, the Assign Selected Pick Tasks To field is shown only if a wave is in Pending Release status.

The Custom subtab serves as the default placeholder when additional custom or standard fields are not displayed on other subtabs or sections of the wave transaction.

Sublist

The Pick Task sublist displays the list of pick task records generated from the wave transaction. On this sublist, the Assigned Picker column can be updated only if a wave is in Pending Release status, and is not scriptable in create. The values in the following columns match the values in fields on the associated pick task records: Status and Recommended Bin.

Code Samples

The following sample code in SuiteScript 2.x includes creating and updating wave transactions.

```

1  require(['N/record'], function (record) {
2
3      var locationId = 9;
4
5      var salesOrderId = 316;
6
7      var pickerId = 77;
8
9
10
11     function createWave() {
12
13         var wave = record.create({type: record.Type.WAVE, isDynamic: true});
14
15         wave.setValue({fieldId: 'location', value: locationId});
16
17         wave.setValue({fieldId: 'wavetype', value: 'SalesOrd'}); // 'TrnfrOrd' for Transfer Orders
18
19         wave.setValue({fieldId: 'priority', value: '1'});
20
21         wave.setValue({fieldId: 'picktype', value: 'MULTI'}); // 'SINGLE' for single order line picking
22
23         wave.setValue({fieldId: 'newwavestatus', value: 'PENDING'}); // "RELEASED" by default

```

```

24
25
26
27
28
29
30     wave.setValue({fieldId: 'searchtemplateid', value: 'default'}); // For searchtemplateid, you must provide the internal ID of a custom
    wave criteria template.
31
32     wave.save();
33
34 }
35
36
37
38 function editWave() {
39
40     var wave = record.load({type: record.Type.WAVE, isDynamic: true, id: 346});
41
42     wave.setValue({fieldId: 'priority', value: '3'});
43
44     wave.setValue({fieldId: 'newwavestatus', value: 'RELEASED'});
45
46
47
48     wave.selectLine({sublistId: "picktasks", line: 0});
49
50     wave.setCurrentSublistValue({sublistId: "picktasks", fieldId: "picker", value: pickerId});
51
52     wave.commitLine({sublistId: "picktasks"});
53
54
55
56     wave.save();
57
58 }
59
60
61
62 function getFirstSalesOrderLineId() {
63
64     var salesOrder = record.load({type: record.Type.SALES_ORDER, id: salesOrderId});
65
66     return salesOrder.getSublistValue({sublistId: 'item', line: 0, fieldId: 'id'}).split('_')[1];
67
68 }
69
70
71
72 createWave();
73
74 // editWave();
75
76 });

```

Work Order

Applies to: SuiteScript 2.x | SuiteCloud Developer

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Work order transactions track the production of assembly items needed for stock or to fill orders. Work orders track the quantities of assemblies that need to be built and the quantities of components, or

member items, needed to do so. This type of transaction is available when the Assembly Items and Work Orders features are enabled and is used when the Allow Purchase of Assembly Items accounting preference is not enabled.

Special order work orders track assemblies for a particular sale, and Production work orders track assemblies to increase stock. Both use the same work order form, but Production work orders do not link to a sales transaction. Production work orders are generated when the back ordered quantity of an assembly reaches its assigned build point. After the build point is reached, a work order is added in the Mass Create Work Orders queue.

For help working with this record in the UI, see the help topic [Assembly Work Orders](#).

The internal ID for this record is `workorder`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The work order record is scriptable in both client and server SuiteScript.

Supported Functions

The work order record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

Fields

estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.

The Multi-Partner Management feature must be enabled in your account for the Partners sublist to appear.

Work Order Close

i Applies to: SuiteScript 2.x | SuiteCloud Developer

If the Manufacturing Work In Process (WIP) feature has been enabled, you can use SOAP web services to interact with work order close records. You can verify that WIP is enabled by going to Setup > Company > Enable Features, and reviewing the Items & Inventory subtab.

With WIP, instead of creating a single assembly build record to denote that a work order has been addressed, you track progress of the work using three records: work order issue, work order completion, and work order close. This approach lets you manage the assembly process in a more granular way, and to keep the General Ledger up to date as materials move through the different phases of assembly.

For details on the benefits of WIP, see the help topic [Manufacturing Work In Process \(WIP\)](#). For information about the process of closing a work order without WIP enabled, see the help topic [Marking Work Orders Closed](#).

For help working with this record in the UI, see the help topic [Entering Work Order Closes](#).

The internal ID for this record is `workorderclose`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The work order close record is scriptable in server SuiteScript only.

Supported Functions

The work order close record is partially scriptable. It can be updated, deleted, and searched using SuiteScript. It cannot be copied or created.

Usage Notes

To use this record you must have the following features enabled: Work Orders and Manufacturing Work in Process.

In the UI, this record is accessed by going to Transactions > Manufacturing > Close Work Order.

Note these additional details:

- You must use [record.transform\(options\)](#) to create a new instance of this record. In this case, `workorder` is the originating record type. For more details, see [Prerequisites for Creating a Work Order Issue Record](#).
- Assembly item should have scrap account, WIP account, and WIP Cost Variance Account specified.
- [record.create\(options\)](#) and [record.copy\(options\)](#) are not supported on this record.

Prerequisites for Creating a Work Order Issue Record

Before you can create a work order issue record, a work order record must already exist, and the work order must be configured to use WIP (the WIP box on the work order record must be selected). This is true regardless of whether you are creating the work order issue record using `initialize` and `add`, or `add by itself`. If you try to create a work order issue record referencing a work order that has not been configured to use WIP, the system generates an error reading in part, "One of the following problems exists: You have an invalid work order < work order ID >, the work order does not use WIP, or the work order is already closed." You can create and modify work orders by choosing **Transactions > Manufacturing > Enter Work Orders**.

You can also interact with work orders using SuiteScript, as described in [Work Order](#).

Note also that the assembly item referenced in the work order must be properly set up for WIP, as described in the [Setting Up Items as WIP Assemblies](#).

Work Order Completion

i Applies to: SuiteScript 2.x | SuiteCloud Developer

If the Manufacturing Work In Process (WIP) feature has been enabled, you can use SOAP web services to interact with work order completion records. You can verify that WIP is enabled by going to **Setup > Company > Enable Features**, and reviewing the **Items & Inventory** subtab.

With WIP, instead of creating a single assembly build record to denote that a work order has been addressed, you track progress of the work using three records: work order issue, work order completion, and work order close. This approach lets you manage the assembly process in a more granular way, and to keep the General Ledger up to date as materials move through the different phases of assembly. For more on the benefits of WIP, refer to [Manufacturing Work In Process \(WIP\)](#).

The work order completion record is used to indicate that assemblies have been built. You can also optionally use this record to record that raw materials — items in the `componentList` sublist — have been consumed as part of the assembly process. This latter option is called entering a completion with backflush. For example, you might enter a completion with backflush if previous records (such as work order issue) did not record the consumption of all the materials you ended up using.

In the UI, you can view the form used for creating the work order completion record by choosing **Transactions > Manufacturing > Enter Completions**, selecting a **Subsidiary** (for OneWorld accounts), then clicking **Complete** that corresponds with one of the listed work orders. An alternate method is to view the work order and click one of two buttons: **Enter Completions** or **Enter Completions With Backflush**.

For help working with this record in the UI, see the help topic [Entering Work Order Completions](#).

The internal ID for this record is `workordercompletion`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The work order completion record is scriptable in server SuiteScript only.

Supported Functions

The work order completion record is partially scriptable. It can be updated, deleted, and searched using SuiteScript. It cannot be copied or created.

Usage Notes

To use this record you must have the following features enabled: Work Orders and Manufacturing Work in Process.

In the UI, this record is accessed by going to Transactions > Manufacturing > Enter Completions.

Note these additional details:

- You must use [record.transform\(options\)](#) to create a new instance of this record. In this case, workorder is the originating record type. For more details, see [Prerequisites for Creating a Work Order Issue Record](#).
- Assembly item should have scrap account and WIP account specified.
- [record.create\(options\)](#) and [record.copy\(options\)](#) are not supported on this record.
- The Component sublist is available only when backflush = true

Prerequisites for Creating a Work Order Issue Record

Before you can create a work order issue record, a work order record must already exist, and the work order must be configured to use WIP (the WIP box on the work order record must be selected). This is true regardless of whether you are creating the work order issue record using initialize and add, or add by itself. If you try to create a work order issue record referencing a work order that has not been configured to use WIP, the system generates an error reading in part, "One of the following problems exists: You have an invalid work order < work order ID >, the work order does not use WIP, or the work order is already closed." You can create and modify work orders by choosing Transactions > Manufacturing > Enter Work Orders.

You can also interact with work orders using SuiteScript, as described in [Work Order](#).

Note also that the assembly item referenced in the work order must be properly set up for WIP, as described in the [Setting Up Items as WIP Assemblies](#).

Work Order Issue

Applies to: SuiteScript 2.x | SuiteCloud Developer

If the Manufacturing Work In Process (WIP) feature has been enabled, you can use SOAP web services to interact with work order completion records. You can verify if WIP is enabled by going to Setup > Company > Enable Features, and reviewing the Items & Inventory subtab.

With WIP, instead of creating a single assembly build record to denote that a work order has been addressed, you track progress of the work using three records: work order issue, work order completion, and work order close. This approach lets you manage the assembly process in a more granular way, and to keep the General Ledger up to date as materials move through the different phases of assembly. For more on the benefits of WIP, refer to [Manufacturing Work In Process \(WIP\)](#).

The work order issue record is used to indicate that particular quantities of raw materials, or component items, have been gathered for the production of the assembly item (or items).

In the UI, you can view the form used for creating the work order issue record by choosing Transactions > Manufacturing > Issue Components, selecting a Subsidiary (for OneWorld accounts), then clicking **Issue** that corresponds with one of listed work orders. An alternate method is to view the work order and click the Issue Components button.

For help working with this record in the UI, see the help topic [Entering Work Order Issues](#).

The internal ID for this record is workorderissue.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The work order issue record is scriptable in server SuiteScript only.

Supported Functions

The work order issue record is partially scriptable. It can be updated, deleted, and searched using SuiteScript. It cannot be copied or created.

Usage Notes

To use this record you must have the following features enabled: Work Orders and Manufacturing Work in Process.

In the UI, this record is accessed by going to Transactions > Manufacturing > Issue Components.

Note these additional details:

- You must use [record.transform\(options\)](#) to create a new instance of this record. In this case, workorder is the originating record type. For more details, see [Prerequisites for Creating a Work Order Issue Record](#).
- [record.create\(options\)](#) and [record.copy\(options\)](#) are not supported on this record.

Prerequisites for Creating a Work Order Issue Record

Before you can create a work order issue record, a work order record must already exist, and the work order must be configured to use WIP (the WIP box on the work order record must be selected). This is true regardless of whether you are creating the work order issue record using initialize and add, or add by itself. If you try to create a work order issue record referencing a work order that has not been configured to use WIP, the system generates an error reading in part, "One of the following problems exists: You have an invalid work order < work order ID >, the work order does not use WIP, or the work order is already closed." You can create and modify work orders by choosing Transactions > Manufacturing > Enter Work Orders.

You can also interact with work orders using SuiteScript, as described in [Work Order](#).

Note also that the assembly item referenced in the work order must be properly set up for WIP, as described in [Setting Up Items as WIP Assemblies](#).

Transactions-related Records

i Applies to: SuiteScript 2.x | SuiteCloud Developer

i Note: Processed lines for this record type do not contribute to the Monthly Transaction Lines metric that counts toward maximum limits for your NetSuite service tier. For more information, see Transaction Types Included in [Transaction Types Included in Monthly Transaction Lines Metric](#).

For information about transactions that do contribute to the Monthly Transaction Lines metric, see [Transactions](#).

- [Advanced Intercompany Journal Entry](#)
- [Balance Transactions by Segments](#)
- [Bin Putaway Worksheet](#)
- [Charge](#)
- [Change Order](#)
- [Charge](#)
- [Credit Card Charge](#)
- [Credit Card Refund](#)
- [Custom Transaction](#)
- [Customer Payment](#)
- [Demand Plan Detail Sublist](#)
- [GL Audit Numbering Sequence](#)
- [Intercompany Journal Entry](#)
- [Intercompany Transfer Order](#)
- [Inventory Detail](#)

- Item Demand Plan
- Item Supply Plan
- Journal Entry
- Landed Cost
- Manufacturing Operation Task
- Manufacturing Planned Time
- Memorized Transaction Definition
- Multi-Book Accounting Transaction
- Order Schedule
- Pick Strategy
- Pick Task
- Statistical Journal Entry
- Subscription
- Subscription Line
- Time
- Transaction Search
- Unlocked Time Period
- Usage
- Vendor Bill
- Vendor Credit
- Vendor Payment
- Weekly Timesheet

Advanced Intercompany Journal Entry

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

An advanced intercompany journal entry includes all the operations of the original journal entries, in addition to new time-saving functions. For SuiteScript details on the original intercompany journals, see [Intercompany Journal Entry](#).

In the UI, you can access this record at Transactions > Financial > Make Advanced Intercompany Journal Entries.

If your account has the Multi-Book Accounting feature enabled, you can also work with book-specific intercompany journal entry records, which are available at Transactions > Financial > Make Book Specific Advanced Intercompany Journal Entries. Although they have different entry forms, both book-specific and regular intercompany journal entries are the same record type. Within SuiteScript, they are differentiated by the accountingbook field. In other words, a record that has a value set for accountingbook is book-specific. Otherwise, the record is not a book-specific intercompany journal entry.

For help working with this record in the UI, see the help topics [Making Advanced Intercompany Journal Entries](#) and [Book-Specific Advanced Intercompany Journal Entries](#).

The internal ID for this record is advintercompanyjournalentry.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The intercompany journal entry record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The advanced intercompany journal entry record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Warning: If you update any type of journal entry that has been applied as a payment to an invoice or vendor bill, the relationship between the journal entry and payment is removed and the payment is no longer applied.

Code Samples

The following sample shows how to create an advanced intercompany journal entry.

```

1  var journalEntry = record.create({
2      type: record.Type.ADV_INTER_COMPANY_JOURNAL_ENTRY,
3      isDynamic: true
4  });
5  journalEntry.setValue({
6      fieldId: 'subsidiary',
7      value: 1
8  });
9  journalEntry.selectNewLine({
10     sublistId: 'line'
11 });
12 journalEntry.setCurrentSublistValue({
13     sublistId: 'line',
14     fieldId: 'subsidiary',
15     value: 1
16 });
17 journalEntry.setCurrentSublistValue({
18     sublistId: 'line',
19     fieldId: 'account',
20     value: 100
21 });
22 journalEntry.setCurrentSublistValue({
23     sublistId: 'line',
24     fieldId: 'debit',
25     value: '1.00'
26 });
27 journalEntry.commitLine({
28     sublistId: 'line'
29 });
30
31 journalEntry.selectNewLine({
32     sublistId: 'line'
33 });
34 journalEntry.setCurrentSublistValue({

```

```

35     sublistId: 'line',
36     fieldId: 'subsidiary',
37     value: 1
38   });
39   journalEntry.setCurrentSublistValue({
40     sublistId: 'line',
41     fieldId: 'account',
42     value: '101'
43   });
44   journalEntry.setCurrentSublistValue({
45     sublistId: 'line',
46     fieldId: 'credit',
47     value: '1.00'
48   });
49   journalEntry.commitLine({
50     sublistId: 'line'
51   });
52
53   journalEntry.selectNewLine({
54     sublistId: 'line'
55   });
56   journalEntry.setCurrentSublistValue({
57     sublistId: 'line',
58     fieldId: 'subsidiary',
59     value: 2
60   });
61   journalEntry.setCurrentSublistValue({
62     sublistId: 'line',
63     fieldId: 'account',
64     value: '200'
65   });
66   journalEntry.setCurrentSublistValue({
67     sublistId: 'line',
68     fieldId: 'debit',
69     value: '1.00'
70   });
71   journalEntry.commitLine({
72     sublistId: 'line'
73   });
74
75   journalEntry.selectNewLine({
76     sublistId: 'line'
77   });
78   journalEntry.setCurrentSublistValue({
79     sublistId: 'line',
80     fieldId: 'subsidiary',
81     value: 2
82   });
83   journalEntry.setCurrentSublistValue({
84     sublistId: 'line',
85     fieldId: 'account',
86     value: '201'
87   });
88   journalEntry.setCurrentSublistValue({
89     sublistId: 'line',
90     fieldId: 'credit',
91     value: '1.00'
92   });
93   journalEntry.commitLine({
94     sublistId: 'line'
95   });

```

Balance Transactions by Segments

Applies to: SuiteScript 2.x | SuiteCloud Developer

The balance transactions by segments record lets you select the criteria for transactions to include in the balancing by segments process. This process generates one or more balancing journals to balance the debits and credits of the included transactions by balancing segment.

This record and process are part of the Balancing Segments feature. For information, see the help topic [Balancing Segments and Journals](#). The Custom Segments feature is a prerequisite for the Balancing Segments feature.

Before you begin working with this record programmatically, see the help topic [Setup for Balancing Segments](#).

For help working with this record in the UI, see the help topic [Balancing Transactions by Segment](#).

The internal ID for this record is `balancetrxbysgments`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The balance transactions by segments record is scriptable in server SuiteScript only.

Supported Functions

The balance transactions by segments record is partially scriptable. You can create, read, and search it using SuiteScript. You cannot update, copy, transform, or delete it.

Usage Notes

The Transactions and Results subtabs, visible in the UI in view mode, are not scriptable.

Code Sample

The following sample shows how to run the balancing by segments process in a NetSuite OneWorld account with the Multi-Book Accounting feature enabled.

```

1  /**
2   * @NApiVersion 2.x
3   */
4  require(['N/record'], function(record) {
5      var balancingTrxBysgments = record.create({
6          type: record.Type.BALANCE_TRX_BY_SEGMENTS,
7          isDynamic: true
8      });
9      balancingTrxBysgments.setValue({
10         fieldId: 'accountingbook',
11         value: [1, 2]
12     });
13     balancingTrxBysgments.setValue({
14         fieldId: 'subsidiary',
15         value: [3, 4]
16     });
17     balancingTrxBysgments.setValue({
18         fieldId: 'includechildren',
19         value: true

```

```

20 });
21 balancingTrxBySegments.setValue({
22     fieldId: 'postingperiod',
23     value: 64
24 });
25 balancingTrxBySegments.setValue({
26     fieldId: 'memo',
27     value: 'My Balancing'
28 });
29 balancingTrxBySegments.save();
30 });

```

Bin Putaway Worksheet

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Use the Bin Putaway Worksheet to print a list of bin numbers for items that need to be re-stocked in your warehouse or stock room.

Items are added to this list any time an item that uses bins has a greater total on-hand count than the combined on-hand count of its associated bins.

How the worksheet looks and functions depends on whether you use basic Bin Management or use Advanced Bin / Numbered Inventory Management.

For help working with this record in the UI, see the help topics [Printing Bin Putaway Worksheets](#) and [Updating Bin Putaway Worksheets](#).

The internal ID for this record is binworksheet.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The bin putaway worksheet record can only be scripted in dynamic mode. For details about dynamic scripting, see the following help topics:

- [SuiteScript 2.x Standard and Dynamic Modes](#)

Note that client (remote object) scripting does not support dynamic scripting.

Supported Functions

The bin putaway worksheet record is partially scriptable — it can be created, deleted, and searched using SuiteScript. It cannot be copied or updated.

Budget Machine

Applies to: SuiteScript 2.x | SuiteCloud Developer

A budget machine is a matrix machine containing amounts for budget categories and calendar months. Exposed budget machine macros implement logic of the header buttons and support a manipulation with the content of the machine. You can find budget machines in the Project, Project Task, and Project template records. Each record contains cost budget (cbudget) and billing budget (bbudget) machines. The record currently supports the [clearBudgetAmounts](#), [distributeBudgetTotalAmount](#), [selectAllBudgetLines](#), [setBudgetAmountsToCalculated](#), and [unselectAllBudgetLines](#) macros.

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

clearBudgetAmounts

Corresponding UI Buttons	Clear Values
Macro Description	Sets amounts to empty value for all selected lines.
Returns	This macro does not return any value.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The [unselectAllBudgetLines](#) macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
requisites.params.sublistId	string	required	Identifies a budget sublist (cbudget bbudget)	2021.1

Sample Return Objects

The following sample shows how to select all lines in Cost Budget and how to reset all amounts in Cost Budget.

```

1 require(['N/currentRecord'], function(currentRecord) {
2     var project = currentRecord.get();
3
4     // reset all amounts in Cost Budget
5     project.executeMacro({
6         id: 'clearBudgetAmounts',
7         params: {

```

```

8      sublistId: 'cbudget'
9    }
10  });
11  });

```

distributeBudgetTotalAmount

Corresponding UI Buttons	Distribute Total
Macro Description	Splits a total amount into all calendar months for all selected lines.
Returns	This macro does not return any value.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The `unselectAllBudgetLines` macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
<code>requisites.params.sublistId</code>	string	required	Identifies a budget sublist (cbudget bbudget)	2021.1

Sample Return Objects

The following sample shows how to use calculated values or distribute total amount.

```

1  require(['N/currentRecord'], function(currentRecord) {
2    var project = currentRecord.get();
3    // use calculated values / distribute total amount
4    project.executeMacro({
5      id: 'setBudgetAmountsToCalculated' // 'distributeBudgetTotalAmount'
6      params: {
7        sublistId: 'cbudget'
8      }
9    });
10 });

```

selectAllBudgetLines

Corresponding UI Buttons	Mark All
Macro Description	Checks all enabled boxes in the Select column.
Returns	This macro does not return any value.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .

Since	2021.1
-------	--------

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The selectAllBudgetLines macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
requisites.params.sublistId	string	required	Identifies a budget sublist (cbudget bbudget)	2021.1

Sample Return Objects

The following sample shows how to select all lines in Cost Budget.

```

1  require(['N/currentRecord'], function(currentRecord) {
2      var project = currentRecord.get();
3
4      // select all lines in Cost Budget
5      project.executeMacro({
6          id: 'selectAllBudgetLines',
7          params: {
8              sublistId: 'cbudget'
9          }
10     });
11 });

```

setBudgetAmountsToCalculated

Corresponding UI Buttons	Set to Calculated
Macro Description	Replaces amounts with calculated values for all selected lines.
Returns	This macro does not return any value.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The unselectAllBudgetLines macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
requisites.params.sublistId	string	required	Identifies a budget sublist (cbudget bbudget)	2021.1

Sample Return Objects

The following sample shows how to use calculated values or distribute total amount.

```

1 require(['N/currentRecord'], function(currentRecord) {
2   var project = currentRecord.get();
3   // use calculated values / distribute total amount
4   project.executeMacro({
5     id: 'setBudgetAmountsToCalculated' // 'distributeBudgetTotalAmount'
6     params: {
7       sublistId: 'cbudget'
8     }
9   });
10 });

```

unselectAllBudgetLines

Corresponding UI Buttons	Unmark All
Macro Description	Clears all enabled boxes in the Select column.
Returns	This macro does not return any value.
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The unselectAllBudgetLines macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
requisites.params.sublistId	string	required	Identifies a budget sublist (cbudget bbudget)	2021.1

Sample Return Objects

The following sample shows how to unselect all lines in Cost Budget and how to select some lines.

```

1 require(['N/currentRecord'], function(currentRecord) {
2   var project = currentRecord.get();
3
4   // unselect all lines in Cost Budget
5   project.executeMacro({
6     id: 'unselectAllBudgetLines',
7     params: {
8       sublistId: 'cbudget'
9     }
10  });
11
12  // select some lines
13  for (var line = 0; line < project.getLineCount({sublistId: 'cbudget'}); line++) {
14    if (...) {
15      project.setSublistValue({
16        sublistId: 'cbudget',
17        fieldId: 'selectline',
18        line: line,
19        value: true
20      });
21    }
22  }
23 }

```

22 | }

Change Order

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer

The change order record is used to transact changes to subscriptions. For example, a change order is required to activate a drafted subscription.

The change order record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab. When the feature is enabled, you can access a list of change order records in the UI by going to Transactions > Subscriptions > View Subscription Change Orders.

For help working with this record in the UI, see the help topic [SuiteBilling Change Orders](#).

The internal ID for this record is `subscriptionchangeorder`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [Conditions for Prospective Merge Creation](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The change order record is scriptable in server and client SuiteScript.

Supported Functions

The change order record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Default Values

When creating a subscription change order in SuiteScript, you must supply the default values like the following code sample:

```
1 | const changeOrder = recordModule.create({
2 |   type: recordModule.Type.SUBSCRIPTION_CHANGE_ORDER,
3 |   defaultValues: {
4 |     action: "ACTIVATE",
5 |     subscription: 208
6 |   }
7 | });
```

Charge

Applies to: SuiteScript 2.x | SuiteCloud Developer

The charge record is used to represent a single billable amount that a client must pay.

The charge record is available only when the Charge-Based Billing feature is enabled at Setup > Company > Enable Features, on the Transactions subtab. When the feature is enabled, you can access the charge record in the UI by choosing Transactions > Customers > Create Charges > List.

For help working with this record in the UI, see the help topic [Generating Charges](#).

The internal ID for this record is charge.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The charge record is scriptable in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The charge record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following sample shows how to create a charge record.

```

1 var charge = record.create({
2   type: record.Type.CHARGE
3 });
4 charge.setText({
5   fieldId: 'stage',
6   text: 'Ready'
7 });
8 charge.setValue({
9   fieldId: 'chargeType',
10  value: 'Time'
11 });
12 charge.setValue({
13   fieldId: 'billto',
14   value: '43'
15 });

```

```

16 charge.setValue({
17     fieldId: 'chargedate',
18     value: '6/10/2012'
19 });
20 charge.setValue({
21     fieldId: 'salesorder',
22     value: '122'
23 });
24 charge.setValue({
25     fieldId: 'salesorderonline',
26     value: '1'
27 });
28 charge.setValue({
29     fieldId: 'currency',
30     value: '1'
31 });
32 charge.setValue({
33     fieldId: 'billingitem',
34     value: '21'
35 });
36 charge.setValue({
37     fieldId: 'timerecord',
38     value: '2'
39 });
40 charge.setValue({
41     fieldId: 'description',
42     value: 'Charge description'
43 });
44 charge.setValue({
45     fieldId: 'rate',
46     value: '2'
47 });
48 charge.setValue({
49     fieldId: 'quantity',
50     value: '3'
51 });
52 charge.setValue({
53     fieldId: 'amount',
54     value: '5'
55 });
56 var recordId = charge.save();

```

Credit Card Charge

Applies to: SuiteScript 2.x | SuiteCloud Developer

The credit card charge record is available only when a credit card account exists in the system.

For help working with this record in the UI, see the help topic [Creating Accounts](#).

The internal ID for this record is `creditcardcharge`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The credit card charge record is scriptable in server SuiteScript only.

Supported Functions

The credit card charge record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The script user must have the Credit Card permission with Full permission level.

Code Samples

The following sample shows how to create a \$30 credit card charge.

```

1  var charge = record.create({
2      type: record.Type.CREDIT_CARD_CHARGE,
3      isDynamic: true
4  });
5
6  charge.setValue({
7      fieldId: 'entity',
8      value: '4'
9  });
10 charge.setValue({
11     fieldId: 'account',
12     value: '128' // Account must be created before (Type = Credit Card)
13 });
14 charge.setValue({
15     fieldId: 'exchangerate',
16     value: '1.00'
17 });
18 charge.setValue({
19     fieldId: 'postingperiod',
20     value: '45'
21 });
22 charge.setValue({
23     fieldId: 'memo',
24     value: 'MEMO'
25 });
26 charge.setValue({
27     fieldId: 'class',
28     value: '1'
29 });
30 charge.setValue({
31     fieldId: 'department',
32     value: '1'
33 });
34 charge.setValue({
35     fieldId: 'location',
36     value: '1'
37 });
38
39 charge.setSublistValue({
40     sublistId: 'expense',
41     fieldId: 'account',
42     line: 1,
43     value: '58'
44 });
45 charge.setSublistValue({
46     sublistId: 'expense',
47     fieldId: 'amount',

```



```

48     line: 1,
49     value: '30.00'
50 });
51 charge.setSublistValue({
52     sublistId: 'expense',
53     fieldId: 'memo',
54     line: 1,
55     value: 'MEMO2'
56 });
57 charge.setSublistValue({
58     sublistId: 'expense',
59     fieldId: 'class_',
60     line: 1,
61     value: '1'
62 });
63 charge.setSublistValue({
64     sublistId: 'expense',
65     fieldId: 'department',
66     line: 1,
67     value: '1'
68 });
69 charge.setSublistValue({
70     sublistId: 'expense',
71     fieldId: 'location',
72     line: 1,
73     value: '1'
74 });
75 charge.setSublistValue({
76     sublistId: 'expense',
77     fieldId: 'customer',
78     line: 1,
79     value: '3'
80 });
81 charge.setSublistValue({
82     sublistId: 'expense',
83     fieldId: 'isbillable',
84     line: 1,
85     value: 'F'
86 });
87 var recordId = charge.save();

```

Credit Card Refund

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The credit card refund record is available only when a credit card account exists in the system. For information about creating a credit card account, see the help topic [Creating Accounts](#).

For help working with this record in the UI, see the help topics [Issuing Credit Card Refunds](#) and [Credit Card Refund Import](#).

The internal ID for this record is `creditcardrefund`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The credit card refund record is scriptable in server SuiteScript only.

Supported Functions

The credit card refund record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

The script user must have the Credit Card Refund permission with Full permission level.

Code Samples

The following sample shows how to create a \$30 credit card refund.

```

1  var ccRefund = record.create({
2      type: record.Type.CREDIT_CARD_REFUND,
3      isDynamic: true
4  });
5  ccRefund.setValue({
6      fieldId: 'entity',
7      value: '4'
8  });
9  ccRefund.setValue({
10     fieldId: 'account',
11     value: '128' // Account must be created before (Type = Credit Card)
12 });
13 ccRefund.setValue({
14     fieldId: 'exchangerate',
15     value: '1.00'
16 });
17 ccRefund.setValue({
18     fieldId: 'postingperiod',
19     value: '45'
20 });
21 ccRefund.setValue({
22     fieldId: 'memo',
23     value: 'MEMO'
24 });
25 ccRefund.setValue({
26     fieldId: 'class',
27     value: '1'
28 });
29 ccRefund.setValue({
30     fieldId: 'department',
31     value: '1'
32 });
33 ccRefund.setValue({
34     fieldId: 'location',
35     value: '1'
36 });
37 ccRefund.setSublistValue({
38     sublistId: 'expense',
39     fieldId: 'account',
40     line: 1,
41     value: '58'
42 });
43 ccRefund.setSublistValue({
44     sublistId: 'expense',
45     fieldId: 'amount',
46     line: 1,
47     value: '30.00'

```

```

48 });
49 ccRefund.setSublistValue({
50     sublistId: 'expense',
51     fieldId: 'memo',
52     line: 1,
53     value: 'MEMO2'
54 });
55 ccRefund.setSublistValue({
56     sublistId: 'expense',
57     fieldId: 'class_',
58     line: 1,
59     value: '1'
60 });
61 ccRefund.setSublistValue({
62     sublistId: 'expense',
63     fieldId: 'department',
64     line: 1,
65     value: '1'
66 });
67 ccRefund.setSublistValue({
68     sublistId: 'expense',
69     fieldId: 'location',
70     line: 1,
71     value: '1'
72 });
73 ccRefund.setSublistValue({
74     sublistId: 'expense',
75     fieldId: 'customer',
76     line: 1,
77     value: '3'
78 });
79 ccRefund.setSublistValue({
80     sublistId: 'expense',
81     fieldId: 'isbillable',
82     line: 1,
83     value: 'F'
84 });
85 var recordId = ccRefund.save();

```

Customer Payment

i Applies to: SuiteScript 2.x | SuiteCloud Developer

When a customer makes a payment, record the payment and apply it to the appropriate invoice or cash sale. Applying a payment decreases the amount due and tracks income.

For help working with this record in the UI, see the help topic [Customer Payments](#).

The internal ID for this record is `customerpayment`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

! Important: When the apply sublist appears on the customer payment record, the Apply subtab is visible, however, all apply sublist data appears on a subtab called Invoice. Also, note that the UI label for the **Amt. Due** (due) field can also appear as **Amount Remaining**.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ [N/record Module](#)

Supported Script Types

The customer payment record is scriptable in server SuiteScript only.

Supported Functions

The customer payment record is partially scriptable — it can be updated, deleted, and searched using SuiteScript. It cannot be created or copied.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
ccnumber	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).
isrecurringpayment	Recurring Payment	A value for this field is stored only if the value for paymentmethod is a credit card.

The Deposits sublist is a **list** sublist.

Demand Plan Detail Sublist

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The internal ID for this sublist is **demandplandetail**.

The Demand Plan Detail sublist appears on the [Item Demand Plan](#) record type.

The Demand Plan Detail sublist is a matrix that is similar to the Pricing sublist. This matrix stores projected quantities demanded by date. Each row in the matrix represents a specific month, week, or day, and each column in the matrix represents an expected quantity demand.

For help working with this record in the UI, see the help topic [Demand Planning](#).

Functionally, this sublist shares many of the characteristics of List Sublists. However, scripting with the Demand Plan Detail sublist is not like scripting with most other sublists in NetSuite. You must use [Matrix APIs for the Demand Plan Detail Sublist](#) to access quantity values on a per-row, per-column basis, similar to the way that item pricing values are accessed. These APIs are a subset of the [N/currentRecord Module](#) and [N/record Module](#) APIs more commonly used for scripting with other sublists.

The format of the Demand Plan Detail sublist depends on the values set in body fields for the start date of the plan, the end date of the plan, and the time period to be used (monthly, weekly, or daily). Because of this dependence, you should work with the Item Demand Plan record and the Demand Plan Detail sublist in dynamic mode. See the help topic [SuiteScript 2.x Standard and Dynamic Modes](#)

Be aware of the following requirements:

- To script with the Item Demand Plan record and the Demand Plan Detail sublist for inventory items, the Demand Planning feature must be enabled. For assembly/BOM items, the Work Orders feature also must be enabled.
- Demand plans are supported only for item(s) that have the **supplyreplenishmentmethod** field set to Time Phased.
- Required body field values must be defined before matrix field values can be edited. In dynamic mode, current values may be retrieved. Start date and end date body fields default to the first day and last day of the current year.

For more details and code samples for each type of demand plan, see the following help topics:

- [Monthly Demand Plan](#)
- [Weekly Demand Plan](#)
- [Daily Demand Plan](#)

Monthly Demand Plan

A monthly demand plan includes a row for each month within the body field start date and end date, and one quantity column for each month.

Manual Item Demand Plan

ListSearchCustomizeMore

SaveCancel

Primary Information

Location*

Unit of Measure

Item*

Memo

Year

Month

Start Date

End Date

View

2024

1.1.2024

31.12.2024

Monthly

Start Date	End Date	Monthly Calculated	Quantity
1.1.2024	31.1.2024		
1.2.2024	29.2.2024		
1.3.2024	31.3.2024		
1.4.2024	30.4.2024		
1.5.2024	31.5.2024		
1.6.2024	30.6.2024		
1.7.2024	31.7.2024		
1.8.2024	31.8.2024		
1.9.2024	30.9.2024		
1.10.2024	31.10.2024		
1.11.2024	30.11.2024		
1.12.2024	31.12.2024		

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the month represented by each row.
 - The enddate is the date of the last day of the month represented by each row.
 - The month for row 1 is the month set in the body field start date, the month for row 2 is the next month, and so on, until the month set in the body field end date is reached.

- The values for the quantity field can be set in SuiteScript. For monthly demand plans, the column parameter for this field is always 1.

Monthly Demand Plan Code Sample

The following sample sets quantities for the months of January and February, 2011:

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
6    const demandplan = record.create({
7      type: record.Type.ITEM_DEMAND_PLAN,
8      isDynamic: true
9    })
10   demandplan.setValue({
11     fieldId: 'demandplancalendartype',
12     value: 'MONTHLY'
13   })
14   demandplan.setValue({
15     fieldId: 'subsidiary',
16     value: 1
17   })
18   demandplan.setValue({
19     fieldId: 'location',
20     value: 1
21   })
22   demandplan.setValue({
23     fieldId: 'item',
24     value: 253
25   })
26   const id = demandplan.save({
27     enableSourcing: true
28   })
29
30   const loadDemandPlan = record.load({
31     type: record.Type.ITEM_DEMAND_PLAN,
32     id: id,
33     isDynamic: true
34   })
35   loadDemandPlan.setValue({
36     fieldId: 'startdate',
37     value: '1/1/2023'
38   })
39   loadDemandPlan.setValue({
40     fieldId: 'enddate',
41     value: '12/31/2023'
42   })
43   loadDemandPlan.selectLine({
44     sublistId: 'demandplandetail',
45     line: '0'
46   })
47   loadDemandPlan.setCurrentSublistValue({
48     sublistId: 'demandplandetail',
49     fieldId: 'quantity_1_',
50     value: 100
51   })
52   loadDemandPlan.selectLine({
53     sublistId: 'demandplandetail',
54     line: '1'
55   })
56   loadDemandPlan.setCurrentSublistValue({
57     sublistId: 'demandplandetail',
58     fieldId: 'quantity_1_',
59     value: 200
60   })
61   loadDemandPlan.commitLine({
62     sublistId: 'demandplandetail'
63   })

```

```
64 | loadDemandPlan.save()  
65 | })
```

Weekly Demand Plan

A weekly demand plan includes a row for each week contained in the time period set by the body field start date and end date, and one quantity column for each week.

Manual Item Demand Plan

ListSearchCustomizeMore

SaveCancel

Primary Information

Location*

Unit of Measure

Item*

Memo

YearMonthStart DateEnd DateView

202411.202411.202431.12.2024Weekly

Start Date	End Date	Weekly Calculated	Quantity
11.2024	71.2024		
8.1.2024	14.1.2024		
15.1.2024	21.1.2024		
22.1.2024	28.1.2024		
29.1.2024	4.2.2024		
5.2.2024	11.2.2024		
12.2.2024	18.2.2024		
19.2.2024	25.2.2024		
26.2.2024	3.3.2024		
4.3.2024	10.3.2024		
11.3.2024	17.3.2024		
18.3.2024	24.3.2024		
25.3.2024	31.3.2024		

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the week represented by each row.
 - The enddate is the date of the last day of the week represented by each row.
- 

Note: The first day of the week by default is Sunday, but may vary according to the company preference set for First Day of the Week at Setup > Company > General Preferences.
- The week for row 1 is the week of the date set in the body field start date. Note that unless the body field start date happens to be the first day of the week, the startdate for this first row may precede the body field start date.
- The week for the final sublist row is the week of the date set in the body field end date. Note that unless the body field end date happens to be the last day of the week, the enddate for this last row may be after the body field enddate.
- The values for the quantity field can be set in SuiteScript. For weekly demand plans, the column parameter for this field is always 1.

Weekly Demand Plan Code Sample

The following sample sets quantities for the first two weeks of 2011:

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
6    const demandplan = record.create({
7      type: record.Type.ITEM_DEMAND_PLAN,
8      isDynamic: true
9    })
10   demandplan.setValue({
11     fieldId: 'demandplancalendartype',
12     value: 'WEEKLY'
13   })
14   demandplan.setValue({
15     fieldId: 'subsidiary',
16     value: 1
17   })
18   demandplan.setValue({
19     fieldId: 'location',
20     value: 1
21   })
22   demandplan.setValue({
23     fieldId: 'item',
24     value: 252
25   })
26   const id = demandplan.save({
27     enableSourcing: true
28   })
29
30   const loadDemandPlan = record.load({
31     type: record.Type.ITEM_DEMAND_PLAN,
32     id: id,
33     isDynamic: true
34   })
35   loadDemandPlan.setValue({
36     fieldId: 'startdate',
37     value: '1/1/2023'
38   })
39   loadDemandPlan.setValue({
40     fieldId: 'enddate',
41     value: '12/31/2023'
42   })
43   loadDemandPlan.selectLine({
44     sublistId: 'demandplandetail',
45     line: '0'
46   })
47   loadDemandPlan.setCurrentSublistValue({
48     sublistId: 'demandplandetail',
49     fieldId: 'quantity_1_',
50     value: 100
51   })
52   loadDemandPlan.selectLine({
53     sublistId: 'demandplandetail',
54     line: '1'
55   })
56   loadDemandPlan.setCurrentSublistValue({
57     sublistId: 'demandplandetail',
58     fieldId: 'quantity_1_',
59     value: 200
60   })
61   loadDemandPlan.commitLine({
62     sublistId: 'demandplandetail'
63   })
64   loadDemandPlan.save()
65 })

```

Daily Demand Plan

A daily demand plan includes a row for each week contained in the time period set by the body field start date and end date, and seven quantity columns for each week, one for each day of the week.

Manual Item Demand Plan

ListSearchCustomizeMore

SaveCancel

▼ Primary Information

Location*

Unit of Measure

Item*

Memo

Year: 2024Month: Start Date: 11.2024End Date: 31.12.2024View: Daily

Start Date	End Date	Weekly Calculated	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
11.2024	7.1.2024								
8.1.2024	14.1.2024								
15.1.2024	21.1.2024								
22.1.2024	28.1.2024								
29.1.2024	4.2.2024								
5.2.2024	11.2.2024								
12.2.2024	18.2.2024								
19.2.2024	25.2.2024								
26.2.2024	3.3.2024								
4.3.2024	10.3.2024								
11.3.2024	17.3.2024								
18.3.2024	24.3.2024								
25.3.2024	31.3.2024								

- The sublist startdate and enddate fields are system-calculated and read-only.
 - The startdate is the date of the first day of the week represented by each row.
 - The enddate is the date of the last day of the week represented by each row.
- Note:

The first day of the week by default is Sunday, but may vary according to the company preference set for First Day of the Week at Setup > Company > General Preferences.
- The week for row 1 is the week of the date set in the body field start date. Note that unless the body field start date happens to be the first day of the week, the startdate for this first row may precede the body field start date.
 - The week for the final sublist row is the week of the date set in the body field end date. Note that unless the body field end date happens to be the last day of the week, the enddate for this last row may be after the body field enddate.
- The values for the quantity fields can be set in SuiteScript.
 - The column parameter for a quantity field is 1,2,3,4,5,6, or 7, depending on the day of the week.
 - In the screenshot above, the week starts with Sunday, which is the default first day of the week, and in this case, maps to a column parameter of 1. However, 1 does not always map to Sunday; it maps to the first day of the week as set in the company preferences.

Daily Demand Plan Code Sample

```
1  /**
2   * @NApiVersion 2.1
3   */
4
5  require(['N/record', 'N/log'], (record, log) => {
```

```

6   const demandplan = record.create({
7     type: record.Type.ITEM_DEMAND_PLAN,
8     isDynamic: true
9   })
10  demandplan.setValue({
11    fieldId: 'demandplancalendartype',
12    value: 'DAILY'
13  })
14  demandplan.setValue({
15    fieldId: 'subsidiary',
16    value: 1
17  })
18  demandplan.setValue({
19    fieldId: 'location',
20    value: 1
21  })
22  demandplan.setValue({
23    fieldId: 'item',
24    value: 254
25  })
26  const id = demandplan.save({
27    enableSourcing: true
28  })
29
30  const loadDemandPlan = record.load({
31    type: record.Type.ITEM_DEMAND_PLAN,
32    id: id,
33    isDynamic: true
34  })
35  loadDemandPlan.setValue({
36    fieldId: 'startdate',
37    value: '1/1/2023'
38  })
39  loadDemandPlan.setValue({
40    fieldId: 'enddate',
41    value: '12/31/2023'
42  })
43  loadDemandPlan.selectLine({
44    sublistId: 'demandplandetail',
45    line: '0'
46  })
47  loadDemandPlan.setCurrentSublistValue({
48    sublistId: 'demandplandetail',
49    fieldId: 'quantity_1_',
50    value: 100
51  })
52  loadDemandPlan.selectLine({
53    sublistId: 'demandplandetail',
54    line: '1'
55  })
56  loadDemandPlan.setCurrentSublistValue({
57    sublistId: 'demandplandetail',
58    fieldId: 'quantity_1_',
59    value: 200
60  })
61  loadDemandPlan.commitLine({
62    sublistId: 'demandplandetail'
63  })
64  loadDemandPlan.save()
65 })

```

Matrix APIs for the Demand Plan Detail Sublist

Use the following matrix APIs with the Demand Plan Detail sublist:

- [Record.getCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.getCurrentMatrixSublistValue\(options\)](#)
- [Record.setCurrentMatrixSublistValue\(options\)](#)
- [CurrentRecord.setCurrentMatrixSublistValue\(options\)](#)

Note: With all APIs listed above, use the [Record.selectLine\(options\)](#) or [CurrentRecord.selectLine\(options\)](#) APIs first to select an existing line.

- [Record.getMatrixSublistField\(options\)](#)
- [CurrentRecord.getMatrixSublistField\(options\)](#)
- [Record.getMatrixSublistValue\(options\)](#)
- [CurrentRecord.getMatrixSublistValue\(options\)](#)
- [Record.findMatrixSublistLineWithValue\(options\)](#)
- [CurrentRecord.findMatrixSublistLineWithValue\(options\)](#)

For more information about APIs, see the help topics [N/currentRecord Module](#) and [N/record Module](#).

GL Audit Numbering Sequence

Applies to: SuiteScript 2.x | SuiteCloud Developer

The GL Audit Numbering feature enables you to set up and run two types of GL audit numbering sequences: permanent and repeatable. When you run a permanent GL audit numbering sequence, the number assigned to a GL impacting transaction cannot be changed. You can, however, modify the details of the original GL impacting transaction. When you run a repeatable GL audit numbering sequence, you can re-run the numbering sequence on GL impacting transactions as often as required to address gaps in numbering that might occur due to adjustments made to your GL.

Note: Re-running a repeatable numbering sequence rennumbers transactions and may assign a number different from the previously assigned number.

In the UI, you can access this record at Transactions > Management > GL Audit Numbering Sequences > New. This page includes fields to set up a GL audit numbering system, including the sequence name, numbering type, available subsidiaries, and the numbering format.

For help working with this record in the UI, see the help topic [Setting Up a GL Audit Numbering Sequence](#).

To use this record in scripts, you must be using the GL Audit Numbering feature and the Manage Accounting Period permission.

Note: The System Notes sublist and the Next Run field in the Recurring subtab for Permanent Numbering are always search-only.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The GL audit numbering sequence record is scriptable in both client and server SuiteScript.

Supported Functions

The GL audit numbering sequence record can be read, created, updated, copied, and searched using SuiteScript.

Code Samples

The following sample shows how to create a new GL audit numbering sequence. The sequencename, period, subsidiaries, numberingtype, accountingbook, initnum, ordertype, and recurringtype fields must all be defined.

```

1  var newGLSequence = record.create({
2      type: record.Type.GL_NUMBERING_SEQUENCE
3  });
4  newGLSequence.setValue({
5      fieldId: 'sequencename',
6      value: 'My sequence'
7  });
8  newGLSequence.setValue({
9      fieldId: 'period',
10     value: 1 // The ID of the accounting period to which this sequence belongs
11 });
12 newGLSequence.setValue({
13     fieldId: 'subsidiaries',
14     value: 1 // The ID of the subsidiary to which this sequence belongs
15 });
16 newGLSequence.setValue({
17     fieldId: 'numberingtype',
18     value: 'REPEATABLE' // Can be set to either 'REPEATABLE' or 'PERMANENT'
19 });
20 newGLSequence.setValue({
21     fieldId: 'accountingbook',
22     value: 1 // ID of the accounting book to which this sequence belongs
23 });
24 newGLSequence.setValue({
25     fieldId: 'initnum',
26     value: 1 // Typically 1 - the starting number of GL numbering
27 });
28 newGLSequence.setValue({
29     fieldId: 'ordertype',
30     value: 'CREATEDDATE' // Can be set to either 'CREATEDDATE' or 'TRANDATE'
31 });
32 newGLSequence.setValue({
33     fieldId: 'recurringtype',
34     value: 'NONE' // Can be set to either 'NONE', 'HOURLY', or 'DAILY'
35 });
36 var id = newGLSequence.save(); // On submission, the system will check all the rules and constraints on
37                               // required fields and produce an error if something is incorrect.

```

Intercompany Journal Entry

Applies to: SuiteScript 2.x | SuiteCloud Developer

Intercompany journal entries are a specialized type of journal available specifically for OneWorld. An intercompany journal entry records debits and credits to be posted to ledger accounts for transactions between two subsidiaries. These records adjust the value of any set of accounts without the need for transactions such as invoices and bills.

In the UI, you can access this record at Transactions > Financial > Make Intercompany Journal Entries.

If your account has the Multi-Book Accounting feature enabled, you can also work with book specific intercompany journal entry records, which in the UI are available at Transactions > Financial > Make Book Specific Intercompany Journal Entries. Although they have different entry forms, both book specific and regular intercompany journal entries are the same record type. Within SuiteScript, they are differentiated

by the accountingbook field. In other words, a record that has a value set for accountingbook is book specific. Otherwise, the record is a regular intercompany journal entry.

For help working with this record in the UI, see the help topics [Making Intercompany Journal Entries](#) and [Book-Specific Intercompany Journal Entries](#).

The internal ID for this record is intercompanyjournalentry.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The intercompany journal entry record is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The intercompany journal entry record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

 **Warning:** If you update any type of journal entry that has been applied as a payment to an invoice or vendor bill, the relationship between the journal entry and payment is removed and the payment is no longer applied.

Code Samples

The following sample shows how to create a book specific intercompany journal entry. The record is book specific because a value has been set for the accountingbook field.

```

1 var journalEntry = record.create({
2   type: record.Type.INTER_COMPANY_JOURNAL_ENTRY,
3   isDynamic: true
4 });
5 journalEntry.setValue({
6   fieldId: 'accountingbook',
7   value: 2 // Setting a value for this field makes the record book-specific.
8 });
9 journalEntry.setValue({
10  fieldId: 'subsidiary',
11  value: 1
12 });
13 journalEntry.setValue({
14  fieldId: 'tosubsidiary',
15  value: 3
16 });
17
18 journalEntry.selectNewLine({
19  sublistId: 'line'
20 });
21 journalEntry.setCurrentSublistValue({
22  sublistId: 'line',
23  fieldId: 'linesubsidiary',
24  value: 1

```

```

25 });
26 journalEntry.setCurrentSublistValue({
27     sublistId: 'line',
28     fieldId: 'account',
29     value: 1
30 });
31 journalEntry.setCurrentSublistValue({
32     sublistId: 'line',
33     fieldId: 'credit',
34     value: '2.00'
35 });
36 journalEntry.commitLine({
37     sublistId: 'line'
38 });
39
40 journalEntry.selectNewLine({
41     sublistId: 'line'
42 });
43 journalEntry.setCurrentSublistValue({
44     sublistId: 'line',
45     fieldId: 'linesubsidiary',
46     value: 1
47 });
48 journalEntry.setCurrentSublistValue({
49     sublistId: 'line',
50     fieldId: 'account',
51     value: 2
52 });
53 journalEntry.setCurrentSublistValue({
54     sublistId: 'line',
55     fieldId: 'debit',
56     value: '2.00'
57 });
58 journalEntry.commitLine({
59     sublistId: 'line'
60 });
61 journalEntry.selectNewLine({
62     sublistId: 'line'
63 });
64 journalEntry.setCurrentSublistValue({
65     sublistId: 'line',
66     fieldId: 'linesubsidiary',
67     value: 3
68 });
69 journalEntry.setCurrentSublistValue({
70     sublistId: 'line',
71     fieldId: 'account',
72     value: 6
73 });
74 journalEntry.setCurrentSublistValue({
75     sublistId: 'line',
76     fieldId: 'credit',
77     value: '2.00'
78 });
79 journalEntry.commitLine({
80     sublistId: 'line'
81 });
82
83 journalEntry.selectNewLine({
84     sublistId: 'line'
85 });
86 journalEntry.setCurrentSublistValue({
87     sublistId: 'line',
88     fieldId: 'linesubsidiary',
89     value: 3
90 });
91 journalEntry.setCurrentSublistValue({
92     sublistId: 'line',
93     fieldId: 'account',
94     value: 149
95 });
96 journalEntry.setCurrentSublistValue({
97     sublistId: 'line',

```

```

98     fieldId: 'debit',
99     value: '2.00'
100  });
101  journalEntry.commitLine({
102      sublistId: 'line'
103  });
104
105  var recordId = journalEntry.save();

```

Intercompany Transfer Order

Applies to: SuiteScript 2.x | SuiteCloud Developer

In accounts using NetSuite OneWorld and the Multi-Location Inventory (MLI) feature, you can use the Intercompany Transfer Order transaction to move inventory from a location for one subsidiary to a location for another subsidiary.

In the UI, you can access this record at Transactions > Inventory > Enter Intercompany Transfer Orders..

For help working with this record in the UI, see the help topic [Intercompany Inventory Transfers - Non-Arm's Length](#).

The internal ID for this record is `intercompanytransferorder`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The intercompany transfer order record is scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The intercompany transfer order record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript. It can also be transformed.

Usage Notes

Intercompany transfer orders are only available when transfer pricing is used, meaning the Use Item Cost as Transfer Cost, at Setup > Accounting > Preferences > Accounting Preferences, on the Order Management subtab, must be disabled.

This record has available transforms. See the SuiteScript Records Browser for available transforms. In the NetSuite Help Center, see the help topic [record.transform\(options\)](#) for examples on how to transform records.

Code Samples

The following sample shows how to create an intercompany transfer order.

```

1 var incoTransfer = record.create({
2   type: record.Type.INTER_COMPANY_TRANSFER_ORDER,
3   isDynamic: true
4 });
5 incoTransfer.setValue({
6   fieldId: 'orderstatus',
7   value: expectedValues.get(orderstatus) //expectedValues is previously defined
8 });
9 incoTransfer.setValue({
10  fieldId: 'subsidiary',
11  value: expectedValues.get(subsidiary) //expectedValues is previously defined
12 });
13 incoTransfer.setValue({
14  fieldId: 'tosubsidiary',
15  value: expectedValues.get(tosubsidiary) //expectedValues is previously defined
16 });
17 incoTransfer.setText({
18  fieldId: 'location',
19  text: sourceLocationName1
20 });
21 incoTransfer.setText({
22  fieldId: 'transferlocation',
23  text: destinationLocationName1
24 });
25 incoTransfer.selectNewLine({
26  sublistId: 'item'
27 });
28 incoTransfer.setCurrentSublistValue({
29  sublistId: 'item',
30  fieldId: 'Item.item',
31  value: itemKey
32 });
33 incoTransfer.setCurrentSublistValue({
34  sublistId: 'item',
35  fieldId: 'Item.rate',
36  value: rate
37 });
38 incoTransfer.setCurrentSublistValue({
39  sublistId: 'item',
40  fieldId: 'quantity',
41  value: quantity
42 });
43 incoTransfer.commitLine({
44  sublistId: 'item'
45 });
46 var key = incoTransfer.save();

```

Inventory Detail

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The inventory detail subrecord is available when the Advanced Bin / Numbered Inventory Management feature is enabled. This type of record stores values relating to bin numbers and serial/lot numbers for items, including line items on transactions. This data includes the quantity on hand and quantity available per bin number, or per serial/lot number, or when both are in use, per bin number/serial lot number combination.

In the UI, the inventory detail subrecord displays as a popup when you click the Inventory Detail body field or sublist field.

For help working with this record in the UI, see the help topic [Advanced Bin / Numbered Inventory Management](#).

The internal ID for this subrecord is `inventorydetail`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inventory detail record is scriptable in client SuiteScript. Server scripts must access through the parent record.

Supported Functions

The inventory detail record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

Inventory Detail is scriptable from both the body field and the line item.

Inventory Detail is considered a subrecord in SuiteScript. For details on working with this subrecord type, see the help topic [Creating an Inventory Detail Sublist Subrecord Example](#). For general details on working with subrecords, see the help topic [SuiteScript 2.x Scripting Subrecords](#).

Item Demand Plan

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The item demand plan record is available for inventory items when the Demand Planning feature is enabled, and for assembly/BOM items when both the Demand Planning and Work Orders features are enabled. Item demand plans can be added for items where the `supplyReplenishMethod` field is set to Time Phased.

An item demand plan transaction stores the quantity expected to be needed, during specified time periods, for an item. NetSuite supports three types of demand plans: monthly, weekly, and daily.

For help working with this record in the UI, see the help topic [Demand Planning](#).

The internal ID for this record is `itemdemandplan`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item demand plan record is scriptable in both client and server SuiteScript.

Supported Functions

The invoice record is partially scriptable — it can be created, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Usage Notes

A demand plan records the expected future demand for an item based on previous or projected demand. When the Demand Planning feature is enabled, demand plans can be created for inventory items. When the Work Orders feature is also enabled, demand plans also can be created for assembly/BOM items. Demand plans can only be created for items that have a value of “Time Phased” for the **supplyreplenishmethod** field.

Each demand plan record includes:

- A set of body fields used to uniquely identify the demand plan, define the time period it covers, and indicate the time period it uses (monthly, weekly, or daily).
Body fields must be defined before matrix field values can be edited.
- A matrix of projected quantities per time period, similar to the matrix used for item pricing.
 - In a monthly demand plan, this matrix includes a row for each month in the time period, and one column with the projected quantity demand for each month.
 - In a weekly demand plan, this matrix includes a row for each week in the time period, and one column with the projected quantity demand for each week.
 - In a daily demand plan, this matrix includes a row for each week in the time period and seven columns with the projected quantity demand for each day of each week.
- Review the following table for details about Item Demand Plan body and matrix sublist fields. For more details and code samples, see [Demand Plan Detail Sublist](#).

Field Internal ID	Field UI Label	Note
Body Fields		
subsidiary	Subsidiary	Required in OneWorld accounts.
location	Location	Required when the Multi-Location Inventory feature enabled.
item	Item	Required. Can only use items with supplyreplenishment method set to Time Phased.
units	Unit of Measure	Optional. Available when the Multiple Units of Measure feature is enabled.
memo	Memo	Optional.
startdate	Start Date	Optional. Defaults to the first day of the current year, for example for 2011, defaults to 1/1/2011.
enddate	End Date	Optional. Defaults to the last day of the current year, for example for 2011, defaults to 12/31/2011.

Field Internal ID	Field UI Label	Note
demandplancalendarstype	View	Required. Valid values are MONTHLY, WEEKLY, or DAILY. (Must use all capital letters.)
Matrix Fields		
quantity	Quantity	- For monthly and weekly demand plans, each row has one quantity column. - For daily demand plans, each row has seven quantity columns.
startdate	Start Date	System-calculated, read-only values. - For monthly plans, the date of the first day of the month that the row represents. - For weekly and daily plans, the date of the first day of the week that the row represents, based on the preference set for First Day of Week at Setup > Company > General Preferences.
enddate	End Date	System-calculated, read-only values. - For monthly plans, the date of the last day of the month that the row represents. - For weekly and daily plans, the date of the last (seventh) day of the week that the row represents, based on the preference set for First Day of Week at Setup > Company > General Preferences.

Note: You should work with the Item Demand Plan record in dynamic mode. See the help topic [SuiteScript 2.x Standard and Dynamic Modes](#).

Item Supply Plan

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `itemsupplyplan`.

This record includes the Orders sublist.

For help working with this record in the UI, see the help topics [Creating Item Supply Plans](#) and [Item Supply Plan Import](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item supply plan is scriptable in both client and server SuiteScript.

Supported Functions

The item supply plan record is fully scriptable — it can be created, copied, updated, deleted, and searched using SuiteScript.

Usage Notes

The item, location, and units body fields cannot be changed in update operations.

An item supply plan's receiptdate cannot be earlier than the orderdate.

The ordercreated field is read-only. It is set to True when an order is generated from an item supply plan.

Code Samples

The following sample creates an item supply plan.

```

1 function createItemSupplyPlanMinimal() {
2     var itemsupplyplan = record.create({
3         type: record.Type.ITEM_SUPPLY_PLAN,
4         isDynamic: true
5     })
6     itemsupplyplan.setValue({
7         fieldId: 'subsidiary',
8         value: 1
9     });
10    itemsupplyplan.setValue({
11        fieldId: 'location',
12        value: 1
13    });
14    itemsupplyplan.setValue({
15        fieldId: 'item',
16        value: 165
17    });
18
19    itemsupplyplan.setValue({
20        fieldId: 'memo',
21        value: 'memotest'
22    });
23    itemsupplyplan.setValue({
24        fieldId: 'unit',
25        value: 3
26    });
27
28    itemsupplyplan.selectNewLine({
29        sublistId: 'order'
30    });
31    itemsupplyplan.setCurrentSublistValue({
32        sublistId: 'order',
33        fieldId: 'orderdate',
34        value: '05/05/2012'
35    });
36    itemsupplyplan.setCurrentSublistValue({
37        sublistId: 'order',
38        fieldId: 'receiptdate',
39        value: '5/8/2012'
40    });
41    itemsupplyplan.setCurrentSublistValue({
42        sublistId: 'order',
43        fieldId: 'quantity',
44        value: 1
45    });
46    itemsupplyplan.setCurrentSublistValue({
47        sublistId: 'order',
48        fieldId: 'ordertype',
49        value: 'PurchOrd'
50    });
51
52    itemsupplyplan.commitLine({

```

```

53     sublistId: 'order'
54   });
55
56   var id = itemsupplyplan.save();
57 }

```

The following sample updates an existing item supply plan.

```

1 function updateItemSupplyPlan()
2 {
3   var itemsupplyplan = record.load({
4     type: record.Type.ITEM_SUPPLY_PLAN,
5     id: 3
6   });
7   itemsupplyplan.setValue({
8     fieldId: 'memo',
9     value: 'memotest2'
10  });
11
12   itemsupplyplan.setSublistValue({
13     sublistId: 'order',
14     fieldId: 'receiptdate',
15     line: 4,
16     value: '11/3/2012'
17   });
18   var id = itemsupplyplan.save();
19 }

```

Journal Entry

Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the journal entry record to adjust balances in your ledger accounts without entering posting transactions.

In the UI, you access this record in the UI at Transactions > Financial > Make Journal Entries.

If your account has the Multi-Book Accounting feature enabled, you can also work with book specific journal entry records, which are available in the UI at Transactions > Financial > Make Book Specific Journal Entries. Although they have different entry forms, both book specific and regular intercompany journal entries are the same record type. Within SuiteScript, they are differentiated by the accountingbook field. In other words, a record that has a value set for accountingbook is book specific. Otherwise, the record is a regular intercompany journal entry.

For help working with this record in the UI, see the help topics [Making Intercompany Journal Entries](#) and [Book-Specific Journal Entries](#).

The internal ID for this record is journalentry.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The journal entry record is scriptable in both client SuiteScript and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The journal entry record is fully scriptable — it can be created, copied, updated, deleted, and searched using SuiteScript.



Warning: If you update any type of journal entry that has been applied as a payment to an invoice or vendor bill, the relationship between the journal entry and payment is removed and the payment is no longer applied.

Usage Notes

With the Advanced Revenue Management (Essentials) feature, you can directly attach a revenue recognition plan to a book specific journal entry or an intercompany journal entry. Before you begin working with advanced revenue management programmatically, see the help topic [Setup for Advanced Revenue Management \(Essentials\)](#).

The following table lists the scriptable field associated with this record and advanced revenue management (essentials).

Field	Type	Internal ID
End Date	Date	enddate
Revenue Recognition Rule	List/Record	revenuerecognitionrule
Start Date	Date	startdate

For help working with this record in the UI, see the help topic [Creating Revenue Elements from Journal Entries](#)

Code Samples

The following sample shows how to create a book-specific journal entry. The record is book-specific because the parameter bookje has been set to T.

```

1 var journalEntry = record.create({
2   type: record.Type.JOURNAL_ENTRY,
3   isDynamic: true,
4   defaultValues: {
5     bookje: 'T' // Setting bookje parameter to T makes the journal book specific.
6   }
7 });
8 journalEntry.setValue({
9   fieldId: 'accountingbook',
10  value: '2'
11 });
12 journalEntry.setValue({
13   fieldId: 'subsidiary',
14   value: '4'
15 });
16 journalEntry.setValue({
17   fieldId: 'trandate',
18   value: new Date(2019, 04, 16)
19 });
20 journalEntry.selectNewLine({
21   sublistId: 'line'
22 });
23 journalEntry.setCurrentSublistValue({
24   sublistId: 'line',

```

```

25     fieldId: 'account',
26     value: '6'
27   });
28   journalEntry.setCurrentSublistValue({
29     sublistId: 'line',
30     fieldId: 'credit',
31     value: '2.00'
32   });
33   journalEntry.commitLine({
34     sublistId: 'line'
35   });
36
37   journalEntry.selectNewLine({
38     sublistId: 'line'
39   });
40   journalEntry.setCurrentSublistValue({
41     sublistId: 'line',
42     fieldId: 'account',
43     value: '149'
44   });
45   journalEntry.setCurrentSublistValue({
46     sublistId: 'line',
47     fieldId: 'debit',
48     value: '2.00'
49   });
50   journalEntry.commitLine({
51     sublistId: 'line'
52   });
53   var id = journalEntry.save();

```

Landed Cost

Applies to: SuiteScript 2.x | SuiteCloud Developer

The landed cost subrecord supports the Landed Cost Allocation per Line functionality, which is part of the Landed Cost feature. Landed costs typically include location-specific expenses such as customs duties and freight fees. The landed cost subrecord can be used in conjunction with several transactions: check, credit card charge, item receipt, and vendor bill. The purpose of the subrecord is to show the landing costs associated with a particular line in the parent transaction's Items sublist.

The subrecord is available only when the Landed Cost feature is enabled at Setup > Company > Setup Tasks > Enable Features, on the Items & Inventory subtab.

For help working with this record in the UI, see the help topic [Using Landed Cost Allocation Per Line on Transactions](#).

The internal ID for this subrecord is landedcost.



Important: Landed cost is considered a subrecord, not a record. For general details on working with subrecords, see the help topic [Subrecord Scripting Overview](#). For details about this finding information for subrecords in the Record Browser, see the help topic [Finding Subrecord Details in the Records Browser](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The landed cost subrecord is scriptable in server SuiteScript only. The user events are not supported.

Supported Functions

The landed cost subrecord is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

To script to the landed cost subrecord, both of the following must be true:

- The parent transaction's Landed Cost per Line option is checked.
- The sublist item being referenced has been configured to use the Track Landed Cost option. For details on configuring the item, see the help topic [Setting up an Item Record for Landed Cost](#).

Code Samples

The following sample shows how to create a landed cost subrecord and perform other basic tasks.

```

1 // Creating a landed cost subrecord
2 var purchaseOrder = record.create({
3   type: record.Type.PURCHASE_ORDER,
4   isDynamic: true
5 });
6 purchaseOrder.setText({
7   fieldId: 'entity',
8   text: 'Acme Medical Supply'
9 });
10 purchaseOrder.setSublistValue({
11   sublistId: 'item',
12   fieldId: 'item',
13   line: 1,
14   value: inventoryItemId // defined outside this snippet
15 });
16
17 var purchaseOrderId = purchaseOrder.save();
18
19 var itemReceipt = record.transform({
20   fromtype: record.Type.PURCHASE_ORDER,
21   fromId: purchaseOrderId, // defined outside this snippet
22   toType: record.Type.ITEM_RECEIPT
23 });
24 itemReceipt.selectLine({
25   sublistId: 'item',
26   line: 1
27 });
28 itemReceipt.setCurrentSublistValue({
29   sublistId: 'item',
30   fieldId: 'location',
31   value: 1
32 });
33 itemReceipt.setValue({
34   fieldId: 'landedcostperline',
35   value: 'T'
36 });
37
38 var landedCost = itemReceipt.getCurrentSublistSubrecord({
39   sublistId: 'item',
40   fieldId: 'landedcost'
41 });
42 landedCost.selectNewLine({
43   sublistId: 'landedcostdata'
44 });
45 landedCost.setCurrentSublistValue({
46   sublistId: 'landedcostdata',
47   fieldId: 'costcategory',
48   value: 1
49 });
50 landedCost.setCurrentSublistValue({

```



```

51     sublistId: 'landedcostdata',
52     fieldId: 'amount',
53     value: 456
54 });
55 landedCost.commitLine({
56     sublistId: 'landedcostdata'
57 });
58 landedCost.selectNewLine({
59     sublistId: 'landedcostdata'
60 });
61 landedCost.setCurrentSublistValue({
62     sublistId: 'landedcostdata',
63     fieldId: 'costcategory',
64     value: 3
65 });
66 landedCost.setCurrentSublistValue({
67     sublistId: 'landedcostdata',
68     fieldId: 'amount',
69     value: 78.96
70 });
71 landedCost.commitLine({
72     sublistId: 'landedcostdata'
73 });
74 itemReceipt.commitLine({
75     sublistId: 'item'
76 });
77
78 var itemReceiptId = itemReceipt(save);
79
80 // Viewing the subrecord
81 itemReceipt = record.load ({
82     type: record.Type.ITEM_RECEIPT,
83     id: itemReceiptId
84 });
85 itemReceipt.selectLine({
86     sublistId: 'item',
87     line: 1
88 });
89 landedCost = itemReceipt.getSublistSubrecord({
90     sublistId: 'item',
91     fieldId: 'landedcost',
92     line : 1
93 });
94 landedCost.getSublistValue({
95     sublistId: 'landedcostdata',
96     fieldId: 'amount',
97     value: 1
98 });
99
100 // Updating the subrecord
101 landedCost = itemReceipt.getCurrentSublistSubrecord({
102     sublistId: 'item',
103     fieldId: 'landedcost'
104 });
105 landedCost.removeLine({
106     sublistId: 'landedcostdata',
107     line: 2
108 });
109
110 landedCost.setSublistValue({
111     sublistId: 'landedcostdata',
112     fieldId: 'costcategory',
113     line: 1,
114     value: 2
115 });
116 landedCost.setSublistValue({
117     sublistId: 'landedcostdata',
118     fieldId: 'amount',
119     line: 1,
120     value: 3.98
121 });
122 landedCost.selectNewLine({
123     sublistId: 'landedcostdata'

```

```

124 });
125 landedCost.setCurrentSublistValue({
126     sublistId: 'landedcostdata',
127     fieldId: 'costcategory',
128     value: 3
129 });
130 landedCost.setCurrentSublistValue({
131     sublistId: 'landedcostdata',
132     fieldId: 'amount',
133     value: 103
134 });
135 landedCost.commitLine({
136     sublistId: 'landedcostdata'
137 });
138 itemReceipt.commitLine({
139     sublistId: 'item'
140 });
141 var receiptId = itemReceipt.save();
142
143 // Deleting the subrecord
144 itemReceipt.selectLine({
145     sublistId: 'item',
146     line : 1
147 });
148 itemReceipt.removeCurrentSublistSubrecord({
149     sublistId: 'item',
150     fieldId: 'landedcost'
151 });
152 itemReceipt.commitLine({
153     sublist: 'item'
154 });
155 var receiptId2 = itemReceipt.save();

```

Manufacturing Operation Task

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Manufacturing Routing and Work Center feature lets you specify a sequence of tasks required for the completion of a Work In Process (WIP) work order. This record represents a job that must be completed by a specific employee group. In the UI, generally these records are created automatically when you save a WIP work order that references a specific routing record — each step described in the routing record becomes an operation task record, viewable on the work order's Operations subtab. You can also manually create an operation task record by clicking the New Operation Task button on the work order's Operations subtab. You can view all existing manufacturing operation task records by going to Transactions > Manufacturing > Manufacturing Operation Tasks.

For help working with this record in the UI, see the help topic [Manufacturing Operation Tasks](#).

The internal ID for this record is manufacturingoperationtask.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The manufacturing operation task record is scriptable in both client and server SuiteScript.

Supported Functions

The manufacturing operation task record is partially scriptable — it can be created, updated, deleted, and searched using SuiteScript. It cannot be copied.

Usage Notes

To work with the manufacturing operation task record, the Manufacturing Routing and Work Center feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.

In the UI, the manufacturing operation task record is accessed by going to Transactions > Manufacturing > Manufacturing Operation Tasks. Alternatively, you can go to the Operations subtab of a WIP work order that uses the routing feature. The Operations subtab lists existing operation task records for that work order and allows you to create new operation task records.

Note these additional details:

- This record supports client and server scripting.
- All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.
- To create a new manufacturing operation task record, you must reference a valid existing WIP work order, as shown in the [Code Samples](#) below.

Code Samples

When creating a manufacturing operation task record, you must use `initValues` to reference a valid existing WIP work order. For example:

```

1  var initValues = new Array();
2  initValues.workorder = '65';
3  var task = record.create({
4      type: record.Type.MANUFACTURING_OPERATION_TASK,
5      defaultValues: initValues
6  });
7  task.setValue({
8      fieldId: 'title',
9      value: 'Some title'
10 });
11 task.setValue({
12     fieldId: 'operationsequence',
13     value: 6
14 });
15 task.setValue({
16     fieldId: 'setuptime',
17     value: 30
18 });
19 task.setValue({
20     fieldId: 'runrate',
21     value: 20
22 });
23 task.setValue({
24     fieldId: 'manufacturingcosttemplate',
25     value: '1'
26 });
27 task.setValue({

```

```

28     fieldId: 'manufacturingworkcenter',
29     value: '113'
30   });
31   var recId = task.save();

```

Manufacturing Planned Time

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Manufacturing Planned Time search type enables you to search for data displayed on the Planned Time subtab of work orders. This subtab is available only when the Show Planned Capacity on Work Orders preference is enabled.

The Planned Time subtab is used to display data about the amount of time being allocated to each work center per day for the work order. This data is derived from the associated operation task records.

You can set the Show Planned Capacity on Work Orders preference at Setup > Accounting > Accounting Preferences. The preference is available only when the Manufacturing Routing and Work Center feature is enabled.

For help working with this record in the UI, see the help topic [Manufacturing Routing](#).

The internal ID for this record is mfgplannedtime.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This search is supported in both client and server SuiteScript.

Supported Functions

Only search is supported for this record.

Usage Notes

Be aware of the following:

- This record is a search record only. You cannot create or copy this record.
- In the UI, you can view this search by going to Reports > New Search and clicking Manufacturing Planned Time.

For help creating search scripts, see the help topics [N/search Module](#) and [N/search Module Script Samples](#).

Memorized Transaction Definition

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A memorized transaction is a transaction you set up to recur in NetSuite, such as recurring journal entries. Memorized transactions eliminate data entry and serve as useful reminders. A memorized transaction consists of two parts:

- The **Memorized Transaction Template** defines the transaction to be created.
- The **Memorized Transaction Definition** sets the parameters for when to create the transaction, whether they are recurring, and how many transactions to create.

Memorized transaction templates and definitions must be created in the NetSuite user interface, but existing memorized transaction definitions can be called using SuiteScript.

The internal ID for this record is memdoc. Note that this record is a search record only. You cannot create or copy this record.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

When you memorize a transaction, you set up the transaction to recur over time. You choose whether the transaction automatically posts or if you are reminded to post it. You also set the frequency the transaction should occur and set posting options. You can specify that the transaction should recur forever or a set number of times.

When you edit or delete a memorized transaction, the change is tracked on the system notes of the memorized transaction.

The following are limitations for posting memorized transactions in locked and closed periods:

- A user must have the Override Period Restrictions permission to enable the Allow Posting in Locked Period option for a memorized transaction. When this option is enabled, memorized transactions can post in locked periods even if users initiating these transactions do not have the Override Period Restrictions permission.
- Transactions can never be posted in closed periods. If the posting date for a memorized transaction is in a period that has been closed, it is posted to the next open period.

In NetSuite, you can memorize the following transactions:

- Checks
- Credit Cards
- Purchase Orders
- Bills
- Sales Orders
- Invoices
- Estimates
- Cash Sales
- Journal Entries
- Statement Charges

- Work Orders
- Purchase Requisitions
- Transfer Orders
- Opportunities
- Custom Transactions

For help working with this record in the UI, see the following articles:

- [Memorized Transactions](#)
- [Memorizing a Transaction](#)
- [Editing Memorized Transactions](#)
- [Customizing the Memorized Transactions List](#)
- [Viewing the Status of Memorized Transactions](#)
- [Deleting Memorized Transactions](#)

Supported Script Types

Memorized transaction definitions are supported in client and server SuiteScript.

 **Note:** Automatically-created transactions do not support server scripting.

Supported Functions

Memorized transaction definitions support only the Read, Update, Delete, and Search functions.

Usage Notes

Be aware of the following:

- The System Notes and Transaction subtabs have not been exposed and cannot be used in scripts.
- Transforming memorized transaction definition records into other types of records is not supported.
- This record has one subrecord: a list with record ID “customdates”.
- You cannot call `nlapiCreateRecord` with client scripts using memorized transaction definitions.
- Scripts which run when a transaction is created may not work as expected if the transaction is automatically created from a memorized transaction. For example, you may have a script which runs when you save a purchase order. However, the script will not run if the purchase order was created from a memorized transaction definition which has “Automatic” set in its Action field.

For information about scripting with this record in SuiteScript, see the following help topics:

- [Working with the SuiteScript Records Browser](#)
- [SuiteCloud Supported Records](#)

Sample Code

Memorized transaction definitions can only be created in the NetSuite user interface. The example script below first loads an existing memorized transaction definition with ID 2. Then, it sets the Action field to

“Reminder”, and sets the Next Transaction Date in the Custom Dates subtab to 12/12/2020. It saves this date, and then saves the memorized transaction definition.

```

1 | var r = nlapiLoadRecord('memdoc', 2)
2 | r.setFieldValue('action', 'REMINDER')
3 | r.selectNewLineItem('customdates')
4 | r.setCurrentLineItemValue('customdates', 'nextdate', '12/12/2020')
5 | r.commitLineItem('customdates')
6 | var id = nlapiSubmitRecord(r);

```

Multi-Book Accounting Transaction

Applies to: SuiteScript 2.x | SuiteCloud Developer

Note: This topic applies to all versions of SuiteScript.

If your account has the Multi-Book Accounting feature enabled, you can use SuiteScript to search for transactions using accounting book as a search filter or a search column. To execute this type of search, you use the multi-book accounting transaction search record.

In the UI, you can view the multi-book accounting transaction search by going to Reports > New Search and clicking Multi-Book Accounting Transaction.

For help working with this record in the UI, see the help topic [Multi-Book Accounting](#).

The internal ID for this record is accountingtransaction. Note that this record is a search record only. You cannot create or copy this record.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The multi-book accounting transaction record is supported in both client and server SuiteScript.

Supported Functions

Only search is supported for this record.

Sample Code

The following sample shows how to search for a specific transaction and include in the results details about the transaction's accounting book and foreign exchange rate, among other data.

```

1 var mySearch = search.create ({
2   type: search.Type.TRANSACTION,
3   columns: [{
4     name: 'accountingbook',
5     sort: search.Sort.ASC
6   }, {
7     name: 'line',
8     join: 'transaction',
9     sort: search.Sort.ASC
10  }, {
11    name: 'account'
12  }, {
13    name: 'amount'
14  }, {
15    name: 'exchangerate'
16  }],
17  filters: [{
18    name: 'internalid',
19    operator: search.Operator.IS,
20    values: 18 // last parameter represents the internal ID of the transaction
21  }],
22 });
23
24 for (var i = 0; mySearch != null && i < mySearch.length; i++) {
25   var searchresult = mySearch[i];
26   var record = searchresult.getId();
27   var rectype = searchresult.getRecordType();
28   var book = searchresult.getValue({
29     name: 'accountingbook'
30   });
31   var line = searchresult.getValue({
32     name: 'line',
33     join: 'transaction'
34   });
35   var account = searchresult.getValue({
36     name: 'account'
37   });
38   var amount = searchresult.getValue({
39     name: 'amount'
40   });
41   var fxrate = searchresult.getValue({
42     name: 'exchangerate'
43   });
44
45   alert(i + ": " + book + " | " + line + " | " + record + " | " + rectype + " | " + account + " | " + amount + " | " + fxrate);
46 }

```

Order Schedule

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Creating Commit Orders Schedule](#).

The internal ID for this subrecord is orderschedule.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

- [N/record Module](#)

Supported Script Types

Server scripts for this subrecord must access through the parent record.

Supported Functions

For details, see the help topic [About Subrecords](#).

Usage Notes

Order Schedule is scriptable from both the body field and the line item.

Order Schedule is considered a subrecord in SuiteScript.

Pick Strategy

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A pick strategy record contains filters that can be applied to the backend process that determines the preferred bin for items in a pick task record. When a wave transaction is saved or released, the backend process uses any available pick strategies to sort bins based on pick strategy filters.

The pick strategy record is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Defining Pick Strategies](#).

The internal ID for this record is pickstrategy.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The pick strategy record is scriptable in both client and server SuiteScript.

Supported Functions

The pick strategy is fully scriptable — it can be created, copied, updated, deleted, and searched using SuiteScript.

Usage Notes

See the [Fields](#) section for more details about working with this record.

Fields

Note the following usage notes for specific fields on this record:

- The Customer field is enabled only for the Sales Order transaction type.
- If the Inventory Status feature is enabled on the account, the Inventory Status field is displayed, containing all inventory status records in the dropdown list.
- The Location dropdown list shows only the location records that have the Use Warehouse Management box checked.

Code Samples

The following sample code in SuiteScript 2.x includes creating and editing a pick strategy record.

```

1  require(["N/record"], function (record) {
2      function createPickStrategy() {
3          var locationId = 1;
4
5          var pickStrategy = record.create({type: record.Type.PICK_STRATEGY, isDynamic: true});
6          pickStrategy.setValue({fieldId: "name", value: "General strategy"});
7          pickStrategy.setValue({fieldId: "description", value: "This will be used if no other strategy is applied"});
8          pickStrategy.setValue({fieldId: "priority", value: 1000000000});
9          pickStrategy.setValue({fieldId: "trantype", value: "TrnfrOrd"});
10         pickStrategy.setValue({fieldId: "location", value: locationId});
11
12         addZoneLine(pickStrategy, {sequence: 100, zoneId: 1});
13         addZoneLine(pickStrategy, {sequence: 200, zoneId: 2});
14         addZoneLine(pickStrategy, {sequence: 300, zoneId: 3});
15
16         setTranslationValues(pickStrategy, {languageName: "Czech", name: "Obecná strategie", description: "Bude použita, pokud nebyla aplikována žádná jiná strategie"});
17
18         pickStrategy.save();
19     }
20
21     function editPickStrategy() {
22         var pickStrategyId = 8;
23         var locationId = 10;
24         var itemProcessfamilyId = 3;
25         var itemProcessGroupId = 10;
26         var itemId = 267;
27         var unitId = 1;
28         var INVENTORY_CLASSIFICATION_B = 2;
29         var orderTypeId = 2;
30         var customerId = 14;
31         var INVENTORY_STATUS_ALL_AVAILABLE = -1;
32
33         var pickStrategy = record.load({type: record.Type.PICK_STRATEGY, id: pickStrategyId, isDynamic: true});
34         pickStrategy.setValue({fieldId: "name", value: "Specific strategy"});
35         pickStrategy.setValue({fieldId: "description", value: "This will be used when all the criteria match the Pick Task"});
36         pickStrategy.setValue({fieldId: "priority", value: 1});
37         pickStrategy.setValue({fieldId: "trantype", value: "SalesOrd"});
38         pickStrategy.setValue({fieldId: "location", value: locationId});
39         pickStrategy.setValue({fieldId: "itemprocessfamily", value: itemProcessfamilyId});
40         pickStrategy.setValue({fieldId: "itemprocessgroup", value: itemProcessGroupId});
41         pickStrategy.setValue({fieldId: "item", value: itemId});
42         pickStrategy.setValue({fieldId: "units", value: unitId});
43         pickStrategy.setValue({fieldId: "invtclassification", value: INVENTORY_CLASSIFICATION_B});
44         pickStrategy.setValue({fieldId: "ordertype", value: orderTypeId});
45         pickStrategy.setValue({fieldId: "customer", value: customerId});
46         pickStrategy.setValue({fieldId: "inventorystatus", value: INVENTORY_STATUS_ALL_AVAILABLE});

```

```

47     pickStrategy.setValue({fieldId: "includeinboundstagingbins", value: true});
48
49     editZoneLine(pickStrategy, {line: 0, sequence: 10, zoneId: 12});
50     editZoneLine(pickStrategy, {line: 1, sequence: 20, zoneId: 13});
51     editZoneLine(pickStrategy, {line: 2, sequence: 30, zoneId: 14});
52
53     setTranslationValues(pickStrategy, {languageName: "Czech", name: "Konkrétní strategie", description: "Bude použita, pokud
všechna kritéria odpovídají úloze vychystání});
54
55     pickStrategy.save();
56 }
57
58 function addZoneLine(pickStrategy, zoneLine) {
59     pickStrategy.selectNewLine({sublistId: "zones"});
60     pickStrategy.setCurrentSublistValue({sublistId: "zones", fieldId: "priority", value: zoneLine.sequence});
61     pickStrategy.setCurrentSublistValue({sublistId: "zones", fieldId: "zone", value: zoneLine.zoneId});
62     pickStrategy.commitLine({sublistId: "zones"});
63 }
64
65 function editZoneLine(pickStrategy, zoneLine) {
66     pickStrategy.selectLine({sublistId: "zones", line: zoneLine.line});
67     pickStrategy.setCurrentSublistValue({sublistId: "zones", fieldId: "priority", value: zoneLine.sequence});
68     pickStrategy.setCurrentSublistValue({sublistId: "zones", fieldId: "zone", value: zoneLine.zoneId});
69     pickStrategy.commitLine({sublistId: "zones"});
70 }
71
72 function setTranslationValues(pickStrategy, translation) {
73     var lineNumber = pickStrategy.findSublistLineWithValue({sublistId: "translations", fieldId: "language", value: transla
tion.languageName});
74     pickStrategy.selectLine({sublistId: "translations", line: lineNumber});
75     pickStrategy.setCurrentSublistValue({sublistId: "translations", fieldId: "name", value: translation.name});
76     pickStrategy.setCurrentSublistValue({sublistId: "translations", fieldId: "description", value: translation.description});
77     pickStrategy.commitLine({sublistId: "translations"});
78 }
79
80 createPickStrategy();
81 // editPickStrategy();
82 });

```

Pick Task

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Pick tasks record the picking and staging details of items for released orders. Item fulfillment records of picked and staged items are generated from the pick task record.

Pick task records can only be generated from wave transactions.

The pick task record is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Releasing Orders to the Warehouse](#).

The internal ID for this record is picktask.

The pick task record contains the Inventory Detail subrecord. For details about subrecords in SuiteScript, see the help topic [SuiteScript 2.x Scripting Subrecords](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The pick task record is scriptable in both client and server SuiteScript.

Supported Functions

The pick task record is partially scriptable — it can be updated, deleted, and searched using SuiteScript. Pick tasks cannot be created or copied using SuiteScript.

Usage Notes

See the following sections for more details about working with this record:

- [Fields](#)
- [Sublists](#)

Fields

Note the following usage notes for specific fields on this record:

Field Names	Read-Only?	Comments
Wave # Location Item	Yes	These fields display the values in their corresponding fields on the associated wave transaction.
Total Picked Quantity Total Remaining Quantity	Yes	These fields are calculated automatically from the values in the Picked Quantity and Remaining Quantity columns in the Line Items sublist.
Assigned Picker	No	By default, this field is populated automatically with the value in its corresponding field on the associated wave transaction. If pickers have a specified warehouse location on their employee records, their name appears in this list only when the pick task belongs to a wave that has the same warehouse location.
Units	Yes	This field is displayed only on accounts that have the Multiple Units of Measure feature enabled. It is populated automatically with the value of item's unit of measure.
Serial/Lot Number	Yes	For serialized or lot items, this field displays the value in its corresponding field on the associated wave transaction.
Status	Yes	Initially, this field is set to the following statuses based on the status of the associated wave transaction ■ Pending – Assigned when the wave is on Pending Release status ■ Ready – Assigned when the wave is on Released status

Field Names	Read-Only?	Comments
		After a wave has been released, this field is updated automatically based on the status of the line items.

Sublists

Note the following usage notes for columns in the Line Items sublist:

- The values in the following columns are populated automatically with values entered using the mobile device: Picked Quantity, Staging Bin, and Inventory Detail.
- The Transaction Number column displays the link to the fulfillment record, which is generated for each pick task line item in Done status.
- If the Multiple Shipping Routes feature is enabled on the account, the following columns are populated automatically with values entered in the line items of the associated order transaction: Ship To and Ship Via.
- The Override Fulfillment Creation box is displayed and enabled for pick task lines with a Failed status and a value of Yes in the Allow Retry column.
- The following fields are displayed and enabled if you set the **Show Additional Pick Task Columns** preference: User Field 1 and User Field 2.

For more information, see the help topic [Setting Warehouse Management Preferences](#).

Code Samples

The following sample code in SuiteScript 2.x includes editing pick tasks.

```

1  require(["N/record"], function (record) {
2      /* Pick Task cannot be created manually. It is always created automatically during creation of a wave! */
3
4      function editPickTask() {
5          var pickTaskId = 14;
6          var pickerId = 103;
7          var GOOD_INVENTORY_STATUS = 1;
8
9          var pickTask = record.load({type: record.Type.PICK_TASK, id: pickTaskId, isDynamic: true});
10         pickTask.setValue({fieldId: "picker", value: pickerId});
11         pickTask.setValue({fieldId: "memo", value: "Call Mike when done, please."});
12
13         editLineItem(pickTask, {
14             line: 0,
15             pickedQuantity: 1.5,
16             stagingBinId: 124,
17             inventoryDetailLines: [
18                 {lotNumber: "L1", binNumberId: 122, inventoryStatusId: GOOD_INVENTORY_STATUS, quantity: 1},
19                 {lotNumber: "L2", binNumberId: 123, inventoryStatusId: GOOD_INVENTORY_STATUS, quantity: 0.5}
20             ]
21         });
22
23         pickTask.save();
24     }
25
26     function editLineItem(pickTask, lineItem) {
27         pickTask.selectLine({sublistId: "pickactions", line: lineItem.line});
28         pickTask.setCurrentSublistValue({sublistId: "pickactions", fieldId: "pickedquantity", value: lineItem.pickedQuantity});
29         createInventoryDetail(pickTask, lineItem.inventoryDetailLines);
30         pickTask.setCurrentSublistValue({sublistId: "pickactions", fieldId: "stagingbin", value: lineItem.stagingBinId});
31         pickTask.commitLine({sublistId: "pickactions"});
32     }
33
34     function createInventoryDetail(pickTask, inventoryDetailLines) {
35         var inventoryDetail = pickTask.getCurrentSublistSubrecord({sublistId: "pickactions", fieldId: "inventorydetail"});
36     }

```

```

37     for (var i = 0; i < inventoryDetailLines.length; i++) {
38         var line = inventoryDetailLines[i];
39
40         inventoryDetail.selectNewLine({sublistId: "inventoryassignment"});
41         inventoryDetail.setCurrentSublistText({sublistId: "inventoryassignment", fieldId: "issueinventorynumber", text:
line.lotNumber});
42         inventoryDetail.setCurrentSublistValue({sublistId: "inventoryassignment", fieldId: "binnumber", value: line.binNum
berId});
43         inventoryDetail.setCurrentSublistValue({sublistId: "inventoryassignment", fieldId: "inventorystatus", value: line.inven
toryStatusId});
44         inventoryDetail.setCurrentSublistValue({sublistId: "inventoryassignment", fieldId: "quantity", value: line.quantity});
45         inventoryDetail.commitLine({sublistId: "inventoryassignment"});
46     }
47 }
48
49 editPickTask();
50 });

```

Statistical Journal Entry

Applies to: SuiteScript 2.x | SuiteCloud Developer

The statistical journal entry record lets you increase or reduce the balance of a statistical account.

To use this record, the Statistical Accounts feature must be enabled at Setup > Enable Features, on the Accounting subtab. Also, you must have already created at least one statistical account. In the UI, you access this record at Transactions > Financial > Make Statistical Journal Entries.

For help working with this record in the UI, see the help topic [Making Statistical Journal Entries](#).

This internal ID for this record is `statisticaljournalentry`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The statistical journal entry record is scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The statistical journal entry record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

Be aware of the following:

- The unitlabel field is read-only. It is populated automatically when you set the unitstype field. (The unitlabel value is the unitstype's base unit value. This value is defined in the corresponding unit of measure record.)
- The unitstype body field cannot be updated after the record is created.
- The field labeled Amount in the UI is called debit in SuiteScript.
- Every statistical journal entry record must have at least one line.
- All sublist lines must use the same unit of measure, which is defined by the unitstype body field.

Code Samples

The following samples show how you can script to the statistical journal entry record.

Adding and Deleting

The following sample shows how you can create the record, add lines, load the record, and delete it.

```

1 // Note: These samples use constants (defined like: var subsidiaryId = '1') which are not included here.
2
3 // Create Record
4 var statisticalJournal = record.create({
5   type: record.Type.STATISTICAL_JOURNAL_ENTRY
6 });
7 statisticalJournal.setValue({
8   fieldId: 'subsidiary',
9   value: subsidiaryId
10 });
11 statisticalJournal.setValue({
12   fieldId: 'externalid',
13   value: externalId
14 });
15 statisticalJournal.setValue({
16   fieldId: 'unitstype',
17   value: unitsTypeId
18 });
19 statisticalJournal.setValue({
20   fieldId: 'unit',
21   value: unitId
22 });
23
24 // Add line to the record
25 statisticalJournal.setSublistValue({
26   sublistId: 'line',
27   fieldId: 'account',
28   line: 1,
29   value: statisticalAccountId
30 });
31 statisticalJournal.setSublistValue({
32   sublistId: 'line',
33   fieldId: 'debit', // field 'debit' has label 'Amount' in UI
34   line: 1,
35   value: amount
36 });
37 statisticalJournal.setSublistValue({
38   sublistId: 'line',
39   fieldId: 'lineunit',
40   line: 1,
41   value: unitId
42 });
43 statisticalJournal.setSublistValue({
44   sublistId: 'line',
45   fieldId: 'memo',
46   line: 1,
47   value: memo
48 });

```

```

49
50 statisticalJournal.setSublistValue({
51     sublistId: 'line',
52     fieldId: 'class',
53     line: 1,
54     value: classId
55 });
56 statisticalJournal.setSublistValue({
57     sublistId: 'line',
58     fieldId: 'department',
59     line: 1,
60     value: departmentId
61 });
62 statisticalJournal.setSublistValue({
63     sublistId: 'line',
64     fieldId: 'location',
65     line: 1,
66     value: locationId
67 });
68
69 // Add record
70 var recId = statisticalJournal.save();
71
72 // Load record
73 var statisticalJournalAddedRec = record.load({
74     type: record.Type.STATISTICAL_JOURNAL_ENTRY,
75     id: recId
76 });
77
78 // Delete record
79 var deleteId = record.delete({
80     type: record.Type.STATISTICAL_JOURNAL_ENTRY,
81     id: recId
82 });

```

Updating the Subsidiary Field

The following sample shows how to update the subsidiary field, which can only be done in dynamic mode.

```

1  var recType = record.Type.STATISTICAL_JOURNAL_ENTRY;
2
3  // 1st: Load record in Dynamic Mode
4  var journalEntry = record.load({
5      type: recType,
6      id: '168',
7      isDynamic: true
8  });
9
10 // 2nd: Remove all lines
11 var lineCount = journalEntry.getLineCount({
12     sublistId: 'line'
13 });
14 for (i = 1; i <= lineCount; i++){
15     var removeId = journalEntry.removeLine({
16         sublistId: 'line',
17         line: i
18     });
19 }
20
21 // 3rd: Change subsidiary
22 journalEntry.setValue({
23     fieldId: 'subsidiary',
24     value: '6'
25 });
26
27 // 4th: Add new line for changed subsidiary
28 // The new account must be available in the new subsidiary
29 journalEntry.selectNewLine({
30     sublistId: 'line'
31 });

```



```

32 | journalEntry.setCurrentSublistValue({
33 |     sublistId: 'line',
34 |     fieldId: 'account',
35 |     value: '286'
36 | });
37 | journalEntry.setCurrentSublistValue({
38 |     sublistId: 'line',
39 |     fieldId: 'debit',
40 |     value: '5'
41 | });
42 | journalEntry.commitLine({
43 |     sublistId: 'line'
44 | });
45 |
46 | var recId = journalEntry.save();

```

Subscription

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The subscription record is used to build recurring subscriptions.

The subscription record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features on the Transactions subtab. When the feature is enabled, you can access the subscription record in the UI by going to Transactions > Subscriptions > Create Subscriptions.

The internal ID for this record is subscription.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subscription record is scriptable in server and client SuiteScript.

Supported Functions

The subscription record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following sample shows how to use this record to build a recurring subscription in SuiteScript 2.x.

```

1 | require(['N/record'],
2 | function(record) {
3 |     var obj = record.create({ type: record.Type.SUBSCRIPTION, isDynamic: true});
4 |     obj.setValue('customer', 5);
5 |     obj.setValue('startdate', new Date('12/4/2020'));

```

```

6 obj.setValue('subscriptionplan', 21);
7 obj.setValue('advancerenewalperiodnumber', 10);
8 obj.setValue('initialterm', -102);
9 obj.setValue('enddate', new Date('12/10/2021'));
10 obj.setValue('defaultrenewalterm', 2);
11 obj.setValue('defaultrenewalplan', 8);
12 obj.save();
13 });

```

Subscription Line

Applies to: SuiteScript 2.x | SuiteCloud Developer

The subscription line record is used to provide subscription item information for subscriptions.

The subscription line record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab.

For help working with this record in the UI, see the help topic [Subscription Management](#).

The internal ID for this record is subscriptionline.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subscription line record is scriptable in server and client SuiteScript.

Supported Functions

The subscription line record is not fully scriptable — create and delete are not supported.

Time

Applies to: SuiteScript 2.x | SuiteCloud Developer

A time transaction, also known as TimeBill, records the hours worked by an employee. This transaction can be used to record billable hours and invoice customers. This transaction is available when the Time Tracking feature is enabled.

For help working with this record in the UI, see the help topic [Managing Time Tracking](#).

The internal ID for this record is timebill.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Type

The time record is scriptable in both client and server SuiteScript.

Supported Functions

The time record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

retract

Corresponding UI Button	Retract
Action Description	Retracts pending timebill with the specified ID.
Returns	{ "id": recordId, "action": "retract", "success": [true or false], "recordCount": number of retracted timebills }
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. RecordId is a default input parameter for this action. No additional input parameters are necessary.

Sample Return Objects

The following sample shows how to use a retract action on timebill.

```

1  {
2
3  {
4
5    "id": recordId,
6
7    "action": "retract",
8
9    "success": [true or false],
10

```

```

11 "recordCount": number of retracted timebills
12
13 },
14
15 "notifications": [ Error, Warning, Information and text]
16
17 }

```

Transaction Search

Applies to: SuiteScript 2.x | SuiteCloud Developer

Nearly all transaction record types use the TransactionSearch record for search.

For help working with this record in the UI, see the help topic [Defining an Advanced Search](#). For help working with this record by using SuiteScript API, see the help topic [N/search Module](#).

The internal ID for this record is transaction.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/search Module](#)
- [N/record Module](#)

Supported Script Types

The transaction search is scriptable in both client and server SuiteScript.

Supported Functions

Only search is supported for the transaction search record.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Search Filters and Search Columns		
ccnumber	Credit Card #	To prevent users from accessing sensitive information such as password and credit card data, this field cannot be read in beforeSubmit user event scripts for external role users (for example, shoppers, online form users (anonymous users), customer center).

Field Internal ID	Field UI Label	Note
entity	Name	The search filter entity is synonymous for the search filter name. Either filter can be used when searching the value of the Name / ID field in the UI.

Unlocked Time Period

Applies to: SuiteScript 2.x | SuiteCloud Developer

The unlocked time period record is available only when the Weekly Timesheets feature is enabled and the Lock Timesheet Period accounting preference is set to lock timesheets after a specified time.

For help working with timesheet locking in the UI, see the help topic [Locking Timesheet Periods](#).

The internal ID for this record is unlockedtimeperiod.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Type

The unlocked time period record is scriptable in both client and server SuiteScript.

Supported Operations

This record can be created, read, updated, deleted, copied, and searched using SuiteScript. It cannot be transformed.

Usage Notes

When creating a new unlocked time period, you should not create a record for an employee that overlaps the date range of an existing active time period for the same employee.

The System Notes sublist is search only.

Field Definitions

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Data about unlocked time periods is also accessible through SuiteAnalytics Connect. The records exposed in SuiteAnalytics Connect are available in the Connect Browser. You can retrieve data for unlocked time periods by using the Unlocked Time Periods table as listed in [Connect Schema](#). For more information, see the help topics [Working with the Connect Browser](#) and [SuiteAnalytics Connect](#).

Code Samples

The following sample shows how to create an unlocked time period.

```

1 require ([N/record', 'N/format'], function (record, format){
2     var unlockTime = record.create ({
3         type: record.Type.UNLOCK_TIME_PERIOD,
4         isDynamic: true
5     });
6     unlockTime.setValue({
7         fieldId: 'entity',
8         value: -5
9     });
10    unlockTime.setValue({
11        fieldId: 'startdate',
12        value: format.parse({
13            value: "6/7/2019",
14            type: format.Type.DATE
15        })
16    });
17    unlockTime.setValue({
18        fieldId: 'enddate',
19        value: format.parse({
20            value: "6/9/2019",
21            type: format.Type.DATE
22        })
23    });
24    unlockTime.setValue({
25        fieldId: 'validuntil',
26        value: format.parse({
27            value: "6/29/2019",
28            type: format.Type.DATE
29        })
30    });
31    unlockTime.save();
32 })

```

Usage

Applies to: SuiteScript 2.x | SuiteCloud Developer

The usage record is used to calculate usage information for subscriptions.

The usage record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features, on the Transactions subtab. When the feature is enabled, you can access the usage record in the UI by going to Transactions > Subscriptions > Create Usages.

For help working with this record in the UI, see the help topic [Creating Usage Records](#).

The internal ID for this record is usage.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The usage record does not support client scripts, server scripts, or workflows.

Supported Functions

The usage record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Vendor Bill

Applies to: SuiteScript 2.x | SuiteCloud Developer

The vendor bill transaction records payables as they arrive from vendors, allowing you to pay bills from the payables list as they are due, and providing an accurate picture of payables at all points of the billing cycle.

For help working with this record in the UI, see the help topic [Vendor Bills](#).

The internal ID for this record is vendorbill.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.


Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor bill record is scriptable in both client and server SuiteScript.

Supported Functions

The vendor bill record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
estgrossprofit	Est. Gross Profit	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.
estgrossprofitpercent	Est. Gross Profit Percent	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search. When this field appears on the sublist line level, this field is not scriptable.

Field Internal ID	Field UI Label	Note
totalcostestimate	Est. Extended Cost	When this field is on the body of the form in edit mode, this field is scriptable and can be returned in a transaction search.
usertotal	Amount	This field is not available using search or lookup for any transactions.

Using Landed Cost Fields

When you create a landed cost category, the associated field IDs for the first category are landedcostamount1 and landedcostsource1. If you create a second category, the IDs will be landedcostamount2 and landedcostsource2.

This pattern increments by one with each additional category. For example, the IDs for the next landed cost category will be landedcostamount3 and landedcostsource3, and so on.

Using the Payee Address Sublist

Payee address data is stored in a subrecord instead of a text field. On the Bill Payment form in the UI, address information is now displayed in a sublist instead of a single field. SuiteScript includes specialized APIs that you must use to script with subrecord data. You should use these subrecord APIs to script with payee address data.

For details about scripting subrecords with SuiteScript 2.x, see:

- [SuiteScript 2.x Scripting Subrecords](#)
- [Record Object Members](#)
- [CurrentRecord Object Members](#)
- [About the Address Subrecord](#)



Warning: This change may impact existing scripts that reference bill payee address data. You should review these scripts to determine whether updates are needed.

Vendor Credit

Applies to: SuiteScript 2.x | SuiteCloud Developer

A vendor credit transaction creates a credit, from a vendor, that can be applied to a payables account. For example, a vendor credit transaction may occur when items are returned to a vendor or when a discount is negotiated with a vendor.

For help working with this record in the UI, see the help topic [Vendor Credits](#).

The internal ID for this record is vendorcredit.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

- [N/record Module](#)

Supported Script Types

The vendor credit record is scriptable in both client and server SuiteScript.

Supported Functions

The vendor credit record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
usertotal	Amount	This field is not available using search or lookup for any transactions.

Vendor Payment

Applies to: SuiteScript 2.x | SuiteCloud Developer

A vendor payment transaction posts to the general ledger as an expense and the amount of the payment is deducted from your accounts payable total. A vendor payment can be applied to one or more vendor bills. Vendor payments can help to track expenditures and total payables due.

For help working with this record in the UI, see the help topic [Vendor Payments](#).

The internal ID for this record is vendorpayment.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See the [Vendor Payment Record Actions](#) for the actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor payment record is scriptable in both client and server SuiteScript.

Supported Functions

The vendor payment record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The Credits sublist, scriptable for the vendor payment record, is a **list** sublist.

Vendor Payment Record Actions

Applies to: SuiteScript 2.x | SuiteCloud Developer

The vendor payment record currently supports the confirm and decline actions.

For help working with this record in the UI, see the help topic [Vendor Payments](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- Overview of Record Action and Macro APIs
- N/action Module

confirm

Corresponding UI Button	Confirm
Action Description	Changes a payment's status from Payment In-Transit to Payment Confirmed.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The confirm action supports the following parameters.

Parameter	Type	Required / Optional	Description	Since
exchangerate	string	optional	The exchange rate at the time of the transaction's confirmation.	2018.2
confirmationdate	string	optional	The date on which the transaction is confirmed and posted.	2018.2
postingperiod	string	optional	The transaction posting period.	2018.2
clearpayment	string	optional	Indicates if the payment should be marked as cleared.	2018.2
recordId	int	required	The record ID.	2018.2

Vendor Payment Confirm Action Syntax

```
1 | require(['N/action'], function(action){
```

```

2 // action & action list loading
3 var actionList = action.find({
4     recordType: 'vendorpayment'
5 });
6 var actionObj = action.get({
7     recordType: 'vendorpayment',
8     id: 'confirm'
9 });
10
11 // action execution 1
12 var result1 = action.execute({
13     recordType: 'vendorpayment',
14     id: 'confirm',
15     params: {
16         recordId: 101,
17         exchangerate: '1.0',
18         trandate: '5/10/2018',
19         postingperiod: '243',
20         clearpayment: 'T'
21     }
22 });
23
24 // action execution 2
25 var result2 = actionObj.execute({
26     recordId: 102,
27     exchangerate : '1.0',
28     trandate: '5/10/2018',
29     postingperiod: '243',
30     clearpayment: 'F'
31 });
32 // action execution 3
33 var result3 = actionObj({
34     recordId: 103,
35     exchangerate : '1.0',
36     trandate: '5/10/2018',
37     postingperiod: '243',
38     clearpayment: 'F'
39 });
40 });

```

decline

Corresponding UI Button	Decline
Action Description	Declines a payment's status from Payment In-Transit to Payment Declined.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	10 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The decline action does not support parameters aside from recordId.

Vendor Payment Decline Action Syntax

```

1 | require(['N/action'], function(action){

```

```

2 // action & action list loading
3 var actionList = action.find({
4     recordType: 'vendorpayment'
5 });
6 var actionObj = action.get({
7     recordType: 'vendorpayment',
8     id: 'decline'
9 });
10
11 // action execution 1
12 var result = action.execute({
13     recordType: 'vendorpayment',
14     id: 'decline',
15     params: {
16         recordId: 101
17     }
18 });
19
20 // action execution 2
21 var result = actionObj.execute({
22     recordId: 102
23 });
24
25 // action execution 3
26 var result = actionObj({
27     recordId: 103
28 });
29 });

```

Weekly Timesheet

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Weekly Timesheets feature works in conjunction with the existing Time Tracking feature to offer a customizable method of capturing time entries in a weekly format. Each timesheet represents a single week, and includes a sublist where each line represents a day of the week.

For help working with this record in the UI, see the help topic [Weekly Timesheets](#).

The internal ID for this record is timesheet. In this newest version of the timesheet record, each sublist line representing a day of the week is a time record. The time record has an internal ID of timebill. In the deprecated version of the timesheet record, the values in the sublist are from a timeentry subrecord. The timeentry subrecord is not used by the newest version of the timesheet record.

See the [SuiteScript Records Browser](#) for all internal IDs associated with the timesheet record, and internal IDs associated with the [Time](#) record. The records browser entry for the timesheet record may include field IDs for the deprecated version of the record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

To track time with weekly timesheets, an administrator should go to Setup > Company > Setup Tasks > Enable Features > Employees, check the Time Tracking and Weekly Timesheets boxes, and click Save. To enable the optional new interface for Weekly Timesheets, go to Setup > Company > Setup Tasks > Enable Features > Employees. Check the New Weekly Timesheets Interface box, and click Save. After this feature is enabled, all users with timesheet permissions use the new UI. For more information, see [Scripting Impact of New Weekly Timesheets Interface](#).

See [Weekly Timesheet Record Actions](#) and [Weekly Timesheet Record Macros](#) for the actions and macros associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Both client and server scripts are supported for the weekly timesheet record.

Supported Functions

The weekly timesheet record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Scripting Impact of New Weekly Timesheets Interface

NetSuite 2018.1 supports an optional new UI for weekly timesheets.

If this new UI is enabled in your account, you may need to update your scripts on timesheets and time records, to ensure that they continue to work properly. The following changes in the new UI impact scripts on timesheet or time records:

- [IDs for Popup Window Fields](#)
- [Changes to User Event Context Types](#)
- [Enforcement of Defined Time Limits](#)

IDs for Popup Window Fields

The new UI allows some fields to be moved to a popup window for each day. Fields that are moved to the popup window have new field IDs for each day. By default, setting the value of a popup window field updates the field's values for all days of the week, in other words, all timebills in a weekly timesheet. You can make it possible to update a field's value for a single day by appending a number 0-6 to the end of the existing field ID. For example, memo0 represents the memo field for the first day of the week.

Client scripts on fields may need to be adjusted. Any scripts intended to change a field located in a popup window should be adjusted to accommodate new field IDs for single days of the week.

In addition, any scripts that are set to run on multiple forms that may or may not have the required fields in the expected locations are likely to fail. If you have client scripts that are set to run on multiple custom forms, you must ensure that the fields are located in the expected place on all forms. The preferred method is to use user event scripts instead of client scripts to update single timebills on a weekly timesheet.

Changes to User Event Context Types

When the new UI is not enabled, there are 3 separate user event execution contexts (context.UserEventType) for approving, rejecting, or submitting a timesheet. With the new UI enabled, all of these actions use the EDIT user event type, so you must update your user event scripts for timesheets

and time records. Instead of checking for a specific execution context type, these scripts should check the Approval Status field to determine whether it has been changed.

Enforcement of Defined Time Limits

When the new UI is not enabled, time limits are only validated when time transactions are submitted through the UI. With the new UI enabled, time limits are validated when an individual time transaction is added using SuiteScript. You must ensure that time transactions submitted using SuiteScript meet defined time limits.

Code Samples

The following samples are for SuiteScript 2.x:

- [Creating a New Weekly Timesheet](#)
- [Verifying Total Hours Upon Submit](#)
- [Updating a Custom Field on a Weekly Timesheet Based on Time Entry Field Values](#)

Creating a New Weekly Timesheet

```

1  /**
2   * @NapiVersion 2.x
3   * @NScriptType ClientScript
4   */
5  define(['N/record'], function(record) {
6      var weeklyTimesheet = record.create({
7          type: record.Type.TIME_SHEET,
8          isDynamic: true
9      });
10     weeklyTimesheet.selectNewLine({
11         sublistId: 'timeitem'
12     });
13     weeklyTimesheet.setCurrentSublistValue({
14         sublistId: 'timeitem',
15         fieldId: 'customer',
16         value: '48'});
17     weeklyTimesheet.setCurrentSublistValue({
18         sublistId: 'timeitem',
19         fieldId: 'item',
20         value: '117'});
21     weeklyTimesheet.setCurrentSublistValue({
22         sublistId: 'timeitem',
23         fieldId: 'location',
24         value: '1'
25     });
26     weeklyTimesheet.setCurrentSublistValue({
27         sublistId: 'timeitem',
28         fieldId: 'hours0',
29         value: '8'
30     });
31     weeklyTimesheet.setCurrentSublistValue({
32         sublistId: 'timeitem',
33         fieldId: 'memo',
34         value: 'timeentry created'
35     });
36     weeklyTimesheet.setCurrentSublistValue({
37         sublistId: 'timeitem',
38         fieldId: 'isbillable',
39         value: true
40     });
41     weeklyTimesheet.setCurrentSublistValue({
42         sublistId: 'timeitem',
43         fieldId: 'payrollitem',

```

```

44     value: '2'
45   });
46   weeklyTimesheet.setCurrentSublistValue({
47     sublistId: 'timeitem',
48     fieldId: 'paidexternally',
49     value: true
50   });
51   weeklyTimesheet.setCurrentSublistValue({
52     sublistId: 'timeitem',
53     fieldId: 'price',
54     value: '1'
55   });
56   weeklyTimesheet.setCurrentSublistValue({
57     sublistId: 'timeitem',
58     fieldId: 'override',
59     value: true
60   });
61   weeklyTimesheet.setCurrentSublistValue({
62     sublistId: 'timeitem',
63     fieldId: 'department',
64     value: '1'
65   });
66   weeklyTimesheet.setCurrentSublistValue({
67     sublistId: 'timeitem',
68     fieldId: 'class',
69     value: '1'
70   });
71   weeklyTimesheet.commitLine({
72     sublistId: 'timeitem'
73   });
74
75   var weeklyTimesheetId = weeklyTimesheet.save();
76 });

```

Verifying Total Hours Upon Submit

```

1  /**
2   * @NapiVersion 2.x
3   * @NScriptType ClientScript
4   */
5  define(['N/ui/dialog', 'N/error'], function(dialog, error) {
6    function saveRecord(context) {
7      var timesheetRecord = context.currentRecord;
8
9      if (timesheetRecord.getValue('totalhours') != '40') {
10       dialog.alert({
11         title: 'Alert',
12         message: 'Total time is not equal to 40'
13       });
14
15       throw error.create({
16         name: 'MISSING_REQ_ARG',
17         message: 'Total time is not equal to 40'
18       });
19     }
20     return true;
21   }
22   return {
23     saveRecord: saveRecord
24   };
25 });

```

Updating a Custom Field on a Weekly Timesheet Based on Time Entry Field Values

```

1  /**

```

```
2  *@NapiVersion 2.x
3  *@NScriptType ClientScript
4  */
5  define(['N/record'], function(record) {
6      /*
7       * Populates custom field 'custrecord_am_timesheet_billable_hours' with sum of time entry 'hours'
8       * values for time entries that are billable. Returns true to continue with record submit;
9       */
10     function timesheet_clientSaveRecord(context) {
11         var timesheetRecord = context.currentRecord;
12
13         var intSum = 0;
14         var intNumberLines = timesheetRecord.getLineCount({
15             sublistId: 'timeitem'
16         });
17         var arTimeGridColumns = ['hours0', 'hours1', 'hours2', 'hours3', 'hours4', 'hours5', 'hours6'];
18
19         for (var intLineCounter = 0; intLineCounter < intNumberLines; intLineCounter++){
20             for (var intDayCounter = 0; intDayCounter < arTimeGridColumns.length; intDayCounter++){
21                 var srecTimeEntry = timesheetRecord.getSublistValue({
22                     sublistId: 'timeitem',
23                     fieldId: arTimeGridColumns[intDayCounter],
24                     line: intLineCounter
25                 });
26                 if (srecTimeEntry != ' '){
27                     var bBillable = timesheetRecord.getSublistValue({
28                         sublistId: 'timeitem',
29                         fieldId: 'isbillable',
30                         line: intLineCounter
31                     });
32                     if (bBillable){
33                         intSum += srecTimeEntry;
34                     }
35                 }
36             }
37         }
38
39         timesheetRecord.setValue({
40             fieldId: 'custrecord_am_timesheet_billable_hours',
41             value: intSum
42         });
43         return true;
44     }
45     return {
46         saveRecord: timesheet_clientSaveRecord
47     };
48 });
```

Weekly Timesheet Record Actions

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The weekly timesheet record type currently supports the approve, reject, and submit actions.

For help working with this record in the UI, see the help topic [Weekly Timesheets](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

approve

Corresponding UI Button	Approve All Pending
-------------------------	---------------------

Action Description	Approve all pending timebills that the user has permission to approve on the timesheet with the specified ID.
Returns	{ "id":recordId }, list containing all timeBills' "notifications": [Error, Warning, Information and text] }
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The approve action does not support any additional parameters.

reject

Corresponding UI Buttons	Reject All Pending Reject All Pending with note
Action Description	Reject all pending timebills that the user has permission to reject on the timesheet with the specified ID. Optionally include a note that explains the reason for the rejection. Available when the Advanced Approvals on Time preference is enabled.
Returns	{ "id":recordId }, list containing all timeBills' "notifications": [Error, Warning, Information and text] }
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The reject action also supports the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
note	string	optional	An explanation of why the pending timebills were rejected.	2018.1

retract

Corresponding UI Button	Retract
Action Description	Retracts pending timebills on the timesheet with the specified ID. Retract changes the record's approval status from PENDING state back to OPEN state.

Returns	{ "id": recordId, "action": "retract", "success": [true or false], "recordCount": number of retracted timebills, "platformResponses": {timeBillId_1: responseObjectForTimeBill_1, timeBillId_2: responseObjectForTimeBill_2, ...}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2021.1

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. RecordId is a default input parameter for this action. No additional input parameters are necessary.

Sample Return Objects

The following sample shows how to use a retract action on weekly timesheet.

```
1  {
2
3  {
4
5  "id": recordId,
6
7  "action": "retract",
8
9  "success": [true or false],
10
11  "recordCount": number of retracted timebills,
12
13  "platformResponses":
14
15  {
16
17  timeBillId_1: responseObjectForTimeBill_1,
18
19  timeBillId_2: responseObjectForTimeBill_2,
20
21  ...
22  }
23  },
24  },
25  },
26
27  "notifications": [ Error, Warning, Information and text]
28
29 }
```

submit

Corresponding UI Buttons	Submit
Action Description	Submit open and rejected timebills on the timesheet with the specified ID. Available when the Advanced Approvals on Time preference is enabled.
Returns	{ "id": recordId }, list containing all timeBills' "notifications": [Error, Warning, Information and text] }
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .

Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the execution of any action. The submit action does not support any additional parameters.

Weekly Timesheet Record Macros

Applies to: SuiteScript 2.x | SuiteCloud Developer

The weekly timesheet record currently supports the checkTimeLimit and copyFromWeek macros.

For help working with this record in the UI, see the help topic [Weekly Timesheets](#).

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

checkTimeLimits

Corresponding UI Button	Check Time Limits
Macro Description	Gathers the contents of the timeitem machine and sends to a service. Based on preferences, returns the results of a server check of time limits: either indicates that limits were not exceeded or provides a list of errors.
Returns	Limit check notifications See Sample Return Objects .
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2018.2

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The checkTimeLimits macro also supports the following additional parameters.

Parameter	Type	Required / Optional	Description	Since
action	string	optional	The action for which the limits are checked. May be one of the following: ■ approve ■ save ■ submit	2018.1

Parameter	Type	Required / Optional	Description	Since
			 Note: Based on preferences enabled in the account, limits may not be checked for the approve action. If the submit action is available limits are not checked for the save action.	
limitcheck	boolean	optional	Checks limits regardless of the action.	2018.1

Sample Return Objects

```

1 // Limit check with some errors but allow the user to continue, with optional preferences setting
2 {
3   notifications: [
4     {
5       title: "Maximum hours per day rule violated",
6       message: "4/17/2017 has more than 10:00 hours",
7       severity: {
8         label = "Warning",
9         value = 2
10      }
11    },
12    {
13      title: "Maximum hours per week rule violated",
14      message: "Week has more than 40:00 hours",
15      severity: {
16        label = "Warning",
17        value = 2
18      }
19    }
20  ],
21  rules: [
22    {
23      name: "MAX_PER_DAY",
24      value: "8",
25      action: "WARN"
26    },
27    {
28      name: "MAX_PER_WEEK",
29      value: "48",
30      action: "DISALLOW"
31    }
32  ]
33 }
34
35 //Limit check with no errors:
36 {
37   notifications: [
38     {
39       title: "Limit check successful",
40       message: "All time records meet their respective rules",
41       severity: {
42         label = "Information",
43         value = 1
44       }
45     }
46   ]
47 }
48
49 Limit check that disallows continuation:
50 {
51   notifications: [
52     {
53       title: "Maximum hours per week rule violated",
54       message: "Week has more than 40:00 hours",
55       severity: {

```

```

56         label = "Error",
57         value = 3
58     }
59 }
60 ]
61 }

```

copyFromWeek

Corresponding UI Button	Copy From Week
Macro Description	Copies all lines from the specified week to the current weekly timesheet. Does not replace any lines in the weekly timesheet. Hours and memos may not be copied.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2018.2

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the execution of any macro. The checkTimeLimits macro also requires the following additional parameters.

Parameter	Type	Required / Optional	Description	Since
weekOf	date	required	The week from which to copy lines to the weekly timesheet.	2018.1
copyExact	boolean	required	Indicates whether hour and memo field values should be copied.	2018.1

Weekly Timesheet Macros Syntax

```

1  require(['N/currentRecord', 'N/format'], function(currentRecord, format){
2      var timesheet = currentRecord.get();
3
4      // Copy time data from the week of 7/10, including hours and memos
5      timesheet.executeMacro({
6          id: 'copyFromWeek',
7          params: {
8              weekOf : '7/10/2017',
9              copyExact : true
10         }
11     });
12
13     // Copy time data from the week of 14/10, but do not include hours and memos
14     timesheet.executeMacro({
15         id: 'copyFromWeek',
16         params: {
17             weekOf : '14/10/2017',
18             copyExact : false
19         }
20     });
21
22     // Check whether the data currently in timesheet meet limits set by the administrator (for example, maximum of 40 hours per week, minimum of 6
23     // hours per working day)
24     var limitCheckResult = timesheet.executeMacro({
25         id: 'checkTimeLimits',
26         params: {

```

```

26 |         action : 'submit'
27 |     }
28 | })
29 | });

```

Support

i Applies to: SuiteScript 2.x | SuiteCloud Developer

i Note: This topic applies to all versions of SuiteScript.

The following records are scriptable in SuiteScript:

- [Case](#)
- [Issue](#)
- [Issue Product](#)
- [Issue Product Version](#)
- [Solution](#)
- [Topic](#)

Case

i Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Creating Case Types](#).

The internal ID for this record is supportcase.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The case record is scriptable in both server and client SuiteScript.

Supported Functions

The case record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Issue

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Logging Issues](#).

The internal ID for this record is `issue`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The issue record is scriptable in both server and client SuiteScript.

Supported Functions

The issue record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Issue Product

Applies to: SuiteScript 2.x | SuiteCloud Developer

The use of products on issue records helps determine which product team is responsible for the resolution of that issue.

For help working with this record in the UI, see the help topic [Working with Products and Modules](#).

The internal ID for this record is `issueproduct`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Supported Script Types

The issue record is scriptable in both server and client SuiteScript.

Supported Functions

The record can be created, read, updated and deleted using SuiteScript. It cannot be copied, transformed or searched.

Code Sample

The following script example illustrates how to create an issue product record.

```

1 | var product = record.create({
2 |     type: record.Type.ISSUE_PRODUCT
3 | });
4 | product.setFieldValue({
5 |     fieldId: 'issueproduct',
6 |     value: 'new product'
7 | });
8 | var productId = product.save();

```

Issue Product Version

Applies to: SuiteScript 2.x | SuiteCloud Developer

If you track multiple versions of the same product, you can create versions and builds using the product version record.

For help working with this record in the UI, see the help topic [Setting Up Product Versions](#).

The internal ID for this record is productversion.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Supported Script Types

The issue record is scriptable in both server and client SuiteScript.

Supported Functions

The record can be created, read, updated and deleted using SuiteScript. It cannot be copied, transformed or searched.

Code Sample

The following script example illustrates how to create an issue product version record for a previously created issue product.

```

1 | var version = record.create({
2 |     type: record.Type.ISSUE_PRODUCT_VERSION
3 | });
4 | version.setFieldValue({
5 |     fieldId: 'issueproduct',
6 |     value: productId
7 | });
8 | version.setFieldValue({
9 |     fieldId: 'productversion',
10 |    value: 'New version'
11 | });
12 | ...
13 | var versionId = version.save();

```

The following script example illustrates how to add IssueProductBuild to IssueProductVersion.

```

1 | var version = record.load({
2 |     type: record.Type.ISSUE_PRODUCT_VERSION,
3 |     id: versionId // defined elsewhere
4 | });
5 | version.selectNewLine({
6 |     sublistId: 'availablebuild'
7 | });
8 | version.setCurrentSublistValue({
9 |     sublistId: 'availablebuild',

```



```

10     fieldId: 'build',
11     value: 'new build'
12 });
13 version.setCurrentSublistValue({
14     sublistId: 'availablebuild',
15     fieldId: 'description',
16     value: 'Build description'
17 });
18 ...
19 version.commitLine({
20     sublistId: 'availablebuild'
21 });
22 var recordId = version.save();

```

Solution

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is solution.

This record is available when the Knowledge Base feature is enabled at Setup > Company > Enable Features, on the CRM subtab. When the feature is enabled, you can access the inventory cost revaluation record in the UI by choosing Lists > Support > Solutions > New.

For help working with this record in the UI, see the help topic [Creating Knowledge Base Solutions](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The solution record is scriptable in both server and client SuiteScript.

Supported Functions

The solution record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Topic

i Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Creating Knowledge Base Topics](#).

The internal ID for this record is *topic*.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The topic record is scriptable only in server SuiteScript.

Supported Functions

The topic record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

File Cabinet

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The following records are scriptable in SuiteScript:

- [File](#)
- [Folder](#)

File

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the user interface, see the help topic [File Cabinet Overview](#).

The internal ID for this record is `file`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The File Cabinet record is scriptable in server SuiteScript only.

Supported Functions

This record is not fully scriptable. Only search is permitted.

Usage Notes

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

The file record is different from other NetSuite records. It has its own SuiteScript API. For information about the API for files, see the help topic [N/file Module](#).

Folder

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [File Cabinet Folders](#).

The internal ID for this record is folder.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The folder record is scriptable in server SuiteScript only.

beforeLoad event is not supported on user event scripts.

Supported Functions

The folder record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Lists

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following list records are scriptable in SuiteScript:

- Account
- Accounting Book
- Accounting Context
- Accounting Period
- Allocation Schedule
- Amortization Schedule
- Amortization Template
- Automated Clearing House
- Billing Account
- Billing Class
- Billing Rate Card
- Billing Schedule
- Bin
- BOM
- BOM Revision
- Budget Category
- Budget Exchange Rate
- Class
- Consolidated Exchange Rate
- Contact Category
- Contact Role
- Cost Category
- Currency
- Currency Rate
- Customer Category
- Customer Message
- Customer-Subsidiary Relationship
- Department
- Employee Change Request
- Employee Change Request Type
- Fair Value Price
- Financial Institution
- Format Profile
- Gift Certificate
- Global Account Mapping
- Global Inventory Relationship
- Goal
- Government-Issued ID Tracking
 - Driver's License
 - Passport
 - Other Government-Issued ID

- Group
- Inbound Shipment
- Intercompany Allocation Schedule
- Inventory Number
- Item Account Mapping
- Item Collection
- Item Collection Item Map
- Item Location Configuration
- Item Process Family
- Item Process Group
- Item Revision
- Kudos Feature Records
 - Kudos Feature Records
 - Organization Value
- Location
- Manufacturing Cost Template
- Manufacturing Routing
- Merchandise Hierarchy Level
- Merchandise Hierarchy Node
- Merchandise Hierarchy Version
- Nexus
- Note Type
- Order Type
- Other Name Category
- Partner Category
- Payment Instruments
- Payment Method
- Payroll Item
- Performance Metric
- Performance Review
- Performance Review Schedule
- Creating Price Books
- Price Level
- Price Plan
- Price Rule
- Price Rule Entry
- Pricing Group
- Project Charge Rule
- Project Expense Type
- Project Revenue Rule
- Project Work Breakdown Structure

- [Revenue Recognition Event](#)
- [Revenue Recognition Field Map](#)
- [Revenue Recognition Plan](#)
- [Revenue Recognition Schedule](#)
- [Revenue Recognition Template](#)
- [Role](#)
- [Recalculate Percent Complete Override](#)
- [Sales Role](#)
- [Sales Tax Item](#)
- [Subsidiary](#)
- [Subsidiary Settings](#)
- [Supply Chain Snapshot](#)
- [System Note](#)
- [Tax Control Account](#)
- [Tax Group](#)
- [Tax Period](#)
- [Tax Type](#)
- [Term](#)
- [Unit of Measure](#)
- [Vendor Category](#)
- [Vendor-Subsidiary Relationship](#)
- [Workplace](#)
- [Zone](#)

Account

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Chart of Accounts Management](#).

The internal ID for this record is account.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The accounting book record is supported in server SuiteScript only.

Supported Functions

The account record is fully scriptable using server SuiteScript.

Accounting Book

i Applies to: SuiteScript 2.x | SuiteCloud Developer

When the Multi-Book Accounting feature has been enabled, you can use the accounting book record to create secondary books.

For help working with this record in the UI, see the help topic [Using Multi-Book Accounting](#).

In the UI, you access the accounting book record at Setup > Accounting > Accounting Books > New.

The internal ID for this record is accountingbook.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The accounting book record is supported in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

This record is partially scriptable — it can be created, updated, deleted, and searched using SuiteScript.

Code Samples

The following samples show how to create accounting book records and perform other basic tasks.

```

1 // Create Record
2 var myACBook = record.create({
3   type: record.Type.ACCOUNTING_BOOK
4 });
5 myACBook.setValue({
6   fieldId: 'name',
7   value: 'test'
8 });
9
10 var subs = new ();
11 subs[0] = '1';
12 subs[1] = '3';
13 subs[2] = '5';
14

```

```

15 myACBook.setValue({
16     fieldId: 'subsidiary',
17     value: subs
18 });
19 myACBook.setValue({
20     fieldId: 'isprimary',
21     value: true
22 });
23
24 var recCreated = myACBook.save();
25
26 var id = 0;
27
28 // Update Record
29 var myACBook = record.load({
30     type: record.Type.ACCOUNTING_BOOK,
31     id: '3'
32 });
33
34 var subs = new ();
35 subs[0] = '1';
36 subs[1] = '3';
37 subs[2] = '5';
38
39 myACBook.setValue({
40     fieldId: 'subsidiary',
41     value: subs
42 });
43 myACBook.setValue({
44     fieldId: 'isprimary',
45     value: true
46 });
47
48 var recUpdated = myACBook.save();
49
50 var id = 0;
51
52 // Delete Record
53 var delrecId = record.delete({
54     type: record.Type.ACCOUNTING_BOOK,
55     id: '2'
56 });

```

Accounting Context

i Applies to: SuiteScript 2.x | SuiteCloud Developer

An accounting context can be a one-to-one relationship between a country's local GAAP (Generally Accepted Accounting Principles) reporting requirements and a statutory chart of accounts (COA). It can also be a unique relationship that meets your company's specific needs. Accounting contexts are useful when users prefer to work in a local GAAP context, rather than in the consolidated context with one centralized COA. Accounting contexts are also useful if you have Multi-Book Accounting provisioned in your account. You can set an accounting context specific for your secondary book and use it for your secondary book reports.

For help working with this record in the UI, see the help topic [Accounting Contexts](#).

The internal ID for this record is accountingcontext.

The accounting context record is available in OneWorld accounts.

The script user must have the Setup Company permission with Full permission level.

In the UI, you configure accounting contexts on the Accounting Contexts subtab on the General Preferences page at Setup > Company > General Preferences. If you do not have a OneWorld account, this subtab is not visible.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

Supported Operations

This record is partially scriptable — it can be read, created, deleted, copied, searched, and edited. It cannot be transformed.

Code Samples

The following samples show how to create an accounting context.

```
1 require(["N/record"], function(record){
2   var rec = record.create({
3     type: record.Type.ACCOUNTING_CONTEXT
4   });
5   rec.setValue({
6     fieldId: 'name',
7     value: 'Context 1'
8   });
9   var recId = rec.save();
10 });
```

Accounting Period

Applies to: SuiteScript 2.x | SuiteCloud Developer

When the Accounting Periods feature has been enabled, this record is available to SuiteScript.

In the UI, you access accounting periods at Setup > Accounting > Manage Accounting Periods.

For help working with this record in the UI, see the help topic [Accounting Period Setup](#).

The internal ID for this record is accountingperiod.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The accounting period record is supported in all client and server scripts.

Supported Functions

Only search and read functions are supported.

Code Sample

```

1 var period = record.load({
2   type: record.Type.ACCOUNTING_PERIOD,
3   id: '1'
4 });
5 var startDate = period.getValue({
6   fieldId: 'startdate'
7 });
8 var endDate = period.getValue({
9   fieldId: 'enddate'
10 });

```

Allocation Schedule

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Allocation schedules transfer balances from expense accounts into one or more other accounts.

The internal ID for this record is allocationschedule.

The allocation schedule record is available only when the Accounting Periods and Expense Allocation features are enabled. An administrator can enable these features at Setup > Company > Enable Features under the Accounting subtab. For information about expense allocation, see the help topic [Expense Allocation Overview](#).

For help working with this record in the UI, see the help topic [Creating Expense Allocation Schedules](#).

If the Statistical Accounting feature is enabled, the allocation schedule record includes fields that enable you to base the weight for the allocation on the balance of a statistical account through statistical journals or as an absolute value. If the Dynamic Allocation feature is also enabled, the weight is dynamically calculated when the allocation journal is generated. An administrator can enable these features at Setup > Company > Enable Features and select Accounting under Advanced Features. For information about statistical allocation schedules, see the help topic [Working with Allocation Schedules Weighted by the Balance of a Statistical Account](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

If the Dynamic Allocation feature is enabled, the currency field is not scriptable. None of the fields from the History subtab on the allocation schedule record in the UI are not scriptable.

Supported Operations

This record is partially scriptable – it can be read, created, deleted, copied, and updated. It cannot be transformed.

Code Samples

The following samples show how to create a statistical allocation schedule that divides the balance of one account and transfers the amounts into two other accounts.

```

1  var myAllocSched = record.create({
2      type: record.Type.ALLOCATION_SCHEDULE,
3      defaultValues: false
4  });
5
6  myAllocSched.setValue({
7      fieldId: 'name',
8      value: 'Allocation Schedule A'
9  });
10 myAllocSched.setValue({
11     fieldId: 'subsidiary',
12     value: '1'
13 });
14 myAllocSched.setSublistValue({
15     sublistId: 'allocationsource',
16     fieldId: 'account',
17     line: 1,
18     value: '24'
19 });
20 myAllocSched.setSublistValue({
21     sublistId: 'allocationdestination',
22     fieldId: 'account',
23     line: 1,
24     value: '6'
25 });
26 myAllocSched.setSublistValue({
27     sublistId: 'allocationdestination',
28     fieldId: 'weight',
29     line: 1,
30     value: '50.00'
31 });
32 myAllocSched.setSublistValue({
33     sublistId: 'allocationdestination',
34     fieldId: 'account',
35     line: 1,
36     value: '32'
37 });
38 myAllocSched.setSublistValue({
39     sublistId: 'allocationdestination',
40     fieldId: 'weight',
41     line: 1,

```

```

42     value: '50.00'
43   });
44
45   recordId = myAllocSched.save();

```

Amortization Schedule

Applies to: SuiteScript 2.x | SuiteCloud Developer

Amortization schedules are automatically created by the system for transactions that contain item or expense lines associated with amortization templates.

The internal ID for this record is `amortizationschedule`.

The amortization schedule record is available only when the Amortization feature is enabled, at Setup > Company > Enable Features, on the Accounting subtab. In the UI, you access this record at Lists > Accounting > Amortization Schedules.

For help working with this record in the UI, see the help topic [Amortization Schedules](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript only.

The user events are not supported.

Supported Functions

This record is partially scriptable — it can be read, searched, and edited. It cannot be copied, created, deleted, or transformed.

Note: Edits are allowed only when the Allow Users to Modify Amortization Schedules preference has been selected. For details, see the help topic [Setting Amortization Preferences](#).

Usage Notes

Be aware of the following:

- As in the UI, you cannot use external ID as part of your criteria when searching for amortization schedule records. You also cannot include external ID in search columns.

- The amortization schedule record is similar to the revenue recognition schedule record — so if you have existing integrations for revenue recognition schedules, you may be able to reuse elements of these scripts. The revenue recognition schedule record is described in [Revenue Recognition Schedule](#).

Amortization Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the amortization template record to define the terms of amortization schedules, which are automatically created by the system. For example, on certain transactions, you can associate an amortization template with a line in the item or expense sublist. When the transaction is saved, an amortization schedule based on the template is automatically created. You can use amortization templates in conjunction with journal entry, vendor bill, and vendor credit records.

The internal ID for this record is `amortizationtemplate`.

The amortization template record is available only when the Amortization feature is enabled, at Setup > Company > Enable Features, on the Accounting subtab. In the UI, you access this record at Lists > Accounting > Amortization Templates > New.

For help working with amortization templates in the UI, see the help topic [Creating Amortization Templates](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The amortization template record is supported in server SuiteScript only.

The user events are not supported.

Supported Operations

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

Be aware of the following:

- If the Method field (recurrencetype) is set to Custom, then you must include at least one line in the recurrence sublist. In this case, the template cannot be deleted.
- As in the UI, you cannot use external ID as part of your criteria when searching for amortization template records. You also cannot include external ID in search columns.

- The amortization template record is similar to the revenue recognition template record — so if you have existing integrations for revenue recognition templates, you may be able to reuse elements of these scripts. The revenue recognition template record is described in [Revenue Recognition Template](#).

Code Sample

The following sample shows how to create amortization templates and perform other basic tasks.

```

1 // Create Record
2 var recAmTemp = record.create({
3   type: record.Type.AMORTIZATION_TEMPLATE
4 });
5 recAmTemp.setValue({
6   fieldId: 'name',
7   value: 'Name'
8 });
9 recAmTemp.setValue({
10  fieldId: 'externalid',
11  value: 'externalId'
12 });
13 recAmTemp.setValue({
14  fieldId: 'isamortization',
15  value: true
16 });
17 recAmTemp.setValue({
18  fieldId: 'amortizationtype', // Type
19  value: 'STANDARD'
20 });
21 recAmTemp.setValue({
22  fieldId: 'recurrencetype',
23  value: 'EVENPERIODSPRORATE' // Method
24 });
25 recAmTemp.setValue({
26  fieldId: 'recognitervalsrc',
27  value: 'RECEIPTDATE' // Term Source
28 });
29 recAmTemp.setValue({
30  fieldId: 'revrecoffset',
31  value: 0
32 });
33 recAmTemp.setValue({
34  fieldId: 'periodoffset',
35  value: 1
36 });
37 recAmTemp.setValue({
38  fieldId: 'acctdeferral',
39  value: '1'
40 });
41 recAmTemp.setValue({
42  fieldId: 'acctcontra',
43  value: '1'
44 });
45 recAmTemp.setValue({
46  fieldId: 'accttarget',
47  value: '1'
48 });
49 recAmTemp.setValue({
50  fieldId: 'amortizationperiod',
51  value: 20
52 });
53 recAmTemp.setValue({
54  fieldId: 'residual',
55  value: '1.00'
56 });
57 recAmTemp.setValue({
58  fieldId: 'initialamount',
59  value: '12.00'
60 });
61 recAmTemp.setValue({
62  fieldId: 'isinactive',

```

```

63     value: false
64   });
65   var recId = recAmTemp.save();
66
67   // Load Record
68   var myAmorTemplate = record.load({
69     type: record.Type.AMORTIZATION_TEMPLATE,
70     id: recId
71   });
72
73   // Copy
74   var recCopied = record.copy({
75     type: record.Type.AMORTIZATION_TEMPLATE,
76     id: recId
77   });
78
79   // Delete Record
80   var delrecId = record.delete({
81     type: record.Type.AMORTIZATION_TEMPLATE,
82     id: recId
83   });

```

Automated Clearing House

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `automatedclearinghouse`. This record is not a subrecord and does not have any subrecords.

Use the automated clearing house record when working with the ACH payment instrument type. For details, see the help topic [Payment Instruments](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Prerequisites

To use the automated clearing house record, you must:

- enable the Payment Instruments feature. For instructions, see the help topic [Payment Instruments](#).
- create an instance of the ACH type payment method. For instructions, see the help topic [Creating a Payment Method](#).

Supported Script Types

The automated clearing house record is scriptable in both client and server SuiteScript.

Supported Operations

The following operations are available for scripting with the automated clearing house record: read, create, update, delete, and search.

The System Notes sublist is search-only.

The following fields cannot be updated after saving:

- Customer (field ID: `entity`)

- Mask (field ID: mask)
- Bank Account Number (field ID: bankaccountnumber)
- Routing Numer (field ID: routingnumber)
- Account Owner Name (field ID: accountownername)
- Account Type (field ID: accounttype)

Code Sample

The following code sample shows how to create an automated clearing house record:

```
1 var record = nlapiCreateRecord('automatedclearinghouse');
2 record.setFieldValue('entity', ID of Customer);
3 record.setFieldValue('paymentmethod', ID of ACH Payment Method);
4 record.setFieldValue('bankaccountnumber', '12345674');
5 record.setFieldValue('routingnumber', '123456789');
6 record.setFieldValue('accountownername', 'John Doe');
7 record.setFieldValue('accounttype', 'CHECKING');
8 record.setFieldValue('bankname', 'KB');
9 record.setFieldValue('memo', 'Memo1234');
10 record.setFieldValue('isinactive', 'F');
11 record.setFieldValue('isdefault', 'T');
12 record.setFieldValue('preserveonfile', 'T');
13 record.setFieldValue('limit', '10');
14 record.setFieldValue('legalconsent', 'This is legal consent created by script');
15 var id = nlapiSubmitRecord(record);
```

Billing Account

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `billingaccount`. Full support is provided for client and server scripting with this record.

For help working with this record in the UI, see the help topic [Creating Billing Accounts](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The billing account record is scriptable in both client and server SuiteScript.

Supported Functions

The billing account record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Billing Class

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the billing class record to create different rates that you can use when calculating the cost of a resource's time on a project. You can use SuiteScript to create, update, and delete billing class records.

The internal ID for this record is `billingclass`.

The billing class record is available when the Per-Employee Billing Rates feature is enabled at Setup > Company > Enable Features, on the Employees subtab. When the feature is enabled, you can access the billing class record in the UI by choosing Setup > Accounting > Billing Classes > New.

For help working with this record in the UI, see the help topic [Using Billing Classes](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The billing class record is scriptable in server SuiteScript only.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The billing class record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Field Definitions

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Code Sample

The following sample shows how to create a billing class record.

```
1 var billingClass = record.create({
2   type: record.Type.BILLING_CLASS
3 });
4
5 billingClass.setValue({
6   fieldId: 'name',
7   value: 'Billing Class'
```

```

8  });
9  billingClass.setValue({
10     fieldId: 'description',
11     value: 'Billing Class Description'
12  });
13  billingClass.setValue({
14     fieldId: 'isinactive',
15     value: false
16  });
17  billingClass.setSublistValue({
18     sublistId: 'pricecost',
19     fieldId: 'price',
20     line: 1,
21     value: 2.56
22  });

```

The following sample shows how to delete a billing class record.

```

1  record.delete({
2     type: record.Type.BILLING_CLASS,
3     id: recId
4  });

```

Billing Rate Card

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `billingratecard`.

If you use billing classes, you can also use billing rate cards to define different billing rates for groups of billing classes. These rate cards can then be used to set billing rates on charge-based projects using time-based charge rules. You can use SuiteScript to read, create, update, delete, and search billing rate card records.

The billing rate card record is available when the Billing Rate Cards feature is enabled at Setup > Company > Enable Features, on the Employees subtab. The Per-Employee Billing Rates and Charge-Based Billing features are also required to use billing rate cards. When the feature is enabled, you can access the billing rate card record in the UI by choosing Setup > Accounting > Billing Rate Cards > New.

For help working with this record in the UI, see the help topic [Using Billing Rate Cards](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Billing Rate Card Record Macros](#) for the macros associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The billing account record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript actions are supported for this record:

- Read
- Create
- Edit
- Delete
- Search

Usage Notes

There are two sublists for rate card pricing: `ratecardpricing` and `ratecardpricingmulti`. These two lists are mutually exclusive. The first list, `ratecardpricing`, is a simple sublist used when the Multiple Currencies feature is not enabled. The second list, `ratecardpricingmulti`, is a matrix sublist used when the Multiple Currencies feature is enabled.

When this feature is enabled, the content of `ratecardpricing` is a subset of `ratecardpricingmulti`. If you enter information into `ratecardpricing` and then enable the Multiple Currencies feature, the original content is in a column in the `ratecardpricingmulti` sublist.

 **Note:** Copy and transform are not supported.

Code Samples

The following sample shows how to create a new billing rate card with a single billing class without multiple currencies.

```

1  var myBillingRate = record.create({
2      type: record.Type.BILLING_RATE_CARD
3  });
4  myBillingRate.setValue({
5      fieldId: 'name',
6      value: 'abcd'
7  });
8  myBillingRate.setSublistValue({
9      sublistId: 'ratecardpricing',
10     fieldId: 'price',
11     line: 1,
12     value: 11
13 });
14 var recId = myBillingRate.save();

```

The following sample shows how to update the billing rate card name and a price in British pounds when multiple currencies is enabled.

```

1  var myBillingRate = record.load({
2      type: record.Type.BILLING_RATE_CARD,
3      id: 3
4  });
5  var name = myBillingRate.getValue({
6      fieldId: 'name'
7  });
8  var updatedName = name + ' - updated';
9  myBillingRate.setValue({
10     fieldId: 'name',
11     value: updatedName
12 });

```

```

13 myBillingRate.setSublistValue({
14     sublistId: 'ratecardpricingmulti',
15     fieldId: 'price_2_',
16     line: 1,
17     value: 5
18 });
19 var recId = myBillingRate.save();

```

The following sample shows how to use a user event script to update a price.

```

1 // To get a new record in SuiteScript 2.x, use the following code in a beforeLoad, beforeSubmit,
2 // or afterSubmit user event script:
3 function afterSubmit(context) {
4     var newRec = context.newRecord;
5 }
6 // then use the new record as follows:
7 var origPrice = newRec.getSublistValue({
8     sublist: 'ratecardpricing',
9     fieldid: 'price',
10    line: 1
11 });
12
13 var newPrice = 11;
14
15 newRec.setSublistValue({
16     sublistId: 'ratecardpricing',
17     fieldId: 'price',
18     line: 1,
19     value: 'newPrice'
20 });
21 log.debug({
22     title: 'Price',
23     details: 'price changed from ' + origPrice + ' to ' + newPrice
24 });

```

Billing Rate Card Record Macros

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The billing rate card record type currently supports the `modifyPricebyPercent` macro.

For help working with this record in the UI, see the help topic [Using Billing Rate Cards](#).

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

modifyPriceByPercent

Corresponding UI Button	Recalculate
Macro Description	Modifies the price list on the record by a specified percentage.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2018.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the of any macro. The modifyPriceByPercent macro also requires the following additional parameter.

Parameter	Type	Required / Optional	Description	Since
percent	number	required	The percentage amount by which the price list is modified. May be positive or negative.	2018.1

Billing Rate Card modifyPriceByPercent Macro Syntax

The following code sample is from a Suitelet that increases the prices on a billing rate card with id 1 by 10%.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptName Script Parameter Test
4   * @NScriptType suitelet
5   */
6  define(['N/record'], function(record) {
7      return Object.freeze({
8          onRequest : function(context) {
9              var rec = record.load({
10                 type: 'billingratecard',
11                 id: 1,
12                 isDynamic: true
13             });
14             rec.executeMacro({
15                 id: 'modifyPriceByPercent',
16                 params: { percent: 10 }
17             });
18             rec.save();
19         }
20     });
21 });

```

Billing Schedule

Applies to: SuiteScript 2.x | SuiteCloud Developer

This record enables you to create a billing schedule that can be applied to a sales order, a line item on a sales order, or a project. SuiteScript supports all five types of billing schedules (charge-based, fixed bid interval, fixed bid milestone, standard, and time and materials).

The internal ID for this record is billingschedule.

To use the billing schedule record, you must enable the Advanced Billing feature, at Setup > Company > Enable Features, on the Transactions subtab. In the UI, you access this record at Lists > Accounting > Billing Schedules > New. You can access a billing schedule of fixed bid milestone type through the project record's Financial subtab.

For help working with this record in the UI, see the help topic [Billing Schedules](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Billing Schedule is scriptable in server SuiteScript only.

Both beforeSubmit, and afterSubmit user events are supported.

Supported Functions

This record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

Additionally, note the following:

- To set a value for schedule type, which is a required body field, you must use `initializeValues`, not `setValue`. For examples, see the following section.
- If you choose a schedule type of fixed bid milestone, you must identify an existing project record (or job record). You do so using `initializeValues`. Then, to create a link between the project and the new billing schedule, you must update the project record — this relationship is not established automatically when you create the billing schedule.
- The Recurrence sublist is available only when schedule type is set to Standard and frequency to Custom.
- The Milestone sublist is available only when schedule type is set to Fixed Bid Milestone.

Code Samples

The following samples show how to create different types of billing schedules.

Creating a Charge-Based Billing Schedule

The following sample shows how to create a charge-based billing schedule.

```

1  var SCHEDULE_TYPE = 'CB'; //Charge-Based
2  var recName = SCHEDULE_TYPE + 'record';
3
4  var initValues = new ();
5  initValues.schedtype = SCHEDULE_TYPE;
6
7  //Create record
8  var myBillingSched = record.create({
9      type: record.Type.BILLING_SCHEDULE,
10     defaultValues: {
11         'schedtype': SCHEDULE_TYPE
12     }
13 });
14 myBillingSched.setValue({
15     fieldId: 'externalid',

```

```

16     value: 'EXTID001'
17   });
18   myBillingSched.setValue({
19     fieldId: 'name',
20     value: recName
21   });
22   myBillingSched.setValue({
23     fieldId: 'frequency',
24     value: 'DAILY'
25   });
26   myBillingSched.setValue({
27     fieldId: 'dayperiod',
28     value: '3'
29   });
30   var recId = myBillingSched.save();

```

Creating a Fixed Bid Milestone Billing Schedule

The following sample shows how to create a billing schedule of fixed bid milestone type. Note that this sample references a particular project record during the creation of the billing schedule. However, you still have to establish the relationship between the billing schedule and the project as a separate step.

```

1  var SCHEDULE_TYPE = 'FBM'; // Fixed Bid, Milestone
2  var P1 = '117';           // Project1
3  var P1M1 = '112';         // Project1 - Milestone1
4  var P1M2 = '113';         // Project2 - Milestone2
5
6  var recId = null;
7  var recName = SCHEDULE_TYPE + " record";
8
9  var initValues = new ();
10 initValues.schedtype = SCHEDULE_TYPE;
11 initValues.project = P1;
12
13 //Create record
14 var myBillingSched = record.create({
15   type: record.Type.BILLING_SCHEDULE,
16   isDynamic: true,
17   defaultValues: initValues
18 });
19
20 myBillingSched.setValue({
21   fieldId: 'name',
22   value: recName
23 });
24 myBillingSched.setValue({
25   fieldId: 'initialamount',
26   value: '10' // 10%
27 });
28
29 //Create the sublist
30 myBillingSched.selectNewLine({
31   sublistId: 'milestone'
32 });
33 myBillingSched.setCurrentSublistValue({
34   sublistId: 'milestone',
35   fieldId: 'milestoneamount',
36   value: '25' // 25%
37 });
38 myBillingSched.setCurrentSublistValue({
39   sublistId: 'milestone',
40   fieldId: 'milestonedate',
41   value: new Date('November 21, 2013')
42 });
43 myBillingSched.setCurrentSublistValue({
44   sublistId: 'milestone',
45   fieldId: 'projecttask',
46   value: P1M1
47 });

```

```

48 myBillingSched.commitLine({
49     sublistId: 'milestone'
50 });
51 recId = myBillingSched.save();
52
53 //Update the project (link it with the newly created Billing Schedule)
54 var project = record.load({
55     type: record.Type.JOB,
56     id: P1
57 });
58 project.setValue({
59     fieldId: 'jobbillingtype',
60     value: SCHEDULE_TYPE
61 });
62 project.setValue({
63     fieldId: 'billingschedule',
64     value: recId
65 });
66 var pId = project.save();

```

Bin

Applies to: SuiteScript 2.x | SuiteCloud Developer

Bins help you track on-hand quantities within a warehouse. Tracking items by bins can help organize receiving items and simplify picking items to fulfill orders.

For example, if you use bins, when you receive a purchase order, the order can tell you which bin to place the items in. Then, your stock level of that item in that bin is tracked.

To use the bin record, the Bin Management feature must be enabled at Setup > Company > Enable Features. On the Items & Inventory subtab, check the Bin Management box.

For help working with this record in the UI, see the help topic [Bin Management](#).

The internal ID for this record is bin.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for the bin record. It is scriptable in server SuiteScript only.

Supported Functions

The bin record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

BOM

Applies to: SuiteScript 2.x | SuiteCloud Developer

A Bill of Materials, or BOM, lists the quantities of raw materials, assemblies, sub-components, and parts needed to manufacture a product. A BOM can be used to communicate between manufacturing partners, multiple facilities within the organization, or with a single manufacturing plant. The BOM that is scriptable in NetSuite is the Advanced BOM.

This record is supported when the Advanced BOM feature is enabled at Setup > Company > Enable Features, on the Items & Inventory subtab. After this feature has been enabled, the Advanced BOM replaces the Assembly/Bill of Materials record.

For help working with this record in the UI, see the help topic [Advanced Bill of Materials](#).

In the UI, this record is accessed by going to Lists > Supply Chain > Bill of Materials.

The internal ID for this record is bom.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The BOM record is scriptable in client and server SuiteScript.

Supported Functions

The BOM record is fully scriptable, which means that it can be read, created, edited, copied, deleted, and searched using SuiteScript.

BOM Code Samples

- [Create a BOM](#)
- [Copy a BOM](#)
- [Delete a BOM](#)
- [Full BOM Code Sample](#)

Create a BOM

The following sample shows how to create a new bill of materials record.

```
1 | require(['N/record'], function(record){
```

```

2      function createBom(values){
3          var rec = record.create({
4              type: record.Type.BOM,
5              isDynamic: true
6          });
7          for (var key in values){
8              if (values.hasOwnProperty(key)){
9                  rec.setValue({
10                     fieldId: key,
11                     value: values[key]
12                 });
13             }
14         }
15         return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
16     }
17     var bomId = createBom({
18         'name': 'SS 2.0 BOM',
19         'subsidiary': [4,5],
20         'includechildren' : true,
21         'usecomponentyield': true
22     });
23 });

```

Copy a BOM

The following sample shows how to copy a bill of materials record.

```

1  require(['N/record'],function(record){
2      function copyBom(bomId, values){
3          var rec = record.copy({
4              type: record.Type.BOM,
5              id: bomId,
6              isDynamic: true
7          });
8          for (var key in values){
9              if (values.hasOwnProperty(key)){
10                 rec.setValue({
11                     fieldId: key,
12                     value: values[key]
13                 });
14             }
15         }
16         return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
17     }
18     var bomId = copyBom(1,{
19         'name': 'SS 2.0 BOM Copy',
20         'subsidiary': [4],
21         'includechildren' : false,
22         'usecomponentyield': false
23     });
24 });

```

Delete a BOM

The following sample shows how to delete a bill of materials record.

```

1  require(['N/record'],function(record){
2      function deleteBom(bomId){
3          record.delete({
4              type: record.Type.BOM,
5              id: bomId
6          });
7      }
8      deleteBom(2);
9  });

```

Full BOM Code Sample

The following sample shows how to create, copy, and delete a bill of materials record, including edits to BOM revisions. For samples specific to BOM revisions, see [BOM Revision Code Samples](#).

```

1  require(['N/record', 'N/search'], function(record, search){
2      // create bill of materials (function)
3      function createBom(values){
4          var rec = record.create({
5              type: 'bom',
6              isDynamic: true
7          });
8          for (var key in values){
9              if (values.hasOwnProperty(key)){
10                 rec.setValue({
11                     fieldId: key,
12                     value: values[key]
13                 });
14             }
15         }
16         return rec.save({
17             enableSourcing: false,
18             ignoreMandatoryFields: false
19         });
20     }
21
22     // create bill of materials revision (function)
23     function createBomRevision(values, components){
24         var rec = record.create({
25             type: 'bomrevision',
26             isDynamic: true
27         });
28         for (var key in values){
29             if (values.hasOwnProperty(key)){
30                 rec.setValue({
31                     fieldId: key,
32                     value: values[key]
33                 });
34             }
35         }
36         for (var i in components){
37             rec.selectNewLine({
38                 sublistId: 'component'
39             });
40             for (var key in components[i]){
41                 if (components[i].hasOwnProperty(key)){
42                     rec.setCurrentSublistValue({
43                         sublistId: 'component',
44                         fieldId: key,
45                         value: components[i][key]
46                     });
47                 }
48             }
49             rec.commitLine({sublistId: 'component'});
50         }
51         return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
52     }
53
54     // update bill of materials revision (function)
55     function updateBomRevision(revisionId, values){
56         var rec = record.load({
57             type: 'bomrevision',
58             id: revisionId,
59             isDynamic: true
60         });
61         for (var key in values){
62             if (values.hasOwnProperty(key)){
63                 rec.setValue({
64                     fieldId: key,
65                     value: values[key]
66                 });

```

```

67     }
68   }
69   return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
70 }
71
72 // copy bill of materials (function)
73 function copyBom(bomId, values){
74   var rec = record.copy({
75     type: 'bom',
76     id: bomId
77   });
78   for (var key in values){
79     if (values.hasOwnProperty(key)){
80       rec.setValue({
81         fieldId: key,
82         value: values[key]
83       });
84     }
85   }
86   return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
87 }
88
89 // delete bill of materials (function)
90 function deleteBom(bomId){
91   record.delete({
92     type: 'bom',
93     id: bomId
94   });
95 }
96
97 // get bill of materials revisions (function)
98 function getBomRevisions(bomId){
99   //search ids for all revisions for given BoM
100   var searchObj = search.create({
101     type: 'bomrevision',
102     filters: [
103       ['billofmaterials', 'is', [bomId]]
104     ],
105     columns: ['name']
106   });
107   var revisions = [];
108   searchObj.run().each(function(result){
109     revisions.push({
110       'id': result.id,
111       'name': result.getValue('name')
112     });
113     return true;
114   });
115   return revisions;
116 }
117
118 // create BoM - call createBom function
119 var bom1Id = createBom({
120   'name': 'SS 2.0 BoM',
121   'subsidiary': [4,5],
122   'includechildren': true,
123   'usecomponentyield': true
124 });
125
126 // create new BoM revision - call createBomRevision function
127 var bom1Rev1Id = createBomRevision({
128   'billofmaterials': bom1Id,
129   'name': 'Rev.1',
130   'effectivestartdate': new Date('1/1/2017')
131 }, [
132   {'item': 63, 'bomquantity': 1},
133   {'item': 64, 'bomquantity': 2}
134 ]);
135
136 // update BoM Revision Effective End Date to prevent a collision - call updateBomRevision function
137 bom1Rev1Id = updateBomRevision(bom1Rev1Id, {
138   'effectiveenddate': new Date('12/31/2018')
139 });

```

```

140
141 // create another new BoM revision - call createBomRevision function
142 var bom1Rev2Id = createBomRevision({
143     'billofmaterials': bom1Id,
144     'name': 'Rev.2',
145     'effectivestartdate': new Date('1/1/2019')
146 }, [
147     {'item': 64, 'bomquantity': 3},
148     {'item': 62, 'bomquantity': 1}
149 ]);
150
151 // copy BoM and update some values - call copyBom function */
152 var bom2Id = copyBom(bom1Id, {
153     'name': 'SS 2.0 BoM Copy',
154     'subsidiary': [4],
155     'usecomponentyield': false,
156     'includechildren': false
157 });
158
159 // find BoM Revisions of BoM - call getBomRevisions function
160 var revisions = getBomRevisions(bom1Id);
161 // update first component's Component Yield value of each found BoM Revision
162 for (var i = 0; i < revisions.length; i++){
163     rec = record.load({
164         type: 'bomrevision',
165         id: revisions[i].id
166     });
167     rec.setSublistValue({
168         sublistId: 'component',
169         fieldId: 'componentyield',
170         line: 0,
171         value: 50
172     });
173     rec.save();
174 }
175
176 // delete BoMs - call deleteBom function
177 deleteBom(bom1Id);
178 deleteBom(bom2Id);
179 });

```

BOM Revision

Applies to: SuiteScript 2.x | SuiteCloud Developer

A BOM revision enables you to update a BOM's details throughout the product lifecycle. A BOM revision also provides an accessible revisions history. Use revisions to compare and track cost savings when many BOM revisions are used in production. This record is available when the Advanced BOM feature is enabled at Setup > Company > Enable Features, on the Items & Inventory subtab. For information about this feature, see the help topic [Advanced Bill of Materials](#).

In the UI, this record is accessed by going to Lists > Supply Chain > Bill of Materials.

For help working with this record in the UI, see [BOM Revision](#).

The internal ID for this record is bomrevision.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The BOM Revision record is scriptable in server SuiteScript only.

Supported Functions

The BOM Revision record is fully scriptable, which means that it can be read, created, edited, copied, deleted, and searched using SuiteScript.

BOM Revision Code Samples

- [Create a BOM Revision](#)
- [Search for a BOM Revision](#)
- [Update a BOM Revision](#)
- [Full BOM Code Sample](#)

Create a BOM Revision

The following sample shows how to create a new bill of materials revision record.

```

1  require(['N/record'], function(record){
2      function createBomRevision(values, items){
3          var rec = record.create({
4              type: 'bomrevision', isDynamic: true
5          });
6          for (var key in values){
7              if (values.hasOwnProperty(key)){
8                  rec.setValue({
9                      fieldId: key,
10                     value: values[key]
11                 });
12             }
13         }
14         for (var itemIndex in items){
15             rec.selectNewLine({
16                 sublistId: 'component'
17             });
18             for (var key in items[itemIndex]){
19                 if (items[itemIndex].hasOwnProperty(key)){
20                     rec.setCurrentSublistValue({
21                         sublistId: 'component',
22                         fieldId: key,
23                         value: items[itemIndex][key]
24                     });
25                 }
26             }
27             rec.commitLine({sublistId: 'component'});
28         }
29         return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
30     }
31
32     var bomRevisionId = createBomRevision({
33         'billofmaterials': 1,
34         'name': 'Rev.1',
35         'effectivestartdate': new Date('1/1/2017')
36     }, [
37         {'item': 63, 'bomquantity': 1},
38         {'item': 62, 'bomquantity': 2}

```

```

39     });
40 });

```

Search for a BOM Revision

The following sample shows how to search for a bills of materials revision record.

```

1  require(['N/search'],function(search){
2      function getBomRevisions(bomId){
3          // search ids for all revisions for given BoM
4          var searchObj = search.create({
5              type: 'bomrevision',
6              filters: [{
7                  name: 'billofmaterials',
8                  operator: search.Operator.IS,
9                  values: [bomId]
10             }],
11             columns: ['name']
12         });
13         var revisions = [];
14         searchObj.run().each(function(result) {
15             revisions.push({
16                 'id': result.id,
17                 'name': result.getValue('name')
18             });
19             return true;
20         });
21         return revisions;
22     }
23     var revisions = getBomRevisions(1);
24 });

```

Update a BOM Revision

The following sample shows how to update a bills of materials revision record.

```

1  require(['N/record'], function(record){
2      function updateBomRevision(revisionId, values){
3          var rec = record.load({
4              type: 'bomrevision',
5              id: revisionId
6          });
7          for (var key in values){
8              if (values.hasOwnProperty(key)){
9                  rec.setValue({
10                     fieldId: key,
11                     value: values[key]
12                 });
13             }
14         }
15         return rec.save({enableSourcing: false, ignoreMandatoryFields: false});
16     }
17     updateBomRevision(1, {
18         'effectiveenddate': new Date('12/31/2018')
19     });
20 });

```

Budget Category

i Applies to: SuiteScript 2.x | SuiteCloud Developer

When the Multiple Budgets feature is enabled, you can use budget category values on budget records to categorize types of budgets. In NetSuite OneWorld, budget category values are used to indicate whether

budgeting is done at the local subsidiary level in local currency or globally using the root subsidiary's currency. Each budget category is associated with a budget category type of local or global.

After the Multiple Budgets feature is enabled, each budget you set up must be assigned a category. Budget category values allow differentiation between budgets for which other criteria, such as Item, Customer or Project, Class, Department, or Location, are identical. For more information about budget categories in NetSuite, see the help topic [Multiple Budgets and Budget Categories](#). For help creating this record in the user interface, see the help topic [Creating Budget Categories for Local Subsidiary Budgeting](#).

The internal ID for this record is `budgetcategory`.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Prerequisites

Budget categories are available only when the Multiple Budgets feature is enabled. Multiple Budgets is on the Enable Features page at Setup > Company > Setup Tasks > Enable Features (Administrator). The Multiple Budgets feature appears on the **Accounting** subtab under Advanced Features. An administrator must enable this feature.

Supported Script Types

The budget category is scriptable in both client and server SuiteScript.

Supported Functions

The budget category record is fully scriptable. It can be created, read, updated, deleted, and searched using SuiteScript.

Usage Notes

The **Global** field is accessed in SuiteScript using the **budgetType** field. For more information, see [Code Sample](#). Like the **Global** field in the user interface, the **budgetType** field can't be edited after the budget category record is saved.

Code Sample

The following sample shows how to create a budget category record:

 **Note:** The **Global** field is accessed using the **budgetType** field below. The **budgetType** field is set as true, which means that budgets assigned this budget category have a type of global, and are defined in the base currency of the root parent subsidiary.

```
1 | require(['N/record'], function(record){
2 |   var budgetCategory= record.create({
3 |     type: record.Type.BUDGET_CATEGORY,
```



```

4      isDynamic: true
5    });
6
7    budgetCategory.setValue({
8      fieldId: 'name',
9      value: 'test budget ss'
10   });
11   budgetCategory.setValue({
12     fieldId: 'budgettype',
13     value: true
14   });

```

Budget Exchange Rate

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Budget Exchange Rates table is available at Lists > Accounting > Budget Exchange Rates. In this table you can maintain exchange rates between the child and parent subsidiaries for use in the budgeting process.

i Note: To access this table, the Multiple Currencies and Multiple Budgets features must be enabled.

The Budget Exchange Rates table is similar to the Consolidated Exchange Rates table. In NetSuite OneWorld, NetSuite uses the Consolidated Exchange Rates table to initially populate the Budget Exchange Rates table. Then, you can update and maintain the Budget Exchange Rates table as necessary.

This table shows the exchange rates for each subsidiary's base currency in relation to the currencies used by its parent subsidiaries, for each period. Table columns include: accounting period, accounting book, whether the period is closed, the From (child) subsidiary, the To (parent) subsidiary, and exchange rates.

Rates in this table are either direct or indirect (derived). Direct rates are rates set between a child and parent subsidiary. Direct rates are set by a user and may be edited. Indirect rates are rates set between a grandchild and grandparent subsidiary. Indirect rates are always calculated by the system and cannot be edited. Budget exchange rates include three different rate types per period, subsidiary, and accounting book: Average, Current, and Historical.

For more information about the budget exchange rate record and working with it in the UI, see the help topic [Budget Exchange Rates](#).

The internal ID for this record is budgetexchangerate.

The budget exchange rate record is available only when the Multiple Currencies feature is enabled. An administrator can enable this feature at Setup > Company > Enable Features under the Company subtab.

The script user must have the Currency permission with Full permission level. To edit the different rate types per period and subsidiary pair, the script user must also have access to both the From and To subsidiary.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The budget exchange rate record is scriptable in both client and server SuiteScript.

The editable elements of the record are the `currentrate`, `averagerate`, and `historicalrate`. The remaining record elements can be read and searched.



Important: Rates are created and deleted when subsidiaries or secondary accounting books are created and deleted. Script users can edit the record only if it is not derived (Element field-> `isderived`) and only when the accounting period to which this rate belongs is not closed (Element field-> `isperiodclosed`). In addition, the script user cannot edit a budget exchange rate of 1 for an elimination subsidiary and its direct parent (Element field-> `iseliminationsubsidiary`), with the following exception. Those customers who already have a non-1 budget exchange rate (current, average, or historical) between an elimination subsidiary and its direct parent subsidiary can edit the budget exchange rates for the elimination subsidiary.

Supported Functions

This record is partially scriptable. It can be read, updated, and searched using SuiteScript. It cannot be created, copied, deleted, or transformed.

Code Samples

The following sample shows how to update a budget exchange rate between two subsidiaries.

```

1  var PARENT_SUBSIDIARY = 1;
2  var CHILD_SUBSIDIARY = 3;
3  var PERIOD = 212;
4  // Search for the budget rate record from CHILD_SUBSIDIARY to PARENT_SUBSIDIARY for a specific PERIOD
5  var mySearch = search.create({
6      type: 'budgetexchangerate',
7      filters: [{
8          name: 'fromsubsidiary',
9          operator: search.Operator.IS,
10         values: CHILD_SUBSIDIARY
11     }, {
12         name: 'tosubsidiary',
13         operator: search.Operator.IS,
14         values: PARENT_SUBSIDIARY
15     }, {
16         name: 'period',
17         operator: search.Operator.IS,
18         values: PERIOD
19     }],
20     columns: [{
21         name: 'internalid'
22     }]
23 });
24 var searchResults = mySearch.run().each(function(result) {
25     var fromSubsidiary = result.getValue({
26         name: 'fromsubsidiary'
27     });
28     var toSubsidiary = result.getValue({
29         name: 'tosubsidiary'
30     });
31     var period = result.getValue({
32         name: 'period'
33     });
34     return true;

```

```

35 });
36
37 // Update the budget rate's current, average, and historical rate
38 var id1 = searchResults[0].getValue({
39     name: 'internalid'
40 });
41 var rateRecord = record.load({
42     type: record.Type.BUDGET_EXCHANGE_RATE,
43     id: id1,
44     defaultValues: true
45 });
46 rateRecord.setValue({
47     fieldId: 'currentrate',
48     value: 1.2
49 });
50 rateRecord.setValue({
51     fieldId: 'averagerate',
52     value: 1.3
53 });
54 rateRecord.setValue({
55     fieldId: 'historicalrate',
56     value: 1.4
57 });
58 recId = rateRecord.save();

```

Class

Applies to: SuiteScript 2.x | SuiteCloud Developer

Classes are categories that you can create to track records such as financials, transactions, and employees. For example, you own a janitorial service and you want to track income and expenses separately for household and commercial accounts. You set up a class for each account type and track the financial performance of each class over any time period.

For help working with this record in the UI, see the help topic [Creating Classes](#).

The internal ID for this record is classification.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. The class record is scriptable in server SuiteScript.

Supported Functions

The class record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Consolidated Exchange Rate

Applies to: SuiteScript 2.x | SuiteCloud Developer

If you use NetSuite OneWorld and your subsidiaries have different base currencies, you can maintain a table of consolidated exchange rates. This table is used to ensure that for consolidation purposes, currency amounts properly roll up from child to parent subsidiaries. Each consolidated exchange rate translates between the base currency of a subsidiary and the base currency of its parent or grandparent subsidiary, for a specified accounting period. Consolidated exchange rates include three different rate types per period, subsidiary, and accounting book: Current, Average, and Historical.

The internal ID for this record is `consolidatedexchangerate`.

The consolidated exchange rate record is available only when the Multiple Currencies feature is enabled. An administrator can enable this feature at Setup > Company > Enable Features under the Company subtab.

For help working with this record in the UI, see the help topic [Consolidated Exchange Rates](#).

The script user must have the Currency permission with Full permission level. To edit the different rate types per period and subsidiary pair, the script user must also have access to both the From and To subsidiary.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The consolidated exchange rate record is scriptable in both client and server SuiteScript.

The editable elements of the record are the `currentrate`, `averagerate`, `historicalrate`, and `externalid`. The remaining record elements can be read and searched.

Important: Rates are created and deleted when subsidiaries or secondary accounting books are created and deleted. Script users can edit the record only if it is not derived (Element field-> `isderived`) and only when the accounting period to which this rate belongs is not closed (Element field-> `isperiodclosed`). In addition, the script user cannot edit a consolidated exchange rate of 1 for an elimination subsidiary and its direct parent (Element field- `iseliminationsubsidiary`), with the following exception. Those customers who already have a non-1 consolidated exchange rate (current, average, or historical) between an elimination subsidiary and its direct parent subsidiary can edit the consolidated exchange rates for the elimination subsidiary.

Supported Functions

This record is partially scriptable. It can be read, updated, and searched using SuiteScript. It cannot be created, deleted, or transformed.

Code Samples

The following sample show how to update a consolidated rate between two subsidiaries.

```

1  var PARENT_SUBSIDIARY = 1;
2  var CHILD_SUBSIDIARY = 3;
3  var PERIOD = 212;
4
5  // Search for the consolidated rate record from CHILD_SUBSIDIARY to PARENT_SUBSIDIARY for a specific PERIOD
6  var mySearch = search.create({
7      type: 'consolidatedexchangerate',
8      filters: [{
9          name: 'fromsubsidiary',
10         operator: search.Operator.IS,
11         values: CHILD_SUBSIDIARY
12     }, {
13         name: 'tosubsidiary',
14         operator: search.Operator.IS,
15         values: PARENT_SUBSIDIARY
16     }, {
17         name: 'period',
18         operator: search.Operator.IS,
19         values: PERIOD
20     }],
21     columns: [{
22         name: 'internalid'
23     }]
24 });
25 mySearch.run().each(function(result){
26     var fromSubsidiary = result.getValue({
27         name: 'fromsubsidiary'
28     });
29     var toSubsidiary = result.getValue({
30         name: 'tosubsidiary'
31     });
32     var period = result.getValue({
33         name: 'period'
34     });
35     return true;
36 });
37 // Update the consolidated rate's current, average, and historical rate
38 var rateRecord = record.load({
39     type: record.Type.CONSOLIDATED_EXCHANGE_RATE,
40     id: searchResults[0].getValue('internalid'),
41     defaultValues: true
42 });
43 rateRecord.setValue({
44     fieldId: 'currentrate',
45     value: 1.2
46 });
47 rateRecord.setValue({
48     fieldId: 'averagerate',
49     value: 1.3
50 });
51 rateRecord.setValue({
52     fieldId: 'historicalrate',
53     value: 1.4
54 });
55 var recId = rateRecord.save();

```

Contact Category

Applies to: SuiteScript 2.x | SuiteCloud Developer

Contact category defines a list of values that are used by the contact record to set the type of contact. In the UI, this is a user defined list at Setup > Sales > CRM Lists > Contact Category.

For help working with this record in the UI, see the help topic [Setting Up Accounting Lists](#).

The internal ID for this record is `contactcategory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The contact category record is scriptable in both client and server SuiteScript.

Supported Functions

The contact category record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Contact Role

Applies to: SuiteScript 2.x | SuiteCloud Developer

Contact roles can be defined for each contact listed on various records including opportunity, customer, vendor, and partner.

For help working with this record in the UI, see the help topic [Contacts](#).

The internal ID for this record is `contactrole`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The contact role record is scriptable in both client and server SuiteScript.

Supported Functions

The contact role record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Cost Category

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Cost category records are used to classify different types of costs associated with your items. Using cost categories helps you to track costs and variances in the manufacturing process. These categories are available when the Standard Costing or Landed Cost feature is enabled.

Material or service cost categories track standard costs for items.

For help working with this record in the UI, see the help topic [Creating Cost Categories](#).

Landed cost category values are used for items on bills and item receipts to categorize the different kinds of expenses incurred when making purchases. For details, see the help topic [Landed Cost Categories](#).

The internal ID for this record is costcategory.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The cost category record is scriptable in both client and server SuiteScript.

Supported Script Functions

The cost category record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Currency

i Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Currency Management](#).

The internal ID for this record is currency.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The currency record is scriptable in server SuiteScript only.

Supported Functions

When the Multiple Currencies feature is enabled, full server scripting is supported for this record type.

When the Multiple Currencies feature is not enabled, scripting does not support create, edit, delete, or search of currency records. Search-based functions such as [search.lookupFields\(options\)](#) are not supported, because search is not supported. Loading of a currency record to get field values is supported if the currency ID is known, as shown in the following sample:

```
1 var rec = record.load({
2   type: record.Type.CURRENCY,
3   id: 1
4 });
5 var symbol=rec.getValue({
6   fieldId: 'symbol'
7 });
```

Usage Notes

The `currencyprecision` field is available for scripting, even when the `ALLOWCURRENCYPRECISIONCHANGE` preference is disabled and the field is read-only. In scripting, this field is always a number and never a list box, behavior which differs from this field's behavior in the UI.

Currency Rate

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

After you have created currency records in NetSuite and defined a base currency, you can set up exchange rates between the base currency and foreign currencies to apply to transactions. If you use NetSuite OneWorld and defined a different base currency per subsidiary, set up different exchange rates between each base currency and its foreign currencies. Exchange rates for a base currency apply to all subsidiaries that use the same base currency.

Currency exchange rates are used to convert foreign currencies to base currencies, providing default rates for transactions in currencies other than the base currency.

The currency exchange rate record is available only when the Multiple Currencies feature is enabled.

To access this record in NetSuite, go to Lists > Accounting > Currency Exchange Rates > New.

For help working with this record in the UI, see the help topic [Currency Exchange Rates](#).

The internal ID for this record is `currencyrate`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The currency rate record is scriptable in both client and server SuiteScript.

Supported Functions

When the Multiple Currencies feature is enabled, this record is partially scriptable. It can be created, read, copied, and searched.

When the Multiple Currencies feature is not enabled, this record is not scriptable.

Code Sample

The following code sample shows how to create a currency exchange rate:

```

1  require(["N/record", "N/format"], (record, format) => {
2
3      var effectivedate = format.parse({ value : '9/20/2023', type : format.Type.DATE});
4
5      let objRecord = record.create({
6          type: "currencyrate",
7          isDynamic: true
8      })
9
10     objRecord.setValue({fieldId: 'basecurrency', value: 4}); // Setting Base Currency, 4 would be the internal Id of the base currency
11     objRecord.setValue({fieldId: 'transactioncurrency', value: 3}); // Setting Transaction Currency, 3 would be the internal Id of the transaction currency
12     objRecord.setValue({fieldId: 'exchangerate', value: 1.08}); // Setting the Exchange Rate
13     objRecord.setValue({fieldId: 'effectivedate', value: effectivedate}); // Setting the Effective Date
14     objRecord.setValue({fieldId: 'externalid', value: 'EXT_01'}); // Setting the External Id
15
16     objRecord.save({
17         enableSourcing: false,
18         ignoreMandatoryFields: false
19     });

```

```

20 |
21 | });

```

Customer Category

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Customer category defines a list of values that are used by the customer record to set the type of customer. In the UI, reference an existing customer category list by going to Setup > Accounting > Accounting Lists.

To create a new customer category list, set the Type list at the top of the page to Customer Category. Next, click the New button that appears at the top of the page.

For help working with this record in the UI, see the help topic [Customers](#).

The internal ID for this record is `customercategory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

i Note: The customer category record does not support custom fields.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer category record is scriptable in both client and server SuiteScript.

Supported Functions

The cost category record is partially scriptable — it can be created, updated, copied, and deleted using SuiteScript. Search is not available for the customer category record.

Customer Message

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Standardized customer messages can be created to be available for selection on transaction records. A selected message is sent to the customer for the transaction. For example, a message of "Thank you for your order!" can be selected on an invoice record to be displayed on the invoice sent to the customer.

For help working with this record in the UI, see the help topic [Setting Up Accounting Lists](#).

The internal ID for this record is `customermessage`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The customer message record is scriptable in both client and server SuiteScript.

Supported Functions

The customer message record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Customer-Subsidiary Relationship

Applies to: SuiteScript 2.x | SuiteCloud Developer

If you use NetSuite OneWorld and have enabled the Multi-Subsidiary Customer feature, you can share the customer and sub-customer records with multiple subsidiaries, and then select those subsidiaries on core transactions. An administrator can enable these features at Setup > Company > Enable Features, on the Company subtab.

For help working with Multi-Subsidiary Customers in the UI, see the help topic [Assigning Subsidiaries to a Customer](#).

The internal ID for this record is `customersubsidiaryrelationship`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For customer-subsidiary and vendor-subsidiary relationships belonging to the same entity, both share the external ID of the entity. For example, Company ABC can be both a vendor and customer. If Company ABC has a vendor and customer relationship with the US subsidiary, both share the same external ID.

Supported Script Types

The customer-subsidiary relationship is scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

The customer-subsidiary relationship can be read, created, updated, and deleted.

Code Sample

The following sample shows how to create a customer-subsidiary relationship.

```

1 var csr = record.create({
2   type: record.Type.CUSTOMER_SUBSIDIARY_RELATIONSHIP
3 });
4 csr.setValue({
5   fieldId: 'entity',
6   value: 42
7 });
8 csr.setValue({
9   fieldId: 'subsidiary',
10  value: 1
11 });
12 var recId = csr.save();

```

Department

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Departments are categories that you can create to separate and track records such as financials, transactions, and employees. For example, you can create a department for each team of employees dedicated to a certain area of business, and then track income and expenses by each department over any time period.

For help working with this record in the UI, see the help topic [Creating Departments](#).

The internal ID for this record is `department`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. The department record is scriptable in server SuiteScript.

Supported Functions

The department record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Employee Change Request

Applies to: SuiteScript 2.x | SuiteCloud Developer



Important: This topic describes a feature that is currently available only in some customer accounts. For information about the availability of this feature for your account, please contact your NetSuite account manager.

Employee Change Requests allow managers to initiate requests for changes to their employees in NetSuite. Employee change request types must be set up and assigned to an employee change reason. Managers can initiate an employee change request for their direct reports in the Employee Center and have it go through an approval process which you can define in SuiteFlow. Approvers can approve or decline employee change requests and approved requests will be reflected in the employee's record on a proposed future date. For more information about employee change requests, see the help topic [Employee Change Requests](#).

The internal ID for this record is `employeechangerequest`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)

Prerequisites

To use the employee change request record, you must enable and set up the Employee Change Requests, Effective Dating, and Advanced Employee Permissions features at Setup > Company > Enable Features, on the Employees subtab. To create an employee change request approval workflow in SuiteFlow, you must also enable the SuiteFlow feature at Setup > Company > Enable Features, on the SuiteCloud subtab.

Supported Script Types

The employee change request record is scriptable in both client and server SuiteScript.

Field IDs

Field Label	Internal ID	Type
Justification	justification	text

Field Label	Internal ID	Type
Requester	requester	select
Proposed Date	proposeddate	date
Next Approver	nextapprover	select
Request Status	requeststatus	select
Inactive	isinactive	checkbox
Change Type	employeechangerequesttype	select
Declining Reason	decliningreason	text
Declined By	declinedby	select

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Delete
- Search

Copy and transform are not supported.

Usage Notes

The following fields cannot be edited and are only available on create:

- requester
- employee
- <requested fields>

Code Samples

Note: The proposeddate field syntax must match the format you have chosen in your company settings. Go to Setup > Company > Preferences > General Preferences to configure your Date Format.

The following sample shows how to create an employee change request.

```

1 | require(['N/record'], function(record){
2 |     var empChangeReq = record.create({
3 |         type : record.Type.EMPLOYEE_CHANGE_REQUEST
4 |     });

```

```

5    empChangeReq.setValue({
6        fieldId : 'employeechangerequesttype',
7        value: '1'
8    });
9    empChangeReq.setText({
10       fieldId : 'proposeddate',
11       text: '10/10/2020'
12    });
13    empChangeReq.setValue({
14       fieldId : 'justification',
15       value: 'Justification'
16    });
17    empChangeReq.setValue({
18       fieldId : 'employee',
19       value: '854'
20    });
21    empChangeReq.save();
22 });

```

Employee Change Request Type

i Applies to: SuiteScript 2.x | SuiteCloud Developer



Important: This topic describes a feature that is currently available only in some customer accounts. For information about the availability of this feature for your account, please contact your NetSuite account manager.

Employee change request types provide managers with the types of changes they can initiate for their direct reports. For more information about employee change request type, see the help topic [Setting Up Employee Change Request Types](#).

The internal ID for this record is `employeechangerequesttype`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)

Prerequisites

To use the employee change request type record, you must enable and set up the Employee Change Requests, Effective Dating, and Advanced Employee Permissions features at Setup > Company > Enable Features, on the Employees subtab.

Supported Script Types

The employee change request type record is scriptable in both client and server SuiteScript.

Field IDs

Field Label	Internal ID	Type
Name	name	text
Description	description	text
Change Reason	changereason	select

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Delete
- Search

Copy and transform are not supported.

Code Sample

The following code sample shows how to create an employee change request type.

```

1  require(['N/record'], function(record){
2      var empChangeReqType = record.create({
3          type : record.Type.EMPLOYEE_CHANGE_REQUEST_TYPE
4      });
5      empChangeReqType.setValue({
6          fieldId : 'name',
7          value: 'Test 2'
8      });
9      empChangeReqType.setValue({
10         fieldId : 'description',
11         value: 'Test'
12     });
13     empChangeReqType.setValue({
14         fieldId : 'changereason',
15         value: "10"
16     });
17     empChangeReqType.save();
18 });

```

Expense Category

Applies to: SuiteScript 2.x | SuiteCloud Developer

Expense categories are used to group expenses. Popular categories include transportation, lodging, mileage, and entertainment. Each expense category is linked to an account. When an employee enters an expense report, they select a category for each expense, and the expense automatically posts to the

associated expense account. Note that new expense categories cannot be created at the time an expense report is entered.

If you use NetSuite OneWorld, be aware that an expense category is available to only those subsidiaries assigned to the account linked with the expense category. If you want to enable intercompany expense transactions, you should set up expense categories linked to expense accounts that are available to all subsidiaries, for use in these transactions. For intercompany expense transactions, users cannot save expense lines unless they contain expense categories available to both the employee subsidiary and customer subsidiary.

To enter expense categories, go to Setup > Accounting > Setup Tasks > Expense Categories > New.

For help working with this record in the UI, see the help topic [Creating an Expense Category](#).

The internal ID for this record is `expensecategory`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The expense category record is scriptable in server SuiteScript only.

Supported Functions

The expense category record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Fair Value Price

Applies to: SuiteScript 2.x | SuiteCloud Developer

Fair Value Price is a list of the records that define the fair value for items. Fair value price is used to allocate revenue in revenue arrangements.

This record is part of the Advanced Revenue Management (Revenue Allocation) feature. You must enable the Accounting Periods feature, Advanced Revenue Management (Essentials) feature, and Advanced Revenue Management (Revenue Allocation) feature to use advanced revenue management (revenue allocation). Before you begin working with advanced revenue management programmatically, see the help topics [Setup for Advanced Revenue Management \(Essentials\)](#) and [Setup for Advanced Revenue Management \(Revenue Allocation\)](#).

For help working with this record in the UI, see the help topic [Fair Value Setup](#).

The internal ID for this record is `fairvaluepricelist`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The fair value price record is scriptable in server SuiteScript only.

Supported Functions

The fair value price record is partially scriptable. It can be created, deleted, and searched using SuiteScript. You cannot copy the fair value price record.

Financial Institution

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The financial institution record enables you to save multiple format profiles, which store configuration details for manual or automatic import of financial institution data. You can configure imports of bank or credit card data for bank reconciliation, and corporate card charges for expense reporting.

For help working with this record in the UI, see the help topic [Financial Institution Records](#).

The internal ID for this record is `financialinstitution`.

This record is available in NetSuite and NetSuite OneWorld accounts.

The script user must have the Financial Institution Records permission with full permissions.

In the UI, you create financial institution records on the Financial Institution page at Setup > Accounting > Financial Institution.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript. This record does not support client SuiteScript.

Supported Functions

This record is partially scriptable — it can be read, created, updated, deleted, and searched. It cannot be copied or transformed.

Usage Notes

The table on the Financial Institution page is not editable or scriptable. Clicking **Add** opens the Format Profile page.

Code Sample

The following sample shows how to create a financial institution record.

Note: Instead of deploying a script, you can use a SuiteApp to automatically create a financial institution record and associated format profiles. You can use a NetSuite-owned SuiteApp, such as the [Bank Feeds SuiteApp](#) and the [AMEX Corporate Card Integration SuiteApp](#). See the help topics [Bank Feeds SuiteApp](#) and [American Express Integration for Expense Reporting](#).

Alternatively, if these SuiteApps do not meet your needs, you can build and deploy your own SuiteApps. To create SuiteApps from an integrated development environment (IDE) on your local computer, see .

You can also build SuiteApps using SuiteBundler, although this feature is no longer updated with new features. See the help topic [SuiteBundler Overview](#).

For details about why to deploy your own script versus use a SuiteApp, see the help topic [Capabilities of Using a Financial Institution Connectivity Plug-in](#).

```

1 // Create a financial institution record
2 var newFIRecord = record.create({
3   type: record.Type.FINANCIAL_INSTITUTION,
4   isDynamic: false,
5   defaultValues: null
6 }).setValue({
7   fieldId: 'financialinstitution',
8   value: 'New XYZ Bank'
9 }).setValue({
10  fieldId: 'description',
11  value: 'Description for XYZ Bank'
12 }).save();
13
14 // Load a financial institution record
15 var createdFIRecord = record.load({
16   type: record.Type.FINANCIAL_INSTITUTION,
17   id: 1,
18   isDynamic: false

```

```

19 });
20 // Get value of the description field
21 var description = createdFIRecord.getValue({
22     fieldId: 'description'
23 });
24
25 // Update the description field
26 createdFIRecord.setValue({
27     fieldId: 'description',
28     value: 'Edited Description for XYZ Bank'
29 });
30 var recId = createdFIRecord.save();

```

Format Profile

Applies to: SuiteScript 2.x | SuiteCloud Developer

Format profiles enable you to configure settings necessary for manual or automatic import of financial institution data. Depending on your format profile type, you can do the following:

- Configure an import of bank and credit card data for reconciliation. Setup involves linking your financial institution accounts to corresponding accounts in NetSuite and mapping custom bank codes to bank data types.
- Configure an import of credit card charges for expense reporting. Setup involves linking credit card charges to employee accounts and mapping expense codes to user-defined expense categories.

You can only associate a format profile with one financial institution record. If you delete a financial institution record, NetSuite deletes the associated format profile as well. You cannot delete a format profile that has associated imports.

For help working with this record in the UI, see the help topic [Creating Format Profiles for Bank Reconciliation](#).

The internal ID for this record is formatprofile.

This record is available in NetSuite and NetSuite OneWorld accounts.

The script user must have the Financial Institution Records permission with full permissions. In addition, if configuring a format profile for expense reporting, the script user must have the Expense Reporting feature enabled.

In the UI, you create a format profile record on the Format Profile page, which you get to by going to Setup > Accounting > Financial Institution and clicking **Add**.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

The format profile record has three sublists.

- [Account Linking](#)
- [Expense Code Mapping](#)

The Account Linking and Code Type Mapping sublists are available in a format profile record with a Bank Reconciliation profile type. The Expense Code Mapping sublist is available in a format profile record with a Corporate Card Expenses profile type.

In the UI, the format profile also contains a Connectivity Configuration and Parser Configuration subtab.

Note: The availability of the Parser Configuration subtab depends on the parser plug-in implementation. The selected plug-in must be one of the following:

- An `fiparserplugin` object that implements the `getConfigurationPageUrl` method
- A `bankstatementparserplugin` object

Additionally, if the format profile has a Corporate Card Expenses profile type, the Import Configuration subtab appears (which replaces the Account Linking subtab).

The Connectivity Configuration, Parser Configuration, and Import Configuration subtabs are not attached to the format profile record and thus cannot be accessed through SuiteScript.

For details about working in the Connectivity Configuration subtab in a Bank Reconciliation format profile, see the following:

- [Setting Up Bank Feeds Integration](#)
- [Setting Up Auto Bank Statement Import](#)
- [Configuring a Connection to a Financial Institution \(Without a SuiteApp\)](#)

For details about working in the Connectivity Configuration subtab in a Corporate Card Expenses format profile, see the help topic [American Express Integration for Expense Reporting](#).

For details about working in the Parser Configuration subtab, see [Configuring Standard Bank Transaction Parsers](#).

For details about working in the Import Configuration subtab, see the help topic [Linking Employee Accounts](#).

Note: Although the Import Configuration subtab is a UI-only form, it contains three fields that are attached to the format profile record. The field names and corresponding SuiteScript IDs are:

- Match Employee By (`matchemployeeby`)
- Expense Type (`expensetype`)
- Expense Source (`expensesourcetype`)

Account Linking

The Account Linking sublist enables you to link financial institution accounts to corresponding accounts in NetSuite.

For help working in this sublist in the UI, see the help topic [Bank Account Linking](#).

The internal ID for this sublist is `accountmapping`.

This sublist is available in a format profile with a Bank Reconciliation profile type, in NetSuite and NetSuite OneWorld accounts.

In the UI, you create a new line in the Account Linking sublist on the Format Profile page by clicking **Add** in the Account Linking subtab.

Transaction Code Mapping

The Code Type Mapping sublist enables you to map custom bank transaction codes to corresponding bank data types in NetSuite.

For help working in this sublist in the UI, see the help topic [Mapping Custom Bank Codes](#).

The internal ID for this sublist is `trancodemapping`.

This sublist is available in a format profile with a Bank Reconciliation profile type, in NetSuite and NetSuite OneWorld accounts.

In the UI, you create a new line in the Code Type Mapping sublist on the Format Profile page by clicking **Add** in the Code Type Mapping subtab.

Expense Code Mapping

The Expense Code Mapping sublist enables you to map custom expense codes to corresponding user-defined expense categories in NetSuite.

For help working in this sublist in the UI, see the help topic [Mapping Custom Expense Codes](#).

The internal ID for this sublist is `expensecodemapping`.

This sublist is available in a format profile with a Corporate Card Expenses profile type, in NetSuite and NetSuite OneWorld accounts.

In the UI, you create a new line in the Expense Code Mapping sublist on the Format Profile page by clicking **Add** in the Expense Code Mapping subtab.

Supported Script Types

The format profile record and its sublists are scriptable in both server and client SuiteScript.

Supported Functions

The format profile record and its sublists are partially scriptable — they can be read, created, updated, and deleted. They cannot be copied, transformed, or searched.

Usage Notes

In the UI, the Financial Institution field is autopopulated on the Format Profile page and cannot be changed. In SuiteScript, users can change this field.

Note: Instead of deploying a script, you can use a SuiteApp to automatically create a financial institution record and associated format profiles. You can use a NetSuite–owned SuiteApp, such as the Bank Feeds SuiteApp and the AMEX Corporate Card Integration SuiteApp. See the help topics [Bank Feeds SuiteApp](#) and [American Express Integration for Expense Reporting](#).

Alternatively, if these SuiteApps do not meet your needs, you can build and deploy your own SuiteApps. To create SuiteApps from an integrated development environment (IDE) on your local computer, see .

You can also build SuiteApps using SuiteBundler, although this feature is no longer updated with new features. See the help topic [SuiteBundler Overview](#).

For details about why to deploy your own script versus use a SuiteApp, see the help topic [Capabilities of Using a Financial Institution Connectivity Plug-in](#).

Code Samples

The following sample shows how to create and load a format profile record with a Bank Reconciliation profile type.

```

1 // Create a format profile record
2 var newFPRRecord = record.create({
3   type: record.Type.FORMAT_PROFILE,
4   isDynamic: true
5 });
6 newFPRRecord.setValue({
7   fieldId: 'formatprofile',
8   value: 'XYZ'
9 });
10 newFPRRecord.setValue({
11   fieldId: 'financialinstitution',
12   value: 2
13 });
14 newFPRRecord.setValue({
15   fieldId: 'transactionparser',
16   value: 201
17 });
18 newFPRRecord.setValue({
19   fieldId: 'description',
20   value: 'Description for XYZ Format Profile'
21 });
22 newFPRRecord.setValue({
23   fieldId: 'profiletype',
24   value: 'bankrec'
25 });
26 newFPRRecord.setValue({
27   fieldId: 'connectivitymethod',
28   value: 202
29 });
30 newFPRRecord.selectNewLine({
31   sublistId: 'accountmapping'
32 });
33
34 newFPRRecord.setCurrentSublistValue({
35   sublistId: 'accountmapping',
36   fieldId: 'accountmappingkey',
37   value: 123
38 });
39 newFPRRecord.setCurrentSublistValue({
40   sublistId: 'accountmapping',

```

```

41     fieldId: 'mappednetsuiteaccount',
42     value: 1
43 });
44 newFPRecord.commitLine({
45     sublistId: 'accountmapping'
46 });
47 var fpID = newFPRecord.save();
48
49 // Load a format profile record
50 var createdFPRecord = record.load({
51     type: record.Type.FORMAT_PROFILE,
52     id: fpID,
53     isDynamic: true
54 });
55
56 // Get line value
57 createdFPRecord.selectLine({
58     sublistId: 'accountmapping',
59     line: 0
60 });
61 console.log(createdFPRecord.getCurrentSublistValue({
62     sublistId: 'accountmapping',
63     fieldId: 'accountmappingkey'
64 }));
65 createdFPRecord.selectNewLine({
66     sublistId: 'trancodemapping'
67 });
68
69 createdFPRecord.setCurrentSublistValue({
70     sublistId: 'trancodemapping',
71     fieldId: 'banktransactioncode',
72     value: '999'
73 });
74 createdFPRecord.setCurrentSublistValue({
75     sublistId: 'trancodemapping',
76     fieldId: 'creditdebit',
77     value: 'CREDIT'
78 });
79 createdFPRecord.setCurrentSublistValue({
80     sublistId: 'trancodemapping',
81     fieldId: 'netsuitetransactiontype',
82     value: 'OTHER'
83 });
84 createdFPRecord.commitLine({
85     sublistId: 'trancodemapping'
86 });
87 createdFPRecord.save();
88
89 var deletedRecord = record.delete({
90     type: record.Type.FORMAT_PROFILE,
91     id: fpID
92 });

```

The following sample shows how to create and load a format profile record with a Corporate Card Expenses profile type.

```

1 // Create new Format Profile
2 const newFPRecord = record.create({
3     type: record.Type.FORMAT_PROFILE,
4     isDynamic: true
5 });
6 newFPRecord.setValue({
7     fieldId: 'formatprofile',
8     value: 'My format profile name'
9 });
10 newFPRecord.setValue({
11     fieldId: 'financialinstitution',
12     value: 1
13 });
14 newFPRecord.setValue({
15     fieldId: 'description',
16     value: 'My format profile description'

```



```

17 });
18     newFPRecord.setValue({
19         fieldId: 'transactionparser',
20         value: 2
21     }); // example parser plugin ID, replace with real one
22
23     newFPRecord.setValue({
24         fieldId: 'connectivitymethod',
25         value: 1
26     }); // example connectivity plugin ID, replace with real one
27
28     newFPRecord.setValue({
29         fieldId: 'profiletype',
30         value: 'expense'
31     }); // accepts values 'bankrec' or 'expense'
32
33     newFPRecord.setValue({
34         fieldId: 'matchemployeeby',
35         value: 'cardHolderName'
36     }); // accepts values 'cardHolderName' or 'employeeId'
37
38     newFPRecord.setValue({
39         fieldId: 'expensetype',
40         value: 'corporateCard'
41     }); // accepts values 'corporateCard' or 'reimbursable'
42
43     newFPRecord.selectNewLine({
44         sublistId: 'expensecodemapping'
45     });
46     newFPRecord.setCurrentSublistValue({
47         sublistId: 'expensecodemapping',
48         fieldId: 'expensecode',
49         value: '999'
50     });
51     newFPRecord.setCurrentSublistValue({
52         sublistId: 'expensecodemapping',
53         fieldId: 'description',
54         value: 'Expense'
55     });
56     newFPRecord.setCurrentSublistValue({
57         sublistId: 'expensecodemapping',
58         fieldId: 'expensecategory',
59         value: '2'
60     });
61     newFPRecord.commitLine({
62         sublistId: 'expensecodemapping'
63     });
64     varformatProfileID = newFPRecord.save();
65
66     // Load record for editing
67     const createdFPRecord = record.load({
68         type: record.Type.FORMAT_PROFILE,
69         id: FP_ID,
70         isDynamic: true
71     });
72     createdFPRecord.selectLine({
73         sublistId: 'expensecodemapping',
74         line: 0
75     });
76     const expenseCodeValue = createdFPRecord.setCurrentSublistValue({
77         sublistId: 'expensecodemapping',
78         fieldId: 'expensecode',
79         value: 'Expense edited'
80     });
81     createdFPRecord.commitLine({
82         sublistId: 'expensecodemapping'
83     });
84     createdFPRecord.save();
85
86     // Load record for viewing
87
88     createdFPRecord = record.load({
89         type: record.Type.FORMAT_PROFILE,

```

```

90     id: FP_ID,
91   });
92   const expenseCodeValue = createdFPRecord.getSublistValue({
93     sublistId: 'expensecodemapping',
94     fieldId: 'expensecode',
95     line: 0
96   });
97   console.log(expenseCodeValue);

```

Gift Certificate

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can create gift certificate items that allow customers to purchase store credit they can send to someone as a gift. The recipient uses the gift certificate code when placing an order through your Web store or entering a transaction with a sales representative. You can set a preference in NetSuite for how you want to generate the gift certificate codes: you can create them yourself, or use a random hash code automatically generated by the system.

Gift certificate codes are not active until the order used to purchase the gift certificate is billed.

For help working with this record in the UI, see the help topic [Gift Certificates](#).

The internal ID for this record is giftcertificate.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The gift certificate record is scriptable in both client and server SuiteScript.

Supported Functions

The gift certificate record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Global Account Mapping

Applies to: SuiteScript 2.x | SuiteCloud Developer

For accounts using Multi-Book Accounting, the global account mapping record enables you to configure secondary accounting books to post to accounts different from the primary book. These mappings are used by transactions where the user can manually select the account to which the transaction posts.

The internal ID for this record is globalaccountmapping.

Using this record requires that, as part of your Multi-Book configuration, you select the Chart of Accounts Mapping option at Setup > Company > Enable Features, on the Accounting subtab, in addition to the other setup steps required for Multi-Book Accounting.

In the UI, you access this record at Setup > Accounting > Global Account Mappings > New.

For help working with this record in the UI, see the help topic [Global Account Mapping](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

This record is fully scriptable, which means it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following samples show how to create global account mapping records and perform other basic tasks.

```

1 // Create Record
2 var gamRec = record.create({
3   type: record.Type.GLOBAL_ACCOUNT_MAPPING
4 });
5 gamRec.setValue({
6   fieldId: 'effectivedate',
7   value: new Date('4/4/2004')
8 });
9 gamRec.setValue({
10    fieldId: 'accountingbook',
11    value: '2'
12 });
13 gamRec.setValue({
14    fieldId: 'sourceaccount',
15    value: '6'
16 });
17 gamRec.setValue({
18    fieldId: 'subsidiary',
19    value: '1'

```

```

20 });
21 gamRec.setText({
22     fieldId: 'class',
23     text: 'Class US'
24 });
25 gamRec.setText({
26     fieldId: 'department',
27     text: 'Department US'
28 });
29 gamRec.setText({
30     fieldId: 'location',
31     text: 'Location US'
32 });
33 gamRec.setValue({
34     fieldId: 'destinationaccount',
35     text: '6'
36 });
37 var id = gamRec.save();
38
39 // Update Record
40 var gamRec = record.load({
41     type: record.Type.GLOBAL_ACCOUNT_MAPPING,
42     id: '102'
43 });
44 gamRec.setValue({
45     fieldId: 'effectivedate',
46     value: new Date('5/5/2005')
47 });
48 gamRec.setText({
49     fieldId: 'sourceaccount',
50     value: 'Advances Paid'
51 });
52 gamRec.setText({
53     fieldId: 'destinationaccount',
54     value: 'ABN Withholding'
55 });
56
57 var id = gamRec.save();
58
59 // Delete Record
60 var gamRec = record.delete({
61     type: record.Type.GLOBAL_ACCOUNT_MAPPING,
62     id: '102'
63 });
64
65 // Copy Record
66 var gamRec = record.copy({
67     type: record.Type.GLOBAL_ACCOUNT_MAPPING,
68     id: '103'
69 });
70 gamRec.setValue({
71     fieldId: 'effectivedate',
72     value: new Date('5/5/2005')
73 });
74 gamRec.setText({
75     fieldId: 'sourceaccount',
76     text: 'Advances Paid'
77 });
78 gamRec.setText({
79     fieldId: 'destinationaccount',
80     text: 'ABN Withholding'
81 });
82 var id = gamRec.save();

```

Global Inventory Relationship

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is globalinventoryrelationship.

The Global Inventory Relationship record is available only when the following features are enabled:

- Multi-Location Inventory
- Advanced Inventory Management
- Intercompany Cross-Subsidiary Fulfillment

Use the Intercompany Cross-Subsidiary Fulfillment feature in your NetSuite OneWorld account to fulfill orders and receive returns across multiple subsidiaries. Use of this feature means that orders are not limited to being fulfilled from locations within the originating sales subsidiary. Instead, you can fulfill a single sales order from locations in multiple subsidiaries.

For help working with this record in the UI, see the help topics [Intercompany Cross-Subsidiary Fulfillment](#), [Advanced Inventory Management](#), and [Multi-Location Inventory](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. See [Supported Operations](#) for additional details about scriptable elements for this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

The Global Inventory Relationship record is scriptable in both client and server SuiteScript.

Supported Operations

The Global Inventory Relationship record can be created, read, edited, deleted, and searched using SuiteScript. It cannot be copied.

Usage Notes

The following elements of the Global Inventory Relationship record have limitations on scripting:

- Only searching is supported for scripts on the System Notes sublist.
- The `originatingsubsidiary` field is required and can no longer be updated after it has been created.
- The `inventorysubsidiary` is required and can no longer be updated after it has been created.
- The `allowcrosssubfulfillment` field is disabled and hidden by default but can be exposed on a Global Inventory Relationship form that has been customized.
- The `allowcrosssubcustomerreturn` is disabled and hidden by default but can be exposed on a Global Inventory Relationship form that has been customized.

Fields

The following fields are scriptable for this record:

- `originatingsubsidiary` (type: select, label: Originating Subsidiary)
- `inventorysubsidiary` (type: select, label: InventorySubsidiary)

- allowcrosssubfulfillment (type: checkbox, label: Allow Cross-Subsidiary Fulfillment)
- allowcrosssubcustomerreturn (type: checkbox, label: Allow Cross-Subsidiary Customer Return)
- alllocationsfulfillment (type: checkbox, label: All Fulfillment Locations)
- alllocationscustomerreturn (type: checkbox, label: All Customer Return Locations)
- externalid (type: text), isinactive (type: checkbox, label: Inactive)

Subtabs

The following subtabs are scriptable for this record:

- invtlocfulfillment (Fulfillment)
- invtlocustreturn (Customer Return)

Sublists

The following sublists are scriptable for this record:

- fulfillment (Location under Fulfillment subtab)
- customerreturn (Location under Customer Return subtab)

Additional Notes

- A Global Inventory Relationship (GIR) record's originatingsubsidiary field cannot be set with the same value as its inventorysubsidiary.field.
- A GIR record's originatingsubsidiary and inventorysubsidiary combination should be unique. Global Inventory Relationship records cannot be duplicated.
- A GIR record cannot have the allowcrosssubfulfillment box and allowcrosssubcustomerreturn box cleared at the same time. At least one should be checked. Both can be checked if applicable.
- A GIR record cannot be deleted when a sales order or return authorization has been created using the relationship record.
- A GIR record's allowcrosssubfulfillment box can be updated only when one of the following are true:
 - No sales order uses the GIR record, or
 - All sales orders using the GIR record have been fully fulfilled and closed.
- A GIR record's allowcrosssubcustomerreturn box can be updated only when of the following are true:
 - No return authorization uses the GIR record, or
 - All return authorizations using the GIR record have been fully received and closed.
- You cannot clear the allowcrosssubfulfillment box when one or more locations are on the fulfillment sublist.
- You cannot clear the allowcrosssubcustomerreturn box when one or more locations are on the customerreturn sublist.
- You cannot remove a location from the fulfillment sublist when there is an existing cross-subsidiary sales order, that is not fully fulfilled and closed, that uses the corresponding GIR with the given location.

- You cannot remove a location from the customerreturn sublist when there is an existing cross-subsidiary return authorization, that is not fully fulfilled and closed, that uses the corresponding GIR with the given location.

Code Sample

The following samples illustrate how to create a Global Inventory Relationship (GIR) record. This code creates two GIR records with the following parameters:

- id1 is a relationship between US Subsidiary as the originating subsidiary and the AU subsidiary as the inventory subsidiary. It uses default values.
- id2 is a relationship between the AU subsidiary as the originating subsidiary and US subsidiary as the inventory subsidiary. It sets specific values such as enabling the Inactive field.

```

1  var US_SUBSIDIARY = 3;
2  var AU_SUBSIDIARY = 1;
3
4  var girRec = record.create({
5      type: record.Type.GLOBAL_INVENTORY_RELATIONSHIP,
6  });
7  girRec.setValue({
8      fieldId: 'originatingsubsidiary',
9      value: US_SUBSIDIARY
10 });
11 girRec.setValue({
12     fieldId: 'inventorysubsidiary',
13     value: AU_SUBSIDIARY
14 });
15 var id1 = girRec.save();
16
17 var girRec2 = record.create({
18     type: record.Type.GLOBAL_INVENTORY_RELATIONSHIP,
19 });
20 girRec2.setValue({
21     fieldId: 'originatingsubsidiary',
22     value: AU_SUBSIDIARY
23 });
24 girRec2.setValue({
25     fieldId: 'inventorysubsidiary',
26     value: US_SUBSIDIARY
27 });
28 girRec2.setValue({
29     fieldId: 'externalid',
30     value: 'AU_US_GIR'
31 });
32 girRec2.setValue({
33     fieldId: 'isinactive',
34     value: true
35 });
36 girRec2.setValue({
37     fieldId: 'allocationsfulfillment',
38     value: true
39 });
40 girRec2.setValue({
41     fieldId: 'allocationscustomerreturn',
42     value: false
43 });
44 var id2 = girRec2.save();

```

The following sample shows how to create a simple search that returns GIR records that are set as Inactive.

```

1  var mySearch = search.create({
2      type: record.Type.GLOBAL_INVENTORY_RELATIONSHIP,

```

```

3   filters: [{
4       name: 'isinactive',
5       operator: search.Operator.IS,
6       values: true
7   }]
8   });

```

Goal

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is goal. This record also includes the subrecords goalcomments and goalevaluations. For more details about this record and how to work with it in the UI, see the help topic [Goal Management for Employees and Managers](#).

This record becomes available when the Performance Management feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Support Script Types

This record is scriptable in both server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Usage Notes

If you copy a goal, the status of the copied goal is set to Planned.

Code Sample

The following sample shows how to open a goal record, enter details for the goal, and then save the goal.

```

1   var goal = record.load({
2       type: record.Type.GOAL,
3       id: 1

```



```

4  });
5
6  goal.setValue({
7      fieldId: 'details',
8      value: 'Work harder than ever before'
9  });
10
11 goal.save();

```

Government-Issued ID Tracking

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Government-Issued ID Tracking feature enables you to track a variety of government-issued identification information about your employees to ensure that you have accurate documentation for your employees' travel, driving qualifications, and tax compliance.

For help working with this record in the UI, see the help topic [Government-Issued ID Tracking](#).

The following record types are related to the Government-Issued ID Tracking feature:

- [Driver's License](#)
- [Passport](#)
- [Other Government-Issued ID](#)

Driver's License

The driver's license record captures the driver's license number for a particular employee.

The internal ID for this record is `driverslicense`. For more details about this record and how to work with it in the UI, see the help topic [Government-Issued ID Tracking](#).

This record becomes available when either the Basic Government-Issued ID Tracking or Advanced Government-Issued ID Tracking features are enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both server and client SuiteScript. However, user event scripts are not supported.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Code Sample

The following sample shows how to create a new driver's license for an employee.

```

1  var dlRecord = record.create({
2      type: 'driverslicense'
3  });
4  dlRecord.setValue({
5      fieldId: 'employee',
6      value: '123'
7  });
8  dlRecord.setValue({
9      fieldId: 'numbervalue',
10     value: '123456789'
11 });
12 dlRecord.setValue({
13     fieldId: 'nameondocument',
14     value: 'John Doe'
15 });
16 dlRecord.setValue({
17     fieldId: 'issuer',
18     value: 'United States Government'
19 });
20 dlRecord.setValue({
21     fieldId: 'class',
22     value: 'M3'
23 });
24 dlRecord.setValue({
25     fieldId: 'dateofissue',
26     value: new Date('10/12/2017')
27 });
28 dlRecord.setValue({
29     fieldId: 'expirationdate',
30     value: new Date('10/12/2027')
31 });
32 dlRecord.setValue({
33     fieldId: 'description',
34     value: 'This is a Drivers License'
35 });
36 var driverslicenseInternalId = dlRecord.save();

```

Passport

The passport record captures the passport number for a particular employee.

The internal ID for this record is passport. For more details about this record and how to work with it in the UI, see the help topic [Government-Issued ID Tracking](#).

This record becomes available when either the Basic Government-Issued ID Tracking or Advanced Government-Issued ID Tracking features are enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both server and client SuiteScript. However, user events are not supported.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Code Sample

The following sample shows how to create a new passport number for an employee.

```

1  var passportRec = record.create({
2      type: 'passport'
3  });
4  passportRec.setValue({
5      fieldId: 'employee',
6      value: '123'
7  });
8  passportRec.setValue({
9      fieldId: 'numbervalue',
10     value: '123456789'
11 });
12 passportRec.setValue({
13     fieldId: 'issuer',
14     value: 'United States Government'
15 });
16 passportRec.setText({
17     fieldId: 'nationality',
18     value: 'American'
19 });
20 passportRec.setValue({
21     fieldId: 'nameondocument',
22     value: 'John Doe'
23 });
24 passportRec.setValue({
25     fieldId: 'dateofissue',
26     value: new Date('10/12/2017')
27 });
28 passportRec.setValue({
29     fieldId: 'expirationdate',
30     value: new Date('10/12/2027')
31 });
32 passportRec.setValue({
33     fieldId: 'description',
34     value: 'This is a Passport'
35 });
36
37 var passportInternalId = passportRecord.save();

```

Other Government-Issued ID

The other government-issued ID record captures ID types other than driver's license and passport.

The internal ID for this record is `othergovernmentissuedid`. For more details about this record and how to work with it in the UI, see the help topic [Government-Issued ID Tracking](#).

This record becomes available when the Advanced Government-Issued ID Tracking feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both server and client SuiteScript. However, user events are not supported.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Code Sample

The following sample shows how to create a new government-issued ID type.

```

1  var otherGovIdRec = record.create({
2    type: 'othergovernmentissuedid'
3  });
4  otherGovIdRec.setValue({
5    fieldId: 'employee',
6    value: '123'
7  });
8  otherGovIdRec.setText({
9    fieldId: 'idtype',
10   value: 'Social Insurance Number (SIN)'
11 });
12 otherGovIdRec.setValue({
13   fieldId: 'numbervalue',
14   value: '123456789'
15 });
16 otherGovIdRec.setValue({
17   fieldId: 'nameondocument',
18   value: 'John Doe'
19 });
20 otherGovIdRec.setValue({
21   fieldId: 'issuer',
22   value: 'United States Government'
23 });
24 otherGovIdRec.setValue({
25   fieldId: 'expirationdate',
26   value: new Date('10/12/2027')
27 });
28 otherGovIdRec.setValue({
29   fieldId: 'description',
30   value: 'This is a Other Government ID'

```

```

31 | });
32 |
33 | var otherGovIdInternalId = otherGovIdRecord.save();

```

Group

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the group record to define groups of contacts, customers, employees, partners, or vendors.

The internal ID for this record is entitygroup.

In the UI, you access this record at Lists > Relationships > Group > New.

For help working with this record in the UI, see the help topic [Working with Groups](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The group record is scriptable in server SuiteScript only. None of the user events are supported.

Supported Functions

This record is fully scriptable, which means it can be created, read, updated, deleted, and searched.

Usage Notes

Be aware of the following details when working with this record:

- The grouptype field is required and must be set either to “static” or “dynamic.”
- If the grouptype is dynamic, you can never manually add to the Members sublist (because the group members are determined by a saved search). This behavior is the same as in the UI.
- If the grouptype is static, you can create lines in the Members sublist during the time that you are creating the record, but you cannot modify the sublist during updates. This behavior differs from the UI, which does allow manual updates after the record is created.

To add members to a group in SuiteScript after it has been created, you can use code like the following:

```

1 record.attach({
2   record: {
3     type: record.Type.CUSTOMER,
4     id: <someCustomerId>
5   },
6   to: {
7     type: 'entitygroup',
8     id: <someGroupId>
9   }
10 });
11 record.detach({
12   record: {
13     type: record.Type.CUSTOMER,
14     id: <someCustomerId>
15   },
16   from: {
17     type: 'entitygroup',
18     id: <someGroupId>
19   }
20 });

```

Code Sample

The following sample shows how to create a static group record.

```

1 var initValues = new ();
2 initValues.grouptype = 'Employee';
3 initValues.dynamic = 'false';
4 var staticRecGroup = record.create({
5   type: 'entitygroup',
6   defaultValues: initValues
7 });
8 staticRecGroup.setValue({
9   fieldId: 'groupname',
10  value: 'WC1'
11 });
12 staticRecGroup.setValue({
13   fieldId: 'subsidiary',
14   value: '1'
15 });
16 staticRecGroup.setValue({
17   fieldId: 'ismanufacturingworkcenter',
18   value: true
19 });
20 staticRecGroup.setValue({
21   fieldId: 'machineresources',
22   value: '11'
23 });
24 staticRecGroup.setValue({
25   fieldId: 'laborresources',
26   value: '22'
27 });
28 var recId = staticRecGroup.save();

```

Inbound Shipment

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Inbound shipments provide visibility of in-transit inventory and the status of a shipment. Items from multiple purchase orders can be assigned to an incoming shipment and bulk received and billed from

within the record. Accounting support is also provided for the transfer of ownership of inbound inventory prior to receipt, through the ownership transfer record.

The internal ID for this record is `inboundshipment`.

The inbound shipment record is available in NetSuite and NetSuite OneWorld accounts when the Inbound Shipment Management feature is enabled.

For help working with this record in the UI, see the help topic [Inbound Shipment Management](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The inbound shipment record is supported in client and server SuiteScript.

Supported Operations

This record can be created, read, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Usage Notes

A script that adds a line item to an inbound shipment should include both the purchase order ID and the line number for the item in the purchase order items sublist.

Field Definitions

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Code Sample

The following sample shows a typical workflow for an inbound shipment. This example does the following:

- Creates the inbound shipment record.
- Adds line items from a purchase order.
- Enters information into the primary information fields.
- Updates the status field.
- Updates the expected shipping date.
- Creates a partial ownership transfer.

- Receives the inbound shipment.

```

1  var inboundShipment = record.create({
2      type: record.Type.INBOUND_SHIPMENT,
3      isDynamic: true
4  });
5  var purchaseOrder = record.load({
6      type: record.Type.PURCHASE_ORDER,
7      id: 1718,
8      isDynamic: true
9  });
10 var itemLineCount = purchaseOrder.getLineCount({
11     sublistId: 'item'
12 });
13 for (var i = 1; i <= itemLineCount; i++) {
14     inboundShipment.selectNewLine({
15         sublistId: 'items'
16     });
17     inboundShipment.setCurrentSublistValue({
18         sublistId: 'items',
19         fieldId: 'purchaseorder',
20         value: purchaseOrder.getId()
21     });
22     inboundShipment.setCurrentSublistValue({
23         sublistId: 'items',
24         fieldId: 'shipmentitem',
25         value: purchaseOrder.getSublistValue({
26             sublistId: 'item',
27             fieldId: 'lineuniquekey',
28             line: i
29         })
30     });
31     inboundShipment.commitLine({
32         sublistId: 'items'
33     });
34 }
35 var inboundShipmentId = inboundShipment.save();
36
37 var inboundShipmentUpdate = record.load({
38     type: record.Type.INBOUND_SHIPMENT,
39     id: inboundShipmentId,
40     isDynamic: true
41 });
42 inboundShipmentUpdate.setValue({
43     fieldId: 'shipmentstatus',
44     value: 'inTransit'
45 });
46 inboundShipmentUpdate.setValue({
47     fieldId: 'externaldocumentnumber',
48     value: 'EDN645'
49 });
50 inboundShipmentUpdate.setValue({
51     fieldId: 'expectedshippingdate',
52     value: new Date('8/2/2017')
53 });
54 inboundShipmentUpdate.selectLine({
55     sublistId: 'items',
56     line: 1
57 });
58 inboundShipmentUpdate.setCurrentSublistValue({
59     sublistId: 'items',
60     fieldId: 'receivinglocation',
61     value: 6
62 });
63 inboundShipmentUpdate.setCurrentSublistValue({
64     sublistId: 'items',
65     fieldId: 'quantityexpected',
66     value: 1
67 });
68 inboundShipmentUpdate.setCurrentSublistValue({
69     sublistId: 'items',
70     fieldId: 'expectedrate',
71     value: 10.5

```



```

72 });
73 inboundShipmentUpdate.commitLine({
74     sublistId: 'items'
75 });
76 var recId = inboundShipmentUpdate.save();
77
78 var takeOwnership = record.load({
79     type: record.Type.BULK_OWNERSHIP_TRANSFER,
80     id: inboundShipmentId,
81     isDynamic: true
82 });
83 takeOwnership.selectLine({
84     sublistId: 'items',
85     line: 2
86 });
87 takeOwnership.setCurrentSublistValue({
88     sublistId: 'items',
89     fieldId: 'process',
90     value: 'F'
91 });
92 takeOwnership.commitLine({
93     sublistId: 'items'
94 });
95 var recId2 = takeOwnership.save();
96
97 var bulkReceive = record.load({
98     type: record.Type.RECEIVE_INBOUND_SHIPMENT,
99     id: inboundShipmentId,
100    isDynamic: true
101 });
102 bulkReceive.selectLine({
103     sublistId: 'receiveitems',
104     line: 2
105 });
106 bulkReceive.setCurrentSublistValue({
107     sublistId: 'receiveitems',
108     fieldId: 'quantitytobereceived',
109     value: 1
110 });
111 bulkReceive.commitLine({
112     sublistId: 'receiveitems'
113 });
114 var recId3 = bulkReceive.save();

```

Intercompany Allocation Schedule

Applies to: SuiteScript 2.x | SuiteCloud Developer

Intercompany allocation schedules transfer a balance from one source subsidiary account to multiple destination subsidiaries for costs that are shared between subsidiaries.

The internal ID for this record is `intercompallocationschedule`.

The intercompany allocation schedule record is available only in OneWorld accounts and when the Accounting Periods and Expense Allocation features are enabled. An administrator can enable these features at Setup > Company > Enable Features under the Accounting subtab. For information about expense allocation, see the help topic [Expense Allocation Overview](#).

For help working with this record in the UI, see the help topic [Creating Intercompany Allocation Schedules](#).

If the Statistical Accounting feature is enabled, the intercompany allocation schedule record includes fields that enable you to base the weight for the allocation on the balance of a statistical account through statistical journals or as an absolute value. If the Dynamic Allocation feature is also enabled, the weight is dynamically calculated when the intercompany allocation journal is generated. An administrator can enable these features at Setup > Company > Enable Features Accounting under Advanced Features. For

information about statistical allocation schedules, see the help topic [Working with Allocation Schedules Weighted by the Balance of a Statistical Account](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in both client and server SuiteScript.

If the Dynamic Allocation feature is enabled, the currency field is not scriptable. None of the fields from the History subtab on the intercompany allocation schedule record in the UI are not scriptable.

Supported Functions

This record is partially scriptable. It can be created, read, updated, and deleted using SuiteScript. It cannot be copied, updated, searched, or transformed.

Code Samples

The following samples show how to create a statistical intercompany allocation schedule that divides the balance of one account and transfers the amounts into two other subsidiaries.

```

1  var schedRec = record.create({
2      type: record.Type.INTERCOMP_ALLOCATION_SCHEDULE,
3      defaultvalues: false
4  });
5  schedRec.setValue({
6      fieldId: 'name',
7      value: 'Allocation Schedule A'
8  });
9  schedRec.setValue({
10     fieldId: 'subsidiary',
11     value: '1'
12 });
13
14 schedRec.setSublistValue({
15     sublistId: 'allocationsource',
16     fieldId: 'account',
17     line: 1,
18     value: '24'
19 });
20 schedRec.setSublistValue({
21     sublistId: 'allocationdestination',
22     fieldId: 'subsidiary',
23     line: 1,
24     value: '6'
25 });
26 schedRec.setSublistValue({
27     sublistId: 'allocationdestination',
28     fieldId: 'weight',
29     line: 1,
30     value: '50.00'
31 });
32 schedRec.setSublistValue({
33     sublistId: 'allocationdestination',

```

```

34     fieldId: 'subsidiary',
35     line: 1,
36     value: '32'
37   });
38   schedRec.setSublistValue({
39     sublistId: 'allocationdestination',
40     fieldId: 'weight',
41     line: 1,
42     value: '50.00'
43   });
44
45   recordId = schedRec.save();

```

Inventory Number

Applies to: SuiteScript 2.x | SuiteCloud Developer

The term inventory number refers to a serial number or a lot number. An inventory number record uniquely identifies an item in physical inventory with a serial number, or uniquely identifies a group of items with a lot number.

For help working with this record in the UI, see the help topics [Serial Numbered Items](#) and [Lot Numbered Items](#).

The internal ID for this record is `inventorynumber`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for the inventory number record. It is scriptable in server SuiteScript only.

Supported Functions

The inventory number record is partially scriptable. It can be updated and searched using SuiteScript. Copy, Create, and Delete are not allowed for this record.

Item Account Mapping

Applies to: SuiteScript 2.x | SuiteCloud Developer

For accounts using Multi-Book Accounting, the item account mapping record enables you to configure secondary accounting books to post to accounts different from the primary book. These mappings are used by transactions where the item determines the account to which the transaction posts.

Using this record requires that, as part of your Multi-Book configuration, you select the Chart of Accounts Mapping option at Setup > Company > Enable Features, on the Accounting subtab, in addition to the other setup steps required for Multi-Book Accounting.

In the UI, you access this record at Setup > Accounting > Item Account Mappings > New.

For help working with this record in the UI, see the help topic [Item Account Mapping](#).

The internal ID for this record is `itemaccountmapping`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript only.

All three user events are supported: before Load, before Submit, and afterSubmit.

Supported Functions

The item account mapping record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Samples

The following samples show how to create item account mapping records and perform other basic tasks.

```

1 // Get Record
2 var itemMapRec1 = record.load({
3     type: record.Type.ITEM_ACCOUNT_MAPPING,
4     id: '105'
5 });
6 // Create Record
7 var itemMapRec2 = record.create({
8     type: record.Type.ITEM_ACCOUNT_MAPPING
9 });
10 itemMapRec2.setValue({
11     fieldId: 'effectivedate',
12     value: new Date('4/4/2004')
13 });
14 itemMapRec2.setValue({
15     fieldId: 'accountingbook',
16     value: '2'
17 });
18 itemMapRec2.setValue({
19     fieldId: 'sourceaccount',
20     value: '118'
21 });
22 itemMapRec2.setText({
23     fieldId: 'itemaccount',
24     text: 'Asset'
25 });
26 itemMapRec2.setValue({
27     fieldId: 'subsidiary',
28     value: '3'
29 });
30 itemMapRec2.setText({
31     fieldId: 'class',
32     text: 'Class US'
33 });

```

```

34 itemMapRec2.setText({
35     fieldId: 'department',
36     text: 'Department US'
37 });
38 itemMapRec2.setText({
39     fieldId: 'location',
40     text: 'Location US'
41 });
42 itemMapRec2.setValue({
43     fieldId: 'destinationaccount',
44     value: '118'
45 });
46 var id = itemMapRec2.save();
47
48 // Update Record
49 var itemMapRec3 = record.load({
50     type: record.Type.ITEM_ACCOUNT_MAPPING,
51     id: '201'
52 });
53 itemMapRec3.setValue({
54     fieldId: 'effectivedate',
55     value: new Date('5/5/2005')
56 });
57 itemMapRec3.setValue({
58     fieldId: 'subsidiary',
59     value: '1'
60 });
61 itemMapRec3.setText({
62     fieldId: 'itemaccount',
63     text: 'Discount'
64 });
65 itemMapRec3.setText({
66     fieldId: 'sourceaccount',
67     text: 'Advertising'
68 });
69 itemMapRec3.setText({
70     fieldId: 'destinationaccount',
71     text: 'Advertising'
72 });
73 var id = itemMapRec3.save();
74
75 // Copy Record
76 var itemMapRec4 = record.copy({
77     type: record.Type.ITEM_ACCOUNT_MAPPING,
78     id: '201'
79 });
80 itemMapRec4.setValue({
81     fieldId: 'effectivedate',
82     value: new Date('6/6/2005')
83 });
84 itemMapRec4.setText({
85     fieldId: 'itemaccount',
86     text: 'Asset'
87 });
88 itemMapRec4.setValue({
89     fieldId: 'sourceaccount',
90     value: '118'
91 });
92 itemMapRec4.setValue({
93     fieldId: 'destinationaccount',
94     value: '118'
95 });
96 var id = itemMapRec4.save();
97
98 // Delete Record
99 var delrecId = record.delete({
100     type: record.Type.ITEM_ACCOUNT_MAPPING,
101     id: '201'
102 });

```

Item Collection

Applies to: SuiteScript 2.x | SuiteCloud Developer

Item collections are a general purpose way of grouping catalog items. They are particularly useful in setting up Personalized Catalog Views.

For help working with this record in the user interface, see the help topic [Item Collections](#).

In the UI, you can go to Lists > Accounting > Item Collections to access this record.

The internal ID for this record is itemcollection.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item collection record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Update
- Delete
- Search

Usage Notes

You must enable the Inventory feature on the Items & Inventory subtab at Setup > Company > Enable Features to be able to script with this record. This record cannot be used to add items to an item collection. Adding items is done using the Item Collection Item Map record.

The Item Collection record has two sublists:

- **Items** – this sublist cannot be accessed directly using SuiteScript. However, it can be accessed using the separate [Item Collection Item Map](#) record.
- **Customer Segments** – this sublist cannot be accessed directly using SuiteScript.

Script Sample

The following code snippet creates an item collection record with the name Item Collection 1.

```

1 // SS 2.0
2 require(["N/record"], function(record){
3     var collection = record.load({type: "itemcollection", id: "1"});
4     collection.setValue({fieldId: "name", value: "ItemCollection 1"});
5     collection.save();
6 });

```

Item Collection Item Map

Applies to: SuiteScript 2.x | SuiteCloud Developer

This record holds the relationship between an item collection and the items it contains.

It is not accessible from the UI.

The internal ID for this record is `itemcollectionitemmap`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item collection record is scriptable in both client and server SuiteScript.

Supported Functions

The following SuiteScript functionality is supported:

- Read
- Create
- Delete
- Search

Usage Notes

You must enable the Inventory feature on the Items & Inventory subtab at Setup > Company > Enable Features to be able to script with this record.

Script Sample

The following code snippets deletes the item collection item map record that has the ID of 1.

```

1 // SS 2.0
2 require(["N/record"], function(record){
3     var mapping = record.load({type: "itemcollectionitemmap", id: 1});
4     record.delete({type: "itemcollectionitemmap", id: 1});

```

```
5 | });
```

Item Location Configuration

① Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `itemlocationconfiguration`.

The item location configuration record is available only when the following features are enabled:

- Multi-Location Inventory
- Advanced Inventory Management
- Advanced Item Location Configuration

The Advanced Item Location Configuration feature supports tracking of location attributes data for items in accounts where the Multi-Location Inventory and Advanced Inventory Management features are enabled. For information about these features, see the help topics [Advanced Inventory Management](#) and [Multi-Location Inventory](#).

For help working with this record in the UI, see the following help topics:

- [Advanced Item Location Configuration](#)
- [Enabling Advanced Item Location Configuration](#)
- [Using Advanced Item Location Configuration](#)
- [Working with Item Location Attributes](#)
- [Duplicate Item Attributes from a Single Location](#)
- [Duplicate Item Attributes from Multiple Locations](#)
- [Edit Item Location Attributes](#)

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item location configuration record is scriptable in both client and server SuiteScript.

Supported Operations

The item location configuration record can be created, read, edited, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Code Sample

The following samples illustrate how to make changes on an item location configuration record.

```
1 // OW account, subsidiary (id=1) "Parent Company", locations: (id=6) "US ONLY Location"
2 // Create Record (OW account, )
3 var itemLocConfigRec = record.create({
4   type: record.Type.ITEM_LOCATION_CONFIGURATION,
```



```

5     defaultValues: {
6         item: '18'
7     }
8 });
9 itemLocConfigRec.setValue({
10     fieldId: 'subsidiary',
11     value: '1'
12 });
13 itemLocConfigRec.setValue({
14     fieldId: 'location',
15     value: '6'
16 });
17 itemLocConfigRec.setValue({
18     fieldId: 'itemid',
19     value: '18'
20 });
21 itemLocConfigRec.setValue({
22     fieldId: 'name',
23     value: 'ilc record1'
24 });
25 var recId = itemLocConfigRec.save();
26
27 // Update Record
28 var itemLocConfigRec2 = record.load({
29     type: record.Type.ITEM_LOCATION_CONFIGURATION,
30     id: '10'
31 });
32 itemLocConfigRec2.setValue({
33     fieldId: 'defaultreturncost',
34     value: '4.3'
35 });
36 itemLocConfigRec2.setValue({
37     fieldId: 'poreceiptcost',
38     value: '7.54'
39 });
40 itemLocConfigRec2.setValue({
41     fieldId: 'safetystocklevel',
42     value: '12'
43 });
44 itemLocConfigRec2.save();
45
46 // Delete Record
47 record.delete({
48     type: record.Type.ITEM_LOCATION_CONFIGURATION,
49     id: '10'
50 });

```

Item Process Family

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The item process family is an item category that uses properties or attributes to identify and group similar items in your warehouse. For example, if you can create a clothing item process family to identify all clothing items.

The item process family record is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Creating Item Process Families](#).

The internal ID for this record is `itemprocessfamily`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item process family record is scriptable in both client and server SuiteScript.

Supported Functions

The item process family record is fully scriptable – it can be created, copied, updated, deleted, and searched using SuiteScript.

Code Samples

The following code sample shows how to create and edit an item process family record using SuiteScript 2.x:

```

1  require(["N/record"], function (record) {
2      function createItemProcessFamily() {
3          var ipf = record.create({type: record.Type.ITEM_PROCESS_FAMILY});
4          ipf.setValue({fieldId: "name", value: "Dairy"});
5          ipf.setValue({fieldId: "description", value: "Products made of milk"});
6          ipf.save();
7      }
8
9      function editItemProcessFamily() {
10         var ipf = record.load({type: record.Type.ITEM_PROCESS_FAMILY, id: 7});
11         ipf.setValue({fieldId: "name", value: "Home Hardware"});
12         ipf.setValue({fieldId: "description", value: "Products for your home"});
13         ipf.save();
14     }
15 });

```

Item Process Group

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The item process group is an item category that groups similar items in your warehouse using a more refined property or attribute than the item process family. For example, if you have a clothing item process family, you can create item process groups such as pants and jeans. You can further identify and group clothing items into either of these item process groups.

The item process group record is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Creating Item Process Groups](#).

The internal ID for this record is `itemprocessgroup`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The item process group record is scriptable in both client and server SuiteScript.

Supported Functions

The item process group record is fully scriptable – it can be created, copied, updated, deleted, and searched using SuiteScript.

Code Samples

The following code sample shows how to create and edit an item process group record using SuiteScript 2.x:

```

1  require(["N/record"], function (record) {
2      function createItemProcessGroup() {
3          var ipf = record.create({type: record.Type.ITEM_PROCESS_GROUP});
4          ipf.setValue({fieldId: "name", value: "Vegetables"});
5          ipf.setValue({fieldId: "description", value: "Carrots, lettuce, etc."});
6          ipf.save();
7      }
8
9      function editItemProcessGroup() {
10         var ipf = record.load({type: record.Type.ITEM_PROCESS_GROUP, id: 17});
11         ipf.setValue({fieldId: "name", value: "Fruit"});
12         ipf.setValue({fieldId: "description", value: "Apples, pears, etc."});
13         ipf.save();
14     }
15 });

```

Item Revision

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Bill of Materials Member Control for Assembly Items](#).

The internal ID for this record is itemrevision.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

Supported Functions

The item revision record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Sample

The following sample shows how to create an item revision record:

```

1 function afterSubmit(type){
2     var itemRev = record.create({
3         type: record.Type.ITEM_REVISION
4     });
5     itemRev.setValue({
6         fieldId: 'name',
7         value: 'revision name 222'
8     });
9     itemRev.setValue({
10        fieldId: 'item',
11        value: '109'
12    });
13    itemRev.setValue({
14        fieldId: 'memo',
15        value: 'revision memo'
16    });
17    itemRev.setValue({
18        fieldId: 'effectivedate',
19        value: new Date('3/4/2012')
20    });
21    var id = itemRev.save();
22 }
```

Kudos Feature Records

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The Kudos feature enables your employees to give their colleagues social-based recognition for projects completed or a job well done.

For help working with this record in the UI, see the help topic [Kudos](#).

The following record types are related to the Kudos feature:

- [Kudos](#)
- [Organization Value](#)

Kudos

A Kudos is an act of recognition from one employee to another.

The internal ID for this record is kudos. For more details about this record and how to work with it in the UI, see the help topic [Kudos](#).

This record becomes available when the Kudos feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript only.

Supported Functions

This record can be read, created, edited, copied, deleted, and searched using SuiteScript. Note that only the inactive field can be edited.

Organization Value

An organization value is a value that you associate with a Kudos when you give a Kudos to a colleague.

The internal ID for this record is organizationvalue. For more details about this record and how to work with it in the UI, see the help topic [Kudos](#).

This record becomes available when the Kudos feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is scriptable in server SuiteScript only.

Supported Functions

This record can be read, created, edited, copied, deleted, and searched using SuiteScript.

Location

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Locations feature enables you to track information about employees and transactions for multiple offices or warehouses. For example, you can create locations for the corporate office and all of the sales offices, or for warehouses in several states. You can associate transactions and employees with each location, and then filter report data by location.

For help working with this record in the UI, see the help topic [Locations Overview](#).

The internal ID for this record is location.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

Supported Functions

The location record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Manufacturing Cost Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

To work with this record, the Manufacturing Routing and Work Center feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.

In the UI, this record is accessed by going to Lists > Supply Chain > Manufacturing Cost Template.

For help working with this record in the UI, see the help topic [Creating Manufacturing Cost Templates](#).

The internal ID for this record is manufacturingcosttemplate.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ [N/record Module](#)

Supported Script Types

The manufacturing cost template record is scriptable in server SuiteScript only.

Supported Functions

The manufacturing cost template record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Manufacturing Routing

i Applies to: SuiteScript 2.x | SuiteCloud Developer

To work with this record, the Manufacturing Routing and Work Center feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.

In the UI, this record is accessed by going to Lists > Supply Chain > Manufacturing Routing.

For help working with this record in the UI, see the help topic [Manufacturing Routing](#).

The internal ID for this record is manufacturingrouting.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The manufacturing routing record is scriptable in server SuiteScript only.

Supported Functions

The manufacturing routing record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Merchandise Hierarchy Level

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the Merchandise Hierarchy feature to help you plan, price, report on, and analyze your product inventory by categorizing inventory items in your enterprise.

A merchandise hierarchy level provides a useful way to categorize hierarchy nodes and determine the relationships between inventory items.

The Merchandise Hierarchy Level record is available only when the following features are enabled:

- Merchandise Hierarchy
- Custom Segments

For help working with this record in the UI, see the help topic [Merchandise Hierarchy](#).

The internal ID for this record is `merchandisehierarchylevel`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

Supported Functions

The Merchandise Hierarchy Level record can be read, created, updated, deleted, and searched using SuiteScript.

Code Sample

The following sample shows how to create a new Merchandise Hierarchy Level record:

```

1  require(['N/record'], function(record){
2      var level = record.create({
3          type: record.Type.MERCHANDISE_HIERARCHY_LEVEL,
4          isDynamic: true
5      });
6      level.setValue({
7          fieldId: 'name',
8          value: 'Level 1'
9      });
10     level.setValue({
11         fieldId: 'description',
12         value: 'Description of Level 1'
13     });
14     level.save();
15 });

```

Merchandise Hierarchy Node

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

You can use the Merchandise Hierarchy feature to help you plan, price, report on, and analyze your product inventory by categorizing inventory items in your enterprise.

A merchandise hierarchy node provides groupings for items that share similar characteristics. They can either be a point in the hierarchy from which other nodes can be created, or the point in the hierarchy where products are assigned to.

A Merchandise Hierarchy Node record contains details of the position of the node in the hierarchy, and the version or versions in which it is included.

The Merchandise Hierarchy Node record is available only when the following features are enabled:

- Merchandise Hierarchy
- Custom Segments

For help working with this record in the UI, see the help topic [Merchandise Hierarchy](#).

The internal ID for this record is `merchandisehierarchy`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

Supported Functions

The Merchandise Hierarchy Node record can be read, created, updated, deleted, and searched using SuiteScript.

Subtabs

The following subtabs are scriptable for this record:

- `hierarchyversions` (Merchandise Hierarchy Versions)

Code Sample

The following sample shows how to create a new Merchandise Hierarchy Node record:

```

1  require(['N/record'], function(record) {
2      var node = record.create({
3          type: record.Type.MERCHANDISE_HIERARCHY_NODE,
4          isDynamic: true
5      });
6      node.setValue({
7          fieldId: 'name',
8          value: 'Node 1'
9      });
10     node.setValue({
11         fieldId: 'description',
12         value: 'Description of Node 1'
13     });
14     var recId = node.save();
15 });

```

Merchandise Hierarchy Version

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the Merchandise Hierarchy feature to help you plan, price, report on, and analyze your product inventory by categorizing inventory items in your enterprise.

A Merchandise Hierarchy Version record contains all the levels included in a hierarchy definition for a specific time period.

The Merchandise Hierarchy Version record is available only when the following features are enabled:

- Merchandise Hierarchy
- Custom Segments

For help working with this record in the UI, see the help topic [Merchandise Hierarchy](#).

The internal ID for this record is `merchandisehierarchyversion`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only.

Supported Functions

The Merchandise Hierarchy Version record can be read, created, updated, deleted, and searched using SuiteScript.

Subtabs

The following subtabs are scriptable for this record:

- `hierarchylevels` (Merchandise Hierarchy Levels)

Usage Notes

To assign levels to a merchandise hierarchy version, you must first create the levels.

Code Sample

The following sample shows how to create a new Merchandise Hierarchy Version record:

```
1 require(['N/record'], function(record){
2     var version = record.create({
3         type: record.Type.MERCHANDISE_HIERARCHY_VERSION,
4         isDynamic: true
5     });
6     version.setValue({
```

```

7      fieldId: 'name',
8      value: 'Version 1'
9    });
10    version.setValue({
11      fieldId: 'description',
12      value: 'Description of Version 1'
13    });
14    version.setValue({
15      fieldId: 'startdate',
16      value: new Date('1/1/2030')
17    });
18    version.selectLine({
19      sublistId: 'hierarchylevels',
20      line: 0
21    });
22    version.setCurrentSublistValue({
23      sublistId: 'hierarchylevels',
24      fieldId: 'isincluded',
25      value: true
26    });
27    version.commitLine({
28      sublistId: 'hierarchylevels'
29    });
30    version.save();
31  });

```

Nexus

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A nexus is a tax jurisdiction where you have a business or taxable presence in. In NetSuite, it is usually defined at the country level. This type of entity is available when the Advanced Taxes feature is enabled, to allow users to manage and calculate taxes for different jurisdictions within the same NetSuite account.

For help working with this record in the UI, see the help topic [Advanced Taxes](#).

The internal ID for this record is nexus.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The nexus record is scriptable in server SuiteScript only.

Supported Functions

The nexus record is partially scriptable. It can be created, updated, deleted, and searched using SuiteScript. It cannot be copied.

Note Type

Applies to: SuiteScript 2.x | SuiteCloud Developer

This type defines a list of values that are used by the note record to set the type of note. In the UI, this is a user defined list at Setup > Sales > Setup Tasks > CRM Lists > New > Note Type.

The internal ID for this record is notetype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The note type record is scriptable in both client and server SuiteScript.

Supported Functions

The note type record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Order Type

Applies to: SuiteScript 2.x | SuiteCloud Developer

The order type is a category that associates attributes to sales and transfer orders. In NetSuite WMS, you can create an accounting list of multiple order types and associate each outbound sales or transfer order with one order type.

The order type record is available when the Warehouse Management feature is enabled. For help working with this record in the UI, see the help topic [Creating Order Types](#).

The internal ID for this record is ordertype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The order type record is scriptable in both client and server SuiteScript.

Supported Functions

The order type record is fully scriptable – it can be created, copied, updated, deleted, and searched using SuiteScript.

Code Samples

The following sample shows how to create and edit an order type record using SuiteScript 2.x:

```

1  require(["N/record"], function (record) {
2      function createOrderType() {
3          var orderType = record.create({type: record.Type.ORDER_TYPE, isDynamic: true});
4          orderType.setValue({fieldId: "name", value: "Rush"});
5          orderType.setValue({fieldId: "description", value: "To be expedited within 24 hours"});
6
7          orderType.save();
8      }
9
10     function editOrderType() {
11         var orderType = record.load({type: record.Type.ORDER_TYPE, id: 3, isDynamic: true});
12         orderType.setValue({fieldId: "name", value: "Low priority"});
13         orderType.setValue({fieldId: "description", value: "To be expedited when all higher priority items have been shipped"});
14
15         orderType.save();
16     }
17
18     createOrderType();
19     // editOrderType();
20 });

```

Other Name Category

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Other name category values are used on other name records to categorize them. The list of other name records is a collection of records for people or companies who are not vendors, customers, or employees. You can use other name records to track other people or companies to whom you write checks or from whom you receive deposits.

For example, your company might donate money to a favorite charity, so you can create an other name record for the charity. You might also list your owners and partners here if they contribute or withdraw equity.

For help working with this record in the UI, see the help topic [Other Name Records](#).

The internal ID for this record is othernamecategory.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The other name category record is scriptable in both client and server SuiteScript.

Supported Functions

The other name category record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Partner Category

Applies to: SuiteScript 2.x | SuiteCloud Developer

To create new partner categories, go to Setup > Sales > Setup Tasks > CRM Lists.

The note type record is scriptable in both client and server SuiteScript.

For help working with this record in the UI, see the help topic [Creating a Partner Record](#).

The internal ID for this record is partnercategory.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The partner category record is scriptable in both client and server SuiteScript.

Supported Functions

The partner category record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Payment Instruments

Applies to: SuiteScript 2.x | SuiteCloud Developer

The **Payment Instruments** feature makes using various payments methods more straightforward and, in the case of payment cards, more secure. Payment instrument is a set of fields that can be used to charge

a shopper without any action on the shopper's side. For more details and instructions on how to enable the feature, see the help topic [Payment Instruments](#).

For help working with this record in the UI, see the help topic [Payment Instruments](#).

The following Payment Instruments records are supported in SuiteScript:

- **General Token** - the internal ID is `generaltoken`
- **Payment Card** - the internal ID is `paymentcard`
- **Payment Card Token** - the internal ID is `paymentcardtoken`

Supported Script Types

The Payment Instruments records are scriptable in both client and server SuiteScript.

Usage Note

You cannot retrieve the value of the **Payment Card Number (cardnumber)** field or the **Token (token)** field from the original form after you save the payment instrument. For security reasons, the values of those fields are masked. The Payment Card Number field is located on the Payment Card record, and the Token field is located on the General Token and Payment Card token records.

Supported Functions

The General Token, Payment Card Token, and Payment Card records can be created, read, updated, deleted, and searched by using SuiteScript. The records cannot be copied or transformed.

Code Sample

The following script sample shows how to create a payment instrument of the Payment Card type. The script sets the compulsory fields, and also enables Dynamic mode, so other compulsory fields are set automatically.

The script sample sets the **Preserve on File** field value to `True`. Therefore, the payment instrument information is saved in the system and can be used in other transactions.

```

1  require(['N/record'], function(record){
2      function createPaymentCard(){
3          var recordObj = record.create({
4              type: record.Type.PAYMENT_CARD,
5              isDynamic: true
6          });
7          recordObj.setValue({
8              fieldId: 'entity',
9              value: '6'
10         });
11         recordObj.setValue({
12             fieldId: 'cardnumber',
13             value: '4111111111111111'
14         });
15         recordObj.setValue({
16             fieldId: 'expirationdate',
17             value: new Date(2023, 11)
18         });
19         recordObj.setValue({
20             fieldId: 'paymentmethod',
21             value: '5'
22         });

```

```

23     recordObj.setValue({
24         fieldId: 'nameoncard',
25         value: 'John Wolfe'
26     });
27     recordObj.setValue({
28         fieldId: 'preserveonfile',
29         value: true
30     });
31     var recordId = recordObj.save({
32         ignoreMandatoryFields: false
33     });
34 }
35 createPaymentCard();
36 });

```

Payment Method

Applies to: SuiteScript 2.x | SuiteCloud Developer

You can create payment methods to add selections to the Payment Method field on transactions and in your Web store.

For help working with this record in the UI, see the help topic [Creating a Payment Method](#).

The internal ID for this record is paymentmethod.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The payment method record is scriptable in both client and server SuiteScript.

Supported Functions

The payment method record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Payroll Item

Applies to: SuiteScript 2.x | SuiteCloud Developer

Payroll items store values for payroll transaction line items. Payroll items are used with payroll transactions generated by the Payroll feature, but these transactions are managed by NetSuite and are generally not used in integrations.

For help working with this record in the UI, see the help topic [Paycheck Journal Feature](#).

The internal ID for this record is payrollitem.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The payroll item record is scriptable in both client and server SuiteScript.

Supported Functions

The payroll item record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The exposure of this record to SuiteScript is intended to enable global payroll solutions. You can use it along with the [Paycheck Journal](#) record to create custom payroll solutions and to support integrations with external payroll systems. For details, see the help topic [Paycheck Journal Feature](#).

Code Samples

The following samples create Deduction type payroll items.

```

1 function createPayrollItemdeductionMinimal(){
2     var payrollItem = record.create({
3         type: record.Type.PAYROLL_ITEM
4     });
5     payrollItem.setValue({
6         fieldId: 'subsidiary',
7         value: 1
8     });
9     payrollItem.setValue({
10        fieldId: 'itemtype',
11        value: 16
12    });
13    payrollItem.setValue({
14        fieldId: 'liabilityaccount',
15        value: 27
16    });
17    payrollItem.setValue({
18        fieldId: 'name',
19        value: 'SSItem-Deduction-Minimal'
20    });
21    payrollItem.setValue({
22        fieldId: 'custrecord_payroll_item',
23        value: 'Cust_field'
24    });

```

```

25 payrollItem.setValue({
26     fieldId: 'externalid',
27     value: 'testingexternalID'
28 });
29 var recId = payrollItem.save();
30 }

```

```

1 function createPayrollItemdeductionComplete(){
2     var payrollItem = record.create({
3         type: record.Type.PAYROLL_ITEM
4     });
5     payrollItem.setValue({
6         fieldId: 'externalid',
7         value: 'SSSitem-Deduction-Completed'
8     });
9     payrollItem.setValue({
10        fieldId: 'subsidiary',
11        value: 1
12    });
13    payrollItem.setValue({
14        fieldId: 'itemtype',
15        value: 16
16    });
17    payrollItem.setValue({
18        fieldId: 'liabilityaccount',
19        value: 22
20    });
21    payrollItem.setValue({
22        fieldId: 'name',
23        value: 'SSSitem-Deduction'
24    });
25    payrollItem.setValue({
26        fieldId: 'vendor',
27        value: 1
28    });
29    payrollItem.setValue({
30        fieldId: 'employeePaid',
31        value: true
32    });
33    payrollItem.setValue({
34        fieldId: 'custrecord_payroll_item',
35        value: 'Cust_field'
36    });
37    var recId = payrollItem.save();
38 }

```

The following samples create Earning:Addition type payroll items.

```

1 function createPayrollItemAdditionMinimal(){
2     var payrollItem = record.create({
3         type: record.Type.PAYROLL_ITEM
4     });
5     payrollItem.setValue({
6         fieldId: 'subsidiary',
7         value: 1
8     });
9     payrollItem.setValue({
10        fieldId: 'itemtype',
11        value: 6
12    });
13    payrollItem.setValue({
14        fieldId: 'expenseaccount',
15        value: 114
16    });
17    payrollItem.setValue({
18        fieldId: 'name',
19        value: 'SSSitem-addition-Minimal'
20    });
21    payrollItem.setValue({
22        fieldId: 'custrecord_payroll_item',
23        value: 'Cust_field'

```

```

24 | });
25 | var recId = payrollItem.save();
26 | }

```

```

1 | function createPayrollItemAdditionComplete() {
2 |   var payrollItem = record.create({
3 |     type: record.Type.PAYROLL_ITEM
4 |   });
5 |   payrollItem.setValue({
6 |     fieldId: 'externalid',
7 |     value: 'SSSitem-Addition-Completed'
8 |   });
9 |   payrollItem.setValue({
10 |    fieldId: 'subsidiary',
11 |    value: 1
12 |   });
13 |   payrollItem.setValue({
14 |    fieldId: 'itemtype',
15 |    value: 6
16 |   });
17 |   payrollItem.setValue({
18 |    fieldId: 'expenseaccount',
19 |    value: 114
20 |   });
21 |   payrollItem.setValue({
22 |    fieldId: 'name',
23 |    value: 'SSSitem-Addition'
24 |   });
25 |   payrollItem.setValue({
26 |    fieldId: 'custrecord_payroll_item',
27 |    value: 'Cust_field'
28 |   });
29 |   var recId = payrollItem.save();
30 | }

```

The following sample shows how to update a payroll item.

```

1 | function updatePayrollItem(){
2 |   var payrollItem = record.create({
3 |     type: record.Type.PAYROLL_ITEM,
4 |     defaultValues: 110
5 |   });
6 |   payrollItem.setValue({
7 |     fieldId: 'liabilityaccount',
8 |     value: 29
9 |   });
10 |   payrollItem.setValue({
11 |     fieldId: 'vendor',
12 |     value: 6
13 |   });
14 |   payrollItem.setValue({
15 |     fieldId: 'custrecord_payroll_item',
16 |     value: 'Updated'
17 |   });
18 |   payrollItem.setValue({
19 |     fieldId: 'inactive',
20 |     value: true
21 |   });
22 |   var recID = payrollItem.save();
23 | }

```

The following sample shows how to delete a payroll item.

```

1 | function deletePayrollItem(){
2 |   var savedSearchInternalId = 2;
3 |   var searchresults = search.load({
4 |     type: search.Type.PAYROLL_ITEM,
5 |     id: savedSearchInternalId
6 |   });

```

```

7   for (var i = 0; searchresults != null && i < searchresults.length; i++) {
8       var searchresult = searchresults[i];
9       if (0 < searchresults[i].getId()) {
10          record.delete({
11              type: searchresults[i].getRecordType(),
12              id: searchresults[i].getId()
13          });
14      }
15  }
16  }

```

Performance Metric

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `performancemetric`. For more details about this record and how to work with it in the UI, see the help topic [Goal Tracking with Performance Metrics](#).

This record becomes available when the Performance Management feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Support Script Types

This record is scriptable in both server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Code Sample

The following sample shows how to open a performance metric record, change the name of the performance metric, and then save the performance metric.

```

1  var metric= record.load({
2      type: record.Type.PERFORMANCE_METRIC,
3      id: 1
4  });
5
6  metric.setValue({
7      fieldId: 'name',
8      value: 'Average Days to Close'
9  });
10
11 metric.save();

```

Performance Review

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `performancereview`. This record also includes the subrecords `performancereviewanswer` and `goal`. For more details about this record and how to work with it in the UI, see the help topic [Performance Reviews](#).

This record becomes available when the Performance Management feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Support Script Types

This record is scriptable in both server and client SuiteScript.

Supported Functions

This record can be created, edited, deleted, and searched.

Usage Notes

The following fields on this record cannot be updated by SuiteScript, because the fields are set by the system:

- `goals`
- `employeeSubmittedDate`
- `reviewSubmittedDate`

Also, depending on the role that you are using, the following limitations apply:

Role	Limitation
Human Resources Generalist	■ You cannot answer performance review questions. ■ You can delete a review only if its status is Launched or Canceled.
Employee Center (as employee)	■ You can only answer questions and select ratings. ■ You can enter employee acknowledgment comments only if the status of the review is HR Approved – Sign-Off Required.
Employee Center (as manager)	■ You cannot edit any date fields. ■ You can only answer questions and select ratings.

Code Sample

The following sample shows how to open a performance review record, change its due date, and then save the record.

```
1 var review = record.load({
2   type: record.Type.PERFORMANCE_REVIEW,
3   id: 1
4 });
5
6 review.setValue({
7   fieldId: 'reviewduedate',
8   value: new Date('6/30/2020')
9 });
10
11 review.save();
```

Performance Review Schedule

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `performancereviewschedule`. For more details about this record and how to work with it in the UI, see the help topic [Scheduling Performance Reviews](#).

This record becomes available when the Performance Management feature is enabled.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Support Script Types

This record is scriptable in both server and client SuiteScript.

Supported Functions

This record is fully scriptable, which means it can be created, edited, copied, deleted, and searched.

Code Sample

The following sample shows how to open a performance review schedule record, change its name, and then save the record.

```
1 var schedule= record.load({
2   type: record.Type.PERFORMANCE_REVIEW_SCHEDULE,
3   id: 1
4 });
```

```

5
6 schedule.setValue({
7   fieldId: 'name',
8   value: 'Engineering Dept, Q2 2020'
9 });
10
11 schedule.save();

```

Price Book

Applies to: SuiteScript 2.x | SuiteCloud Developer

The price book record, along with the price plan record, is used to build pricing information for subscriptions.

The price book record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab.

For help working with this record in the UI, see the help topic [Creating Price Books](#).

The internal ID for this record is pricebook.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The price book record is scriptable in server and client SuiteScript.

Supported Functions

The price book record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Price Level

Applies to: SuiteScript 2.x | SuiteCloud Developer

Price level defines a list of values that are used by the opportunity and item records to set the price level for a specific item. Items can be assigned different price levels — such as Employee Price or Corporate Discount Price.

When the Multiple Pricing feature is enabled, you can create up to 1000 price levels for items. Setting up multiple price levels allows greater flexibility to set different pricing for different customers.

For help working with this record in the UI, see the help topics [Creating Price Levels](#) and [Setting Up Items for Multiple Price Levels](#).

The internal ID for this record is `pricelevel`. Search is not available on this record type.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The price level record is scriptable in both client and server SuiteScript.

Supported Functions

The price level record is partially scriptable. It can be created, updated, copied, and deleted using SuiteScript. It cannot be searched.

Price Plan

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The price plan record, along with the price book record, is used to build pricing information for subscriptions.

The price plan record is available only when the SuiteBilling feature is enabled at Setup > Company > Enable Features (Administrator), on the Transactions subtab.

For help working with this record in the UI, see the help topic [Creating Price Plans](#).

The internal ID for this record is `priceplan`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record. For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The price plan record is scriptable in server and client SuiteScript.

Supported Functions

The price plan record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Code Sample

The following code sample shows a tiered price.

```
1 var pricePlan = nlapiCreateRecord('priceplan');
2 pricePlan.setFieldValue('priceplantype', '2'); // CUMULATIVE_TIER
3 pricePlan.selectNewLineItem('pricetiers');
4 pricePlan.setCurrentLineItemValue('pricetiers', 'fromval', '0');
5 pricePlan.setCurrentLineItemValue('pricetiers', 'pricingsoption', '-101'); // RATE
6 pricePlan.setCurrentLineItemValue('pricetiers', 'value', '100');
7 pricePlan.commitLineItem('pricetiers')
8 var newPricePlan = nlapiSubmitRecord(pricePlan);
```

Price Rule

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Use a price rule to set prices for groups of items. For example, you could specify a group of items to sell at a discount during an upcoming sale event.

The record ID for a price rule record is **pricerule**. This record contains one subrecord, with record ID **customerdimension**.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The price rule record is scriptable in server and client SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The price rule record is partially scriptable. It can be read, created, updated, copied, and deleted using SuiteScript. Search and transform scripting is not supported.

Prerequisites

You must enable Advanced Pricing before you can use this record through SuiteScript.

Usage Notes

Base Price Rule and Creating Price Rule cannot be inactive when you use this record through SuiteScript. There can be only one purchase price for an item.

Code Sample

The following sample shows how to create a new price rule.

```

1  require(['N/record'], function(record) {
2      var priceRule = record.create({
3          type: "pricerule",
4          isDynamic: true
5      });
6      priceRule.setValue({
7          fieldId: "name",
8          value: "TestPriceRule"
9      });
10     priceRule.setValue({
11         fieldId: "priority",
12         value: 1
13     });
14     priceRule.insertLine({
15         sublistId: "customerdimension",
16         line: 0
17     });
18     priceRule.setCurrentSublistValue({
19         sublistId: "customerdimension",
20         fieldId: "isincluded",
21         value: true
22     });
23     priceRule.setCurrentSublistValue({
24         sublistId: "customerdimension",
25         fieldId: "customer",
26         value: 4
27     });
28     priceRule.commitLine({sublistId: "customerdimension"});
29     log.debug(priceRule.save());
30 });

```

Price Rule Entry

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Use a price rule entry to set automatic price adjustments. For example, you could set up a price reduction that comes into effect when the customer orders a specified item quantity.

The record ID for a price rule entry record is **priceruleentry**. This record contains one subrecord, with record ID **quantityBreaks**.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)

■ N/record Module

Supported Script Types

The purchase price rule entry record is scriptable in server and client SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The purchase price rule entry record is partially scriptable. It can be read, created, updated, copied, and deleted using SuiteScript. Search and transform scripting is not supported.

Prerequisites

You must enable Advanced Pricing before you can use this record through SuiteScript.

Usage Notes

The following fields on this record can't be updated by SuiteScript, because the fields are set by the system:

- item
- pricelist

You can't edit or create a price rule entry record that references a currently inactive price rule.

Code Sample

The following sample shows how to create a new price rule.

```

1  require(['N/record'], function(record) {
2      var rec = record.create({
3          type: "priceruleentry",
4          isDynamic: true
5      });
6      rec.setValue({
7          fieldId: "pricerule",
8          value: 1
9      });
10     rec.setValue({
11         fieldId: "item",
12         value: 34
13     });
14     rec.insertLine({
15         sublistId: "quantitybreaks",
16         line: 0
17     });
18     rec.setCurrentSublistValue({
19         sublistId: "quantitybreaks",
20         fieldId: "rate",
21         value: 10.99
22     });
23     rec.commitLine({sublistId: "quantitybreaks"});
24     log.debug(rec.save());
25 });

```

Pricing Group

Applies to: SuiteScript 2.x | SuiteCloud Developer

Price groups allow you to assign customer-specific price levels for groups of items. For example, you could create a pricing group called Laptops and associate the pricing group with all of your laptop items.

For help working with this record in the UI, see the help topic [Pricing Groups](#).

The internal ID for this record is pricinggroup.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The pricing group record is scriptable in both client and server SuiteScript.

Supported Functions

The pricing group record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Project Charge Rule

Applies to: SuiteScript 2.x | SuiteCloud Developer

The project charge rule record is available only when the Project Management and Charge-Based Billing features are enabled. In the UI, you access this record on the Financial subtab of charge-based billing projects.

The internal ID for this record is chargerule.

Project charge rules determine the billing rate, the timing of project charges, and the stage of a charge when it is generated.

For help working with this record in the UI, see the help topic [Understanding Charge Rules](#).

See [Project Charge Rule Record Macros](#) for macros associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)

- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project charge rule record is supported in client and server SuiteScript.

Supported Operations

This record can be created, read, updated, deleted, copied, and searched using SuiteScript. It cannot be transformed.

Usage Notes

If charges generated from a project charge rule have been billed, the project charge rule record is read-only. The `chargeruletype` field is required.

Field Definitions

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Code Samples

The following samples show how to create a time-based project charge rule.

```

1  var chargeRule= record.create({
2      type: record.Type.CHARGE_RULE,
3      isDynamic: true,
4      defaultValues: {
5          'chargeruletype': 'TIMEBASED'
6      }
7  });
8  chargeRule.setValue({
9      fieldId: 'name',
10     value: 'ChargeRuleName'
11 });
12 chargeRule.setValue({
13     fieldId: 'project',
14     value: '96'
15 });
16 var id = chargeRule.save();

```

Project Charge Rule Record Macros

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The charge rule record type currently supports the `copyResources` macro.

For help working with this record in the UI, see the help topic [Understanding Charge Rules](#).

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

copyResources

Corresponding UI Button	Copy Resources from Tasks
Macro Description	As part of a time—based charge rule defined on a project, copies to the charge rule all project resources that have a project task assignment. This includes resources that have time for the project or for one of its project tasks. If the Resource Allocations feature is enabled, this also includes resources that have a resource allocation for a project task. If the Allow Allocated Resources to Enter Time to All Tasks project preference is enabled, this also includes resources that have a resource allocation for the project.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Since	2018.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the of any macro. The copyResources macro does not support any additional parameters.

Charge Rule copyResources Macro Syntax

```

1 // On a current record (when a time-based charge rule is opened in a browser)
2 require(['N/currentRecord'], function(current) {
3     var rec = current.get();
4     rec.executeMacro({
5         'id': 'copyResources'
6     });
7 });

```

```

1 //Creating new time-based charge rule on an existing project, and executing the macro on it
2 require(['N/record'], function(record) {
3     var chargeRule = record.create({
4         type: record.Type.CHARGE_RULE,
5         isDynamic: true,
6         defaultValues: {
7             'chargeruletype': 'TIMEBASED',
8             'project': '968'
9         }
10    });
11    chargeRule.setValue({
12        fieldId: 'name',
13        value: 'my time-based charge rule'
14    });
15    chargeRule.setValue({
16        fieldId: 'ratesourcetype',
17        value: 'RESOURCES'
18    });

```

```

19     chargeRule.executeMacro({
20         id: 'copyResources'
21     });
22     chargeRule.save();
23 });

```

Project Expense Type

i Applies to: SuiteScript 2.x | SuiteCloud Developer



Important: The project expense type record is part of the Job Costing and Project Budgeting feature. For information about the availability of this feature, please contact your account representative.

This record is available when the Job Costing and Project Budgeting feature is enabled at Setup > Company > Enable Features, on the Company subtab. If you do not see this option in the UI, contact your account representative for assistance.

When the feature is enabled, you can access the project expense type record in the UI by choosing Setup > Accounting > Setup Tasks > Project Expense Types > New.

For help working with this record in the UI, see the help topic [Creating Project Expense Types](#).

The project expense type record lets you create expense classifications that you can apply to projects. The internal ID for this record is projectexpensetype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project expense type record is not scriptable in client SuiteScript. It is scriptable in server SuiteScript only.

Supported Functions

The project expense type record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The Language sublist is available only when the Multi-Language feature is enabled at Setup > Company > Enable Features, on the Company subtab. The sublist's records can be updated, but you cannot add records to the sublist through SuiteScript.

Project Revenue Rule

Applies to: SuiteScript 2.x | SuiteCloud Developer

The project revenue rule record is available only when the Project Management, Charge-Based Billing, and Advanced Revenue Management (Essentials) features are enabled. In the UI, you access this record on the Financial subtab of charge-based billing projects. There are three types of project revenue rules.

The internal IDs for these records are `laborbasedprojectrevenue`, `fixedamountprojectrevenue`, and `pctcompleteprojectrevenue`.

Project revenue rules rely on charge rules to determine the amount of revenue to be distributed. The type of project revenue rule you create is determined by how you want the revenue to be recognized. A labor based project revenue rule recognizes the revenue created from time tracked on your project. A fixed amount project revenue rule can recognize revenue based on a date, task, or milestone. A percent complete project revenue rule recognizes revenue at a rate equivalent to the project's progress. Expense-based charge rules are not available to use with Project Revenue Recognition.

For help working with this record in the UI, see the help topics [Charge-Based Project Billing](#) and [Advanced Revenue Management \(Essentials\) and \(Revenue Allocation\)](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The project revenue rule record is supported in client and server SuiteScript.

Supported Functions

The project revenue rule record is partially scriptable. It can be created, read, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Usage Notes

For labor based project revenue rules, the filters sublist is not exposed to SuiteScript. For fixed amount project revenue rules, the recurrence field is not exposed.

When you create a new project revenue rule you must set the Project (company) field to the project/job ID. Project revenue rules are not standalone records, and therefore must be associated with a specific project.

Code Samples

The following sample shows how to create a project revenue rule.

```
1 var projRevRule = record.create({
2   type: 'pctcompleteprojectrevenue'
3 });
4 projRevRule.setValue({
5   sublistId: 'name',
6   value: 'Percent Complete Rule'
7 });
```



```

8 projRevRule.setValue({
9     sublistId: 'project',
10    value: '25'
11 });
12 projRevRule.setValue({
13     sublistId: 'serviceitem',
14     value: '1'
15 });
16 var internalId = projRevRule.save();

```

Project Work Breakdown Structure

Applies to: SuiteScript 2.x | SuiteCloud Developer

The project work breakdown structure is available only when the Job Costing and Project Budgeting, Advanced Project Accounting, Custom Records, Custom Segments, and Activity Codes features are enabled. In the UI, you access this record at Lists > Relationships > Projects.

The internal ID for this record is `wbs`.

A work breakdown structure (WBS) provides a new subtab for project records. The WBS separates work into mutually exclusive parts. You can choose a global or monthly timeline type. Global timeline is a default option. The monthly timeline lets you to select a start and end date of the WBS. You can change the definition of the WBS lines. The monthly timeline type does not show the actuals outside of the defined time range. Unmatched revenue actuals show entries from revenue transactions with a global impact, which do not match any line of the WBS. The monthly timeline type does not display the actuals outside of the defined time range.

While selecting a task on the WBS line, add the actuals to the given line. If matching actuals by the task field fails, use the activity code. Add only one task or activity code to each line, or the system can not determine which line to use.

For help working with this record in the UI, see the help topics [Project Work Breakdown Structure \(WBS\)](#) and [Budget vs. Actual Report](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

See [Project Work Breakdown Structure](#) for the macros associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

The project work breakdown structure event record is supported in client and server SuiteScript.

Supported Functions

The project work breakdown structure record is partially scriptable. It can be created, read, updated, deleted, and searched using SuiteScript.

Usage Notes

Prefer using SuiteScript 2.x over SuiteScript 1.0 in favor of macro capabilities. You do not have to fill the Project field in UI. Set a value to the project field when scripting, because there is no context set.

Code Samples

The following sample shows how to create a work breakdown structure record, how to create a line and how to use macros for ETC and EAC.

```

1  require(['/N/record'], function(record) {
2      function createAndAssertWbs() {
3          var wbs = record.create({
4              type: record.Type.WBS,
5              isDynamic: true
6          });
7
8          wbs.setValue({
9              fieldId: 'project',
10             value: '43'
11         });
12
13         wbs.setValue({
14             fieldId: 'timeline',
15             value: 'Global'
16         });
17
18         wbs.selectNewLine({
19             sublistId: 'lines'
20         });
21
22         wbs.setCurrentSublistValue({
23             sublistId: 'lines',
24             fieldId: 'name',
25             value: 'Line 1'
26         });
27
28         wbs.commitLine({
29             sublistId: 'lines'
30         });
31
32         wbs.executeMacro({
33             id: 'setAmountFieldValue',
34             params: { lineIndex: 0,
35                     fieldId: 'cost_etc',
36                     value: 123.45
37             }
38         });
39
40         var wbsId = wbs.save();
41
42         console.log(wbsId);
43     }
44
45     createAndAssertWbs();
46 });
47
48 require(['/N/record'], function(record) {
49     record.delete({
50         type: record.Type.WBS,
51         id: '18'

```

```

52 | });
53 | });

```

Project Work Breakdown Structure Macros

Applies to: SuiteScript 2.x | SuiteCloud Developer

The project work breakdown structure record type currently supports the `getAmountsFields`, `setAmountFieldValue`, `getAmountFieldValue`, and `getUnmatchedActuals` macros.

For help working with this record in the UI, see the help topics [Project Work Breakdown Structure \(WBS\)](#) and [Budget vs. Actual Report](#).

For information about SuiteScript 2.x record macros, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/record Module Members](#)
- [Macro Object Members](#)

getAmountsFields

Corresponding UI Button	Get Amounts Fields
Macro Description	Gathers the contents of all amounts field names.
Returns	Objects
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2020.1

Parameters

The `getAmountsFields` macro does not require any additional parameter.

Work Breakdown Structure getAmountsFields Macro Syntax

The following code sample returns objects:

```

1 | {
2 |     fields: [
3 |         '11_2019_cost_etc',
4 |         '11_2019_revenue_etc',
5 |         '11_2019_actual_etc',
6 |         '11_2019_actual_etc',]
7 | }

```

The following code sample returns all field IDs which can be used in other macros:

```

1 | require(['N/currentRecord'], function (currentRecord){
2 |     var record = currentRecord.get();
3 |
4 |     var amountsFields = record.executeMacro({
5 |         id: 'getAmountsFields'
6 |     });

```

```
7 | });
```

setAmountFieldValue

Corresponding UI Button	Set Amount Field Value
Macro Description	Sets amount field.
Returns	No object
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2020.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the of any macro. The setAmountFieldValue macro also requires the following additional parameters:

Errors

See the following table for details about the setAmountFieldValue macro errors:

Error	Thrown If
INVALID_FLD_VALUE	Invalid value type
SSS_MISSING_REQD_ARGUMENT	Required argument is missing or undefined
SSS_COLUMN_DOES_NOT_EXIST	Invalid fieldId
YOU_HAVE_ATTEMPTED_AN_UNSUPPORTED_ACTION	Trying set value for unsupported fields (EAC,)
SSS_INVALID_SUBLIST_OPERATION	Invalid lineIndex argument

Work Breakdown Structure setAmountFieldValue Macro Syntax

The following code sample updates ETC amount for the first line:

```
1 | require(['N/currentRecord'], function (currentRecord){
2 |     var record = currentRecord.get();
3 |
4 |     record.executeMacro({
5 |         id: 'setAmountFieldValue',
6 |         params: {
7 |             lineIndex: 0,
8 |             fieldId: '01_2020_cost_etc',
9 |             value: 100
10 |         }
11 |     });
12 | });
```

getAmountFieldValue

Corresponding UI Button	Get Amount Field Value
--------------------------------	------------------------

Macro Description	Gets amount field.
Returns	Objects
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2020.1

Parameters

See the help topic [Record.executeMacro\(options\)](#) for details about parameters required for the of any macro. The `getAmountFieldValue` macro also requires the following additional parameters:

Parameter	Type	Required / Optional	Description	Since
lineIndex	number	required	Line's index (indexed from 0)	2020.1
fieldId	string	required	field's id (obtained from the <code>getAmountFields</code> macro, only etc field is supported)	2020.1
value	number	required	Value	2020.1

Errors

See the following table for details about the `getAmountFieldValue` macro errors:

Error	Thrown If
SSS_MISSING_REQD_ARGUMENT	Required argument is missing or undefined
SSS_COLUMN_DOES_NOT_EXIST	Invalid fieldId
SSS_INVALID_SUBLIST_OPERATION	Invalid lineIndex argument

Work Breakdown Structure getAmountFieldValue Macro Syntax

The following code sample returns an object:

```

1 | {
2 |     field: '01_2020_cost_etc',
3 |     value: 25.5
4 | }
```

The following code sample returns the ETC amount for the first line:

```

1 | require(['N/currentRecord'], function (currentRecord){
2 |     var record = currentRecord.get();
3 |
4 |     var etcCost = record.executeMacro({
5 |         id: 'getAmountFieldValue'
6 |         params: {
7 |             lineIndex: 0,
8 |             fieldId: '01_2020_cost_etc'
9 |         }
10 |     });
11 | });
```

getUnmatchedActuals

Corresponding UI Button	Get Unmatched Actuals
Macro Description	Gets unmatched actuals.
Returns	Objects
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Since	2020.1

Parameters

The getUnmatchedActuals macro does not require any additional parameter.

Work Breakdown Structure getUnmatched Actuals Macro Syntax

The following code sample returns an object:

```
1 {
2     cost: 800
3     revenue: 1000
4 }
```

The following code sample returns unmatched cost and revenue:

```
1 require(['N/currentRecord'], function (currentRecord){
2     var record = currentRecord.get();
3
4     var unmatchedActuals = record.executeMacro({
5         });
6 });
```

Recalculate Percent Complete Override

Applies to: SuiteScript 2.x | SuiteCloud Developer

A new action is available on project status records to enable automatic percent complete recalculation.

For help working with this record in the UI, see the help topic [Using the Percent Complete Override Subtab](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

recalculatepercentcompleteoverride

Corresponding UI Button	Recalculate % Complete Override
--------------------------------	---------------------------------

Action Description	A new action is available on the Project Revenue Rules record to enable automatic percent complete recalculation. Check the Percent Complete Automatic Recalculation box to prompt NetSuite to recalculate the percent complete every eight hours for projects with this status. For the In Progress project status, this box is checked by default. You can also manually trigger a percent complete override recalculation from the Actions menu at the top of project records. For more information, see the help topic Using the Percent Complete Override Subtab.
Returns	No return object
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types.
Since	2023.1

Supported Script Types

The Recalculate Percent Complete Override record is supported in client and server SuiteScript.

The user events are not supported.

Supported Functions

The recalculate action performs simple SQL updates of sublist.

Code Samples

The following sample shows how your customers can call the action API that you are exposing.

```

1  require(['N/action'], function(actionMod) {
2
3      // client side example
4      actionMod.find.promise({recordType: 'job', recordId: 1}).then(function(actions) {
5          actions.recalculatepercentcompleteoverride.promise().then(function(result) {
6              console.log("Percent complete override machine has been updated.");
7          });
8      });
9
10     // server side example
11     var result = actionMod.perform({id: 'recalculatepercentcompleteoverride', recordType: 'job', params: {recordId: 1}});
12 });

```

Revenue Recognition Event

Applies to: SuiteScript 2.x | SuiteCloud Developer

The revenue recognition event record is available only when the Advanced Revenue Management (Essentials) feature is enabled. In the UI, you access this record at Lists > Accounting > Revenue Recognition Events > New.

The internal ID for this record is `billingrevenueevent`.

Revenue recognition events trigger the creation of revenue recognition plans. Each revenue recognition event must have a revenue recognition event type. Revenue recognition event types are not scriptable. To create a revenue recognition event type, go to Lists > Accounting > Revenue Recognition Event Types

> New. You must create at least one revenue recognition event type before you can create a revenue recognition event.

For help working with this record in the UI, see the help topic [Creating a Custom Revenue Recognition Event](#).

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

The revenue recognition event record is supported in client and server SuiteScript.

The user events are not supported.

Supported Functions

The revenue recognition event record is partially scriptable. It can be created, read, updated, deleted, and searched using SuiteScript. It cannot be copied or transformed.

Usage Notes

The fields of this record that must be used in combination are fully described in the [Revenue Recognition Event](#) section of the help topic for the UI. For details, see the help topic [Creating a Custom Revenue Recognition Event](#).

Code Samples

The following sample shows how to create revenue recognition events and perform other basic tasks.

```

1 // Create Record
2 var revRecEvent = record.create({
3   type: record.Type.BILLING_REVENUE_EVENT
4 });
5 revRecEvent.setValue({
6   fieldId: 'transactionline',
7   value: 12 // Transaction Line ID
8 });
9 revRecEvent.setValue({
10  fieldId: 'eventtype',
11  value: 2 // Event Type internal ID
12 });
13 revRecEvent.setValue({
14  fieldId: 'eventpurpose',
15  value: 'FORECAST'
16 });
17 revRecEvent.setValue({
18  fieldId: 'eventdate',
19  value: new Date('12/31/2016')
20 });

```



```

21 revRecEvent.setValue({
22     fieldId: 'quantity',
23     value: 1
24 });
25 var recID = revRecEvent.save();
26
27 //Load Record
28 var rec = record.load({
29     type: record.Type.BILLING_REVENUE_EVENT,
30     id: recID
31 });

```

Revenue Recognition Field Map

Applies to: SuiteScript 2.x | SuiteCloud Developer

The revenue recognition field map is used to map custom and standard fields from source records to custom and standard transaction column fields in revenue arrangements.

This record is available only when the Advanced Revenue Management (Essentials) feature is enabled.

If you want Project Revenue Line to appear as a Source Record Type option, you must enable the Projects, Project Management, and Charge-Based Billing features. For more information, see the help topics [Enabling Project Features](#) and [Setting Up Charge-Based Billing](#).

If you want Subscription Line to appear as a Source Record Type option, you must complete the following:

- Check the Consolidate Jobs on Sales Transactions accounting preference,
- If it is enabled, disable the Employee Commissions feature.
- Enable the Subscription Billing feature and all of its prerequisites.

For more information, see the help topics [Using the Project Consolidation Preference](#) and [Enabling SuiteBilling Features](#).

For help working with this record in the UI, see the help topic [Mapping Revenue Recognition Fields](#).

The internal ID for this record is revrecfieldmapping.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The revenue recognition field map record is supported in client and server SuiteScript.

Supported Functions

The revenue recognition field map record is fully scriptable. It can be created, read, updated, copied, deleted, and searched.

Code Sample

The following sample shows how to create a new revenue recognition field map:

```

1 require(["N/record", "N/format"], (record,format) => {
2
3     var startdate = format.parse({ value : '9/20/2023', type : format.Type.DATE});
4     var enddate = format.parse({ value : '9/20/2024', type : format.Type.DATE});
5
6     let objRecord = record.create({
7         type: "revrecfieldmapping",
8         isDynamic: true
9     })
10
11     objRecord.setValue({fieldId: 'sourcerecordtype', value: 'TRANSACTION_LINE'}); // Source Record Type as Transaction Line
12     objRecord.setValue({fieldId: 'transactiontype', value: 'SalesOrd'}); // Transaction Type as Sales Order
13     objRecord.setValue({fieldId: 'customtxform', value: '68'}); // Custom Form as Standard Sales Order
14     objRecord.setValue({fieldId: 'sourcefield', value: '-3078'}); // Source Field as Start Date
15     objRecord.setValue({fieldId: 'targetfield', value: '-33602'}); // Source Field as End Date
16     objRecord.setValue({fieldId: 'startdate', value: startdate}); // Start Date
17     objRecord.setValue({fieldId: 'enddate', value: enddate}); // End Date
18     objRecord.setValue({fieldId: 'autoupdatetargetfieldvalue', value: true}); // Automatically Update Target Field Value
19     objRecord.setValue({fieldId: 'externalid', value: '15'}); // External Id
20     objRecord.setValue({fieldId: 'description', value: 'First Field Map'}); // Description
21
22     objRecord.save({
23         enableSourcing: false,
24         ignoreMandatoryFields: false
25     });
26
27 });

```

Revenue Recognition Plan

Applies to: SuiteScript 2.x | SuiteCloud Developer

This record is used to indicate the posting periods in which revenue should be recognized and the amount to be recognized in each period. Revenue plans are derived from revenue recognition rules. Each revenue element has a forecast plan and one or more plans. The revenue plans control the posting of revenue.

This record is part of advanced revenue management. To use advanced revenue management (essentials), the Accounting Periods feature and Advanced Revenue Management (Essentials) feature must be enabled.

For help working with this record in the UI, see the help topic [Setup for Advanced Revenue Management \(Essentials\)](#).

The internal ID for this record is revrecplan.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The revenue recognition plan record is supported in server SuiteScript only.

The user events are not supported.

Supported Functions

The revenue recognition plan record is partially scriptable. It can be read, updated, and searched using SuiteScript. It cannot be created, copied, deleted, or transformed.

Planned Revenue Sublist

The internal id for this sublist is plannedrevenue. The sublist type is inline editor.

The following table lists the scriptable sublist fields associated with the Planned Revenue sublist.

Field	Type	Internal ID
Amount	Decimal Number	amount
Planned Period	List/Record	plannedperiod

For help working with this record in the UI, see the help topic [Revenue Recognition Plans](#).

Revenue Recognition Schedule

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A revenue recognition schedule indicates the posting periods in which revenue should be recognized, and the amount to be recognized in each period, for an item sale. A revenue recognition schedule is generated for any sales transaction item that has an associated revenue recognition template. The point at which a revenue recognition schedule is generated for an item sale depends upon the type of sales transaction and enabled features and preferences set in your account. The schedule could be generated when a transaction is first saved, when it is approved, or when it is billed. Revenue recognition schedules provide a basis for the generation of journal entries that record the impact of item sales. This record is available when the Revenue Recognition feature is enabled.

For help working with this record in the UI, see the help topic [Working with Revenue Recognition Schedules](#).

Revenue recognition schedules are system-generated.

The internal ID for this record is revrecschedule.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for the revenue recognition schedule record. It is scriptable in server SuiteScript only. Also, user event scripts are not supported.

Supported Functions

The revenue recognition schedule record is partially scriptable. It can be read, updated, and searched using SuiteScript. It cannot be created, copied, deleted, or transformed.

Revenue Recognition Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

A revenue recognition template indicates how revenue from associated items should be posted. For each template: you can select from a choice of standard terms or define your own custom terms, set the time period over which recognition occurs, define an offset to delay the start of recognition, and set up an initial amount to be recognized. This record is available when the Revenue Recognition feature is enabled.

For help working with this record in the UI, see the help topics [Revenue Recognition Template](#) and [Creating Revenue Recognition Templates](#).

The internal ID for this record is revrectemplate.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for this record. It is scriptable in server SuiteScript only. Also, user event scripts are not supported.

Supported Functions

The revenue recognition template record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Role

Applies to: SuiteScript 2.x | SuiteCloud Developer

A role is a defined access configuration. To set up and manage user access to your NetSuite account, you need to set up roles that can be assigned to users. Roles include sets of permissions for viewing and editing data. Roles and their permissions determine the pages that users can see in the NetSuite interface.

and the tasks that they can complete. Each role is associated with a center, a UI modeled for a particular business area.

For help working with this record in the UI, see the help topic [NetSuite Roles Overview](#).

The internal ID for this record is `role`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is partially scriptable in both client and server SuiteScript.

This record is scriptable in client with browser console only. You cannot deploy any client script on this record.

Supported Functions

This record is not fully scriptable. Only search is permitted. The Role record is supported in all search APIs, excluding the duplicate search APIs.

Usage Notes

To work with this record, a user must have Manage Roles permissions, at Setup > Users/Roles > Manage Roles > Search.

When passing a role into SuiteScript 2.x search functions, the internal ID is case-sensitive and must be capitalized. The search type is `role = search.ROLE`. This is unique to the search APIs,

Sales Role

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Sales roles can be defined for each member of a sales team. To use this record the Team Selling feature must be enabled. To enable team selling, go to Setup > Company > Enable Features > CRM. Click the Team Selling box, and then click **Save**.

For help working with this record in the UI, see the help topic [NetSuite Roles Overview](#).

The internal ID for this record is `salesrole`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The sales role record is supported in client and server SuiteScript.

Supported Functions

The sales role record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Sales Tax Item

i Applies to: [SuiteScript 2.x](#) | [SuiteCloud Developer](#)

For help working with this record in the UI, see the help topic [Sales Taxes](#).

The internal ID for this record is `saletaxitem`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

Client SuiteScript is not supported for the sales tax item record. It is scriptable in server SuiteScript only.

Supported Functions

The sales tax item record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Subsidiary

i Applies to: [SuiteScript 2.x](#) | [SuiteCloud Developer](#)

A NetSuite OneWorld account enables you to manage data for a hierarchical structure of separate legal entities, or subsidiaries. This structure is organized as a tree that rolls up to a root, or top-level parent subsidiary. The root subsidiary is the highest-level subsidiary in your account, and all other subsidiaries are below it in the hierarchy. If an account that is upgraded to OneWorld has preexisting data, this data is used for the root subsidiary.

Each subsidiary represents a separate company within your global organization. Subsidiaries can be international or domestic. When you create a subsidiary record in NetSuite, the country you define for its address determines the NetSuite edition and tax nexus associated with that subsidiary.

For help working with this record in the UI, see the help topics [Subsidiaries in OneWorld](#) and [Subsidiary Setup](#).

The internal ID for this record is subsidiary.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The subsidiary record is supported in client and server SuiteScript.

Supported Functions

The subsidiary record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The Company Information page and the root Subsidiary record share information. In OneWorld accounts, if you update shared fields on the Company Information page, the corresponding fields on the root Subsidiary record are also updated. An update to the Company Information page also triggers all user event scripts deployed on the Subsidiary record. If your script updates a shared field on the Subsidiary record, the corresponding field on the Company Information page is also updated.

The Subsidiary record is hidden in accounts that are not OneWorld, but the record still exists. You cannot deploy a stand-alone user event script on the Subsidiary record in accounts that are not OneWorld. You can, however, bundle your script and distribute it to these accounts. In this scenario, updates to the Company Information page trigger the of the bundled script.

You can create up to 249 subsidiary records in addition to the root subsidiary, for a total of 250.

On some records, the Subsidiary field is a multi-select field type. When executing a search containing multi-select fields, SuiteScript treats commas as a delimiter of values. Therefore, you should avoid using commas in subsidiary record names. If you do not, subsidiary record names containing a comma can skew your search results.

Subsidiary Settings

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

Some subsidiary-specific features require additional setup before you can use them. The Subsidiary Settings Manager page enables you to manage these subsidiary-specific features that require additional

setup. After you enable a subsidiary-specific feature on the Enable Features page, that feature is available on the Subsidiary Settings Manager page.

From the Subsidiary Settings Manager page, you can access the Subsidiary Settings page for a specific subsidiary. On this page you can enable subsidiary-specific features and define their additional settings for this subsidiary. For information about using the Subsidiary Settings Manager page and its Subsidiary Settings pages, see the help topic [Subsidiary Settings Manager](#).

The internal ID for this record is `subsidiarysettings`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

Supported Script Types

The Subsidiary Settings record is scriptable in server SuiteScript.

All server script types are supported.

Supported Operations

The Subsidiary Settings record can be read, updated, and searched using SuiteScript. However, searching is available only through SuiteAnalytics Workbook by searching for the Subsidiary Settings record type. For information about SuiteAnalytics Workbook, see the help topic [SuiteAnalytics Workbook Overview](#).

The System Notes subtab on the Subsidiary Settings record is search-only. For information about the System Note record, see [System Note](#).

Usage Notes

Your user role must have the Subsidiary Settings Manager edit permission to use the Subsidiary Settings record. For information about this permission, see the help topic [Subsidiary Settings Manager Permissions](#).

 **Important:** The Subsidiary Settings page has a one-to-one relationship with its subsidiary record. When you create a subsidiary, NetSuite creates a Subsidiary Settings page for that subsidiary. When you delete a subsidiary, NetSuite deletes its corresponding Subsidiary Settings page.

The Subsidiary Settings record has the same Internal ID and External ID as its corresponding subsidiary record. For information about the Subsidiary record, see [Subsidiary](#).

Code Sample

The following sample shows how to enable the Period End Journal Entries feature for a specific subsidiary.

```
1 var parentSubsidiarySettings = record.load({
2   type: 'subsidiarysettings',
3   id: 1
4 });
5
```



```

6 // Enable the Period End Journal Entries feature
7 parentSubsidiarySettings.setValue({
8     fieldId: 'allowperiodendjournalentries',
9     value: true
10 });
11
12 // Set some settings for the feature
13 parentSubsidiarySettings.setValue({
14     fieldId: 'createbscloseandopenjournals',
15     value: true
16 });
17 parentSubsidiarySettings.setValue({
18     fieldId: 'balancesheetopeningaccount',
19     value: 45
20 });
21 parentSubsidiarySettings.setValue({
22     fieldId: 'balancesheetclosingaccount',
23     value: 46
24 });
25
26 var recId = parentSubsidiarySettings.save();

```

Supply Chain Snapshot

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is `supplychainsnapshot`.

The Supply Chain Snapshot record is available only when the following features are enabled:

- Multi-Location Inventory
- Advanced Inventory Management
- Demand Planning
- Supply Chain Control Tower

Use the Supply Chain Control Tower feature in your NetSuite OneWorld account to simulate inventory supply and demand across your supply chain. These simulations, or Snapshots, are used to analyze whether inventory levels are in line with demand or planned levels. This feature can help you to juggle how to match customer requests with supply availability.

For help working with this record in the UI, see the help topic [Supply Chain Control Tower](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

See [Supply Chain Snapshot Record Actions](#) for actions associated with this record. For more information about actions and macros, see the help topic [Overview of Record Action and Macro APIs](#).

Supported Script Types

The Supply Chain Snapshot record is scriptable in both client and server SuiteScript.

Supported Operations

The Supply Chain Snapshot record can be created, read, edited, deleted, and searched using SuiteScript. It cannot be copied.

Usage Notes

Supply Chain Snapshot details are based on a saved search. The record is scriptable in the same way that saved searches are scriptable. Because it is a snapshot of transactional data, you can refresh this view or run the snapshot details search as needed.

There are two types of saved searches that are associated with Supply Chain Snapshots:

- Supply chain snapshot details search – This is the line-by-line information as displayed by the snapshot record.
- Supply chain snapshot search – This is header-level information such as replenishment method, future horizon date, and past horizon date.

Field values on the record header can be set using scripts. Some fields are read-only or have special behaviors. Please see the table below for details.

The following are applicable elements of the Supply Chain Snapshot record:

Fields

Field Id	Field Label	Type	Description
memo	Memo	textarea	A typical memo field that can be set on create or edit.
item	Item Name / Number	select	On create, value is selected or passed in as parameter. On edit, snapshot record is tied to item id, so the item field is read-only.
replenishmentmethod	Replenishment Method	select	This field is read-only. Its value is derived from the item record.
daterun	Date Run	datetime	This field is read-only. If refresh = true, it is the datetime stamp from when the data on the record was last refreshed. If refresh = false, it has not been updated.
pasthorizon	Past Horizon	select	Can be set on create. If refresh = T, can be set on edit. Options (in days): 7, 14, 30, 45, 60 Defaults to the value set in Setting Supply Chain Control Tower Preferences .
futurehorizon	Future Horizon	integer	This field is read-only. Its value is derived from the item record. If the item record value is empty, it derives its value from the value set in Setting Supply Chain Control Tower Preferences . The value must be between 0 and 365 days.
futurehorizondate	Future Horizon Date	date	This field is read-only. It is calculated as follows:

Field Id	Field Label	Type	Description
			current date + the number of days set in the Past Horizon field.
refresh	Refresh Summary	checkbox	This is a non-persisted value. On create, this field cannot be set because data is fetched when the record is first created. On edit, this field determines whether to refresh the search results of snapshot details.
externalid	ExternalId	text	Hidden by default in the UI.
stockunit	Stock Unit	select	This field is read-only and shows only when the Multiple Units of Measure feature is enabled . Its value is derived from the value for Stock Units on the item record.

Additional Notes

- You can use the SuiteScript saved search API (the N/search module in SuiteScript 2.x) to script supply chain snapshots.
- Type is 'supplychainsnapshot' for searching general snapshot record info (header-level fields).
- Type is 'supplychainsnapshotdetails' for searching line-level snapshot data.
- Search can be customized. Searches of 'supplychainsnapshot' correspond to header fields, as shown in the table above. Searches of 'supplychainsnapshotdetails' access the transactional data in the details table, such as the following:
 - item, subsidiary, location, date, sourcetype (Source), docnum (transaction), status, invbalance (Inventory Balance), demandqty, supplyqty, purchaseorderqty, salesorderqty, transferorderqty, workorderqty, otherdemandqty, othersupplyqty
 - This above list includes labels in parentheses for potentially confusing column identifiers.
- Scripts can be scheduled. Additional logic in these scripts is flexible. For example, it is possible to automatically analyze the inventory balance for an item to see if it is expected to be underwater, and then complete specific actions like creating purchase orders.
- You can use an automated script for refreshing your supply chain snapshots. For example, every Sunday you could run a script to refresh all snapshots.
- It is your responsibility to build, maintain, and export snapshot data to match your needs.

Code Samples

The following sample shows how to create an item and set the future horizon to 10 days.

```

1  var item = record.create({
2      type: record.Type.INVENTORY_ITEM
3  });
4  item.setValue({
5      fieldId: 'itemid',
6      value: 'SS item test'
7  });
8  item.setValue({
9      fieldId: 'taxschedule',
10     value: '1'
11  })
12  item.setValue({

```

```

13     fieldId: 'futurehorizon',
14     value: '10'
15 });
16 var itemId = item.save();

```

The following sample shows how to generate snapshot for that item with past horizon of 14 days.

```

1  var snap = record.create({
2      type: record.Type.SUPPLY_CHAIN_SNAPSHOT
3  });
4  snap.setValue({
5      fieldId: 'item',
6      value: '7'
7  });
8  snap.setValue({
9      fieldId: 'pasthorizon',
10     value: '14'
11 });
12 var snapId = snap.save();

```

The following sample shows how to refresh the snapshot for an item with a past horizon of 7 days.

```

1  var loadSnap = record.load({
2      type: record.Type.SUPPLY_CHAIN_SNAPSHOT,
3      id: snapId,
4      defaultValues: true
5  });
6  loadSnap.setValue({
7      fieldId: 'refresh',
8      value: 'T'
9  });
10 loadSnap.setValue({
11     fieldId: 'pasthorizon',
12     value: '7'
13 });
14 loadSnap.setValue({
15     fieldId: 'memo',
16     value: 'updated ss memo'
17 });
18 var recID = loadSnap.save();

```

The following sample shows how to create a saved search for a snapshot record.

```

1  var mySearch = search.create({
2      type: search.Type.SUPPLY_CHAIN_SNAPSHOT,
3      title: 'Snapshot header search for test item2',
4      id: 'customsearch_scct',
5      columns: [{
6          name: 'item'
7      }, {
8          name: 'memo'
9      }, {
10         name: 'daterun'
11     }],
12     filters: [{
13         name: 'item',
14         operator: search.Operator.IS,
15         values: itemId
16     }]
17 });
18 var customsearch_scctId = mySearch.save();

```

The following sample shows how to create a saved search for snapshot details.

```

1  var mySearch = search.create({
2      type: 'supplychainsnapshotdetails',
3      title: 'Snapshot details search for test item2',

```

```

4      id: 'customsearch_scct_details',
5      columns: [{
6          name: 'docnum'
7      }, {
8          name: 'location'
9      }, {
10         name: 'invbalance'
11     }],
12     filters: [{
13         name: 'item',
14         operator: search.Operator.IS,
15         values: itemId
16     }]
17 });
18 var customsearch_scct_detailsId = mySearch.save();

```

The following sample shows how to load a search for snapshot record.

```

1 var loadSearch= search.load({
2     type: search.Type.SUPPLY_CHAIN_SNAPSHOT,
3     id: 'customsearch_scct'
4 });

```

The following sample shows how to load search for snapshot details.

```

1 var loadSearchDetails= search.load({
2     type: 'supplychainsnapshotdetails',
3     id: 'customsearch_scct_details'
4 });

```

Supply Chain Snapshot Record Actions

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The supply chain snapshot record currently supports the refresh action.

For help working with this record in the UI, see the help topic [Supply Chain Control Tower](#).

For information about SuiteScript 2.x record actions, see the following help topics:

- [Overview of Record Action and Macro APIs](#)
- [N/action Module](#)

refresh

Corresponding UI Button	Refresh
Action Description	Takes a snapshot of the transactions executed for an item during a specified time period that includes past and future.
Returns	{notifications:[], response:{}}
Supported Script Types	Client and server scripts For additional information, see the help topic SuiteScript 2.x Script Types .
Governance	5 usage units
Since	2018.2

Parameters

See the help topic [action.execute\(options\)](#) for details about parameters required for the of any action. The refresh action does not support any additional parameters.

Supply Chain Snapshot Refresh Action Syntax

```

1  require(['N/action'], function(action){
2      // action & action list loading
3      var actionList = action.find({recordType: 'supplychainsnapshot'});
4      var actionObj = action.get({recordType: 'supplychainsnapshot', id: 'refresh'})
5
6      // action
7      var result = actionObj.perform({recordId: 1});
8      var result = actionObj({recordId: 2});
9
10     var result = action.perform({recordType: 'supplychainsnapshot', id: 'refresh', params: {recordId: 1 }});
11 });

```

System Note

i Applies to: SuiteScript 2.x | SuiteCloud Developer

i Note: System Notes only. For information about System Notes v2, see the help topic [System Notes v2 Overview](#).

System notes track changes made to a record and to general configuration settings that have a financial impact. A system note for a change on a record captures the date when the change was made, who made the change, the role of the user who made the change, the type of change, and the old and new value in the record. System notes are logged on custom records as well as standard records. Anyone with view access to a record can view the system notes for that record. When searching system notes, you can search for certain records, use advanced search filters, create a saved search, or export the information for additional analysis.

For help working with this record in the UI, see the help topics [System Notes Overview](#) and [Searching System Notes](#).

System notes are only exposed as a search.

In the UI, users who are not Administrators can only see system notes for which they are the authors. The same restriction applies to scripting.

The internal ID for this record is systemnote.

In SuiteScript 2.x, when using the [search.Type](#) enum, use the value SYSTEM_NOTE.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The system note record is scriptable in both client and server scripts.

Supported Functions

The system note record is partially scriptable. Only search scripting is permitted.

Code Samples

The following sample shows how you can script with system note records.

```

1  require(['/N/search'], function(search){
2      function createSearch(){
3          var systemNotesSearch = search.create({
4              type: search.Type.SYSTEM_NOTE,
5              title: 'Changes made in Employee Center',
6              id: 'customsearch_emp_center_changes',
7              columns: ['date', 'recordtype', 'record', 'field', 'oldvalue', 'newvalue'],
8              filters: [{
9                  name: 'role',
10                 operator: search.Operator.IS,
11                 values: '15'
12             }]
13         });
14         systemNotesSearch.save();
15     }
16     createSearch();
17 });

```

Tax Control Account

i Applies to: SuiteScript 2.x | SuiteCloud Developer

A tax control account is an account to which the amounts computed for indirect taxes, such as sales tax and VAT, are posted.

In the UI, you can create a tax control account, and view existing ones, at Setup > Accounting > Tax Control Accounts. Note also that a tax control account is essentially an account record, with a few differences. Because they are accounts, tax control accounts also show up in the full list of accounts at Lists > Accounting > Accounts.

For help working with this record in the UI, see the help topic [Tax Control Accounts Overview](#).

The internal ID for this record is taxacct.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The tax control account record is scriptable in both client and server SuiteScript. However, user event scripts are not supported.

Supported Functions

The tax control account record is partially scriptable. Only creating, copying, and some updates are permitted.

Updating

Updating is permitted, but only of certain fields: [Description](#), [External ID](#), [IsInactive](#), and [Name](#). However, you may be able to change other fields when interacting with the tax control account as an account. For details, see [Account](#).

Alternatives to Deleting

[record.delete\(options\)](#) is not supported, because a tax control account cannot be deleted (either through the UI or through SuiteScript). If the Advanced Taxes feature is enabled, you can effectively remove the tax control account by deleting the nexus that the account is associated with. If the Advanced Taxes feature is not enabled, you cannot remove the account.

An alternative to deleting the account is to make it account inactive, by setting the [IsInactive](#) field to true.

For more details on accounts that cannot be deleted in NetSuite, refer to the help topic [Deleting Accounts and Making Accounts Inactive](#).

Field Definitions

This section describes some of the key fields on the tax control account.

Country

The value for country is derived from the nexus value and is read-only. If your account does not have the Advanced Taxes feature enabled, the value for country will always be the same for all your tax control accounts.

Description

The description of the record can include only up to 50 characters. Otherwise, the operation fails with an error reading "The field description contained more than the maximum number (50) of characters allowed."

Description is one of the few fields that can be modified during an update.

External ID

ExternalID is one of the few fields that can be modified during an update.

IsInactive

Because a tax control account cannot be deleted, you might want to make it inactive.

IsInactive is one of the few fields that can be modified during an update.

Name

The name of the record must be unique. If you try to add a tax control account using a non-unique value for the name field, the system returns an error reading “This record already exists,” even if you included a unique external ID.

Name is one of the few fields that can be modified during an update.

Nexus

You can set a value for nexus only if the Advanced Taxes feature is enabled. If your NetSuite account does not use the Advanced Taxes feature, the value for nexus is set automatically and will always be the same for all your tax control accounts.

When Advanced Taxes is enabled, initialization of a nexus value is required. (For an example, refer to [Code Samples](#).) However, after the account is created, the value for nexus cannot be changed.

Note that Advanced Taxes is enabled in all OneWorld accounts and cannot be turned off.

Tax Account Type

All tax control accounts have a tax account type, but you only actively choose a tax account type for accounts in certain countries. For example:

- If you are creating a tax code for the United Kingdom, valid choices include “sales” and “purchase.”
- When creating a tax code for the United States, only one option exists (“sales”), so you do not set a value for this field.

In countries where both “sales” and “purchase” are valid choices, the tax account type field is required. After an account has been created, the tax account type cannot be changed.

In countries where only one choice is allowed, tax account type is automatically set, and you cannot change it through SuiteScript (or through the UI).

Note that if your account uses Advanced Taxes and has nexuses in many countries, you may need to set this field for some tax control accounts and not others.

For More Information

See the SuiteScript Records Browser for all internal IDs associated with this record.

Usage Notes

This section includes additional details on interacting with the tax control account record.

Finding the Internal ID

If you need the internal ID for an existing tax control account, note that you cannot find it through Setup > Accounting > Tax Control Accounts. However, you can find it at Lists > Accounting > Accounts. Make sure you have configured your NetSuite preference to “Show Internal IDs.” (You can find this choice at Home > Set Preferences.)

When sorting accounts at Lists > Accounting > Accounts, be aware that:

- Tax control accounts of type “sale” show up as Other Current Liability.

- Tax control accounts of type “purchase” show up as Other Current Asset.

Be aware that you can also add and update values for the external ID field.

Relationship to the Account Record

You can interact with the tax control account in many of the same ways you can interact with any account. However, some exceptions exist. For example, a tax control account cannot be deleted, and there are limits to what you can update, as described in [Updating](#). However, when you interact with the record as an account, you may be able to update additional fields.

For details on interacting with the account record, see [Account](#).

Code Samples

The following samples show how you can script with tax control account records.

Get

The following sample shows how to retrieve a tax account record with the internal ID of '186':

```
1 var taxAcct = record.load({
2   type: record.Type.TAX_ACCT,
3   id: '186'
4 });
```

Add

The following sample shows how to create a tax control account when the Advanced Taxes feature is enabled. In this scenario, a value for nexus is required:

```
1 var taxAcct = record.create({
2   type: record.Type.TAX_ACCT,
3   defaultValues: {
4     'nexus': 'internal ID of nexus'
5   }
6 });
7 taxAcct.setValue({
8   fieldId: 'name',
9   value: 'Name of account'
10 });
11 taxAcct.setValue({
12   fieldId: 'description',
13   value: 'Description of account'
14 });
15 taxAcct.setValue({
16   fieldId: 'externalid',
17   value: 'Your external ID value'
18 });
19 taxAcct.setValue({
20   fieldId: 'taxaccttype',
21   value: 'Type'
22 });
23 var recId = taxAcct.save();
```

Update

The following sample shows how to update the name, description, and external ID of a tax control account, and to make it active. Note that these fields are the only ones that can be updated.

```

1 var taxAcct = record.load({
2   type: record.Type.TAX_ACCT,
3   id: 'internal ID of tax control account'
4 });
5 taxAcct.setValue({
6   fieldId: 'name',
7   value: 'Updated name'
8 });
9 taxAcct.setValue({
10  fieldId: 'description',
11  value: 'Updated description'
12 });
13 taxAcct.setValue({
14  fieldId: 'isinactive',
15  value: false
16 });
17 taxAcct.setValue({
18  fieldId: 'externalid',
19  value: 'Updated external ID of tax control account'
20 });
21 var recId = taxAcct.save();

```

Tax Group

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the tax group record to combine several tax codes, even if the taxes are paid to different jurisdictions. For example, a tax group in the US might include a state tax, a city tax, and a transit tax. The advantage of using a tax group is that, when you create a sales invoice, you can apply one tax group to the transaction, instead of several separate tax codes.

In the UI, you go to this record by choosing Setup > Accounting > Tax Groups.

For help working with this record in the UI, see the help topic [Tax Groups Overview](#).

The internal ID for this record is taxgroup.

See the [SuiteScript Records Browser](#) for the internal IDs of fields, search filters, and search columns associated with this record.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The tax group record is scriptable in both client SuiteScript and Server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The tax group record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

Code Samples

The following sample shows how to create a US tax group.

```

1  var initValues = new ();
2  initValues.nexuscountry = 'US';
3
4  var taxgroup = record.create({
5      type: record.Type.TAX_GROUP,
6      isDynamic: true,
7      defaultValues: {
8          'nexuscountry' : 'US'
9      }
10 });
11 taxgroup.setValue({
12     fieldId: 'itemid',
13     value: 'Test US Tax Group'
14 });
15 taxgroup.setValue({
16     fieldId: 'description',
17     value: 'Tax group description'
18 });
19 taxgroup.selectNewLine({
20     sublistId: 'taxitem'
21 });
22 taxgroup.setCurrentSublistValue({
23     sublistId: 'taxitem',
24     fieldId: 'taxname',
25     value: '-1425' // -1425 is ID of TX_BUFFALO Tax Code
26 });
27 taxgroup.commitLine({
28     sublistId: 'taxitem'
29 });
30 recId = taxgroup.save();

```

The following code sample shows how to create a tax group for Canada.

```

1  var newgroup = record.create({
2      type: record.Type.TAX_GROUP,
3      defaultValues: {
4          'nexuscountry': 'CA'
5      }
6  });
7  newgroup.setValue({
8      fieldId: 'itemid',
9      value: 'CA-T5'
10 });
11 newgroup.setValue({
12     fieldId: 'piggyback',
13     value: 'T'
14 });
15 newgroup.setValue({
16     fieldId: 'taxitem1',
17     value: 21 // Canadian tax code
18 });
19 newgroup.setValue({
20     fieldId: 'taxitem2',
21     value: 24 // Canadian tax code
22 });
23 newgroup.setValue({
24     fieldId: 'subsidiary',
25     value: 2 // Canadian subsidiary
26 });
27 newgroup.setValue({
28     fieldId: 'state',
29     value: 'AB' // Canadian subsidiary
30 });
31 newgroup.setValue({
32     fieldId: 'description',
33     value: 'New Canada Tax Group'

```

```

34 });
35 var recID = newgroup.save();

```

Tax Period

Applies to: SuiteScript 2.x | SuiteCloud Developer

A tax period defines a period of time over which your company tracks tax. A tax period can be a month, a quarter, or a year, depending on the frequency of your tax submissions or what your business has agreed with the local tax agency. You can run monthly, quarterly, or annual VAT/GST reports in NetSuite.

For help working with this record in the UI, see the help topic [Tax Periods Overview](#).

The internal ID for this record is taxperiod.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The tax period record is scriptable in server SuiteScript only.

Supported Functions

The tax period record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Tax Type

Applies to: SuiteScript 2.x | SuiteCloud Developer

A tax type determines where the tax paid or collected is tracked on the balance sheet. The balance sheet account to which NetSuite posts the collection or payment of tax is called the tax control account.

In NetSuite, the tax types are either already set up by default, or set up for you by Professional Services. When you add a subsidiary in a new country, the tax types for that country become available in the system automatically. However, an administrator can create new tax types if necessary.

For help working with this record in the UI, see the help topic [Tax Types Overview](#).

The internal ID for this record is taxtype.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The tax type record is scriptable in server SuiteScript only.

Supported Functions

The tax type record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Term

Applies to: SuiteScript 2.x | SuiteCloud Developer

Terms are used to specify when payment is due on your customers' invoices. Define the specific requirements of a term of payment by creating a term record. You can create different payment terms for different customers. In the UI, this is a user defined list at Setup > Accounting > Setup Tasks > Accounting Lists > Term > New.

For help working with this record in the UI, see the help topic [Creating Terms of Payment](#).

The internal ID for this record is term. Search is not available on this record type.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The term record is scriptable in both client and server SuiteScript

Supported Functions

The term record is partially scriptable. It can be created, updated, copied, and deleted using SuiteScript. It cannot be searched.

Unit of Measure

Applies to: SuiteScript 2.x | SuiteCloud Developer

The Multiple Units of Measure feature enables you to define various units used to stock, purchase, and sell inventory items, and track non-monetary accounts.

For help working with this record in the UI, see the help topics [Multiple Units of Measure](#) and [Setting Up Units of Measure](#).

The internal ID for this record is `unitstype`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The unit of measure record is scriptable in server SuiteScript only.

Supported Functions

The unit of measure record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Vendor Category

Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Vendor Records Overview](#).

The internal ID for this record is `vendorcategory`. Search is not available on this record type.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The vendor category record is scriptable in client and server SuiteScript.

Supported Functions

The vendor category record is fully scriptable, which means that it can be created, updated, copied, deleted, and searched using SuiteScript.

Vendor-Subsidiary Relationship

i Applies to: SuiteScript 2.x | SuiteCloud Developer

If you use NetSuite OneWorld, you can share the vendor record with multiple subsidiaries, and then select those subsidiaries on core transactions.

For help working with Multi-Subsidiary Vendors in the UI, see the help topic [Assigning Subsidiaries to a Vendor](#).

The internal ID for this record is vendorsubsidiaryrelationship.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For customer-subsidiary and vendor-subsidiary relationships belonging to the same entity, both share the external ID of the entity. For example, Company ABC can be both a vendor and customer. If Company ABC has a vendor and customer relationship with the US subsidiary, both share the same external ID.

Supported Script Types

The vendor-subsidiary relationship is scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The vendor-subsidiary relationship can be read, created, updated, and deleted.

Code Sample

The following sample shows how to create a vendor-subsidiary relationship.

```
1 var vsr = record.create({
2   type: record.Type.VENDOR_SUBSIDIARY_RELATIONSHIP
3 });
4 vsr.setValue({
5   fieldId: 'entity',
6   value: 2
7 });
8 vsr.setValue({
9   fieldId: 'subsidiary',
```



```

10 |     value: 6
11 |   });
12 |   var recId = vsr.save();

```

Workplace

Applies to: SuiteScript 2.x | SuiteCloud Developer

To use SuitePeople U.S. Payroll, you must create workplace records for each location where you have a tax ID and maintain paid employees. NetSuite uses workplace records to define the taxes the employer must withhold and pay. Each employee you include in a payroll must have a designated workplace.

For help working with this record in the UI, see the help topic [Entering Workplace Records for Payroll](#).

This record is available only when the Payroll feature is enabled at Setup > Company > Enable Features on the Employees subtab.

In the UI, you can access this record at Lists > Employees > Workplaces.

The internal ID for this record is workplace.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The workplace record is scriptable in both client and server SuiteScript.

Supported Functions

The workplace record is fully scriptable, which means that the record can be created, updated, copied, deleted, and searched using SuiteScript.

Zone

Applies to: SuiteScript 2.x | SuiteCloud Developer

Zone records specify areas in your warehouse where you putaway, store, or pick items. In NetSuite WMS, you can associate zones with warehouse locations.

The zone record is available when the Warehouse Management feature is enabled. For help working with this record, see the help topic [Creating Zones](#).

The internal ID for this record is zone.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The zone record is scriptable in both client and server SuiteScript.

Supported Functions

The zone record is fully scriptable – it can be created, copied, updated, deleted, and searched using SuiteScript.

Code Samples

```

1  require(["N/record"], function (record) {
2      function createZone() {
3          var locationId = 1;
4
5          var zone = record.create({type: record.Type.ZONE, isDynamic: true});
6          zone.setValue({fieldId: "name", value: "Freezers"});
7          zone.setValue({fieldId: "description", value: "Temperature less then 0°C"});
8          zone.setValue({fieldId: "location", value: locationId});
9
10         setTranslationValues(zone, {languageName: "Czech", name: "Mrazáky", description: "Teplota pod 0°C"});
11
12         zone.save();
13     }
14
15     function editZone() {
16         var zoneId = 17;
17         var locationId = 2;
18
19         var zone = record.load({type: record.Type.ZONE, id: zoneId, isDynamic: true});
20         zone.setValue({fieldId: "name", value: "Coolers"});
21         zone.setValue({fieldId: "description", value: "Temperature between 0°C and 6°C"});
22         zone.setValue({fieldId: "location", value: locationId});
23
24         setTranslationValues(zone, {languageName: "Czech", name: "Lednice", description: "Teplota mezi 0°C a 6°C"});
25
26         zone.save();
27     }
28
29     function setTranslationValues(zone, translation) {
30         var lineNumber = zone.findSublistLineWithValue({sublistId: "translations", fieldId: "language", value: translation.language
31         Name});
32         zone.selectLine({sublistId: "translations", line: lineNumber});
33         zone.setCurrentSublistValue({sublistId: "translations", fieldId: "name", value: translation.name});
34         zone.setCurrentSublistValue({sublistId: "translations", fieldId: "description", value: translation.description});
35         zone.commitLine({sublistId: "translations"});
36     }
37
38     createZone();
39     // editZone();

```

39 | });

Customization

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following customization records are scriptable in SuiteScript:

- [Custom List](#)
- [Custom Record](#)
- [Custom Segment Fields and Values](#)
- [Custom Transaction](#)
- [Prompt](#)
- [Scheduled Script Instance](#)
- [Script](#) (individual script types)
- [Script Deployment](#)
- [Text Enhance Action](#)

Custom List

Applies to: SuiteScript 2.x | SuiteCloud Developer

A custom list is a list of values that you can use in custom fields on your forms and records. Custom lists enable you to set up predefined choices for your employees and customers to select when entering transactions and records. Custom lists can be searched using search filters and search columns.

For help working with this record in the UI, see the help topic [Custom Lists](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with custom lists, see the following help topics:

- [SuiteScript 2.x Custom List Pages](#)
- [Custom List IDs](#)
- [Search Filters](#)
- [Search Columns](#)

Custom List IDs

Each custom list has a unique ID. This value shown in the ID field on the custom list record. When creating or interacting with an instance of a custom record type, you must use this ID.

Every custom list ID is prefaced by `customlist`. If the ID was entirely system generated, it ends with a number (for example, `customlist100`). If the ID was customized when the record was created, the ID may be more descriptive. To view a list of all these IDs, go to Customization > Lists, Records, & Fields > Lists. These values are shown in the ID column.

Search Filters

Field Internal ID	Field UI Label	Field Type
internalid	Internal ID	select
internalidnumber	Internal ID (Number)	integer
isinactive	Inactive	checkbox
name	Name	text

Search Columns

Field Internal ID	Field UI Label	Field Type
internalid	Internal ID	select
isinactive	Inactive	checkbox
name	Name	text

Custom Record

Applies to: SuiteScript 2.x | SuiteCloud Developer

A custom record can be used to collect information specific to the needs of your business. For example, you may want to keep track of training courses your employees have taken. Since a record type specific for this purpose does not exist in NetSuite, you could create a custom record type, titled Employee Courses, specifically to store the training course data.

The NetSuite UI enables you to create custom record types and work with instances of those types. You cannot use SuiteScript to create a custom record type or make changes to an existing custom record type. However, you can use SuiteScript to interact with instances of an existing custom record type.

For example, suppose your system had a custom record type called Feature. Each instance of that record type would be a feature record. You could use SuiteScript to update an existing feature record, delete a feature record, or create a new feature record. However, you could not use SuiteScript to alter the Feature custom record type.

For help working with this record in the UI, see the help topic [Creating Custom Record Types](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record, which includes a list of fields commonly available on instances of all custom records.

To interact with fields that were created for a specific record custom type, use the IDs for those fields. These IDs have a prefix of custrecord. For help finding these IDs, see the help topic [Internal IDs for Standard and Custom Fields in Field Level Help Window](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with records in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)

■ SuiteScript 2.x Scripting Records and Subrecords

Custom Record IDs

Each custom record has a unique ID. This value is shown in the ID field on the custom record. When creating or interacting with an instance of a custom record type, you must use this ID.

Every custom record ID is prefaced by `customrecord`. If the ID was entirely system generated, it ends with a number (for example, `customrecord100`). If the ID was customized when the record was created, the ID may be more descriptive. To view a list of all IDs, go to Customization > Lists, Records, & Fields > Record Types. The values are shown in the ID column.

Record Types		
New Type		
+ Filters		
☐ Show Inactives		
Edit	From Bundle ^	ID
Training Area	1220	<code>customrecord_area</code>
Training Class	1220	<code>customrecord_class</code>
Training Course	1220	<code>customrecord_course</code>
Training Level	1220	<code>customrecord_level</code>

You can also see the ID on the record that represents the custom record type.

Custom Record Type

Training Course

[Save](#)
[Cancel](#)
[Change ID](#)

Name *

ID

`customrecord_course`

Internal ID

47

In SuiteScript 2.x, you use this value to set the `Record.type` or `CurrentRecord.type` field. Note that this guidance differs from the way you set this field if you are working with a standard NetSuite record type. When working with a standard record type, you set this field by using either the `record.Type` or `CurrentRecord.type` enum.

See the [Code Sample](#) for examples.

Supported Script Types

Custom records (instances of a custom record type) are scriptable in both client and server SuiteScript.

All three user events are supported: `beforeLoad`, `beforeSubmit`, and `afterSubmit`.

Supported Functions

Custom records are fully scriptable — they can be created, updated, copied, deleted, and searched using SuiteScript.

Code Sample

In SuiteScript 2.x, you use the [N/record Module](#) and [N/currentRecord Module](#) to interact with custom records, the same as you would with a standard record type. Note that you use the custom record type's string ID when setting the [Record.type](#) field, as shown in the following example. For help finding this ID, see [Custom Record IDs](#).

The following sample shows how to work with instances of a custom record type with the script ID `customrecord_feature`. This sample assumes that this custom record type has fields with the field IDs `custrecord_priority` and `custrecord_risklevel`.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ], function(record) {
7      function afterSubmit (context) {
8          // Use a string internal ID to identify the custom record type.
9          var rec = record.create({
10             type: 'customrecord_feature',
11             isDynamic : true
12         });
13
14         rec.setValue({
15             fieldId: 'name',
16             value: 'entry form redesign'
17         });
18
19         rec.setValue({
20             fieldId: 'custrecord_priority',
21             value: 1
22         });
23
24         rec.setValue({
25             fieldId: 'custrecord_risklevel',
26             value: 3
27         });
28
29         try {
30             var callId = rec.save();
31             log.debug('Call record created successfully', 'Id: ' + callId);
32         } catch (e) {
33             log.debug('sigh');
34             log.error(e.name);
35         }
36     }
37     return {
38         afterSubmit: afterSubmit
39     };
40 });

```

Custom Segment Fields and Values

Applies to: SuiteScript 2.x | SuiteCloud Developer

Custom segments are custom classification fields similar to class, department, and location. When the Custom Segments feature is enabled, an administrator working in the UI can create custom segments,

define possible values for each segment, and apply the segments to specific record types. A custom segment appears as a list or multi-select field on instances of record types where the segment was applied. Users can classify records by selecting the appropriate value for each segment.

For help working with this record in the UI, see the help topic [Custom Segments](#).

You cannot use SuiteScript to create a custom segment. However, you can use SuiteScript to create values for existing custom segments. You can also use SuiteScript to set values for segments that appear as fields on record instances. For details, see the following:

- [Using SuiteScript to Create Values for Existing Custom Segments](#)
- [Using SuiteScript to Set Values for Custom Segment Fields](#)

Using SuiteScript to Create Values for Existing Custom Segments

Applies to: SuiteScript 2.x | SuiteCloud Developer

Your NetSuite account contains a custom record type for every custom segment that has been defined. To add a value to the custom segment, you add an instance of the corresponding custom record type.

You can find the internal ID of the appropriate custom record type on the custom segment definition, under the Record ID label.

The screenshot shows the 'Custom Segment' configuration page. At the top, there are buttons for 'Save', 'Cancel', 'Change ID', 'Manage Values', and 'Actions'. Below these is a section titled 'Primary Information'. It contains several fields: 'Label' (with the value 'Profit Center A'), 'ID' (with the value 'cseg_profitcentera'), 'Record ID' (with the value 'customrecord_cseg_profitcentera', which is highlighted with a red box), 'Type' (a dropdown menu set to 'List/Record'), 'Inactive' (an unchecked checkbox), 'Display Type' (a dropdown menu set to 'Normal'), and 'Show in List' (an unchecked checkbox). At the bottom, there is a label 'Custom Record Type' with the value 'Profit Center A'.

For help working with this record in the UI, see the help topic [Custom Segments](#).

Code Sample

The following sample shows how to create a value for a segment called Profit Center.

```

1  ...
2
3  // Create new segment value
4  var newSegmentValue = record.create({
5      type: 'customrecord_cseg_profitcenter'
6  });
7
8  // Set a name value on segment value record
9  newSegmentValue.setValue ({
10     fieldId: 'name',
11     value: stItemIdValue
12 });

```

```

13
14 // If appropriate, you may want to set a parent value for the new value.
15 // For example, you might have a static value that you use for all
16 // segment values created through a script.
17 newSegmentValue.setText ({
18   fieldId: 'parent',
19   text: 'Created from automated script'
20 });
21
22 ...

```

Using SuiteScript to Set Values for Custom Segment Fields

i Applies to: SuiteScript 2.x | SuiteCloud Developer

When a segment is applied to a record type, it appears as a field on instances of that type. You can interact with this field as you would a custom field. To do this, you need to be able to identify both the segment and the relevant values.

A unified script ID is available for custom segment definitions. You can use a single ID to refer to a custom field related to a particular custom segment on any record type the segment is applied to. In this case, the script ID refers to a custom field and not to a particular record type. With the unified script ID, you do not need to distinguish among script ID values based on the record type you are working with.

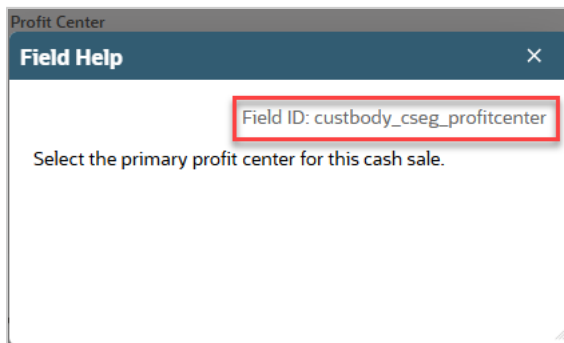
For help working with this record in the UI, see the help topic [Creating a Custom Segment](#).

For details, see the following topics:

- [Custom Segments](#)
- [Identifying a Segment Field when Using the Legacy Script ID](#)
- [Identifying a Segment Field when Using the Unified Script ID](#)
- [Identifying a Segment Value](#)
- [Code Sample](#)

Identifying a Segment Field when Using the Legacy Script ID

You identify a segment field in the same way you would a custom field: by using its field ID. To find the field ID, view an instance of the record type that you want to interact with. Click the label of the field to display a window that includes the field ID.

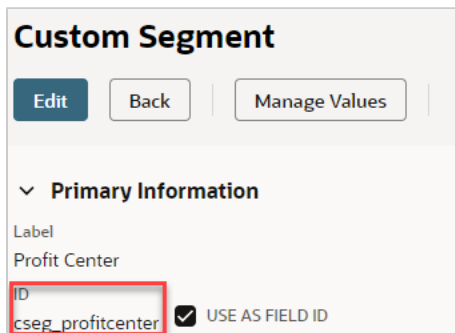


If you have permission to view custom segment definitions, you might notice that the segment definition lists a script ID and record ID. You should **not** refer to either of these IDs when setting a value for the segment on a record instance.

Additionally, be aware that a single custom segment can appear on instances of multiple record types. The field ID for the segment can vary among record types. You must use the field ID as it appears on an instance of the **same record type** where you want your script to set a segment value.

Identifying a Segment Field when Using the Unified Script ID

If you use the unified script ID for a custom segment, the script ID refers to **all** custom fields for record types that the segment is applied to. In this case, you must use the ID as it appears on the custom segment record to identify any custom field on a custom segment.



Custom Segment

Edit Back Manage Values

▼ **Primary Information**

Label
Profit Center

ID
cseg_profitcenter ☒ USE AS FIELD ID

If the Use as Field ID box is checked, it means that you are using the unified script ID for the particular custom segment only.

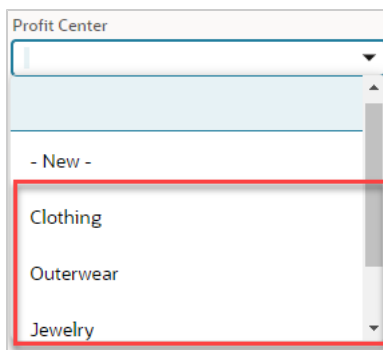
Identifying a Segment Value

Typically, each custom segment includes a list of possible values. To set a value for a segment by using SuiteScript, at least one value must already have been defined for the segment.

You can reference a custom segment value by specifying either of the following:

- The value's text label
- The value's internal ID

The text label is shown in the custom segment definition and on instances of the record type where the segment has been applied. On the record instance, the segment's values are shown in either a multi-select box or a list.



Profit Center

- New -

Clothing

Outerwear

Jewelry

To find a segment value's internal ID, open the segment definition at Customization > Lists, Records, & Fields > Custom Segments. To see a list of all possible values, refer to the Values sublist. This list includes a column labeled ID, which identifies each value's internal ID.

Values	Application & Sourcing	Validation & Defaulting	Permissions
Display Order	X	Sublist	Alphabetical
ID	Value	Translation	
1	Clothing		
3	Outerwear		
5	Jewelry		
6	Shoes		

Code Sample

The following snippet shows how to set a value for a segment with the field ID of `custbody_cseg_profitcenter`. This uses the legacy script ID to refer to a transaction body field on a custom segment.

```
1 objRecord.setText({
2   fieldId: 'custbody_cseg_profitcenter',
3   text: 'Outerwear',
4   ignoreFieldChange: false
5 });
```

The following snippet shows how to set a value for a segment that is included on a transaction line.

```
1 objRecord.setCurrentSublistText({
2   sublistId: 'item',
3   fieldId: 'custbody_cseg_profitcenter',
4   text: 'Outerwear',
5   ignoreFieldChange: false
6 });
```

Custom Transaction

Applies to: SuiteScript 2.x | SuiteCloud Developer

You use the custom transaction record to interact with instances of existing custom transaction types. For example, suppose you had a custom transaction type named Non-Operational Income Entry. In this case, you could use SuiteScript to create and modify non-operational income entries.

For help working with this record in the UI, see the help topic [Custom Transactions](#).

The internal ID for a custom transaction record varies depending on the ID value of the custom transaction type. For example, if your custom transaction type had an ID value of `_noie`, the SuiteScript internal ID would be `customtransaction_noie`. You can view the custom transaction type's ID value by opening it editing (go to Customization > Lists, Records, & Fields > Transaction Types) and selecting the appropriate type.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with records in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)

Supported Script Types

The custom transaction record (instance of a custom transaction type) are scriptable in both client and server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The custom transaction record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

You use the standard transaction search to search custom transactions. For more information, see [Transaction Search](#).

Custom transactions can be voided if configured to support the void option. Direct voids are never permitted with custom transactions. Only voids through reversal journals are permitted. See the help topic [transaction.void\(options\)](#).

Usage Notes

In the UI, custom transactions can have a Status field. In SuiteScript, this field has an internal ID of transtatus. Values for transtatus are set by the system when the statuses are created. They cannot be modified. Possible values are A through Z. In the UI, you can view a transaction type's available statuses (and their transtatus values) by opening the transaction type and going to the Statuses subtab. The transtatus value for each status is listed in the Code column.

Code Sample

The following sample shows how to create an instance of a custom transaction type.

```

1  var transaction = record.create ({
2      type: 'customtransaction_mytype'
3  });
4  transaction.setValue ({
5      fieldId: 'subsidiary',
6      value: '1'
7  });
8  // Add debit line
9  transaction.selectNewLineItem ({
10     sublistId: 'line'
11 });
12 transaction.setCurrentSublistValue ({
13     sublistId: 'line',
14     fieldId: 'account',
15     value: '1'
16 });
17 transaction.setCurrentSublistValue ({
18     sublistId: 'line',
19     fieldId: 'debit',
20     value: 10
21 });
22 transaction.setCurrentSublistValue ({
23     sublistId: 'line',

```

```

24     fieldId: 'credit',
25     value: 0
26   });
27   transaction.setCurrentSublistValue ({
28     sublistId: 'line',
29     fieldId: 'memo',
30     value: 'My first custom transaction line'
31   });
32   transaction.commitLine ({
33     sublistId: 'line'
34   });
35   // Add credit line
36   transaction.selectNewLineItem ({
37     sublistId: 'line'
38   });
39   transaction.setCurrentSublistValue ({
40     sublistId: 'line',
41     fieldId: 'account',
42     value: '2'
43   });
44   transaction.setCurrentSublistValue ({
45     sublistId: 'line',
46     fieldId: 'debit',
47     value: 0
48   });
49   transaction.setCurrentSublistValue ({
50     sublistId: 'line',
51     fieldId: 'credit',
52     value: 10
53   });
54   transaction.setCurrentSublistValue ({
55     sublistId: 'line',
56     fieldId: 'memo',
57     value: 'My second custom transaction line'
58   });
59
60   transaction.commitLine ({
61     sublistId: 'line'
62   });
63
64   transaction.save();

```

Prompt

Applies to: SuiteScript 2.x | SuiteCloud Developer

A prompt record represents a generative artificial intelligence (AI) prompt. Generative AI prompts are the instructions given to a large language model (LLM). The Text Enhance feature uses prompts to provide instructions to the LLM to generate or modify text. For more information, see the help topic [Text Enhance](#). You can also manage prompt records using SuiteScript. For more information, see the help topic [N/llm Module](#).

A prompt record includes information about the prompt and the LLM to send the prompt to, such as:

- The prompt type (for example, Text Enhance or Generic)
- The record type and field the prompt applies to (for example, the description field on an expense item record)
- The model family to use when sending the prompt text to the LLM (for example, Cohere Command R)
- The prompt template, which represents the text to send to the LLM, including any variables that the prompt uses

The internal ID for this record is prompt.

For help working with this record in the UI, see the help topic [Prompt Studio](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)
- [N/llm Module](#)

Supported Script Types

The prompt record is scriptable in both client and server SuiteScript.

Supported Functions

The prompt record is partially scriptable — it can be created, updated, and deleted using SuiteScript.

Certain fields cannot be updated after a prompt record is created. You can update the values of these fields in your scripts, but you will receive an error if you try to save the record with the updated values.

Code Sample

The following code sample demonstrates how to create a prompt record, set field values, and save the prompt record. The sample also calls `llm.evaluatePrompt(options)` and `llm.evaluatePrompt.promise(options)` to test the new prompt.

```

1  require(['N/record', 'N/llm'], function(record, llm) {
2      const rec = record.create({
3          type: "prompt"
4      });
5
6      rec.setValue({
7          fieldId: "name",
8          value: "Test"
9      });
10     rec.setValue({
11         fieldId: "prompttype",
12         value: "CUSTOM"
13     });
14     rec.setValue({
15         fieldId: "modelfamily",
16         value: "COHERE_COMMAND_R"
17     });
18     rec.setValue({
19         fieldId: "template",
20         value: "${mandatoryVariable} <#if optionalVariable?has_content>${optionalVariable}<#else>World</#if>"
21     });
22
23     const id = rec.save();
24
25     try {
26         llm.evaluatePrompt({
27             id: id
28         });
29     }
30     catch (e){
31         if (e.name === "TEMPLATE_PROCESSING_EXCEPTION")
32             log.debug("Exception", "Expected exception was thrown");
33     }
34
35     const response = llm.evaluatePrompt({
36         id: id,

```

```

37     variables: {
38         mandatoryVariable: "Hello",
39         optionalVariable: "People"
40     }
41 });
42 if ("Hello People" === response.chatHistory[0].text)
43     log.debug("Evaluation", "Correct prompt got evaluated");
44
45 llm.evaluatePrompt.promise({
46     id: id,
47     variables: {
48         mandatoryVariable: "Hello", optionalVariable: "World"
49     }
50 }).then(function(response) {
51     if ("Hello World" === response.chatHistory[0].text)
52         log.debug("Evaluation", "Correct prompt got evaluated");
53     record.delete({
54         type: "prompt",
55         id: id
56     });
57     debugger;
58 })
59 });

```

Scheduled Script Instance

i Applies to: SuiteScript 2.x | SuiteCloud Developer

You can use the scheduled script instance search to return information about individual scheduled script and map/reduce script jobs. A subset of this information is displayed on the Scheduled Scripts and Map/Reduce Scripts pages in the NetSuite UI:

- **Scheduled Scripts:** Each entry on the Scheduled Script Status page contains information about a single scheduled script instance. Each scheduled script instance is handled by one scheduled script job. To access the Scheduled Script Status page in the UI, go to Customization > Scripting > Scheduled Script Status.
- **Map/Reduce Scripts:** Each entry on the Map/Reduce Script Status page contains information about a single map/reduce script instance. Each map/reduce script instance is handled by multiple map/reduce script jobs. To access the Map/Reduce Script Status page in the UI, go to Customization > Scripting > Map/Reduce Script Status. You can access information about the individual map/reduce jobs for a particular map/reduce script instance from the map/reduce script details page. To access this page from the Map/Reduce Script Status page, click **Details** for the appropriate script instance.

For more information about the Scheduled Script Status page, see the help topic [Scheduled Script Status Page](#). For more information about the Map/Reduce Script Status page, see the help topic [Map/Reduce Script Status Page](#).

For help working with this record in the UI, see the help topic [SuiteScript Monitoring, Auditing, and Logging](#).

The internal ID for this record is `scheduledscriptinstance`.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with records in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)

Supported Script Types

The scheduled script instance record is scriptable in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

The scheduled script instance record is partially scriptable — it can only be searched. It cannot be created, updated, copied, or deleted using SuiteScript.

Usage Notes

You can script searches on `scheduledscriptinstance` with all server script types. Scheduled script instance searches are not exposed to client scripts.

For a list of SuiteScript supported `scheduledscriptinstance` fields, see the [SuiteScript Records Browser](#). In addition, see the following table for supplemental information about a few supported fields.

Field	Notes
Original Priority (originalpriority)	This field holds the original priority setting (low, standard, or high) for the scheduled script job or map/reduce job. The priority setting is the value set when the scheduled script job or map/reduce job is initially created. ■ If a job is created when a deployment is submitted for processing, the original priority is the priority set on the deployment record at the time it was submitted for processing. ■ If a job is created during script processing, the original priority is the current priority of the job that created the new job. For more information about deployment record priority settings, see the help topic SuiteCloud Processors Priority Levels.  Note: For scheduled scripts, one scheduled script job is created when a script deployment is submitted for processing. For map/reduce scripts, the initial map/reduce jobs are created when the script deployment is submitted for processing. In addition, more map/reduce jobs can be created during script processing.
Original Priority Number (originalprioritynumber)	This field holds a numeric value that corresponds to the original priority value: ■ High priority: 1 ■ Standard priority: 2 ■ Low priority: 3 You cannot group your search results by the original priority field because it holds a text value. The original priority number field is provided as an alternative.
Priority Timestamp (prioritytimestamp)	This field holds the time that the current priority is set for the scheduled script job or map/reduce job. This field can reflect the timestamp of a user-defined priority setting change or a system-defined priority setting change. User-defined priority changes do not impact the value of this field for existing jobs. However, system-defined priority changes may impact this value for existing jobs.

Field	Notes
	For example, if you create a scheduled script deployment and immediately submit the deployment for processing, the priority timestamp for that job reflects the deployment submission time. If you then change the priority on the deployment record, neither the priority nor the priority timestamp for the existing job is changed. However, If you enable priority elevation, and the priority for the existing job is changed by the system, the priority timestamp value is changed accordingly. For more information about priority elevation, see the help topic SuiteCloud Processors Priority Elevation .

Code Sample

The following sample shows how to perform a search for all schedule script instance records that are in progress in queue 2. It uses search filters and columns to return all available information in the filtered results. Note that the first two search columns are joins on the script and script deployment record types.

```
1 var searchResults = search.create ({
2   type: 'scheduledscriptinstance',
3   filters: [
4     {name: 'queue',
5       operator: IS,
6       values: '2'
7     },
8     {name: 'status',
9       operator: ANYOF,
10      values: 'PROCESSING'
11    }
12  ],
13  columns: [
14    {name: 'name', join: 'script'},
15    {name: 'internalid', join: 'scriptdeployment'},
16    {name: 'datecreated'},
17    {name: 'status'},
18    {name: 'startdate'},
19    {name: 'enddate'},
20    {name: 'queue'},
21    {name: 'percentcomplete'},
22    {name: 'queueposition'},
23    {name: 'percentcomplete'}
24  ]
25 });
26 var mySearchResults = searchResults.run();
```

Script

Applies to: SuiteScript 2.x | SuiteCloud Developer

A script record allows you to deploy your entry point script. The SuiteScript Records Browser lists the script record for each script type separately (see table below).

For help working with script records in the UI, see the help topic [Script Record Creation](#).

Each script record has a unique internal ID. The following table lists each internal id and provides links to the SuiteScript Records Browser for each type of script record. Refer to these records for all other internal IDs associated with each record.

Script Record	Internal ID	SuiteScript Records Browser Link
Bundle Installation Script	bundleinstallationscript	Bundle Installation Script

Script Record	Internal ID	SuiteScript Records Browser Link
Client Script	clientscript	Client Script
Map/Reduce Script	mapreducescript	Map/Reduce Script
Massupdate Script	massupdatescript	Massupdate Script
Portlet	portlet	Portlet
Restlet	restlet	Restlet
Suitelet	suitelet	Suitelet
User Event Script	usereventscript	User Event Script
Workflow Action Script	workflowactionscript	Workflow Action Script

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with records in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)

Supported Script Types

Script records are scriptable in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Supported Functions

Each script record is partially scriptable — it can be updated and searched. It cannot be created, copied, or deleted.

Usage Notes

You can load, update and save, and search script records with all server script types. Script records are not exposed to client scripts.

The following objects are not scriptable on any script record:

- The Change ID button (in Edit view of UI)
- The Unhandled Errors subtab
- The History subtab
- The System Notes subtab
- The Execution Log subtab
- The Libraries subtab

The following objects are read-only (editing is not supported):

- The Type field
- The Parameters subtab

To access script records within a script, the script owner must belong to a role assigned with SuiteScript permissions.



Important: Scripts can only access script records when the records are part of the same bundle.

Scripts that are not part of a bundle cannot access script records that are part of a bundle.

For the `id` parameter in `record.load(options)`, `search.load(options)`, or `search.create(options)` methods, you must use the script record's internal ID (for example, 191); the script record's ID (for example, `customscript_csalert`) is not supported. You can find a script record's internal ID at Customization > Scripting > Scripts. If you have the Show Internal IDs preference enabled at Home > Set Preferences, internal IDs are listed in the Internal ID column.

Scripts

New Script

View Deployments

Filters

Type

User Event

API Version

- All -

Script File

- All -

From Bundle

Show Inactives

Customscript1442 — Customscript_edp_ue_pu_validations

<

>

Total: 142

Edit View	Name	From Bundle	ID ^	API Version	Deployments	Script	Library Script	Internal ID
Edit View	User Event Module Test		customscript1442	2.0	Deployments	UserEventModuleTest.js		1442
Edit View	testResetButton		customscript1447	2.0	Deployments	testResetButton.js		1447
Edit View	AddSubTabtoForm		customscript190	1.0	Deployments	addsubtabs.js		190

For record and search methods that require the type parameter, use one of the following `record.Type` or `search.Type` values:

- BUNDLE_INSTALLATION_SCRIPT
- CLIENT_SCRIPT
- MAP_REDUCE_SCRIPT
- MASSUPDATE_SCRIPT
- PORTLET
- RESTLET
- SUITELET
- USEREVENT_SCRIPT
- WORKFLOW_ACTION_SCRIPT

Code Samples

The following samples show how to load and read a script record, how to create a script record, and how to search a script record.

```

1 // Read a SUITELET script record
2 var rec = record.load ({
3     type: record.Type.SUITELET,
4     id: 605
5 });
6 var a = rec.getValue({
7     fieldId: 'scriptfile'
8 });
9 var count = rec.getLineCount ({
10    sublistId: 'parameters'
11 });
12 var pLabel = rec.getSublistValue ({
13    sublistId: 'parameters',
14    fieldId: 'label',

```

```

15     line: 1
16 });
17 var pId = rec.getSublistValue ({
18     sublistId: 'parameters',
19     fieldId: 'internalid',
20     line: 1
21 });
22 var pType = rec.getSublistValue ({
23     sublistId: 'parameters',
24     fieldId: 'fieldtype',
25     line: 1
26 });
27 var pRecordType = rec.getSublistValue ({
28     sublistId: 'parameters',
29     fieldId: 'selectrecordtype',
30     line: 1
31 });

```

```

1 // Edit a user event script record
2 var rec = record.load ({
3     type: record.Type.USEREVENT_SCRIPT,
4     id: 302
5 });
6 rec.setValue ({
7     fieldId: 'name',
8     value: 'userevent_001'
9 });
10 rec.setValue ({
11     fieldId: 'scriptfile',
12     value: '227'
13 });
14 rec.setValue ({
15     fieldId: 'notifyadmins',
16     value: 'T'
17 });
18 rec.setValue ({
19     fieldId: 'aftersubmitfunction',
20     value: 'afterSubmitFunction'
21 });
22 rec.save();

```

```

1 // Search a script record
2 var searchFilters = search.createFilter ({
3     name: 'defaultfunction',
4     operator: search.Operator.IS,
5     values: 'myfunctionname'
6 });
7 var searchColumns = search.createColumn ({
8     name: 'name'
9 });
10 var mySearch = search.create ({
11     type: search.Type.SUITELET,
12     filters: searchFilters,
13     columns: searchColumns
14 });
15 var mySearchResults = mySearch.run();

```

Script Deployment

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Script deployment records are required to run your script in your NetSuite account.

For help working with this record in the UI, see the help topic [Script Deployment](#).

The internal ID for the script deployment record is scriptdeployment.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.



Important: The **recurrenceminutes** and **starttime** fields are not scriptable.



Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with records in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)

Supported Script Types

Script records are scriptable in server SuiteScript only.

Supported Functions

The script deployment record is fully scriptable — it can be created, copied, updated, deleted, and searched using SuiteScript.

Usage Notes

You can access script deployment records with all server script types. Script deployment records are not exposed to client scripts.

The following objects are not scriptable on the script deployment record:

- Change ID button (in Edit view of UI)
- The History subtab
- The System Notes subtab
- The Execution Log subtab
- The Scheduling subtab

The following object is read-only on the script deployment record (editing is not supported):

- The Parameters subtab

To access script deployment records within a script, the script owner must belong to a role assigned with SuiteScript permissions.



Important: Scripts can only access script deployment records when the records are part of the same bundle.

Scripts that are not part of a bundle cannot access script deployment records that are part of a bundle.

You can use script deployment records to set up context filtering. Context filtering lets you specify how and when the associated script runs. For more information, see the help topics [Execution Contexts](#) and [Record Localization Context](#).

For methods that require the ID parameter, you must use the script deployment record's internal ID (for example, 119). The script deployment record's script ID (for example, customdeploy_newsfeed_email) is

not supported. You can find a script deployment record's internal ID at Customization > Scripting > Script Deployments. If you have the Show Internal IDs preference enabled at Home > Set Preferences, internal IDs are listed in the Internal ID column.

Script Deployments

Scripts

Filters

Type

Scheduled

Status

- All -

Record Type

- All -

API Version

- All -

Script

- All -

Show Undeployed

In — Not Scheduled

Total: 152

Internal ID	Edit View	ID	Script	API Version	Status	Title	Schedule	Priority	Queue	Last Modified
139	Edit View	customdeploy1	NLTestSuiteLauncher	1.0	In Queue	NLTestSuiteLauncher	occurs every day starting on 27.5.2010	Standard		13.6.2012 7:01 pm
9485	Edit View	customdeploy_pd_bundle_install_ss	PD Bundle Install Scheduled	2.0	Not Scheduled	PD Bundle Install Scheduled	one time event on 19.2018	Standard	1	28.7.2024 8:40 pm
119	Edit View	customdeploy1	testinactive	1.0	Not Scheduled	testScheduledScript2	occurs every Thursday from 15.6.2006 until 15.6.2006	Standard		21.2.2009 11:56 pm

For record and search methods that require the type parameter, use the `record.Type.SCRIPT_DEPLOYMENT` or `search.Type.SCRIPT_DEPLOYMENT` value.

Code Samples

The following samples show how to create a script deployment record, how to edit a script deployment record, how to read a script deployment record, how to copy and edit a script deployment record, and how to search a script deployment record.

```

1 // Create a script deployment record
2 var rec = record.create ({
3   type: record.Type.SCRIPT_DEPLOYMENT,
4   defaultValues: {
5     script: 2 //scriptId
6   }
7 });

```

```

1 // Edit a script deployment record
2 var rec = record.load ({
3   type: record.Type.SCRIPT_DEPLOYMENT,
4   id: 65
5 });
6 rec.setValue({
7   fieldId: 'allroles',
8   value: 'T'
9 });
10 rec.save();

```

```

1 // Read a script deployment record
2 var rec = record.load ({
3   type: record.Type.SCRIPT_DEPLOYMENT,
4   id: 203
5 });
6 var recStatus = rec.getValue ({
7   fieldId: 'status'
8 });

```

```

1 // Copy and edit a script deployment record
2 var rec = record.copy ({
3   type: record.Type.SCRIPT_DEPLOYMENT,
4   id: 41
5 });
6 rec.setValue ({
7   fieldId: 'isdeployed',

```

```

8     value: 'F'
9   });
10  rec.setValue ({
11    fieldId: 'title',
12    value: 'Feature 4141 - Suitelet Scratch 4'
13  });
14  rec.save();

```

```

1  // Search a script deployment record
2  var searchFilters = search.createFilter ({
3    name: 'internalidnumber',
4    operator: search.Operator.ANYOF,
5    values: '43'
6  });
7  var searchColumns = search.createColumn ({
8    name: 'title'
9  });
10 var mySearch = search.create ({
11   type: search.Type.SCRIPT_DEPLOYMENT,
12   filters: searchFilters,
13   columns: searchColumns
14 });
15 var mySearchResults = mySearch.run();

```

Text Enhance Action

Applies to: SuiteScript 2.x | SuiteCloud Developer

A Text Enhance action record represents a Text Enhance action that can be added to supported text fields. For more information, see the help topic [Text Enhance](#). The following types of text fields support Text Enhance actions:

- Long text
- Text area
- Rich text

A Text Enhance action record includes information about the action, such as:

- Name and script ID
- Language
- Whether the action is a partial Text Enhance action

The internal ID for this record is `textenhanceaction`.

For help working with this record in the UI, see the help topic [Prompt Studio](#).

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following help topics:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [N/record Module](#)
- [N/llm Module](#)

Supported Script Types

The Text Enhance action record is scriptable in both client and server SuiteScript.

Supported Functions

The Text Enhance action record is partially scriptable — it can be created, updated, and deleted using SuiteScript.

Certain fields cannot be updated after a Text Enhance action record is created. You can update the values of these fields in your scripts, but you will receive an error if you try to save the record with the updated values.

Code Sample

The following code sample demonstrates how to create a Text Enhance action record, set field values, and save the Text Enhance action record.

```

1  require(['N/record'], function(record) {
2      var action = record.create({
3          type: "textenhanceaction"
4      });
5
6      action.setValue({
7          fieldId: "name",
8          value: "Summarize"
9      });
10     action.setValue({
11         fieldId: "language",
12         value: "en"
13     });
14     action.setValue({
15         fieldId: "scriptid",
16         value: "custteaction_summarize"
17     });
18
19     action.save();
20 });

```

Marketing

Applies to: SuiteScript 2.x | SuiteCloud Developer

The following marketing records are scriptable in SuiteScript:

- [Campaign](#)
- [Campaign Response](#)
- [Campaign Template](#)
- [Coupon Code](#)
- [Email Template](#)
- [Promotion](#)

Campaign

Applies to: SuiteScript 2.x | SuiteCloud Developer

Note: This topic applies to all versions of SuiteScript.

Campaign records are used to manage your marketing initiatives. With campaign records, you can manage your online marketing as well as your offline efforts. You can create and send email marketing

messages, manage paid keywords, send promotions through the mail, and manage your advertising campaigns.

For help working with this record in the UI, see the help topics [Campaign Overview](#) and [Creating a Campaign Record](#).

The internal ID for this record is campaign.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The campaign record is scriptable in both client and server SuiteScript.

Supported Functions

The campaign record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Campaign Response

 **Applies to:** SuiteScript 2.x | SuiteCloud Developer

The campaign response record helps you refine how you deliver marketing campaigns.

To view this record in the UI, go to an entity record and click the Marketing subtab. On the Campaigns subtab, click the Add Response button.

For help working with this record in the UI, see the help topic [Tracking Campaign Responses](#).

The internal ID for this record is campaignresponse.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [Tracking Campaign Responses](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The campaign response record is scriptable in both client and server SuiteScript.

Supported Functions

The campaign response record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Campaign Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

The campaign template record is used for both CRMSDK templates and scriptable templates. In the UI, you find this record by choosing Documents > Templates > Marketing Templates.

For help working with this record in the UI, see the help topic [Email Marketing Campaigns](#).

Scriptable templates are not supported with SuiteScript.

The internal ID for this record is campaigntemplate.

See the [SuiteScript Records Browser](#) for the internal IDs of all fields associated with this record, their corresponding labels in the UI, and more details. Fields not listed in the Records Browser are not supported and should not be used.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The campaign template record is scriptable in server SuiteScript only.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Field Definitions

This section describes some of the key fields on the campaign template record.

Content and Mediaitem

When accessing this record, only one of the following fields contains a value:

- **content** — This field maps to the text editor that appears on the Template subtab in the UI. This field contains plain text or HTML.
- **mediaitem** — This field maps to the File text box that appears on the Template tab in the UI. This field identifies the file that is used as the basis for the template.

Code Sample

The following sample shows how to access template content:

```

1  var template = record.load({
2    type: record.Type.CAMPAIGN_TEMPLATE,
3    id: 154
4  });
5
6  if (template.getValue({fieldId: 'mediaitem'}) != null) {
7    var media = template.getValue({fieldId: 'mediaitem'});
8    // do something with the media...
9  } else {
10   var content = template.getValue({fieldId: 'content'});
11   // do something with the content...
12 }

```

Coupon Code

Applies to: SuiteScript 2.x | SuiteCloud Developer

Promotions allow you to track the source of revenue and to offer discounts in the form of coupons. Each promotion has a promotion code that can be applied to transactions and campaigns. Enable Promotion Codes on the Transactions subtab at Setup > Company > Setup Tasks > Enable Features.

For help working with this record in the UI, see the help topic [Selling with Promotion Codes](#).

The internal ID for this record is couponcode.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The coupon code record is scriptable in both client and server SuiteScript.

Supported Functions

The coupon code record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Email Template

Applies to: SuiteScript 2.x | SuiteCloud Developer

The email template record is used for both CRMSDK templates and scriptable templates. In the UI, you find this record by choosing Documents > Templates > Email Templates. This record is available only if the

Customer Relationship Management feature has been enabled at Setup > Company > Enable Features, on the CRM subtab.

For help working with this record in the UI, see the help topic [Working with Email Templates](#).

Scriptable templates are not supported with SuiteScript.

The internal ID for this record is emailtemplate.

See the [SuiteScript Records Browser](#) for the internal IDs of all fields associated with this record, their corresponding labels in the UI, and more details. Fields not listed in the Records Browser are not supported and should not be used.

 **Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#).

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

This record is only supported in server SuiteScript.

All three user events are supported: beforeLoad, beforeSubmit, and afterSubmit.

Field Definitions

This section describes some of the key fields on the email template record.

Content and Mediaitem

When accessing this record, only one of the following fields contains a value:

- **content** — This field maps to the text editor that appears on the Template subtab in the UI. This field contains plain text or HTML.
- **mediaitem** — This field maps to the File text box that appears on the Template tab in the UI. This field identifies the file that is used as the basis for the template.

Code Sample

The following sample shows how to access template content:

```

1 var template = record.load({
2   type: record.Type.EMAIL_TEMPLATE,
3   id: 124
4 });
5
6 if (template.getValue({fieldId: 'mediaitem'}) != null) {
7   var media = template.getValue({fieldId: 'mediaitem'});
8   // do something with the media...
9 } else {
10  var content = template.getValue({fieldId: 'content'});
11  // do something with the content...

```

Promotion

Applies to: SuiteScript 2.x | SuiteCloud Developer

Promotions allow you to create special offers to motivate your customers to purchase products in higher quantities. Promotions can help you to move out-of-date stock, increase sales, and reward valuable clients. Promotions provide discounts that customers can apply to Web store orders and orders placed with sales reps. Furthermore, NetSuite promotions enable you to target specific customers, locations or channels, and time periods.

For help working with this record in the UI, see the help topics [Promotions Overview](#), [Configuring Promotions](#), and [The Promotion Record](#).

The internal ID for this record is promotioncode.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

**Note:** For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The promotion record is scriptable in both client and server SuiteScript.

Supported Functions

The promotion record is fully scriptable — it can be created, updated, copied, deleted, and searched using SuiteScript.

Usage Notes

The following table provides usage notes for specific fields on this record.

Field Internal ID	Field UI Label	Note
Body Fields		
discounttype	radio	Valid values in scripts are: ■ percent ■ flat

When applying a free gift promotion to a transaction using SuiteScript, the free gift is not automatically added or removed from the order.

The following SuiteScript example shows how to apply the free gift and remove it when the transaction is no longer eligible:

```

1 // Create Sales Order
2 var mySalesOrder = record.create({
3     type: record.Type.SALES_ORDER,
4     isDynamic: true
5 });
6 mySalesOrder.setValue({
7     fieldId: 'entity',
8     value: '41'
9 });
10
11 // Add free gift promotion to the promotion machine
12 mySalesOrder.selectNewLine({
13     sublistId: 'promotions'
14 });
15 mySalesOrder.setCurrentSublistValue({
16     sublistId: 'promotions',
17     fieldId: 'promocode',
18     value: '103'
19 });
20 mySalesOrder.commitLine({
21     sublistId: 'promotions'
22 });
23
24 // Add eligible item with quantity 4 to the item machine
25 mySalesOrder.setCurrentSublistValue({
26     sublistId: 'item',
27     fieldId: 'item',
28     value: '233'
29 });
30 mySalesOrder.setCurrentSublistValue({
31     sublistId: 'item',
32     fieldId: 'quantity',
33     value: '4'
34 });
35 mySalesOrder.commitLine({
36     sublistId: 'item'
37 });
38
39 // ***** Manually add free gift and link it to the free gift promotion *****
40 mySalesOrder.setCurrentSublistValue({
41     sublistId: 'item',
42     fieldId: 'item',
43     value: '244'
44 });
45 mySalesOrder.setCurrentSublistValue({
46     sublistId: 'item',
47     fieldId: 'quantity',
48     value: '2'
49 });
50 mySalesOrder.setCurrentSublistValue({
51     sublistId: 'item',
52     fieldId: 'freegiftpromotion',
53     value: '103'
54 });
55 mySalesOrder.commitLine({
56     sublistId: 'item'
57 });
58
59 // Check Free Gift Promotion columns on Promotion Machine:
60 // firstFreeGiftQuantity should say 2
61 // firstEligibleFreeGifts should say 2 // firstFreeGiftsAdded should say 2
62 var firstFreeGiftQuantity= mySalesOrder.getSublistValue({
63     sublistId: 'item',
64     fieldId: 'quantity',
65     line: 2
66 });
67 var firstEligibleFreeGifts = mySalesOrder.getSublistValue({
68     sublistId: 'promotions',
69     fieldId: 'eligiblefreegifts',

```

```

70     line: 1
71   });
72   var firstFreeGiftsAdded = mySalesOrder.getSublistValue({
73     sublistId: 'promotions',
74     fieldId: 'freegiftsadded',
75     line: 1
76   });
77
78   // ***** Reduce eligible item quantity to 3 *****
79   mySalesOrder.selectLine({
80     sublistId: 'item',
81     line: '1'
82   });
83   mySalesOrder.setCurrentSublistValue({
84     sublistId: 'item',
85     fieldId: 'quantity',
86     value: '3'
87   });
88   mySalesOrder.commitLine({
89     sublistId: 'item'
90   });
91
92   // Verify that the columns have been updated but the user still needs to manually remove 1 Free Gift
93   // secondFreeGiftQuantity should say 2
94   // secondEligibleFreeGifts should say 1
95   // secondFreeGiftsAdded should say 2
96   var secondFreeGiftQuantity= mySalesOrder.getSublistValue({
97     sublistId: 'item',
98     fieldId: 'quantity',
99     line: 2
100  });
101  var secondEligibleFreeGifts = mySalesOrder.getSublistValue({
102    sublistId: 'promotions',
103    fieldId: 'eligiblefreegifts',
104    line: 1
105  });
106  var secondFreeGiftsAdded = mySalesOrder.getSublistValue({
107    sublistId: 'promotions',
108    fieldId: 'freegiftsadded',
109    value: 1
110  });
111
112  // ***** Manually reduce Free Gift quantity *****
113  mySalesOrder.selectLine({
114    sublistId: 'item',
115    line: '2'
116  });
117  mySalesOrder.setSublistValue({
118    sublistId: 'item',
119    fieldId: 'quantity',
120    value: '1'
121  });
122  mySalesOrder.commitLine({
123    sublistId: 'item'
124  });
125
126  // Verify that the promotions machine columns have been updated again
127  // Verify that the columns have been updated but the user still needs to manually remove 1 Free Gift
128  // firstFreeGiftQuantity should say 1
129  // firstEligibleFreeGifts should say 1
130  // firstFreeGiftsAdded should say 1
131  var thirdFreeGiftQuantity= mySalesOrder.getSublistValue({
132    sublistId: 'item',
133    fieldId: 'quantity',
134    line: 2
135  });
136  var thirdEligibleFreeGifts = mySalesOrder.getSublistValue({
137    sublistId: 'promotions',
138    fieldId: 'eligiblefreegifts',
139    line: 1
140  });
141  var thirdFreeGiftsAdded = mySalesOrder.getSublistValue({
142    sublistId: 'promotions',

```

```

143 |     fieldId: 'freegiftsadded',
144 |     line: 1
145 | });

```

Working with the Discount Amount Search Column

When you use the Discount Amount search column, instead of using the discountamount internal ID, use discountrate.

Website

i Applies to: SuiteScript 2.x | SuiteCloud Developer

The following website records are scriptable in SuiteScript:

- [CMS Content](#)
- [CMS Content Type](#)
- [CMS Page](#)
- [Commerce Category](#)
- [Website Setup](#)
- [Shopping Cart](#)

CMS Content

i Applies to: SuiteScript 2.x | SuiteCloud Developer

CMS content enables you to create things such as text and images for your website.

For help working with this record in the UI, see the help topic [Site Management Tools](#).

The internal ID for this record is cmscontent.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The CMS content record is scriptable in server SuiteScript only.

Supported Functions

The CMS content record is partially scriptable. It can be created, updated, and searched using SuiteScript. It cannot be copied, attached, or transformed.

Usage Notes

You must enable the Site Management Tools feature on the Web Presence subtab at Setup > Company > Enable Features to be able to script with this record.

Script Sample

The following sample code snippet creates a script that will search for a CMS content record.

```

1 function searchCmsContentRecords(siteId){
2     var mySearch = search.create({
3         type: search.Type.CMS_CONTENT,
4         filters: [{
5             name: 'site',
6             operator: search.Operator.IS,
7             values: 'siteId'
8         }],
9         columns: [{
10             name: 'name'
11         }, {
12             name: 'site'
13         }, {
14             name: 'cmscontenttype'
15         }],
16     });
17     mySearch.run().each(function(result){
18         var name = result.getValue({
19             name: 'name'
20         });
21         var label = result.getValue({
22             name: 'site'
23         });
24         var customRecordId = result.getValue({
25             name: 'cmscontenttype'
26         });
27         return true;
28     });
29 }
```

CMS Content Type

Applies to: SuiteScript 2.x | SuiteCloud Developer

CMS page enables you to create different content types for your website. For more information, see the help topic [Site Management Tools](#).

For help working with this record in the UI, see [CMS Content](#).

The internal ID for this record is cmspage.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The CMS content type record is scriptable in server SuiteScript only.

Supported Functions

The CMS content type record is partially scriptable. It can be created, updated, and searched using SuiteScript. It cannot be copied, attached, or transformed.

Usage Notes

You must enable the Site Management Tools feature on the Web Presence subtab at Setup > Company > Enable Features to be able to script with this record.

Script Samples

The following sample code snippets show how to create and search for CMS content type records. All CMS content type records are associated with a custom record. For more information, see [Custom Record](#) and [Creating Custom Record Types](#)

Create CMS Content Type Record

The following sample code snippet creates a basic CMS content type record and associates it with the specified custom record.

```

1  var cmsContentTypeRecord = record.create({
2      type: record.Type.CMS_CONTENT_TYPE
3  });
4  cmsContentTypeRecord.setValue({
5      fieldId: 'name',
6      value: 'cms content type name'
7  });
8  cmsContentTypeRecord.setValue({
9      fieldId: 'label',
10     value: 'content type label'
11 });
12 cmsContentTypeRecord.setValue({
13     fieldId: 'description',
14     value: 'My cms content type description'
15 });
16 cmsContentTypeRecord.setValue({
17     fieldId: 'customrecordid',
18     value: 123 // example value
19 });
20
21 var contentTypeID = cmsContentTypeRecord.save();

```

Search CMS Content Type Records

The following sample code snippet searches for all CMS content type records for a given SCA site. It returns the name, site, pagetype, and url fields as search result columns.

```

1 function searchCmsContentTypeRecords(){
2     var searchColumns = search.create({
3         type: search.Type.CMS_CONTENT_TYPE,
4         columns: [{
5             name: 'name'
6         }, {
7             name: 'label'
8         }, {
9             name: 'customrecordid'
10        }]
11    });
12    searchColumns.run().each(function(result){
13        var name = result.getValue({
14            name: 'name'
15        });
16        var label = result.getValue({
17            name: 'label'
18        });
19        var customRecordId = result.getValue({
20            name: 'customrecordid'
21        });
22        return true;
23    });
24 }

```

CMS Page

i Applies to: SuiteScript 2.x | SuiteCloud Developer

CMS page enables you to create different pages for hosting content on your website. For more information, see the help topic [Pages](#).

For help working with this record in the UI, see the help topic [CMS Page Type Record](#).

The internal ID for this record is cmspage.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The CMS page record is scriptable in server SuiteScript only.

Supported Functions

The CMS page record is partially scriptable. It can be created, updated, and searched using SuiteScript. It cannot be copied, attached, or transformed.

Usage Notes

You must enable the Site Management Tools feature on the Web Presence subtab at Setup > Company > Enable Features to be able to script with this record.

Script Samples

Create CMS Page Record

The following sample code snippet creates a basic landing page record (pagetype = 1) for the specified SCA site.

```

1  var recordObj = record.create({
2      type: record.Type.CMS_PAGE,
3      isDynamic: false
4  });
5  recordObj.setValue({
6      fieldId: 'name',
7      value: 'Cms Page Name'
8  });
9  recordObj.setValue({
10     fieldId: 'site',
11     value: 1
12 });
13 recordObj.setValue({
14     fieldId: 'pagetype',
15     value: 1
16 });
17 recordObj.setValue({
18     fieldId: 'cmspagetype',
19     value: 1
20 });
21 recordObj.setValue({
22     fieldId: 'url',
23     value: 'PageUrl'
24 });
25 return = recordObj.save({
26     enableSourcing: false,
27     ignoreMandatoryFields: false
28 });

```

Search CMS Page Records

The following sample code snippet searches for all CMS page records for a given SCA site. It returns the name, site, pagetype, and url fields as search result columns.

```

1  function searchCmsPageRecords(siteId){
2      var searchColumns = search.create({
3          type: search.Type.CMS_PAGE,
4          columns: [{
5              name: 'name'
6          }, {
7              name: 'site'
8          }, {
9              name: 'pagetype'
10         }, {
11             name: 'url'
12         }],
13         filters: [{
14             name: 'site',
15             operator: search.Operator.IS,
16             values: 1 // example siteId
17         }]
18     });
19     searchColumns.run().each(function(result){
20         var name = result.getValue({

```

```

21     name: 'name'
22   });
23   var site = result.getValue({
24     name: 'site'
25   });
26   var pageType = result.getValue({
27     name: 'pagetype'
28   });
29   var url = result.getValue({
30     name: 'url'
31   });
32   return true;
33 });
34 }

```

Commerce Category

i Applies to: SuiteScript 2.x | SuiteCloud Developer

Commerce Categories enable you to create a hierarchical structure of product categories, subcategories, and products.

For help working with this record in the UI, see the help topic [Commerce Categories](#).

In the UI, you can go to Commerce > Content Management > Catalog & Categories > Commerce Categories to access this record.

The internal ID for this record is commercecategory.

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

i Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The commerce category record is scriptable in server SuiteScript only.

Supported Functions

The commerce category record is partially scriptable. It can be created, updated, and searched using SuiteScript. It cannot be copied, attached, or transformed.

Usage Notes

You must enable the Commerce Category feature on the Web Presence subtab at Setup > Company > Enable Features to be able to script with this record. There is now a new sublist that includes the

customersegment and customersegmentsize fields. Note that you cannot create, edit, or delete commerce category sublists through SuiteScript.

Script Sample

The following sample code snippet searches for a commerce category record with a fullurl of "/cat-0" and siteid of 2. It returns fullurl, addtohead, and subcataddtoheadoverride as search result columns.

```

1  var fullurl = '/cat-0';
2  var siteid = 2;
3
4  var mySearch = search.create({
5    type: search.Type.COMMERCE_CATEGORY,
6    filters: [{
7      name: 'fullurl',
8      operator: search.Operator.IS,
9      values: fullurl
10   }],
11   columns: [{
12     name: 'fullurl'
13   }, {
14     name: 'addtohead'
15   }, {
16     name: 'subcataddtoheadoverride'
17   }]
18 });
19 mySearch.run().each(function(result){
20   var name = result.getValue({
21     name: 'fullurl'
22   });
23   var site = result.getValue({
24     name: 'addtohead'
25   });
26   var pageType = result.getValue({
27     name: 'subcataddtoheadoverride'
28   });
29   return true;
30 });

```

Website Setup

Applies to: SuiteScript 2.x | SuiteCloud Developer

The internal ID for this record is websi te. In the UI, you can find this record by going to Commerce > Websites > Website List.

For help working with this record in the UI, see the help topic [Site Builder Web Site Record Settings](#).

See the [SuiteScript Records Browser](#) for all internal IDs associated with this record.

Note: For information about using the SuiteScript Records Browser, see the help topic [Working with the SuiteScript Records Browser](#) in the NetSuite Help Center.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The web site setup record is scriptable in server SuiteScript only.

The web site record is not supported in beforeLoad user event scripts.

Supported Functions

The web site setup record is fully scriptable. It can be created, copied, updated, deleted, and searched using SuiteScript.

Usage Notes

Developers can use SuiteScript with the web site setup record whether they are working in accounts using Site Builder, Commerce web stores, or SCIS.

Creating a Web Site Setup Record in SuiteScript

The following code sample shows how to create a SuiteCommerce Web Site Setup record using SuiteScript 2.X:

```

1  /**
2   * @NApiVersion 2.x
3   */
4
5  require(['N/record'], function(record) {
6      var initValues = new Array();
7      initValues.displayName = "name_of_your_website";
8      initValues.internalname = "name_of_your_website";
9
10     initValues.sitetype = "ADVANCED";
11     initValues.websitescope = "SUITE_COMMERCE";
12
13     var objRec = record.create({
14         type: record.Type.WEBSITE,
15         defaultValues: initValues
16     });
17
18     var recId = objRec.save({
19         enableSourcing: false,
20         ignoreMandatoryFields: false
21     });
22 })

```

To set up any other kind of website, the same code is applicable but the `sitetype` and `websitescope` parameters must contain different values. The required values for the different site types are as follows:

Product	sitetype value	websitescope value
SuiteCommerce	ADVANCED	SUITE_COMMERCE
SuiteCommerce Advanced	ADVANCED	SUITE_COMMERCE_ADVANCED
SuiteCommerce In Store	ADVANCED	SUITE_COMMERCE_IN_STORE
SuiteCommerce My Account	ADVANCED	SUITE_COMMERCE_MY_ACCOUNT
Site Builder	STANDARD	One of the following: FULL_WEB_STORE

Product	sitetype value	websitescope value
		INFO_CATALOG_PRICING INFO_CATALOG INFO_ONLY For more information about these options, see the help topic Multi-Site Settings .



Note: Websitescope field for ADVANCED sites is read-only by default. This means that you cannot modify websitescope for an existing ADVANCED site.

Setting Values for Web Site Setup Dropdown Fields

The following table lists the dropdown fields on the Web Site Setup record. When writing SuiteScript, if you are setting the value of a dropdown field, use the IDs listed in the column called Internal IDs for Dropdown Values.

Subtab	Dropdown Field	UI Labels for Dropdown Values	Internal IDs for Dropdown Values
Setup subtab			
	- Web Site Scope (websitescope) Note: Web Site Scope can be changed only for Site Builder sites.	- Full Web Store - Information And Catalog, With Pricing - Information And Catalog - Information Only	- FULL_WEB_STORE - INFO_CATALOG_PRICING - INFO_CATALOG - INFO_ONLY
	Default Customer Category (defaultcustomercategory)	- Corporate - Individual - Employee	- CORPORATE - INDIVIDUAL - EMPLOYEE
Appearance subtab – applies only to Site Builder			
	Web Site Logo Alignment (websitelogoalign)	- Align Left - Align Right - Align Center	- LEFT - RIGHT - CENTER
	Page Alignment (pagealign)	- Align Left - Align Right - Align Center	- LEFT - RIGHT - CENTER
	Display Order of Cart Items (cartdisplayorder)	- Most Recently Added First - Most Recently Added Last	- RECENT_FIRST - RECENT_LAST
Upsell subtab			
	Items to Upsell (upsellitems)	- Show Related Items First and Upsell Items Next - Show Upsell Items First and Related Items Next - Show Only Related Items	- RELATED_FIRST_UPSELL_NEXT - UPSELL_FIRST_RELATED_NEXT - ONLY_RELATED_ITEMS

Subtab	Dropdown Field	UI Labels for Dropdown Values	Internal IDs for Dropdown Values
		Show Only Upsell Items	ONLY_UPSELL_ITEMS
	Items to Upsell in Cart (cartupsellitems)	- Show Related Items First and Upsell Items Next - Show Upsell Items First and Related Items Next - Show Only Related Items - Show Only Upsell Items	- RELATED_FIRST_UPSELL_NEXT - UPSELL_FIRST_RELATED_NEXT - ONLY_RELATED_ITEMS - ONLY_UPSELL_ITEMS
Legacy subtab – applies only to Site Builder			
	Site Tab Alignment (sitetabalignment)	- Align Left - Align Right - Align Center	- LEFT - RIGHT - CENTER
Shopping subtab			
	Sales Order Type (salesordertype)	Per Customer Basis	PER_CUSTOMER

Shopping Cart

i Applies to: SuiteScript 2.x | SuiteCloud Developer

For help working with this record in the UI, see the help topic [Shopping Cart and Checkout Setup](#).

The internal ID for this record is shoppingcart. You can use the UI to create saved searches with data from the shopping cart. Go to Lists > Search > Saved Searches > New > Shopping cart.

For information about scripting with this record in SuiteScript, see the following:

- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Forms](#)
- [N/record Module](#)

Supported Script Types

The shopping cart record is scriptable in server SuiteScript only.

Supported Functions

The shopping cart record is not fully scriptable. Only search is permitted.

Usage Notes

You can use SuiteScript to create sophisticated marketing campaigns based upon the data linked to the shopping cart. The following data points are exposed through SuiteScript: customer ID, date created, date edited, item IDs for cart items, and website ID.

Note: Because shopping carts can be created by anonymous shoppers, in some search results the customer ID may be null.

Script Samples

The following example uses the [search.create\(options\)](#) method to get shopping cart information.

To get carts from a certain website (id:1) which contains a particular item (item id:352), use the following code sample:

```

1  var mySearch = search.create({
2    type: search.Type.SHOPPING_CART,
3    filters: [{
4      name: 'websiteid',
5      operator: search.Operator.EQUALTO,
6      values: '1'
7    }, {
8      name: 'itemid',
9      operator: search.Operator.EQUALTO,
10     values: '352'
11   }],
12   columns: [{
13     name: 'itemid'
14   }, {
15     name: 'itemqty'
16   }, {
17     name: 'websiteid'
18   }, {
19     name: 'customerid'
20   }]
21 });
22 mySearch.run().each(function(result){
23   var name = result.getValue({
24     name: 'fullurl'
25   });
26   var site = result.getValue({
27     name: 'addtohead'
28   });
29   var pageType = result.getValue({
30     name: 'subcataddtoheadoverride'
31   });
32   return true;
33 });

```